

面试资料合集

Golang 后端开发工程师 · 技术面试备考资料

共 22 个章节

整理日期：2026 年 07 月 16 日

目录

- 00-00 | 整体知识图谱
- 01-01 | Go语言核心与类型系统
- 01-02 | Go并发与运行时
- 01-03 | Go工程化与代码架构
- 01-04 | Go Web与RPC工程实践
- 01-05 | Go性能优化与生产排障
- 02-00 | 核心数据结构与存储引擎索引对比
- 02-01 | Redis缓存
- 02-02 | MySQL数据库
- 02-03 | MongoDB文档数据库
- 02-04 | Elasticsearch搜索引擎
- 03-01 | 消息队列MQ
- 04-01 | 微服务与RPC治理
- 04-02 | 分布式与高并发架构
- 05-01 | Linux操作系统与系统排障
- 05-02 | 计算机网络与HTTP工程
- 06-01 | 系统架构设计方法论
- 06-02 | 项目复盘与资深面试表达
- 07-00 | 算法面试总纲与Golang实现
- 07-01 | 算法70题高频分类清单
- 07-02 | 算法70题高频母模板 | Golang
- 07-03 | 动态规划与图遍历高频背诵清单 | Golang

纲领 | 整体知识图谱思维导图

目标岗位：9 年+ Golang Web 后端开发工程师，主攻业务后端，不求管理岗。

表达策略：用“结论先行 -> 分层展开 -> 面试追问”的金字塔结构回答，突出业务落地、复杂系统治理、线上稳定性和技术判断力。

一、候选人定位

1. 核心标签

结论先行：

你应该被包装成“能独立负责复杂业务域、能扛高并发和稳定性、能带动局部技术方案落地的资深 Go 后端”，而不是纯管理型候选人。

分层展开：

主线定位

- 9 年+ Web 后端开发经验。
- Golang 技术栈为主。
- 能负责复杂业务模块从需求分析、技术方案、编码实现、上线治理到故障复盘的完整闭环。

技术优势

- Go 语言、并发模型、运行时、性能优化。
- MySQL、Redis、消息队列、分布式事务、缓存一致性。
- 微服务、RPC、网关、限流熔断、可观测性。
- 高并发系统设计、稳定性治理、线上排障。

管理经验表达

- 1 年多管理经验作为补充优势：能做任务拆解、方案评审、风险识别、跨团队协作。
- 面试中不要强调“带团队规模”和“管理意愿”，而要强调“能推动复杂业务落地、能辅导协作、能提升交付质量”。

岗位偏好

- 明确表达仍希望做业务后端一线 Owner。
- 可以承担技术方案、核心模块、稳定性治理和项目推进，但不以纯管理岗为目标。

二、技术栈知识图谱

1. Go 语言核心与类型系统

核心模块

- 类型和值语义、string、array、slice、map。
- struct、方法集、interface、function、error。
- 泛型、反射、unsafe、编译与工具链。

对应文档

- [01-01 | Go语言核心与类型系统.md]

2. Go 并发与运行时

核心模块

- goroutine、channel、select、同步原语和内存模型。
- GMP、Netpoller、sysmon、系统调用和 CGO。
- 栈、逃逸分析、内存分配器、GC 和有界并发。

对应文档

- [01-02 | Go并发与运行时.md]

3. Go 工程化与代码架构

核心模块

- package、模块边界、领域分层、接口和依赖方向。
- error、context、配置、日志、测试和代码质量。
- 依赖治理、兼容迁移、CI/CD 和遗留系统重构。

对应文档

- [01-03 | Go工程化与代码架构.md]

4. Go Web 与 RPC 工程实践

核心模块

- net/http、HTTP Client、Transport、连接池和超时。
- REST、gRPC、Protobuf、数据客户端和服务韧性。
- 认证安全、健康检查、优雅停机和业务服务实战。

对应文档

- [01-04 | Go Web与RPC工程实践.md]

5. Go 性能优化与生产排障

核心模块

- metrics、logs、trace、pprof 和 runtime trace。
- CPU、内存、goroutine、锁、连接和依赖排障。
- 容量、压测、过载保护、灰度、恢复和故障复盘。

对应文档

- [01-05 | Go性能优化与生产排障.md]

推荐学习顺序

语言核心与类型系统

- > 并发与运行时
- > 工程化与代码架构
- > Web 与 RPC 工程实践
- > 性能优化与生产排障

6. 缓存与 Redis

核心模块

- Redis 数据结构、缓存一致性、穿透击穿雪崩。
- 分布式锁、持久化、主从、哨兵、Cluster。
- 大 key、热 key、慢查询、线上治理。

对应文档

- [02-01 | Redis缓存.md]

7. MySQL 与数据库

核心模块

- 索引、事务、MVCC、锁、隔离级别。
- SQL 优化、执行计划、分库分表、主从复制。
- 数据一致性、慢 SQL 治理、容量设计。

对应文档

- [02-02 | MySQL数据库.md]

8. 消息队列

核心模块

- Kafka/RocketMQ/RabbitMQ 基础模型。
- 顺序消息、延迟消息、事务消息、重复消费、消息丢失。
- 削峰填谷、异步解耦、最终一致、积压治理。

对应文档

– [03-01 | 消息队列MQ.md]

9. MongoDB 与文档数据库

核心模块

- 文档模型、集合设计、嵌入与引用。
- 索引、聚合管道、事务、读写 concern。
- 副本集、分片集群、容量治理、慢查询排查。

对应文档

– [02-03 | MongoDB文档数据库.md]

10. Elasticsearch 与搜索引擎

核心模块

- 倒排索引、分词、mapping、相关性排序。
- 写入链路、refresh、segment、merge、深分页。
- 集群分片、副本、搜索性能、数据同步和线上治理。

对应文档

– [02-04 | Elasticsearch搜索引擎.md]

11. 微服务与 RPC

核心模块

- HTTP、gRPC、服务注册发现、配置中心。
- 网关、限流、熔断、降级、重试、超时。
- 链路追踪、日志、指标、灰度发布。

对应文档

– [04-01 | 微服务与RPC治理.md]

12. 分布式与高并发架构

核心模块

- CAP、BASE、幂等、分布式事务。
- 秒杀、库存、订单、支付、Feed、搜索、排行榜。
- 热点治理、限流降级、多级缓存、异步化。

对应文档

– [04-02 | 分布式与高并发架构.md]

13. Linux 操作系统与系统排障

核心模块

- 进程、线程、调度、系统调用与上下文切换。
- 虚拟内存、Page Cache、OOM、Cgroup 与容器资源隔离。
- CPU、内存、磁盘 I/O、负载和系统级故障定位。

对应文档

- [05-01 | Linux操作系统与系统排障.md]

14. 计算机网络与 HTTP 工程

核心模块

- TCP、UDP、QUIC、DNS、TLS、HTTP/1.1、HTTP/2 与 HTTP/3。
- 长连接、连接池、超时、重试、负载均衡与网络安全。
- epoll、Socket、NAT、抓包分析和网络故障定位。

对应文档

- [05-02 | 计算机网络与HTTP工程.md]

15. 系统架构设计方法论

核心模块

- 需求澄清、业务不变量、领域建模与接口契约。
- 容量评估、一致性、可用性、性能、安全和成本权衡。
- 架构演进、数据迁移、灰度发布、容灾与方案评审。

对应文档

- [06-01 | 系统架构设计方法论.md]

16. 项目复盘与资深面试表达

核心模块

- 项目背景、个人职责、关键决策、量化结果与业务价值。
- 事故复盘、跨团队协作、技术影响力和管理经验转译。
- 面向 9 年以上业务后端岗位的自我介绍与追问防守。

对应文档

- [06-02 | 项目复盘与资深面试表达.md]

17. 算法与 Golang 编码

核心模块

- 复杂度、数据结构、双指针、滑动窗口、二分、树、图和动态规划。

- 题型识别、循环不变量、边界分析、现场表达与测试方法。
- Golang 工程化实现、并发题、后端场景题和高频母模板。

对应文档

- [07-00 | 算法面试总纲与Golang实现.md]
- [07-01 | 算法70题高频分类清单.md]
- [07-02 | 算法70题高频母模板 | Golang.md]
- [07-03 | 动态规划与图遍历高频背诵清单 | Golang.md]

三、面试优先级

1. 一面重点

结论先行：

一面通常看基本功和编码深度，Go、MySQL、Redis、MQ、网络必须扎实。

重点模块

- Go slice/map/channel/GMP/GC。
- MySQL 索引、事务、锁、SQL 优化。
- Redis 缓存一致性和稳定性治理。
- MQ 可靠性、顺序性、重复消费。
- TCP/HTTP、超时重试、连接池。

2. 二面重点

结论先行：

二面通常看系统设计和业务 Owner 能力，要能把技术点串成完整业务方案。

重点模块

- 高并发读写链路。
- 多级缓存和缓存一致性。
- 分布式事务与最终一致。
- 限流熔断降级。
- 可观测性和故障治理。

3. 三面或交叉面重点

结论先行：

高阶面更关注判断力、取舍、复杂项目推动和长期稳定性。

重点模块

- 为什么这么设计，而不是只说怎么做。
- 成本、性能、一致性、可用性的权衡。

- 项目失败风险和兜底方案。
- 管理经验如何帮助业务交付，但本人仍希望负责业务后端。

四、回答风格模板

1. 技术原理题

结论：

一句话给核心判断。

原理：

从数据结构、执行流程、复杂度、并发模型解释。

场景：

说明适合什么业务。

风险：

指出线上坑。

治理：

给出工程方案、监控指标和取舍。

2. 系统设计题

目标：

先澄清 QPS、数据量、一致性、延迟、可用性。

链路：

拆读链路、写链路、异步链路、补偿链路。

稳定性：

考虑限流、降级、熔断、隔离、幂等、重试。

可观测性：

指标、日志、trace、告警、压测和复盘。

3. 项目深挖题

背景：

业务问题和量级。

职责：

你负责的核心模块和决策。

方案：

技术架构、关键设计、难点处理。

结果：

量化收益。

反思：

方案边界和后续优化。

五、字节红果定向复习路线

1. 你的面试主线

一句话定位：

资深 Golang 后端 IC / 技术骨干，主线是内容平台、广告商业化平台和高并发稳定性治理。

三段经历映射：

- 枫叶互动：出海短剧广告投放平台，映射红果商业化、素材平台、投放平台、数据回流。
- 知乎：动态模板和统一发布，映射红果内容生产、内容发布、客户端动态化、增长实验。
- 360：设备事件消费和统一广告/运营平台，映射高并发事件链路、MQ 削峰、缓存和平台化能力。

2. 技术栈优先级

P0 必须熟：

- Go 并发、GMP、GC、context、pprof、race。
- 算法手写：LRU、限流器、worker pool、TopK、滑动窗口、链表、二叉树、二分。
- Redis：锁、限流、缓存一致性、热 key、大 key、Cluster。
- MySQL：索引、事务、MVCC、锁、慢 SQL、乐观锁。
- MQ：幂等、可靠性、积压、最终一致。
- 项目表达：枫叶互动、知乎、360 各准备 2 分钟版本和深挖版本。

P1 重点准备：

- MongoDB：广告任务文档模型、素材同步幂等、聚合管道。
- ES：素材检索、DB/Mongo 到 ES 同步、灰度切换、fallback 补偿。
- Linux 排障：CPU、内存、网络、P99、MQ lag、Redis/ES 故障。
- 分布式系统：短剧发布系统、广告投放平台、播放页读链路、埋点回流。

P2 有边界地准备：

- Redis zskiplist 等底层数据结构。
- MongoDB 分片、副本集深层原理。
- ES 集群调参细节。
- 微服务注册发现底层实现。

3. 每天复习节奏

上午：算法 + Go

- 1 道中等算法题。

- 1 个手写工程题。
- 复盘复杂度和边界。

下午：技术栈项目化

- 每天选一个技术栈。
- 用“原理 -> 项目场景 -> 风险 -> 治理 -> 追问”复述。

晚上：项目深挖 + 系统设计

- 枫叶互动、知乎、360 轮流讲。
- 每天练一个系统设计题。
- 重点练红果相关：短剧发布、播放页、广告投放、素材检索、埋点回流。

4. 面试前 3 天冲刺

第 1 天：项目

- 枫叶互动广告投放平台完整讲 3 遍。
- 知乎动态模板和统一发布各讲 2 遍。
- 准备“最大难点”“线上故障”“如果重做怎么优化”。

第 2 天：技术

- Redis、MySQL、MQ、Go 并发各过一遍。
- 每个技术栈准备 2 个项目例子。
- 复习 ES/Mongo 的选型边界。

第 3 天：算法和状态

- LRU、限流器、worker pool 必须能默写。
- 复习 10 道高频算法。
- 准备自我介绍、离职原因、不想做组长、为什么红果。

5. 面试回答的统一模板

技术题：

结论 -> 原理 -> 项目场景 -> 风险 -> 治理。

项目题：

背景 -> 职责 -> 方案 -> 难点 -> 结果 -> 反思。

系统设计题：

目标 -> 数据模型 -> 读链路 -> 写链路 -> 异步链路 -> 一致性 -> 稳定性 -> 监控。

管理/职业题：

真实经历 -> 正向收获 -> 下一阶段定位 -> 和岗位匹配。

01-01 | Go语言核心与类型系统

技术 | Go 语言核心与类型系统面试题

面向 9 年以上 Golang Web 后端业务研发岗位。本篇回答“Go 代码为什么具有这些语义、底层怎样表示、工程中哪里会出错”。并发调度、内存分配和生产排障分别在其他 Go 文档深入展开。

一、Go 语言学习地图与资深回答框架

1. Go 语言核心知识图谱是什么？

类型与值语义

- > String / Array / Slice / Map
- > Struct / Method / Interface
- > Function / Closure / Defer / Error
- > Generic / Reflection / Unsafe
- > Package / Init / Compile / Link
- > 性能边界与业务实践

主线不是背语法，而是回答：值怎样复制、数据怎样共享、类型怎样抽象、对象是否逃逸、错误怎样跨层传播，以及代码怎样在长期演进中保持清晰。

2. 资深后端如何回答 Go 语言题？

采用“结论 -> 规范语义 -> 当前实现 -> 工程风险 -> 选择依据”的结构。

例如回答 Slice：先说它是包含指针、长度、容量的描述符；再解释传参复制 Header 但可能共享底层数组；然后说明 append 可能扩容；最后落到并发、内存滞留、预分配和 API 所有权。

3. 语言规范和 Runtime 实现为什么必须区分？

- 规范保证赋值、比较、方法集、Channel 等可观察语义。
- 编译器和 Runtime 可以改变 Map、Slice 扩容、ABI、分配和调度实现。
- 面试可讲当前源码帮助理解，但不能把 Bucket 数、扩容阈值等实现细节当永久契约。
- 业务代码只能依赖规范和公开 API，不能依赖 Map 遍历顺序或内部布局。

4. Go 的设计取舍是什么？

Go 用较少语言特性、显式错误、组合和工具链一致性换取可读性、构建速度和团队协作。代价是部分表达较啰嗦、抽象能力刻意受限、运行时和 GC 带来额外成本。

好的 Go 代码通常“无聊但清楚”：控制依赖、显式处理失败、不过度抽象，并用 Benchmark 和 Profile 证明性能选择。

二、类型、零值、赋值、指针与内存布局

5. Go 的类型系统有哪些核心特点？

- 静态类型、名义类型与结构化接口组合。
- 定义类型与底层类型分离。
- 不支持传统类继承，使用组合和接口。
- 泛型约束描述类型集合，接口描述行为。
- 类型转换显式，避免大量隐式规则。

6. 定义类型、类型别名和底层类型有什么区别？

```
type UserID int64    // 新的定义类型
type ID = int64      // int64 的别名
```

定义类型具有独立身份和方法集，可阻止不同业务 ID 混用；别名主要用于兼容迁移。底层类型决定允许的转换和泛型 `~T` 约束。

生产中推荐为金额、状态、用户 ID 定义明确类型，但不要为每个字段制造无意义包装。

7. Go 为什么强调零值可用？

零值降低初始化分支：`Mutex`、`WaitGroup`、空 Slice、nil Map、nil Channel 都有定义明确的状态。但“有定义”不等于“可以完成所有操作”：nil Map 可读不可写，nil Channel 收发永久阻塞。

类型设计应让零值安全，无法做到时提供构造函数并隐藏字段。

8. Go 是值传递还是引用传递？

Go 只有值传递。传指针会复制指针值；传 Slice、Map、Channel、Function 和 Interface 会复制各自描述符，它们内部可能指向共享数据。

判断副作用不能只看参数是否带 `*`，要看复制后的值是否仍引用同一底层对象。

9. new 和 make 有什么区别？

- `new(T)` 分配零值 T，返回 `*T`。
- `make` 只初始化 Slice、Map、Channel，返回其值而不是指针。
- 字面量和局部变量通常更直观，编译器会决定栈或堆，不必为了“性能”强行使用 `new`。

10. 什么时候使用指针接收者？

- 方法需要修改接收者。
- Struct 较大，不希望复制。
- 类型包含 Mutex 等不可复制状态。
- 希望方法集保持一致。

小型不可变值、时间值或业务 ID 可用值接收者。不要在同一类型上无理由混用；还要考虑 nil 接收者语义和并发共享。

11. Struct 内存对齐为什么重要？

CPU 按对齐边界访问，编译器会在字段之间加入 Padding。字段顺序可能影响 Struct 大小，海量对象中会放大内存。

可用 `unsafe.Sizeof/Alignof/Offsetof` 观察，但调整前要确认对象数量和 Profile；可读性和序列化契约通常比省几个字节重要。

12. 哪些类型可以比较？

布尔、数字、字符串、指针、Channel、Interface 在满足条件时可比较；数组要求元素可比较；Struct 要求所有字段可比较。Slice、Map、Function 除与 nil 比较外不可比较。

Interface 比较会比较动态类型和值，如果动态值不可比较会 Panic。这也是 `map[any]V` 的风险之一。

三、String、Byte、Rune、Array 与 Slice

13. String、[]byte 和 []rune 如何选择？

- String 是只读字节序列，可包含任意字节，不保证 UTF-8。
- `[]byte` 适合网络、文件、编码和可变二进制数据。
- `[]rune` 按 Unicode Code Point 操作，方便字符级处理但占用更大。

`len(s)` 返回字节数；`range` 解码 UTF-8，索引是当前 Rune 的字节偏移。用户感知字符还可能由多个 Rune 组成。

14. String 为什么不可变，转换有什么成本？

不可变便于共享和作为 Map Key，也允许编译器优化。String 与 `[]byte` 的普通转换通常需要复制，热路径反复转换会增加分配。

网络处理中应明确所有权并复用 Buffer；禁止为省一次复制随意用 Unsafe 绕过不可变性。

15. Array 和 Slice 有什么区别？

Array 长度属于类型，赋值会复制全部元素；Slice 是底层数组的视图，包含 data、len、cap。Slice Header 被复制后，多个 Slice 仍可能共享数组。

固定长度密钥、哈希值适合 Array；业务动态集合通常使用 Slice。

16. Slice 的底层结构是什么？

概念上包含：

```
data -> 底层数组
len  -> 当前可访问元素数
cap  -> 从 data 起可扩展的最大元素数
```

这解释了切片 O(1)、共享修改、append 是否覆盖原数据和小切片持有大数组。

17. Slice 共享底层数组有哪些陷阱？

```
a := []int{1, 2, 3, 4}
b := a[:2]
b = append(b, 9) // cap 足够，可能覆盖 a[2]
```

需要隔离 append 时使用完整切片表达式 `a[:2:2]` 或显式 `copy`。API 若保留调用方 Slice，应复制或明确所有权，防止调用方后续修改产生隐蔽 Bug。

18. Slice 扩容规则是什么？

当 append 后长度超过容量，Runtime 分配新数组并复制元素。增长策略和 Size Class 属于实现细节，会随版本调整；工程上只需依赖“可能重新分配和复制”。

能估算数量时 `make([]T, 0, n)` 可减少分配，但过度预分配会浪费内存，必须以 `allocs/op` 和 Profile 验证。

19. Slice 为什么会造成内存滞留？

小 Slice 只要仍指向大数组，大数组就无法回收。读取大文件后只保留一小段、缓存大响应的子 Slice 都可能发生。

需要长期保存小片段时使用 `bytes.Clone`、`strings.Clone` 或 `copy` 到合适大小的新对象。

20. nil Slice 和空 Slice 有什么区别？

两者 `len/cap` 都为 0，均可 append；nil Slice 等于 nil，空 Slice 不一定。JSON 编码可能分别输出 `null` 和 `[]`，因此 API 契约必须明确。

内部代码优先零值 nil Slice；外部响应若要求数组，序列化前规范化。

21. 如何安全删除 Slice 元素？

保持顺序可用 `copy(s[i:], s[i+1:])` 后缩短；不保持顺序可用末尾覆盖。元素含指针时应清零被移除位置，避免底层数组继续持有大对象。

批量过滤推荐原地复用 `dst := s[:0]`，但要明确是否允许修改原数组。

四、Map 语义、实现演进与工程边界

22. Map 的语言语义是什么？

Map 是无序键值集合，Key 必须可比较。读取不存在 Key 返回 Value 零值；comma-ok 区分不存在与零值；nil Map 可读和删除，但写入会 Panic。

遍历顺序未定义且可能变化，业务排序必须提取 Key 后显式排序。

23. Map 的底层设计目标是什么？

核心目标是通过哈希将 Key 定位到小范围槽位，在负载升高时扩容，同时平衡查询、写入、内存局部性和迁移成本。

具体实现已发生过重大演进。学习源码应以当前 Go 版本为准；面试重点是哈希、控制信息、冲突、负载、渐进扩容和缓存局部性，而非死背旧 Bucket 字段。

24. 经典 Bucket Map 如何解决哈希冲突？

经典实现将哈希高位作为快速比较信息，低位选择 Bucket；同 Bucket 多个槽保存 Key/Value，溢出时链接 Overflow Bucket。扩容期间旧 Bucket 渐进迁移，避免一次 Stop-the-world 搬完。

这些知识有助理解历史版本和渐进扩容，但业务不能依赖 Bucket 数和阈值。

25. 新式表组设计为什么改善 Map？

新式设计借鉴 Swiss Table 思想，使用控制字节组、并行匹配和开放寻址，提高缓存局部性并减少指针追逐；大 Map 可拆为多个表并渐进增长。

应该掌握“连续内存 + 控制信息 + 探测”的方向，同时明确源码字段和扩容细节仍可能变化。

26. Map 为什么不能并发读写？

写入可能修改表结构、迁移状态和槽位；无同步的并发读写产生 Data Race，Runtime 还可能检测到并直接 Fatal。没有 Panic 不代表安全。

使用 `Mutex/RWMutex + map`、分片 Map、`sync.Map` 或单 Goroutine 所有权；选择依据是读写比例、Key 稳定性和复合操作原子性。

27. sync.Map 为什么不是默认选择？

它针对写一次读多和不同 Goroutine 操作不相交 Key 等场景优化，API 使用 `any`，复合不变量不自然。普通业务状态用带类型的 Map + Lock 通常更清晰。

必须通过并发 Benchmark 比较，不要因“并发安全”四个字替代设计。

28. Map 作为参数会复制数据吗？

复制的是 Map 值描述符，底层表共享。函数内修改元素对调用方可见；给局部变量重新赋一个新 Map 不会改变调用方变量。

跨层传 Map 易产生隐式共享，配置和响应数据需要不可变约定或防御性复制。

29. 为什么 Map 元素不能直接取地址？

元素位置可能因扩容和迁移变化，语言不允许对 `m[k]` 直接取稳定地址。需要修改 Struct 元素时先取出、修改、写回，或存 `*T`。

存指针减少复制但增加分配、GC 和共享可变状态，不能机械选择。

30. Map 如何预估容量和降低 GC 成本？

已知规模时 `make(map[K]V, n)` 可降低增长成本。Key/Value 含大量指针会增加 GC 扫描；紧凑值、索引 ID 或分层结构可能更省内存。

优化前用 Heap Profile、对象数量和实际延迟验证，避免牺牲代码清晰度。

五、Struct、组合、方法集与 Interface

31. Go 为什么使用组合而不是继承？

组合显式表达“拥有”和能力复用，减少深层继承和脆弱基类。匿名嵌入会提升字段和方法，但不等于子类型；冲突时仍需显式选择。

业务领域优先小对象组合，避免用嵌入模拟复杂继承树。

32. 方法集如何影响接口实现？

`T` 的方法集包含值接收者方法；`*T` 包含值和指针接收者方法。因此若接口方法由指针接收者实现，通常只有 `*T` 满足接口。

可寻址变量调用时编译器可能自动取地址，但接口赋值遵循方法集规则，二者不要混淆。

33. Interface 的底层概念是什么？

Interface 值包含动态类型和动态值。空接口和带方法接口内部表示不同，但工程语义相同：只有二者都为空时 Interface 才等于 nil。

```
var p *MyError
var err error = p
fmt.Println(err == nil) // false
```

返回 Error 时不要把 nil 具体指针装进接口。

34. 接口应该由谁定义？

通常由消费者在需要的地方定义最小接口，生产者返回具体类型。这样接口围绕使用场景，减少“一个大接口服务所有人”。

跨模块稳定契约可以集中定义，但不应为每个 Struct 机械生成接口。

35. 接口越小越好吗？

小接口易实现、测试和组合，但过度拆成大量单方法接口也会增加导航成本。边界应表达完整业务能力，同时遵循 Interface Segregation。

36. 类型断言和 Type Switch 有什么风险？

单返回值断言失败会 Panic，生产代码通常使用 comma-ok。大量 Type Switch 往往说明抽象层级不当；插件、解码和边界适配可合理使用。

any 不应成为逃避领域类型设计的通用容器。

37. Interface 调用一定很慢吗？

可能涉及动态分派并影响内联和逃逸，但实际成本常小于 IO、序列化和分配。编译器也可能去虚拟化。

只有 Profile 显示热路径受影响时，才考虑泛型、具体类型或代码生成。

六、Function、Closure、Defer、Panic 与 Error

38. Function 为什么是一等值？

函数可赋值、传参和返回，适合策略、回调和中间件。函数值可能携带环境形成 Closure，也可能影响内联和逃逸。

业务中函数参数应表达局部策略，不要用大量回调隐藏控制流。

39. Closure 如何捕获变量？

闭包捕获变量而非某次快照，变量可能被提升到堆。循环启动 Goroutine 时要确认当前 Go 版本的循环变量语义，并对外层复用变量、指针和并发修改保持警惕。

查看 `-gcflags='-m=2'` 和 Race Detector，不靠经验判断逃逸与竞争。

40. Defer 的执行规则是什么？

- 注册时计算函数值和参数。
- 当前函数返回前按后进先出执行。
- 可读取或修改命名返回值。
- 即使 Panic 也会沿栈执行已注册 Defer。

编译器已优化常见 Defer，但热循环内大量注册仍需 Benchmark。资源获取成功后立即 Defer 释放通常最安全。

41. Defer 在循环中为什么可能出问题？

Defer 在外层函数返回时才执行，循环中打开大量文件/结果集会长期占资源。将单次迭代抽成函数，让 Defer 在每轮结束时执行。

42. Panic/Recover 应该怎样使用？

Panic 表示程序不变量破坏或无法继续的异常，不替代业务 Error。Recover 只能在同一 Goroutine 的 Defer 中捕获。

HTTP/RPC、消息 Worker 边界可 Recover 防进程退出，但必须记录 Stack、返回明确错误并保持数据一致性；不能吞掉后继续假装成功。

43. Go Error 应如何分类？

- 参数和校验错误。
- 业务冲突与状态错误。
- 认证授权错误。
- 依赖瞬时错误与永久错误。
- 内部不变量和数据一致性错误。

分类服务于上层决策：状态码、重试、告警、补偿和用户提示，而不只是日志文本。

44. errors.Is/As/Join 如何使用？

- `%w` 保留错误链上下文。
- `errors.Is` 判断哨兵或自定义 `Is` 语义。
- `errors.As` 提取类型化错误。

- `errors.Join` 聚合多个独立失败。

不要依赖错误字符串判断逻辑；包装时增加“哪一步、哪个资源”，同时避免敏感数据。

45. 错误应该在哪一层记录日志？

通常由最了解最终影响并决定处理方式的边界层记录一次。底层包装上下文并返回；每层都打印会产生重复噪声和多次堆栈。

日志包含 `error_code`、`operation`、`resource_id`、`trace_id` 和重试次数，业务参数脱敏。

七、泛型、反射、Unsafe 与序列化边界

46. 泛型解决什么问题？

泛型为容器、算法和基础库提供编译期类型安全，减少 `any` 和重复代码。它不替代接口：泛型表达一组类型上的通用实现，接口表达运行时行为抽象。

业务代码若只有两个简单类型，复制几行可能比复杂约束更易维护。

47. `any`、`comparable` 和 `~T` 如何理解？

- `any` 是空接口别名，允许任意类型。
- `comparable` 表示可使用 `==/!=` 的类型集合，常用于 Map Key。
- `~T` 包含底层类型为 T 的定义类型。

约束应只包含实现真正需要的操作，避免建立庞大“万能数字”体系。

48. 泛型的性能一定比 Interface 好吗？

不一定。编译器实现可能字典传递、共享代码或按形状生成代码，内联和逃逸结果也不同。泛型的首要收益是类型安全；性能必须 Benchmark 和看生成代码。

49. 反射的核心对象是什么？

`reflect.Type` 描述类型，`reflect.Value` 描述值及可设置性。反射适合序列化、ORM、DI 框架和通用工具；普通业务热路径优先静态类型或代码生成。

反射要处理 `Invalid`、`nil`、`Kind`、可导出字段和 `Panic` 边界，并缓存类型元数据。

50. Unsafe 可以用在哪里？

只在边界清晰、收益可测、测试充分的底层库中使用，例如系统调用、内存映射和特定零拷贝优化。必须遵守指针转换和对象生命周期规则，并用 `checkptr`、`Race`、`Fuzz` 和多架构测试。

普通 Web 业务不应为了少一次转换牺牲 GC 正确性和可维护性。

51. JSON 序列化有哪些常见坑？

- 未导出字段不会编码。
- `omitempty` 与零值、指针、空集合语义要明确。
- `interface{}` 中数字默认解码精度风险。
- 时间、金额和大整数需要稳定格式。
- nil Slice 与空 Slice 输出不同。
- 未知字段是否拒绝取决于兼容策略。

外部契约使用 DTO，不直接暴露数据库 Struct；高 QPS 优化先减少字段和次数，再更换库。

八、Package、初始化、编译、链接与工具链

52. Package 的设计原则是什么？

围绕单一业务能力组织，名称短且表达职责；避免 `util/common/base` 无限膨胀；依赖由业务层指向抽象和基础设施边界，禁止循环依赖。

Internal Package 用于限制模块外访问，Public API 应小而稳定。

53. init 的顺序和风险是什么？

依赖包先初始化，同包按编译器确定的文件顺序完成变量初始化后执行各 `init`。业务不应依赖文件顺序或隐藏副作用。

注册驱动等框架场景可用 `init`；网络连接、启动 Goroutine 和复杂配置应显式放入构造和启动流程，便于测试和失败处理。

54. Go 编译大致经历什么？

词法/语法与类型检查 -> 中间表示 -> 内联、逃逸、边界检查等优化 -> SSA/机器码 -> 链接。编译器还生成 GC 元数据、方法包装和运行时所需信息。

面试应把编译优化与可观察结果连接：是否内联、是否逃逸、是否消除 Bounds Check，而非背阶段名称。

55. 内联为什么影响性能和逃逸？

内联消除调用开销并让跨函数优化可见，也可能帮助逃逸分析。函数过大、复杂控制流或某些特性会限制内联。

使用 `-gcflags='-m=2'` 观察，不要为了强行内联破坏代码结构。

56. Bounds Check Elimination 如何理解？

Slice/Array 索引默认检查越界；编译器能证明范围安全时会消除。合理的循环形态通常足够，手工改写前应看编译诊断和 Benchmark。

即使追求性能也不能用 Unsafe 绕过不可信输入的边界检查。

57. CGO 的收益和代价是什么？

CGO 可复用 C 库和系统能力，但跨边界调用、线程、调度、内存所有权、构建、交叉编译和部署更复杂。Go Pointer 不能被 C 任意长期持有。

高频小调用应批量化；能用纯 Go 满足时通常更易部署和排障。

九、语言级性能、API 设计与生产代码边界

58. 如何判断一次语言级优化是否值得？

先定义业务指标，写 Benchmark，查看 ns/op、B/op、allocs/op，再结合 CPU/Heap Profile 确认热点。优化后用真实负载验证 P99、CPU 和内存，不以微基准替代端到端结果。

59. 常见无意义分配来自哪里？

String/Byte 转换、Interface 装箱、反射、闭包捕获、`fmt`、临时 Slice/Map、JSON 中间对象和逃逸。处理顺序是减少工作量、批量化、预分配、复用，最后才考虑 Unsafe。

60. API 参数应该传值还是指针？

依据语义而非统一规则：小不可变值传值；需要修改、区分缺失与零值、对象较大或不可复制时用指针。Slice/Map 本身已是小描述符，不要机械传 `*[]T` 或 `*map[K]V`。

61. 如何设计不可变数据和所有权？

Go 没有不可变关键字，需要靠未导出字段、构造函数、返回副本和文档约定。跨 Goroutine 传递后不再修改，或明确单一 Owner，能显著降低锁和竞态。

62. 代码评审中如何检查类型和内存风险？

- 是否错误共享 Slice/Map/Pointer。
- Interface 是否可能 Typed nil 或比较 Panic。
- 大对象是否被闭包、缓存或子 Slice 持有。
- Error 是否保留语义并避免重复日志。
- 反射/Unsafe 是否有必要和测试。
- API 是否明确 nil、空集合、单位和所有权。

十、业务架构与项目实战

63. 广告素材批量处理如何使用 Slice?

分页读取时按当前批次预分配 Slice，处理完立即释放引用；不要一次加载全量素材。传给异步 Worker 前明确复制还是转移所有权，防止主协程复用 Buffer 后数据被覆盖。

优化用批次耗时、峰值 Heap、allocs/op 和 GC CPU 验证。

64. 第三方 SDK Map 并发问题如何表达?

场景是高并发调用下偶发 Fatal/Data Race。通过 `go test -race` 和压力场景定位 SDK 内共享 Map 并发写；短期在适配层串行或加锁，长期升级/修复 SDK，并增加并发回归测试。

亮点不是“加了一把锁”，而是识别所有权、限定影响边界并用工具验证。

65. 如何设计稳定的业务 Error?

领域层返回类型化业务错误；基础设施层包装原始原因；应用层映射 API/MQ 语义；入口统一记录一次日志和指标。第三方错误保存平台 code、request_id 和 retryable，但不泄露 Token。

66. Map 新旧实现面试时应该怎样回答?

Map 的稳定语义是 Key 可比较、遍历无序、并发读写不安全。底层目标是哈希定位、冲突处理和渐进增长。历史实现用 Bucket 和 Overflow，新实现更强调控制字节组、开放寻址和缓存局部性。源码细节随 Go 版本演进，因此我不会让业务依赖内部布局；工程上更关注容量预估、Key 分布、共享所有权、锁策略和 Heap/GC 成本。

67. 如何用两分钟回答 Go 语言核心题?

我理解 Go 的核心不是语法清单，而是值语义与共享边界。所有参数都是值传递，但 Slice、Map、Interface 等描述符会引用底层数据，所以 API 设计要明确所有权、复制和并发。容器方面关注 Slice 扩容与内存滞留、Map 无序和并发边界；类型抽象方面关注方法集、消费者定义小接口和 Typed nil；失败处理使用可分类、可包装的 Error，Panic 只在边界 Recover。泛型用于编译期通用代码，反射和 Unsafe 只在框架或经验证的热点。实现细节会随编译器和 Runtime 演进，我用编译诊断、Benchmark、Race 和 Profile 验证，而不依赖内部布局。最终目标是让业务代码语义清楚、可维护并能在网上证明性能。

01-02 | Go并发与运行时

技术 | Go 并发与运行时面试题

面向 9 年以上 Golang Web 后端业务研发岗位。本篇沿“代码语义 -> Runtime 执行 -> 资源成本 -> 生产模式”展开，重点深入 Channel、GMP、内存模型、逃逸、分配器、GC 和有界并发。

一、Go 并发与 Runtime 学习地图

1. Go 并发与 Runtime 的知识主线是什么？

```
Goroutine / Channel / Select
-> Mutex / Atomic / Memory Model
-> G / M / P / Run Queue / Netpoller
-> Stack / Escape Analysis
-> mcache / mcentral / mheap
-> GC / Write Barrier / Pacer / Scavenger
-> Context / Worker Pool / Backpressure
-> Race / Deadlock / Leak / Production Practice
```

2. 资深后端如何回答并发题？

采用“并发目标 -> 共享状态 -> 同步关系 -> 生命周期 -> 资源上限 -> 故障与观测”。不要只说“开 Goroutine 加 Channel”，而要说明谁创建、谁取消、谁关闭、错误如何传播、并发多少、下游能否承受。

3. 规范语义和 Runtime 源码如何结合？

先回答语言保证，例如 Channel 收发、锁和 Atomic 建立的 happens-before；再用 `hchan`、`sudog`、GMP 等源码解释成本。源码字段可变，规范语义和工程约束才是业务依赖。

4. 并发、并行和异步有什么区别？

- 并发：多个任务在时间上重叠推进。
- 并行：多个任务同一时刻在不同 CPU 上执行。
- 异步：发起后不阻塞等待最终结果的交互方式。

Goroutine 提供并发抽象，是否并行取决于可用 P、CPU 和阻塞状态；异步任务还需要持久化状态和可靠队列。

二、Goroutine、Channel、Select 与通信语义

5. Goroutine 为什么比线程轻？

Goroutine 初始栈较小且可增长，由 Runtime M:N 调度到 OS Thread，创建和切换通常低于线程。但它仍占栈、调度元数据和所引用对象；百万泄漏 Goroutine 一样会拖垮服务。

6. Channel 解决什么问题？

Channel 同时传递数据和建立同步关系，适合所有权转移、任务队列、结果汇聚和事件通知。它不是“比锁高级”；共享状态的短临界区通常用 Mutex 更直接。

7. hchan 的核心结构是什么？

概念字段包括：

```
qcount/dataqsiz -> 当前数量与容量
buf              -> 环形缓冲区
sendx/recvx     -> 写入与读取位置
sendq/recvq     -> 等待发送/接收的 sudog 队列
closed          -> 关闭状态
elemtype/size   -> 元素类型和尺寸
lock            -> 保护 Channel 状态
```

因此 Channel 不是无锁队列；锁保证内部一致性，Runtime 再优化直接传递和快速路径。

8. 无缓冲 Channel 的发送链路是什么？

1. `chansend` 检查是否有等待 Receiver。
2. 有 Receiver 时直接从 Sender 栈复制到 Receiver 目标并唤醒。
3. 没有 Receiver 时构造 `sudog`，记录当前 G 和元素地址。
4. 将 `sudog` 放入 `sendq`，调用 `gopark` 挂起当前 G。
5. Receiver 到来后配对复制，通过 `goready` 使 Sender 可运行。

它表达同步交接，不提供排队容量。

9. 有缓冲 Channel 的发送和接收如何执行？

- 有等待对端时优先直接配对。
- 无等待 Receiver 且缓冲未滿时，复制到 `buf[sendx]` 并推进环形索引。
- 缓冲滿时 Sender 进入 `sendq`。
- Receiver 取缓冲头元素；若有等待 Sender，可将其元素补入缓冲并唤醒。

元素按值复制，大 Struct 会增加复制成本；传指针减少复制但引入共享和 GC 扫描。

10. sudog、gopark 和 goready 分别做什么？

`sudog` 表示 Goroutine 在某个同步对象上的等待关系；`gopark` 将当前 G 置为等待状态并让出 M；`goready` 将目标 G 变为 Runnable，放入运行队列等待调度，不代表立即执行。

11. 关闭 Channel 时 Runtime 做什么？

`closechan` 标记关闭，唤醒所有等待 Receiver：它们得到零值和 `ok=false`；唤醒所有等待 Sender：恢复后发生 send-on-closed Panic。重复关闭和关闭 nil Channel 都会 Panic。

关闭是广播完成信号，不负责释放 Channel 内存；对象不可达后才由 GC 回收。

12. Channel 的关闭原则是什么？

通常由唯一 Sender 或明确 Owner 关闭，Receiver 不关闭不拥有的 Channel。多 Sender 场景由协调者等待所有 Sender 结束后关闭，或只通过 Context 通知退出而不关闭共享输入。

不要用 Recover 掩盖重复关闭，应该修正所有权。

13. nil、open、closed Channel 的行为是什么？

操作	nil	open	closed
send	永久阻塞	正常/可能阻塞	Panic
receive	永久阻塞	正常/可能阻塞	立即零值、ok=false
close	Panic	成功	Panic

将 Channel 设 nil 可在 Select 中动态禁用 Case，但误用会造成永久阻塞。

14. Select 的底层和选择语义是什么？

Select 先随机化轮询顺序，再按稳定锁顺序锁定涉及 Channel，检查就绪 Case；无就绪且无 Default 时把当前 G 的 `sudog` 注册到多个等待队列并挂起。某个 Case 唤醒后清理其他队列登记。

多 Case 就绪时伪随机选择，不保证业务公平；带 Default 的空转循环会烧 CPU。

15. Channel Buffer 应该设多大？

容量代表允许的突发和在途任务，不是越大越好。依据生产速率、消费速率、可接受等待、单元素内存和故障恢复设计。Buffer 只能吸收短期波动，长期生产大于消费最终仍会满。

必须监控长度、等待时间、拒绝数和最老任务年龄，并提供背压。

16. Channel 与 Mutex 如何选择？

- 所有权转移、任务流水线和事件通知：Channel。

- 保护内存中小块共享状态：Mutex。
- 单值计数/状态：Atomic。
- 复杂状态机可由单 Owner Goroutine 串行处理，但要评估队列和故障。

以语义清晰和压测为准，不奉行“不要通过共享内存通信”的字面教条。

三、同步原语、Atomic 与 Go Memory Model

17. Mutex 的工作状态和饥饿模式如何理解？

Mutex 有快速 CAS 路径，竞争时进入自旋或信号量等待；长期等待会切换更公平的交接倾向，降低尾部饥饿。具体位字段属于实现细节。

临界区应短，不在锁内做网络 IO；用 Mutex Profile 证明竞争。

18. Mutex 和 RWMutex 如何选择？

读多不等于 RWMutex 一定更快。读临界区足够长、并发高、写少时才可能受益；写到来会阻止新读进入。普通短临界区优先 Mutex，再 Benchmark。

19. 复制 Mutex 为什么危险？

Mutex、RWMutex、WaitGroup、Once 等使用后不可复制，复制会产生两个表面独立但状态错误的同步对象。含锁 Struct 使用指针接收者，通过 `go vet copylocks` 检查。

20. WaitGroup 常见错误有哪些？

- `Add` 与 `Wait` 生命周期竞争。
- Goroutine 内才 Add，主协程可能先 Wait 返回。
- 忘记 Done 或 Panic 导致永不结束。
- 计数变负 Panic。
- 复用轮上一轮 Wait 尚未结束。

任务创建前 Add，Goroutine 开头 `defer Done`；需要错误传播和取消优先 `errgroup`。

21. Once、Cond 和 Pool 分别适合什么？

- Once：并发只执行一次初始化；函数 Panic 也视为执行过。
- Cond：在锁保护条件变化时唤醒等待者，必须循环检查条件。
- Pool：跨 GC 周期可被清空的临时对象复用，不是缓存和资源池。

22. Atomic 适合什么场景？

独立计数、只读配置指针、简单状态位。多个字段构成不变量时用锁或不可变对象 + `atomic.Pointer/Value` 整体替换。

Atomic 正确不代表业务原子，CAS 循环还可能争用和 ABA。

23. 什么是 Data Race？

两个 Goroutine 并发访问同一内存，至少一个写，且没有同步关系。结果可能受编译器优化和 CPU 重排影响，不能用“机器上一直正常”证明安全。

`go test -race` 只能发现执行覆盖路径上的 Race，不能证明不存在。

24. Go Memory Model 中重要的 happens-before 有哪些？

- Channel send happens-before 对应 receive 完成。
- close happens-before Receiver 观察到关闭。
- Mutex Unlock happens-before 后续成功 Lock。
- Once 函数完成 happens-before 任意 Do 返回。
- Atomic 操作提供文档定义的同步语义。
- Goroutine 创建本身不保证其完成对外可见，必须同步等待。

25. Double-Checked Locking 为什么容易错？

无同步读取初始化标志和对象会产生 Race，可能看到标志但对象尚未完整可见。使用 `sync.Once` 或 `atomic.Pointer` 的完整发布模式，不手写不受证明的双检锁。

四、GMP、调度、Netpoller、系统调用与 CGO

26. GMP 模型是什么？

- G: Goroutine，保存栈、状态和调度信息。
- M: OS Thread，真正执行机器指令。
- P: 执行 Go 代码所需的调度资源，持有本地 Run Queue 和 mcache。

M 必须绑定 P 才能执行 Go 代码；阻塞系统调用时 P 可转交其他 M。

27. 新 Goroutine 放到哪里？

通常进入当前 P 的本地队列，部分情况下进入全局队列或 `runnext`。调度器综合本地、全局、Netpoller 和 Work Stealing 寻找 Runnable G。

具体频率会变化，理解局部性和负载均衡即可。

28. Work Stealing 如何工作？

空闲 P 从其他 P 的本地队列窃取一部分 G，避免某个 P 堆积而其他 CPU 空闲。它提高利用率，但大量短任务、频繁唤醒和锁竞争仍会产生调度成本。

29. Goroutine 在哪些时机发生调度？

Channel/Lock 阻塞、系统调用、网络等待、主动 `Gosched`、函数调用安全点的抢占、GC 协作等。现代 Go 支持更强的异步抢占，降低计算循环长期独占 P 的风险，但业务仍应避免无界 CPU 循环。

30. Sysmon 做什么？

后台监控长时间运行 G、系统调用中的 P、Netpoller、定时器和 GC 相关任务，触发抢占与资源回收。它是 Runtime 维持调度和时间进展的重要组件。

31. Netpoller 如何让网络 IO 不阻塞线程？

`net` 包使用非阻塞 FD 和操作系统多路复用。G 等待 IO 时由 Netpoller 登记并 Park，事件就绪后变为 Runnable；M 可执行其他 G。

这解释了 Go 适合高并发 IO，但每个连接仍消耗 FD、Buffer、G 和业务内存。

32. 阻塞系统调用发生什么？

Runtime 可将 P 从阻塞 M 分离，让新 M 接管 P 执行其他 G；调用返回后原 M 重新获取 P 或把 G 放回队列。大量不可感知的阻塞 CGO 调用仍可能创建许多线程。

33. GOMAXPROCS 应如何设置？

它限制同时执行 Go 代码的 P 数，不等于 Goroutine 或线程数。CPU 密集任务通常接近可用 CPU；容器中应与 CPU 配额协调。调高超过实际 CPU 可能增加调度，调低会限制并行。

用负载测试观察吞吐、P99、CPU throttling 和 Run Queue。

34. CGO 对调度有什么影响？

调用 C 需要跨边界，可能阻塞 OS Thread；C 持有 Go Pointer 和回调还受严格规则。高频小调用要批量化，监控线程数和 CGO 调用耗时，部署上处理动态库和交叉编译。

五、Goroutine 栈、逃逸分析与编译器决策

35. Goroutine 栈如何增长？

G 使用可增长连续栈。函数入口检查空间，不足时分配更大栈、复制内容并修正栈内指针，随后继续执行；栈也可收缩。

Go Pointer 在合法代码中不会因栈移动失效，Runtime 和编译器维护指针元数据。

36. 什么是逃逸分析？

编译器判断对象生命周期是否超出当前栈帧、地址是否可能被外部保留，从而决定栈或堆。返回局部变量地址在 Go 中安全，编译器会让对象活得足够久。

37. 常见逃逸场景有哪些？

- 返回指针或被长生命周期对象保存。
- Closure 捕获变量。
- Interface/反射使具体类型和生命周期难确定。
- 发送指针到可能更长生命周期的 Goroutine。
- 对象过大或编译器无法安全放栈。

这些是倾向，不是永久规则，应查看当前编译结果。

38. 如何查看逃逸分析？

```
go build -gcflags='all=-m=2' ./...  
go test -gcflags='all=-m=2' ./path
```

输出很嘈杂，关注业务 Package 和热点函数；再用 Benchmark 的 B/op、allocs/op 与 Heap Profile 验证真实影响。

39. 逃逸一定需要优化吗？

不一定。对象数量小或不在热路径时，清晰代码更重要。优化优先减少算法工作量和对象生命周期，不要为“零逃逸”改成全局池或 Unsafe，反而增加共享和泄漏。

40. Stack Overflow 和深递归如何处理？

栈可增长但不是无限，深递归会持续扩栈并最终崩溃。图遍历和不可信深度数据应考虑迭代写法、深度限制和输入校验。

六、mcache、mcentral、mheap 与内存分配

41. Go 堆分配器的整体层次是什么？

```
当前 P 的 mcache  
-> 对应 Size Class 的 mspan  
-> mcentral 获取可用 span
```

-> mheap 管理 Page、Arena 和大对象
-> OS 虚拟/物理内存

P-local mcache 让小对象常见分配避免全局锁。

42. Span、Page 和 Size Class 是什么？

Heap 按 Page 管理，连续 Page 组成 Span；小对象按 Size Class 放入固定槽位。相同 Class 对象共享 Span，分配是找到空槽。

对象实际占用可能向上取整，尺寸分布不佳会产生内部碎片。

43. Tiny、小对象和大对象如何分配？

无指针的极小对象可经 Tiny Allocator 合并；普通小对象从 mcache 对应 Class 分配；超过小对象范围的大对象直接向 mheap 申请合适 Page。

阈值属于实现细节，工程关注大对象和高频小对象对 GC、碎片与延迟的影响。

44. 为什么每个 P 有 mcache？

执行 Go 代码必须持有 P，因此分配可从 P-local Cache 快速完成，减少多线程竞争。G 切换 P 后使用新 P 的 mcache，不拥有固定 Allocator。

45. sync.Pool 如何降低分配？

Pool 保存临时可复用对象并利用 P-local 结构降低竞争，GC 时对象可能被丢弃。适合 `bytes.Buffer`、编码临时对象；取出必须 Reset，放回前清除大引用并限制异常大 Buffer。

不能放数据库连接、业务缓存和必须存在的对象。

46. 为什么释放对象后 RSS 不立即下降？

GC 证明对象不可达后，内存先成为 Runtime 可复用空间；Scavenger 再按策略将空闲 Page 归还 OS。HeapAlloc 下降、HeapIdle 上升而 RSS 暂时不降可能正常。

判断泄漏要看 GC 后存活对象趋势和 Profile，而非一次 RSS。

47. 常见内存指标如何区分？

- HeapAlloc：当前已分配存活/未回收堆字节。
- HeapInuse：标记为堆使用的 Span 字节。
- HeapIdle：保留但当前空闲的堆内存。
- HeapReleased：已归还 OS 的空闲内存。
- Sys：Runtime 从 OS 获取的各类内存总量。
- RSS：操作系统观察的驻留物理内存，包含 Go Heap 之外内容。

七、GC、写屏障、Pacer、GOGC 与 GOMEMLIMIT

48. Go GC 的主要目标是什么？

并发、低延迟、非分代标记清扫。它在吞吐、内存和暂停之间取舍：更多 Heap 可降低 GC 频率，更多 GC CPU 可降低内存峰值。

49. 三色标记如何理解？

白色未发现，灰色已发现待扫描，黑色已扫描。GC 从 Root 出发并发标记可达对象，最终白色对象可回收。颜色是理解模型，源码实现使用位图和工作队列。

50. 为什么需要写屏障？

并发标记期间业务仍修改指针，可能破坏“可达对象不会漏标”的不变量。写屏障在指针写入时记录必要关系，使 GC 正确追踪；它增加写操作成本。

51. GC 哪些阶段需要 STW？

开始和终止标记等阶段需要短暂 STW 完成状态切换和 Root 准备，大部分标记与业务并发。暂停短不代表 GC 无成本，Mark Worker 和 Assist 会消耗 CPU 并影响 P99。

52. Mark Assist 为什么会让接口变慢？

当分配速度超过 GC 标记进度，分配 Goroutine 被要求协助标记，业务请求承担 GC 工作。表现为高分配率、GC CPU 上升和尾延迟抖动。

根治是减少分配、缩小活对象和合理容量，不只是调参数。

53. GOGC 控制什么？

它按上一轮存活堆比例设置下一轮 Heap Goal。值高：GC 少、内存多；值低：GC 多、内存少。设置 Off 会停止按增长触发，但内存限制仍可能触发。

54. GOMEMLIMIT 控制什么？

它为 Runtime 管理内存设软上限，包含范围可近似理解为 Runtime 总内存减已归还 Heap。接近限制时 GC 更积极；若限制低于真实活对象和非 Go 内存需求，会发生 GC Thrashing 甚至 OOM。

容器中要为线程栈、CGO、mmap、内核 Buffer 和 Sidecar 留余量。

55. Scavenger 做什么？

后台识别长期空闲 Page，并通过 OS 接口降低其物理驻留。它影响 RSS 回落但不决定对象可达性；GC 负责对象，Scavenger 负责 Page 归还。

56. 指针密度为什么影响 GC?

含指针对象需要扫描其指针图；大量 `map[string]*T`、链表和 Interface 会增加扫描。连续无指针数据更友好，但优化应由 GC CPU 和 Profile 驱动，不能为了少指针破坏模型。

57. 如何设置容器内存预算?

从 Limit 扣除非 Go 内存、线程栈、CGO、文件映射、内核和安全余量，再设置 GOMEMLIMIT。通过压测观察峰值 RSS、Heap Goal、GC CPU、P99 和 OOM；Limit 不能等于 GOMEMLIMIT。

八、Context、Worker Pool、Pipeline 与有界并发

58. Context 解决什么问题?

传播取消、Deadline 和请求级元数据。它是协作信号，不会强制终止代码；数据库、HTTP、循环和 Worker 必须主动监听。

Context 作为首参，不存长期 Struct，不传 nil，Value 只放跨进程请求元信息而非业务参数。

59. WithCancel、WithTimeout、WithDeadline 和 Cause 如何选择?

父流程主动停止用 Cancel；相对预算用 Timeout；已有绝对结束时间用 Deadline；需要保留取消原因用 Cause API。创建后尽快安排 Cancel，释放 Timer 和子树。

60. Worker Pool 怎样设计才完整?

- 有界 Worker 和 Queue。
- 提交支持 Context、等待或明确拒绝。
- 每个任务 Panic 隔离和错误分类。
- Owner 控制停止提交、关闭队列、等待 Worker。
- 指标覆盖活跃数、队列长度、等待、执行、拒绝和失败。
- 下游并发上限按承载能力，不按 CPU 随意设置。

61. errgroup 适合什么场景?

同一请求内有限并发子任务，需要首错取消和等待收敛。使用 `WithContext` 和 `SetLimit`；循环变量、共享结果和错误语义仍需正确处理。

长生命周期 Job 系统不应只靠内存 errgroup，应持久化状态。

62. Pipeline 如何避免泄漏?

每一级监听 Context；下游提前退出时上游 Send 能取消；Owner 负责关闭输出；错误有独立收敛路径；所有 Stage 最终可 Wait。

没有取消路径的 Fan-out/Fan-in 很容易因无人接收而泄漏。

63. Semaphore 和 Worker Pool 如何选择？

Semaphore 限制同时在途数量，任务仍由调用 Goroutine 执行；Worker Pool 同时限定执行者和排队。请求内批量调用适合 Semaphore/errgroup，长期任务入口适合有界 Queue + Worker。

64. 背压和丢弃策略怎样设计？

队列满时可阻塞等待、快速失败、丢最新、丢最旧或降级，必须根据业务语义选择。支付不能静默丢；指标采样可以丢。所有等待受 Deadline 限制，防止无限堆积。

65. 定时器和 Ticker 有哪些坑？

长期 Ticker 要 Stop；循环中反复 `time.After` 会创建计时对象，应复用 Timer；Reset 前按 API 契约处理状态；回调不能假设准点执行。定时任务跨实例还要做幂等和租约。

66. 如何优雅停机？

摘除 Readiness -> 停止接收新请求/拉消息 -> 取消根 Context -> 等待在途请求和 Worker -> Flush/提交 Offset -> 超时强制退出。宽限时间必须覆盖内部等待并小于平台终止期限。

九、Race、死锁、活锁、泄漏与运行时诊断

67. Goroutine 泄漏的常见原因是什么？

无人接收的 Send、无人发送的 Receive、无 Deadline 网络调用、Ticker 未停止、Pipeline 下游退出、锁永不释放、后台循环无取消。现象是 Goroutine 和 Heap 随时间上涨、P99 抖动。

68. 如何排查 Goroutine 泄漏？

对比两个时间点的 Goroutine Profile，按 Stack 聚合增长；结合创建位置、Trace、队列和连接池指标；用测试前后 `runtime.NumGoroutine` 或专用工具做回归，但允许 Runtime 背景波动。

69. 死锁、活锁和饥饿有什么区别？

- 死锁：参与者永久等待，系统无进展。
- 活锁：参与者持续动作但相互避让，无有效进展。
- 饥饿：某个任务长期得不到资源，其他任务仍推进。

锁顺序、Try/Backoff、公平队列和缩短临界区分别治理不同问题。

70. 如何使用 Race Detector?

```
go test -race ./...  
go test -race -run TestName -count=20 ./path
```

它有明显 CPU/内存开销，适合 CI、测试和受控压测，不默认生产全量开启。报告应定位两条冲突访问和 Goroutine 创建栈。

71. Block、Mutex 和 Goroutine Profile 分别看什么?

- Goroutine：当前堆栈和等待状态。
- Block：Channel、Select、锁等发生的阻塞累计。
- Mutex：锁竞争造成的等待热点。

采样率会影响开销，故障前没有开启可能缺历史；生产按需短时启用并保护端点。

72. Runtime Trace 适合解决什么?

观察 Goroutine 调度、阻塞/唤醒、网络、Syscall、GC 和 Processor 利用率，适合“CPU 不高但延迟大”“并行度不足”“G 长时间等待”。热点函数优先 CPU Profile，内存对象优先 Heap Profile。

十、业务架构与项目实战

73. 广告素材同步如何设计有界并发?

分页读取任务，按媒体账号设置独立并发 Semaphore，外层 Worker Pool 有界；每个调用继承批次 Deadline，错误分可重试/永久；结果逐条落库，队列和媒体限流可观测。取消后不再取新任务，在途任务按 Deadline 收敛。

74. Kafka Consumer 中 Goroutine 如何组织?

消费组控制 Partition 并行度，Partition 内若要求顺序不能无约束并发。可按业务 Key 分片到固定 Worker；完成处理和幂等落库后提交 Offset。Rebalance 时停止取数、等待在途或安全取消，防止旧实例继续提交。

75. 外部 SDK 并发不安全如何治理?

先用 Race 和压测证明共享状态冲突；适配层给每账户实例独立 Owner 或加锁，禁止业务绕过；评估锁内是否包含 IO；升级 SDK 后保留并发回归。用 P99、Race 结果和锁 Profile 验证。

76. Go 服务内存问题如何串联回答？

编译器先通过逃逸分析决定对象是否进入堆；堆对象由 P-local mcache、mcentral 和 mheap 分配；GC 根据可达性标记，Pacer 用 GOGC 和 GOMEMLIMIT 控制目标，Scavenger 再尝试归还空闲页。线上我们区分分配速率、GC 后活对象、Runtime 保留内存和 RSS；用 allocs/op、Heap Profile、runtime/metrics 与容器指标定位。优化优先缩短对象生命周期、限制缓存和批次、修复 Goroutine/子 Slice 持有，再考虑 Pool 和参数。

77. Channel 设计原理如何用两分钟回答？

Channel 是带同步语义的类型化队列，底层 hchan 维护环形 Buffer、收发索引、sendq/recvg、关闭状态和锁。发送时若已有等待 Receiver 就直接复制并唤醒；有 Buffer 空间则入队；否则当前 G 通过 sudog 登记到 sendq 并 gopark，接收方到来后 goready。无缓冲 Channel 是同步交接，有缓冲 Channel 允许有限解耦。close 会广播唤醒 Receiver，并让等待 Sender 恢复后 Panic。Select 会在多个 Channel 注册等待并清理未选分支。工程上重点不是源码字段，而是谁拥有关闭权、Buffer 表示多少突发、元素复制与 GC 成本、取消和背压是否完整。

78. 如何用两分钟回答 Go Runtime？

Go Runtime 用 G、M、P 将大量 Goroutine 调度到 OS Thread，P 持有本地运行队列和 mcache；阻塞 IO 通过 Netpoller 挂起 G，系统调用时 P 可转交其他 M，Sysmon 负责抢占和进展。G 使用可增长栈，编译器逃逸分析决定栈或堆；堆由 mcache、mcentral、mheap 分层管理，GC 并发标记并通过写屏障保持正确，GOGC 和 GOMEMLIMIT 平衡 CPU 与内存。生产中我关注 Goroutine、队列、锁、分配率、GC CPU、Run Queue 和 P99，用 Race、pprof 和 Trace 定位，并用 Context、有界并发、背压和优雅停机控制生命周期。

01-03 | Go工程化与代码架构

技术 | Go 工程化与代码架构面试题

面向 9 年以上 Golang Web 后端业务研发岗位。本篇关注代码怎样被团队长期修改、测试、发布和治理，重点是业务边界、依赖方向、错误模型、测试体系、兼容演进和遗留系统重构。

一、Go 工程化学习地图与高级 IC 框架

1. Go 工程化知识图谱是什么？

业务边界

- > Module / Package / Dependency
- > Domain / Usecase / Adapter / Infrastructure
- > Error / Context / Config / Log
- > Test / Fuzz / Benchmark / Contract
- > Module / Tool / Generate / CI
- > Schema Evolution / Release / Refactor
- > 项目治理与高级 IC 表达

2. 资深后端如何回答工程化题？

用“现状痛点 -> 设计原则 -> 规则落地 -> 自动化检查 -> 迁移路径 -> 指标结果”。只画分层图不够，要解释谁能依赖谁、谁拥有数据、如何测试、违规如何在 CI 被发现。

3. 好工程的判断标准是什么？

- 新需求的修改范围可预期。
- 核心业务规则可独立测试。
- 基础设施可替换但不追求虚假抽象。
- 失败语义、配置和日志一致。
- 发布、回滚和数据迁移可执行。
- 新人能通过代码和文档理解关键路径。

4. 高级 IC 与管理岗的工程职责如何区分？

高级 IC 可以推动契约、评审、质量门禁和复盘，但价值仍来自关键设计、编码、排障和技术结果。面试中以亲自解决的复杂问题为主，协作经验只说明方案能跨团队落地。

二、Module、Package、目录与依赖方向

5. Go 项目是否有标准目录？

没有强制标准。目录服务于模块边界和可见性，不应复制模板。小服务可保持扁平；复杂业务再按领域或能力拆 Package。

常见入口是 `cmd/`，内部实现放 `internal/`，但 `pkg/`、`service/`、`util/` 是否存在取决于真实复用。

6. Package 应按技术层还是业务能力拆？

优先按业务能力聚合变化，例如 `order`、`campaign`、`asset`；每个能力内部再组织应用、领域和基础设施。全局 `controller/service/dao` 容易让一次需求跨多个巨大目录并形成公共模型。

7. 如何避免循环依赖？

- 让依赖指向更稳定的业务抽象。
- 将共同概念提升为小型独立 Package，而非 `common` 垃圾场。
- 消费者定义接口，基础设施实现。
- 通过事件或应用层编排解除领域双向调用。
- 循环频发通常说明边界错误，不用代码生成绕过。

8. `internal` 解决什么问题？

Go 工具链限制父目录树外导入 Internal Package，可建立真实编译期边界。它适合服务内部实现和不稳定 API，减少被其他仓库意外依赖。

9. 一个仓库应该包含几个 Module？

默认单 Module 最简单。只有发布周期、版本、依赖和消费者真正独立时才拆多 Module。多 Module 会增加 Replace、版本联动、工具和本地开发复杂度。

10. Monorepo 和 Multirepo 如何选择？

Monorepo 便于原子重构、统一工具和全局搜索；Multirepo 权限和发布边界清晰。选择取决于团队、代码规模和平台能力，不由 Go 决定。

关键是接口兼容、所有权、CI 范围和依赖版本可治理。

11. Public Package API 如何控制？

导出最少符号，使用稳定语义和文档；返回具体类型、接收最小接口；避免导出内部 Struct 字段成为永久契约。破坏性变化通过新 API、废弃周期和版本管理。

三、领域分层、接口、依赖注入与设计模式

12. Go Web 服务如何分层？

- Transport：协议解析、鉴权、错误映射。
- Application/Usecase：编排业务用例和事务。
- Domain：业务规则、状态机和不变量。
- Repository/Adapter：持久化和外部系统适配。
- Infrastructure：客户端、配置、日志和运行环境。

层数按复杂度裁剪，简单 CRUD 不需要模拟大型 DDD。

13. Domain Model 和数据库 Model 为什么要分开？

数据库模型服务存储布局，领域模型服务规则，API DTO 服务协议。三者生命周期不同。复杂业务分离可避免字段标签、空值和存储变化污染领域；简单系统可以合并，但要知道未来耦合成本。

14. 接口应该定义在哪一侧？

消费者定义它所需的最小行为，基础设施或生产者实现。这降低对“大一统 Repository 接口”的依赖，也让测试替身更自然。

15. 是否所有 Struct 都需要 Interface？

不需要。只有存在多实现、边界隔离或测试替换需求时定义。为未来假想变化提前制造接口会增加跳转和 Mock 成本。

16. 构造函数应该负责什么？

检查必要依赖和配置，建立可用对象，不做慢网络操作或启动隐藏 Goroutine。资源连接与服务启动最好有显式 Lifecycle，使失败、测试和关闭可控。

17. Dependency Injection 如何选择？

手写构造注入最透明；代码生成适合依赖图较大；反射容器运行时灵活但错误晚、调试难。Go 业务服务通常手写或生成足够，不必引入重量级容器。

18. Functional Options 适合什么场景？

可选配置多、需要默认值且构造 API 要演进时适用。必需依赖仍应显式参数；Option 要校验冲突并避免任意修改内部状态。

19. Repository 应该返回什么？

返回领域需要的数据和类型化错误，不暴露 SQL Row、Mongo Cursor 等基础设施对象。事务边界由 Usecase 明确，避免 Repository 内部偷偷开启多个无法组合的事务。

20. 如何避免“贫血 Service + 巨型 Util”？

让状态转换和业务不变量靠近领域对象或明确 Policy；Usecase 负责流程；通用函数只保留真正无业务语义的能力。按变更原因拆分，而不是把代码都放 Service。

四、Error、Context、配置、日志与可观测规范

21. 工程级 Error Model 应包含什么？

稳定 Code、类别、可重试性、HTTP/gRPC 映射、用户消息、内部 Cause 和必要元数据。领域层不依赖 HTTP 状态码，入口层统一转换。

22. 哨兵错误和类型化错误如何选择？

固定无上下文状态可用 Sentinel；需要字段和行为用类型化 Error；底层原因用 `%w` 包装。调用方通过 `errors.Is/As`，不比较字符串。

23. 哪些错误可以重试？

只有瞬时依赖错误、操作幂等、仍在 Deadline 内且系统未过载时。Error 类型应显式表达 Retryable 或由边界适配器分类，不能对所有 Error 统一重试。

24. Context 应如何穿过分层？

从入口传到所有可能阻塞的 RPC、DB、缓存和队列操作；作为首参，不塞进对象长期保存。领域纯计算函数若不阻塞，可不接受 Context，避免污染核心模型。

25. 配置如何设计？

启动时将环境变量/文件解析成强类型 Config，校验范围和依赖关系；动态配置走显式订阅和原子快照；敏感 Secret 不进入普通配置和日志。

默认值、单位、环境覆盖和废弃字段必须清楚。

26. 日志规范包含什么？

结构化字段：service、version、env、trace_id、request_id、business_id、operation、error_code、duration。边界记录一次最终错误，底层返回上下文；敏感字段脱敏，限制大对象和高频成功日志。

27. Metrics 应该在哪里埋点？

入口 RED、资源池饱和、依赖延迟/错误、队列、业务状态与不变量。Label 不使用 user_id/order_id 等无界高基数字段；详细定位放 Trace 和 Log。

28. 如何把 Trace 与业务日志关联？

Middleware/Interceptor 从 Context 提取 Trace ID，日志器显式接收 Context 或绑定 Request Scope；MQ Header 传播 Trace Context。禁止用全局可变日志字段串请求。

五、Module、版本、依赖与供应链治理

29. Go Module 的核心对象是什么？

`go.mod` 定义 Module Path、Go 版本、依赖；`go.sum` 校验下载内容；MVS 选择满足图中要求的最小版本集合。`go mod tidy` 维护依赖图，但执行前后要审 Diff。

30. 依赖升级如何降低风险？

小步升级、阅读 Release/Breaking Change、运行单测/Race/集成测试、灰度观察。关键 SDK 建适配层和契约测试，避免业务散落直接调用。

31. replace 应怎样使用？

本地联调和临时 Fork 可用，但发布前必须确认是否残留。长期 Replace 会隐藏真实版本和供应链，CI 应检查。

32. Go Workspace 适合什么？

本地同时开发多个 Module，不修改各自 go.mod。它不是生产依赖管理方案，CI 是否使用要明确，避免本地 Workspace 掩盖缺失版本。

33. 如何治理第三方依赖？

维护 Owner、用途、License、安全扫描和升级策略；核心链路优先成熟小依赖；封装不稳定外部 SDK；禁止为了简单工具引入巨型依赖树。

34. 如何保证可复现构建？

固定 Go Toolchain、Module 版本和生成工具；CI 干净环境构建；记录 Commit、版本和构建参数；镜像使用 Digest；不依赖开发机缓存和未提交生成文件。

35. 私有 Module 如何管理？

配置私有代理/`GOPRIVATE`、最小权限凭证、可信校验策略和缓存。CI 使用短期凭证，不将 Token 写入 URL、go.mod 或镜像层。

六、单元测试、集成测试、契约测试与 Fuzz

36. 测试金字塔如何落到 Go？

大量快速领域单测，适量数据库/MQ/HTTP 集成测试，少量关键端到端。每层解决不同风险，覆盖率不是唯一目标。

37. Table-Driven Test 的价值和风险是什么？

适合多个输入边界共享 Arrange/Act/Assert；Case 名称表达业务场景。若每个 Case 流程差异很大，硬塞表格会降低可读性。

38. Mock、Fake 和 Stub 如何选择？

- Stub 返回固定结果。
- Fake 有轻量可运行实现，如内存仓库。
- Mock 验证交互。

领域规则优先 Fake/直接测试状态；跨边界协议可 Mock。过度 Mock 内部调用会让重构困难。

39. 数据库集成测试怎样做？

使用真实兼容版本，独立 Schema/容器，执行 Migration，按测试事务或唯一数据隔离。验证约束、锁、事务和查询，不用内存 Map 冒充数据库语义。

40. 契约测试解决什么问题？

验证 Provider 与 Consumer 对 API/事件 Schema 的共同理解，包括字段、错误、兼容和边界。它不能替代端到端，但能在发布前发现跨服务破坏。

41. Fuzz Test 适合哪里？

解析器、序列化、金额/时间转换、协议边界、去重和状态机。先定义永不 Panic、不越界、往返一致等不变量，保留触发 Crash 的 Corpus。

42. 并发测试怎样减少偶然性？

使用 Barrier/Channel 控制交错，Race Detector，多次运行和 Deadline；避免 Sleep 猜时序。测试最终不变量和退出，不只验证某次调度顺序。

43. 测试如何处理时间？

注入 Clock/Now 函数或可控 Timer，避免真实 Sleep。业务时间与超时机制分开抽象，测试跨时区、闰日和边界。

44. 覆盖率应该怎样使用？

作为未测试区域的线索，不作为质量代理。重点看核心不变量、错误路径、并发、迁移和补偿；生成代码和简单 Getter 无需追求同等覆盖。

七、Benchmark、静态检查、代码生成与 CI

45. Benchmark 如何避免失真？

隔离 Setup、消费结果避免优化消除、报告分配、使用代表性数据，多次比较并关注方差。微基准只说明局部，不代表服务 P99。

46. go test 常用质量参数有哪些？

```
go test ./...
go test -race ./...
go test -shuffle=on -count=1 ./...
go test -coverprofile=cover.out ./...
go test -bench=. -benchmem ./path
```

CI 分层执行，Race 和集成测试可独立 Job，不能因耗时永久跳过。

47. go vet、Staticcheck 和 Linter 的职责是什么？

Vet 检查可疑语言/API 用法；Staticcheck 做更广静态分析；Linter 统一团队规则。规则要可解释、渐进启用，禁止用大量 `noLint` 掩盖问题。

48. 代码生成如何治理？

固定工具版本和 `go:generate` 命令，生成输出可复现；CI 重新生成并检查 Diff；生成代码不手改；输入 Schema 和 Owner 清楚。

49. CI 的最小质量门禁是什么？

Format、Vet/Static、Unit、关键 Integration、Race、Module Tidy Diff、安全扫描、Build。主分支保护和失败不可绕过；耗时任务按变更范围优化而非删除。

50. 如何处理 Flaky Test?

先隔离并记录 Owner/期限，保存失败日志和随机种子；根因常是共享状态、真实时间、端口、顺序和外部依赖。重复执行掩盖失败不是修复。

八、Schema、API、数据迁移与兼容演进

51. 什么是 Expand-Contract?

先新增兼容字段/接口，发布能同时处理新旧的数据路径；回填并切换；观察后再停止旧写和删除旧结构。每一步可回滚，不把 DDL、回填和代码切换绑成一次发布。

52. Struct 字段变更为什么可能破坏系统?

会影响 JSON/Proto、数据库扫描、反射、配置和下游消费者。字段重命名、类型变化和零值语义都需兼容窗口；内部 Struct 不应直接成为外部契约。

53. 如何做双写迁移?

明确主写、版本和失败补偿；双写事件可由 Outbox/CDC 保证；影子读对比；按租户灰度切读；保留回切；最终停止旧写并下线。永久双写会成为长期复杂度。

54. Feature Flag 如何进入代码架构?

在 Usecase 边界选择策略，避免散落 if；配置有 Owner、过期时间、默认和审计；两侧都测试；稳定后删除 Flag 和旧路径。

55. 如何废弃一个 Public API?

标记 Deprecated、发布替代方案、监控调用者、通知迁移、提供兼容周期、最后在 Major Version 或受控窗口删除。内部 API 也应遵循，不直接让消费者在发布后崩溃。

56. 如何重构一个没有测试的核心模块?

先用 Characterization Test 锁定当前行为和关键流量，补监控；建立 Adapter 隔离；小步提取纯函数和边界；双跑/影子对比；灰度切换并保留回滚。不要边重构边大改业务语义。

九、开发流程、Review、发布与工程治理

57. Code Review 重点检查什么?

正确性和业务不变量、失败路径、并发/所有权、数据兼容、安全、资源释放、可观测性和测试。风格交给自动工具，Reviewer 把时间用于系统风险。

58. ADR 有什么价值？

记录背景、约束、备选、决策、后果和复审条件，防止半年后只剩“当时为什么这样”。适合数据库选型、服务拆分、协议和高风险迁移，不为每个小改动写长文。

59. 发布前检查什么？

兼容性、Migration 顺序、配置/Secret、容量、灰度指标、Feature Flag、回滚、告警和数据副作用。Rollback 不能只回镜像，还要考虑新数据是否被旧代码读取。

60. 如何控制技术债？

把债务具体化为风险、成本和触发场景，进入 Backlog 和 Owner；新需求顺带小步偿还；高风险核心债单独立项。不要用“大重构”替代持续治理。

61. 文档应该写什么？

写代码无法表达的边界：系统上下文、关键流程、数据所有权、不变量、运行手册、迁移和决策。API 从 Schema 生成，命令确保可执行，过期文档比没有更危险。

62. 如何衡量工程化收益？

Lead Time、发布频率、Change Failure Rate、MTTR、Flaky Rate、构建时间、线上回滚、跨模块修改范围和新人上手时间。不能只报告“接入了某 Linter”。

十、业务项目与高级 IC 实战

63. 广告平台如何划分 Package？

按 campaign、asset、publish、account 等业务能力组织；媒体 SDK 位于 Adapter，Usecase 编排发布，Domain 维护状态机，Repository 隐藏 MySQL/Mongo。新增媒体主要修改 Adapter 和映射，不污染核心流程。

64. 如何封装不稳定第三方 SDK？

定义消费侧小接口和内部 DTO；Adapter 完成鉴权、限流、错误分类、重试与响应转换；契约测试固定关键行为；业务层不直接依赖平台异常类型。升级时灰度 Adapter 并对比指标。

65. 如何推进统一 Error 规范？

先盘点现有错误和入口协议，定义少量类别与映射；提供包装/转换工具和示例；从核心链路渐进迁移；CI 禁止字符串判断；监控按 error_code 聚合。保留 Cause 供排障，不把内部错误直接返回用户。

66. 如何表达遗留服务重构？

讲清原问题，如巨型 Service、共享 Map、测试困难和发布高风险；先加 Characterization Test 和观测，再提取媒体 Adapter、状态机和 Repository；影子对比和灰度；用修改文件数、测试时间、事故率和交付周期证明收益。

67. 如何用两分钟回答 Go 工程化？

我把工程化理解为让业务变化可控，而不是套目录。Package 围绕业务能力，依赖指向稳定边界，消费者定义最小接口；Transport、Usecase、Domain 和 Adapter按复杂度分离。Error 有统一类别和协议映射，Context 贯穿阻塞调用，配置强类型校验，日志和 Trace 用业务 ID 关联。测试以领域单测为主，数据库和协议用真实集成/契约测试，并通过 Race、Fuzz、Staticcheck 和可复现构建进入 CI。Schema 变更采用 Expand-Contract，发布可灰度、可回滚、可对账。遗留系统通过测试锁行为、小步提取和影子流量迁移。最终用发布失败率、MTTR、交付周期和变更范围证明工程收益。

技术 | Go Web 与 RPC 工程实践面试题

面向 9 年以上 Golang Web 后端业务研发岗位。本篇围绕“一次请求的一生”组织：进入服务、协议解析、业务调用、访问依赖、返回响应，以及整个链路的超时、幂等、过载、安全和生命周期治理。

一、Go Web/RPC 学习地图与回答框架

1. Go Web 服务的知识主线是什么？

```
Listener / Connection / TLS
-> net/http Server / Router / Middleware
-> Authentication / Validation / Idempotency
-> Usecase / DB / Redis / MQ / ES
-> HTTP Client / gRPC / Third-party API
-> Deadline / Retry / Limit / Circuit / Backpressure
-> Metrics / Trace / Security / Lifecycle
```

2. 资深后端如何回答 Web 工程题？

采用“请求链路 -> 资源边界 -> 失败语义 -> 生产参数 -> 观测验证”。例如连接池不能只列参数，要说明单实例并发、下游容量、等待时间、超时和扩容连接风暴。

3. Go Web 服务为什么不应只关注 Handler？

Handler 只是业务入口。生产问题往往来自 Listener、TLS、Header、Body、Context、连接池、下游慢、重试、响应写入和优雅停机。必须端到端理解请求生命周期。

4. 同步请求和异步任务如何划分？

用户必须立即知道的校验与受理同步完成；耗时、可重试、可削峰的工作持久化为任务并异步执行。异步接口返回 task_id 和明确状态，不用内存 Goroutine 假装可靠任务系统。

二、net/http Server、Router 与 Middleware

5. 一条 HTTP 请求在 Go 服务中如何执行？

Listener Accept 连接，TLS/HTTP 协议解析，Server 为连接和请求组织处理，Router 匹配 Handler，Middleware 执行鉴权/观测等横切逻辑，Handler 调用业务与依赖，最后写 Header、Status 和 Body。

HTTP/1.1 常按连接顺序处理请求，HTTP/2 可多路复用；具体并发和流控应结合协议版本理解。

6. http.Server 哪些超时必须设置？

- ReadHeaderTimeout：限制读取 Header，防 Slowloris。
- ReadTimeout：限制读取整个请求，流式/大上传需谨慎。
- WriteTimeout：限制响应写出，但流式响应语义特殊。
- IdleTimeout：限制 Keep-Alive 空闲。
- Handler 内再用 Context Deadline 控制业务链路。

不能只设置一个大超时覆盖所有阶段。

7. 如何限制请求资源？

限制 Header、URL、Body 字节、Multipart、JSON 深度/字段、分页和批量条数；先鉴权和基本校验再执行昂贵解析；使用 `MaxBytesReader` 等能力防止恶意请求消耗内存和磁盘。

8. Router 应该承担什么？

路径和方法匹配、参数提取、Middleware 组合。业务规则、数据库事务和复杂协议转换不放 Router。路由冲突、404/405 和尾斜杠语义要有测试。

9. Middleware 顺序为什么重要？

推荐外到内大致为 Panic Recovery、Request ID/Trace、访问日志与 Metrics、限流、认证授权、业务 Handler。Metrics 要看到最终状态，Recovery 要覆盖后续层；Body 读取和响应包装要避免破坏流式接口。

10. Panic Recovery 怎样才正确？

在每个请求边界 Recover，记录 Panic 值、Stack、Trace 和版本，若尚未写响应则返回通用 500。已部分写出时不能伪装完整成功；Panic 还要计入告警。

Recovery 只保护进程，不修复可能已发生的数据副作用。

11. ResponseWriter 有哪些陷阱？

第一次 `WriteHeader/Write` 后状态和 Header 基本确定；默认 `Write` 会发送 200。包装 `ResponseWriter` 统计状态时要兼容 `Flusher`、`Hijacker`、`Pusher` 等可选接口，否则 `WebSocket/Streaming` 可能失效。

12. 如何处理大文件上传和下载？

流式读取、总大小限制、内容类型校验、临时文件配额、病毒/安全扫描和 `Context` 取消；不要 `io.ReadAll` 全部入内存。下载使用流式、Range、对象存储签名 URL 或反向代理卸载。

13. WebSocket/SSE 与普通 HTTP 有什么治理差异？

长连接占 FD、内存和 Goroutine，需要心跳、空闲超时、写队列上限、慢客户端剔除、重连游标和发布停机迁移。普通请求级负载均衡不能完全代表连接级分布。

三、HTTP Client、Transport、DNS 与连接池

14. 为什么应该复用 `http.Client` 和 `Transport`？

`Transport` 管理连接池、DNS/TLS 和 Keep-Alive。每请求创建会失去复用、增加握手和 `TIME_WAIT`；默认 `Client` 又没有整体 Timeout。通常按依赖复用一组受治理 `Client`。

15. HTTP Client 的超时层次有哪些？

- 请求 `Context`：端到端 Deadline，首选。
- `Client Timeout`：包含重定向和读取 Body 的总上限。
- `Dial Timeout`、`TLS Handshake Timeout`、`ResponseHeaderTimeout`。
- `Idle Connection Timeout`。

每层解决不同阶段；流式响应不适合简单 `Client` 总超时。

16. `MaxIdleConns`、`MaxIdleConnsPerHost` 和 `MaxConnsPerHost` 怎么配？

前两者控制可复用空闲连接，总连接上限控制对单 Host 的在途和拨号。根据实例峰值并发、协议、下游容量和实例数设置；过低导致排队/握手，过高在扩容时形成连接风暴。

必须监控池等待、建立连接、复用率和下游活跃连接。

17. 为什么必须关闭并读取 `Response Body`？

Body 不关闭会泄漏连接和资源；HTTP/1.1 下未读到 EOF 可能妨碍连接复用。按响应上限读取或丢弃必要内容后关闭，错误路径也必须处理。

18. DNS 问题如何影响 Go Client?

解析延迟、缓存、TTL、搜索域和失效 IP 会表现为拨号慢或连接失败。长连接可能长期不重新解析；服务发现更新后要考虑连接轮换。记录 DNS、Connect、TLS、TTFB 分阶段耗时。

19. 如何避免 HTTP Client 重试放大?

只对幂等请求和明确瞬时错误重试，使用总 Deadline、有限次数、指数退避与抖动；尊重 429/Retry-After；避免网关、SDK、业务层同时重试。

POST 可通过 Idempotency-Key 和服务端状态查询变得可安全重试。

20. 如何处理第三方 API 限流?

按平台、账号、Endpoint 做令牌桶和并发限制，解析平台 Rate-Limit Header；任务队列吸收短期波动；429 按建议时间退避；配额耗尽时暂停而不是快速重试。

21. 如何记录外部调用日志?

记录 dependency、operation、status/error_code、attempt、duration、request_id、quota，不打印 Token 和完整敏感 Body。Trace Span 标注 DNS/Connect/TTFB 可帮助定位网络阶段。

四、REST API、契约、错误、分页与幂等

22. REST API 如何表达资源和动作?

稳定资源使用名词路径和 HTTP 方法；复杂业务动作可用子资源或命令 Endpoint，不必为形式强行 CRUD。关键是语义、幂等、错误和状态转换清楚。

23. API DTO 为什么不能直接复用数据库 Model?

外部协议、领域规则和存储结构变化周期不同。直接复用会泄露字段、标签和零值，数据库迁移被迫成为 API Breaking Change。使用显式 DTO 和映射。

24. API 错误响应应包含什么?

稳定 code、用户可理解 message、trace_id、可选 field violations 和 retry hint。HTTP Status 表达类别；内部 Cause 和 Stack 不返回客户端。

25. 创建接口如何实现幂等?

客户端生成 Idempotency-Key；服务端在同一事务中记录 Key、请求摘要、处理状态和响应/资源 ID。重复相同请求返回原结果；同 Key 不同参数拒绝；处理中返回明确状态。

26. PUT、PATCH 和 POST 的幂等性如何理解？

HTTP 方法语义不自动保证业务实现。PUT 目标状态相同通常幂等；PATCH 取决于操作；POST 可借助业务 ID 幂等。重试决策必须依据服务端实际语义。

27. Cursor Pagination 如何设计？

按稳定索引排序，Cursor 包含最后一条排序值、唯一 Tie-breaker、查询版本和签名。限制 page_size；条件变化后旧 Cursor 失效。不要暴露可篡改原始内部状态。

28. API 版本如何演进？

优先向后兼容地新增字段；破坏性语义才引入版本。监控旧版本消费者，提供废弃周期。URL/Header/媒体类型只是载体，兼容流程更重要。

29. 批量 API 如何设计？

限制条数和总字节，定义全成全败还是逐项结果；每项有业务 ID 和错误；服务端控制并发并支持部分重试。不要把一万个请求简单包装成一个超大请求。

30. 如何防止重复提交和并发更新覆盖？

创建用唯一业务键，更新用 ETag/Version 条件，状态迁移带当前状态，客户端超时后按业务 ID 查询。前端按钮禁用只是体验优化，不是正确性。

五、gRPC、Protobuf、Interceptor 与流式调用

31. gRPC 的请求链路是什么？

客户端 Stub 序列化 Protobuf，经 HTTP/2 Stream 发送；Resolver 提供地址、LB 选择 Subchannel；服务端解码，Interceptor 执行横切逻辑，Handler 返回 Status 和 Message；Deadline/Metadata 沿链路传播。

32. gRPC 相比 HTTP/JSON 的核心优势是什么？

强 IDL、代码生成、紧凑二进制、HTTP/2 多路复用和原生流式。代价是浏览器/调试、网关、Schema 管理和跨团队生成工具。内部高频调用常合适，外部开放 API 未必。

33. Protobuf 字段如何兼容演进？

字段号永不复用，删除后 `reserved`；新增字段有默认语义；不随意改类型；Enum 处理未知值；消费者先兼容再让生产者使用。CI 做 Breaking Check。

34. Unary 和 Streaming 如何选择？

普通短请求用 Unary；连续大结果、实时事件或双向交互才用 Streaming。流式需要消息上限、背压、心跳、半关闭、断线续传和长连接负载均衡。

35. gRPC Deadline 为什么必须显式设置？

默认可能无限等待。客户端根据端到端预算设置，服务端观察 Context 取消并停止工作；跨语言传递时注意 Deadline 转换和时钟偏差。

36. gRPC 重试有哪些边界？

Service Config 可按方法配置尝试、退避和可重试状态；但应用必须保证幂等，并理解透明重试和负载放大。INVALID_ARGUMENT 等永久错误不重试，UNAVAILABLE 也需受 Deadline/预算约束。

37. Unary/Stream Interceptor 适合做什么？

认证授权、Trace、Metrics、日志、Recovery、限流和错误规范化。顺序需明确；业务补偿、复杂重试和方法特有规则放 Usecase/Client Wrapper。

38. gRPC Health Check 如何使用？

服务注册标准 Health Service，启动完成置 SERVING，摘流量/停机先置 NOT_SERVING。可按 Service 粒度；健康不等于所有下游完美，避免深度探测引发级联。

39. gRPC 长连接为什么可能负载不均？

HTTP/2 多路复用让大量 RPC 固定在少量连接，扩容后旧 Client 未建立新 Subchannel。使用正确 Resolver/LB Policy、连接生命周期和慢启动，观察每实例请求分布。

40. gRPC 消息大小和压缩怎么治理？

限制收发大小，避免大对象撑爆内存；批量按字节而非条数；压缩节省带宽但消耗 CPU，按消息类型和压测开启。超大文件走对象存储/流式，不塞单个 Protobuf。

六、数据库、Redis、Kafka、ES 客户端工程实践

41. Go 数据客户端的通用治理框架是什么？

单例复用、连接池、Context Deadline、并发上限、错误分类、幂等、Metrics/Trace、优雅关闭。每个依赖按 SLA 和容量独立配置，不能共享一个万能超时。

42. database/sql 连接池如何配置？

`MaxOpenConns` 控制总并发，`MaxIdleConns` 保留复用，`ConnMaxLifetime/IdleTime` 轮换。按数据库总连接预算分给服务实例，并预留运维；监控 `WaitCount/WaitDuration`、`InUse`、`Idle`。

43. 数据库事务在 Go 中有哪些坑？

使用 `BeginTx(ctx)`，所有语句通过 Tx；错误后 Rollback，Commit Error 也要处理；事务短小，不在内部调用外部 HTTP；不要混用 DB 和 Tx 导致部分操作跑出事务。

44. Cursor/Rows 为什么必须关闭？

未关闭会长期占连接；遍历后检查 `rows.Err/cursor.Err`；限制返回和 Batch。`defer Close` 放在合适函数作用域，循环中不要积累。

45. Redis Client 如何治理？

复用连接池，设置 Dial/Read/Write/Pool Timeout；Pipeline 控制批次；大 Key/响应有限制；只对幂等命令重试；监控 Pool Wait、Timeout、命中率和 Hot Key。

46. Kafka Consumer 的并发边界是什么？

Partition 决定消费组并行上限；处理要幂等；按 Key 顺序时同一分区不可随意并行；Rebalance 期间收敛在途；提交 Offset 与业务结果顺序正确；队列有界并保护下游数据库。

47. ES Bulk Worker 如何设计？

按字节/时间组成有限 Batch，有限并发，逐 Item 检查错误；429 退避，Mapping 错误隔离；Context 取消时停止接收并 Flush；监控 Queue、Bulk P99、拒绝和同步延迟。

48. 如何防止客户端连接风暴？

启动抖动、慢启动、限制并发建连、复用 Client、每实例连接上限和全局数据库预算。扩容平台应考虑下游容量；Readiness 在连接和预热完成后开启。

49. 依赖错误怎样统一分类？

Adapter 将驱动错误转换为 Timeout、Unavailable、Conflict、NotFound、RateLimited、Permanent 等内部类别，同时保留 Cause。Usecase 根据类别决定重试/补偿，Transport 映射协议。

七、Deadline、重试、限流、熔断、降级与背压

50. 端到端 Deadline 如何分配？

入口预算扣除网关、排队和响应预留，再分给串行依赖；下游使用剩余时间而非创建更长 Context。并行分支也受总预算和并发上限。

51. 如何实现 Retry Budget？

限制重试流量占正常请求的比例或令牌数量；只有初次调用成功率允许时补充预算。这样依赖异常时不会所有请求都乘倍。按依赖和方法监控 attempt 与 retry success。

52. 本地限流和全局限流如何组合？

Gateway 做租户/用户全局配额，实例做本地 QPS/并发保护，资源层做账号/数据库等细粒度限制。本地近似换高可用，全局精确承担额外依赖。

53. 熔断和限流有什么区别？

限流控制进入速率/并发，熔断根据下游失败和慢调用暂时停止请求。二者与 Deadline、重试和降级共同构成韧性，不能互相替代。

54. 如何做 Bulkhead 隔离？

不同媒体、租户或优先级使用独立 Semaphore、Queue、Client/Pool 或实例组。一个慢平台不能占满全部 Goroutine 和连接。隔离份额要允许动态调整和闲时借用。

55. 服务降级如何避免假成功？

读请求可返回缓存/简化数据并标识陈旧；非核心字段可缺失；写请求若仅受理异步任务就返回 Accepted/Processing，不能把未执行伪装成功。降级后补偿和恢复条件明确。

56. 请求合并解决什么问题？

同一进程内多个相同 Key 的缓存 Miss 可共享一次下游查询，降低热点击穿。要限制等待人数、传播 Context 和错误，不能让一个短 Deadline 取消所有其他调用者。

57. 有界队列满了怎么办？

根据业务阻塞等待、快速 429/503、降级、丢低优先级或持久化到 MQ。所有等待有 Deadline；返回 Retry-After；监控拒绝和最老任务年龄。无限 Queue 只是推迟 OOM。

八、认证、授权、输入与数据安全

58. Authentication 和 Authorization 如何区分？

认证确认是谁，授权确认能对哪个资源做什么。Gateway 可验证 Token，服务仍需资源级授权和租户过滤，不能只信前端或内网 Header。

59. JWT 使用有哪些风险？

验证算法、Issuer、Audience、Expiration、NotBefore 和 Key ID；密钥轮换；Token 不放敏感可变数据；撤销和权限变更需短有效期/状态检查。禁止接受 `none` 或混淆算法。

60. mTLS 解决什么？

加密传输并认证双方工作负载身份。它不等于业务授权，服务仍检查主体是否可访问资源；证书签发、轮换、过期和时钟要监控。

61. 如何防 SSRF？

不直接请求用户给定 URL；使用允许的 Scheme/Host/Port，解析后检查 IP，禁止内网/Metadata 地址，处理重定向和 DNS Rebinding；出网代理和网络策略再做一层限制。

62. 如何防注入和越权？

SQL 参数化、禁止拼 DSL/Shell；字段白名单；租户 ID 从可信身份注入而非请求自报；对象级授权；批量接口逐项校验。日志和错误不泄露查询与 Secret。

63. JSON/Protobuf 反序列化如何保护？

限制 Body/Message 大小、嵌套和重复字段策略；验证必填、范围、Enum 和字符串长度；对外 JSON 可拒绝未知字段或按兼容策略处理；反序列化后仍需业务校验。

64. Secret 和凭证如何进入 Client？

从 Secret Manager/工作负载身份获取，内存中最小范围持有，支持轮换；不进入 URL、日志、Trace、错误和镜像。不同依赖/环境使用最小权限账号。

九、启动、健康检查、优雅停机与发布生命周期

65. 服务启动顺序是什么？

读取并校验配置 -> 初始化日志/Trace -> 建立必要客户端 -> 执行轻量连通性和缓存预热 -> 启动 Listener/Consumer -> Readiness=true。失败明确退出，避免半初始化服务接流量。

66. Liveness、Readiness 和 Startup Probe 如何区分？

Liveness 判断进程是否需要重启；Readiness 判断是否接流量；Startup 保护慢启动。临时数据库故障通常让 Readiness 降级，不应让所有实例同时 Liveness 失败重启。

67. 优雅停机的完整步骤是什么？

收到 SIGTERM -> Readiness=false -> 等负载均衡传播 -> 停止新 HTTP/RPC/MQ 任务 -> Server Shutdown/GracefulStop -> 取消后台 Context -> 等待在途 -> Flush/提交 -> 超时强退。

68. gRPC GracefulStop 有什么风险？

等待现有 RPC 完成，长流可能永不结束。应先停止新流并通知 Stream 关闭，外层设置宽限时间，到期使用 Stop。所有 Handler 必须尊重 Context。

69. Consumer 如何优雅退出？

停止 Poll/拉取，等待已分配 Partition 的在途任务完成或安全取消，提交已完成 Offset，关闭 Session/Client。Rebalance 回调不能无限等待，幂等保证重复处理安全。

70. 滚动发布为什么会出现连接不均？

旧长连接仍指向少量实例，新实例虽 Ready 但无流量。客户端/Proxy 需要连接年龄、Endpoint 更新和慢启动；发布监控按版本和实例观察，而不只看副本数。

71. 如何设计快速回滚？

应用版本向后兼容 Schema/事件，配置和 Feature Flag 可独立回退，镜像不可变；发布前验证旧版本能读取新数据。回滚后检查任务积压和已产生副作用，不止恢复进程。

十、业务架构与项目实战

72. 广告媒体 API 适配层如何设计？

每个平台实现内部统一接口；Adapter 负责协议、鉴权、错误分类、限流和幂等号；账号级 Semaphore 隔离；平台响应映射领域状态；原始 request_id 脱敏保存供排障。业务不直接依赖 SDK 类型。

73. 批量素材同步接口如何设计？

API 幂等创建 Batch/Item 任务并返回 task_id；Outbox 发布 Kafka；Worker 分页、有界并发调用媒体和 ES；每项状态落库；可取消、重试和人工处理；指标覆盖排队、成功、429、同步延迟和内存峰值。

74. 外部 API 超时结果未知怎么办？

请求携带平台支持的幂等 ID；超时后先调用查询接口确认，不能直接重复创建；本地状态标记 Unknown 并进入查询任务；最终按平台结果推进状态机，无法确认时人工处理。

75. 如何保护数据库不被 Go 服务扩容打爆？

数据库总连接预算按服务和实例分配；实例本地并发小于连接池；扩容慢启动并限制 MaxConnecting；入口限流；慢 SQL 治理；池等待告警。扩服务前同时评估数据库和第三方容量。

76. 如何回答一次完整 Web 请求？

请求经过 Listener/TLS 和 HTTP 解析，Server 在受控超时与大小限制下交给 Router；Middleware 完成 Trace、恢复、限流、认证和指标；Handler 校验 DTO 后调用 Usecase。Context Deadline 传播到 DB、Redis 和 RPC，各客户端复用连接池并有独立并发预算；写请求用业务 ID 幂等，错误按类别映射协议。响应只返回必要字段。线上监控各阶段耗时、池等待和依赖错误，发布时先 Ready 再接流量，停机时先摘流量再等待在途。核心是每个资源都有上限，每个失败都有语义，每个步骤可观测。

77. 如何用两分钟回答 Go Web/RPC 工程实践？

我从端到端请求而不是单个 Handler 设计 Go 服务。Server 设置 Header、请求体、空闲和业务 Deadline，Middleware 统一 Trace、认证、限流、Recovery 和错误；HTTP/gRPC Client 单例复用连接，按依赖设置连接池、超时、并发和重试预算。API 使用 DTO、稳定错误码、Cursor 和 Idempotency-Key；gRPC 通过 Protobuf 兼容规则、Interceptor、Health 和正确 LB 治理。DB、Redis、Kafka、ES 都传播 Context、限制队列并分类错误。安全上做工作负载身份、资源授权、输入上限和 Secret 轮换。启动完成才 Ready，停机先摘流量再收敛请求和消费者。生产效果用 P99、错误率、池等待、拒绝、依赖可用性和任务积压验证。

01-05 | Go性能优化与生产排障

技术 | Go 性能优化与生产排障面试题

面向 9 年以上 Golang Web 后端业务研发岗位。本篇不把 pprof 当答案本身，而是建立从业务现象、系统指标、Runtime 证据到代码热点的完整诊断链，并强调止血、恢复和防复发。

一、生产诊断地图、SLO 与排障框架

1. Go 生产排障的知识主线是什么？

业务影响与时间线

```
-> RED / USE / Business Invariant
-> CPU / Heap / Goroutine / Block / Mutex / Trace
-> HTTP/RPC / Pool / DB / Redis / MQ
-> Code / Config / Release / Traffic
-> Limit / Degrade / Rollback / Recover
-> Load Test / Capacity / Postmortem
```

2. 标准排障六步是什么？

1. 确认影响范围、时间和用户症状。
2. 保存版本、日志、Profile、Trace、配置和依赖证据。
3. 限流、降级、回滚或隔离，先控制损失。
4. 从入口到 Runtime 和依赖分层定位。
5. 修复后渐进恢复并校验业务数据。
6. 补监控、容量、测试、预案和演练。

3. 为什么先看业务 SLI 而不是机器指标？

CPU 高可能是健康高吞吐，CPU 低也可能全部等待下游。先看成功率、P99、吞吐、积压、数据正确性和用户影响，再用资源指标解释原因。

4. RED 和 USE 分别是什么？

- RED：请求 Rate、Error、Duration。
- USE：资源 Utilization、Saturation、Errors。

Go 服务还要关注 Goroutine、GC、Heap Goal、线程、FD、池等待和队列。饱和比使用率更能解释排队与尾延迟。

5. 如何建立故障时间线？

对齐告警开始、发布、配置、流量、扩容、定时任务、GC、依赖异常和人工操作。所有事件使用统一时区；保留新旧版本差异和第一条异常，而不是从大量重复错误开始猜。

6. 止血时最容易犯什么错？

盲目重启丢失证据、无限扩容打爆下游、调大超时增加在途、开启更多重试放大流量、未经确认清缓存/重置 Offset。每个动作先判断压力会转移到哪里和如何回滚。

二、Metrics、Logs、Trace、pprof 与 Runtime Trace

7. Go 服务最小监控集是什么？

- 请求 QPS、成功率、状态码、P50/P95/P99。
- Goroutine、CPU、RSS、Heap、GC CPU/Pause。
- HTTP/gRPC 连接与请求阶段。
- DB/Redis 等 Pool InUse/Wait/Timeout。
- MQ Lag、Queue Length、Reject、Oldest Age。
- 发布版本、实例、租户和业务不变量。

8. 常用 pprof Profile 分别解决什么？

- CPU：采样执行热点。
- Heap：当前存活与历史分配热点。
- Goroutine：当前栈和等待状态。
- Block：Channel、Select、锁等阻塞累计。
- Mutex：锁竞争等待。
- Threadcreate：线程创建来源。

9. alloc_space 和 inuse_space 有什么区别？

alloc_space 看累计分配，适合高分配/GC 压力；inuse_space 看当前仍存活，适合内存占用和滞留。一个函数累计分配很高但对象短命，未必是泄漏；存活持续增长更值得追踪。

10. pprof 的 Top、List、Web 和 Diff 如何使用？

Top 找主要成本，List 映射源码行，Web/火焰图看调用路径，Diff 比较变更前后或故障/正常。分析必须带采样时段、流量和版本，不能拿不同负载 Profile 直接下结论。

11. 生产开启 pprof 有什么风险？

暴露内部路径和 Stack；CPU/Trace/高采样 Block 产生开销；Heap Dump 可能很大并含敏感数据。应内网鉴权、短时采集、限制并发、保存到安全位置，不直接公网开放。

12. Runtime Trace 什么时候比 pprof 更有用？

CPU 不高但延迟大、Goroutine 长期等待、并行度不足、Syscall/Netpoller/GC 交互复杂时。它展示调度时间线；热点计算仍用 CPU Profile，存活对象用 Heap。

13. 分布式 Trace 与 Runtime Trace 有什么区别？

分布式 Trace 看跨服务业务调用和依赖耗时；Runtime Trace 看单进程调度、阻塞、GC 和 P。先用前者找到慢服务，再用后者解释进程内部为什么慢。

14. 日志怎样帮助性能诊断？

结构化记录 operation、duration、queue_wait、pool_wait、attempt、dependency、result、version 和 Trace ID。慢日志按采样/阈值记录，不在高峰打印完整请求和大对象导致二次故障。

三、Benchmark、压测、基线与性能方法论

15. 性能优化的正确顺序是什么？

定义目标 -> 建立基线 -> Profile 定位 -> 提出假设 -> 最小改动 -> Benchmark -> 端到端压测 -> 灰度观测 -> 记录回归门槛。未经测量的“优化”可能只增加复杂度。

16. 微基准容易犯哪些错误？

包含 Setup、结果被编译消除、数据过小、只跑一次、并行度失真、缓存热度与生产不同、忽略分配和方差。使用 `benchstat` 等方式多次比较，并固定环境。

17. 如何设计服务压测？

使用真实请求比例、对象大小、热点分布和依赖延迟；覆盖稳态、突发、长稳、阶梯、故障和恢复；记录入口与下游全部指标。压测环境容量和网络要已知，不能把压测机瓶颈归因服务。

18. 开环和闭环压测有什么区别？

闭环客户端等待响应再发，请求变慢时自动降低到达率，可能掩盖过载；开环按目标速率到达，更能观察排队和崩溃点。两者结合，报告排队和协调遗漏问题。

19. 为什么必须看 P99？

平均值掩盖排队、GC、锁和慢依赖。多级串行/扇出系统中尾延迟会累积；P99 对用户超时和容量更相关。同时给出样本量，低 QPS P99 波动大。

20. 如何建立性能回归门禁？

对稳定核心函数保存 Benchmark，对关键接口保存同环境负载基线；设置合理容差；性能敏感变更附 Profile 和前后数据。微小噪声不阻塞，显著回退需解释。

21. PGO 适合什么时候使用？

有代表性 CPU Profile、CPU 热路径稳定、编译与部署可复现时。PGO 可能改善内联和布局，但不替代算法、分配和 IO 优化；升级 Go 或流量变化后重新评估。

四、CPU 飙高、热点函数、GC 与锁竞争

22. Go 服务 CPU 飙高如何排查？

确认单实例/全局、QPS 是否变化和版本；看 User/System/Throttling、GC CPU；采集同窗口 CPU Profile；检查序列化、压缩、加密、正则、循环、日志、锁自旋和重试。

止血可限流、关闭非核心计算、回滚和扩容，但扩容前确认下游容量。

23. CPU 高但 QPS 没涨有哪些原因？

新版本算法退化、错误重试、日志暴增、GC 因分配上升、锁竞争/自旋、后台任务、健康检查、数据分布变化和死循环。用版本、操作维度和 Profile 区分。

24. System CPU 高说明什么？

可能大量 Syscall、网络、小包、文件日志、线程调度、锁 Futex、Page Fault 或 CGO。结合 syscall/网络指标、Thread、Trace 和系统工具，不只看 Go CPU Profile。

25. JSON/Proto 序列化为什么可能成为热点？

大对象、反射、重复转换、无用字段、Buffer 分配和日志二次序列化。优先减少响应字段和次数、分页/批量、缓存结果，再考虑代码生成或 Pool。

26. GC CPU 高如何处理？

判断是分配速率高还是活对象大：alloc Profile、Heap Goal、GC 周期和 Mark Assist。减少临时对象、反复 String/Byte、过大 Batch 和高指针结构；限制缓存；谨慎 Pool；最后调整 GOGC/Memory Limit。

27. 锁竞争如何定位？

Mutex Profile 找等待累计热点，Block Profile/Trace 看阻塞，CPU Profile 看自旋；检查临界区内 IO、全局锁、热点 Key 和读写锁误用。优化可拆分状态、分片锁、缩短临界区或不可变快照。

28. 为什么加 RWMutex 可能更慢？

读锁本身有原子/管理成本，写者会阻塞新读，读临界区太短时并行收益不足；高核下共享计数器还可能竞争。用真实读写比例 Benchmark。

29. CPU Throttling 如何影响 P99？

容器达到 CPU Quota 后在周期内被限制，平均 CPU 看似未满足但请求暂停，Run Queue 和延迟上升。看 Cgroup Throttled Time、Quota、GOMAXPROCS 和实例请求分布。

五、内存上涨、GC、对象滞留与 OOM

30. 内存问题必须区分哪些概念？

累计分配、当前 Heap 存活、Runtime 保留、已归还、线程栈、CGO/mmap、Page Cache 和 RSS。VSS 通常不代表物理压力；容器 OOM 看 RSS/Working Set 与 Limit。

31. 内存持续上涨如何排查？

对齐流量和 GC 后水位；比较多个时间点 inuse Heap；按对象/调用路径找增长；同时看 Goroutine、缓存、队列、Batch、子 Slice、Map 和外部内存。必要时受控触发 GC 仅用于诊断，不作为修复。

32. Go 有“内存泄漏”吗？

GC 语言仍会逻辑泄漏：对象被全局 Map、缓存、Goroutine、Timer、Closure、Slice 底层数组等可达引用持有。GC 只能回收不可达对象，不知道业务已不需要。

33. Slice 内存滞留怎样在 Profile 中表现？

少量小 Slice 可能使大底层数组存活，Profile 显示大 `[1]byte` 分配仍 inuse，保留路径来自解析/缓存函数。长期保留小片段时 Clone/Copy，并限制单对象大小。

34. Map/缓存增长如何治理？

设置容量、TTL、淘汰和租户配额；监控 Entry/Bytes/Hit/Eviction；值包含大引用时清理；并发安全和定时清理不阻塞主路径。无界 `sync.Map` 仍是无界缓存。

35. Goroutine 泄漏为什么导致内存上涨？

每个 G 持有栈、等待对象、Closure 和调用链引用；泄漏数量增长会使 Heap 和调度元数据一起增加。Heap 与 Goroutine Profile 联合分析，不能只优化对象。

36. RSS 高但 HeapAlloc 低有哪些原因？

HeapIdle 尚未 Released、Goroutine/OS Thread 栈、CGO 分配、mmap、Race Detector、共享库和内核 Buffer。看 MemStats/runtime metrics、线程、CGO 指标和进程映射。

37. OOMKilled 前为什么不一定有 Go Panic？

内核/Cgroup 直接杀进程，Runtime 无机会输出。保留容器指标、事件、Heap/GC 趋势和退出码；设置低于 Limit 的 GOMEMLIMIT 与告警，但为非 Go 内存留余量。

38. 调低 GOGC 为什么可能让系统更差？

GC 更频繁，Heap 降低但 CPU/Mark Assist 增加，P99 上升；活对象过大时收益有限。若 Memory Limit 过低还会 Thrash。调参必须与流量和分配优化一起压测。

39. sync.Pool 为什么可能造成反优化？

对象 Reset 不完整导致引用滞留，异常大 Buffer 被长期复用，池操作和类型断言有成本，GC 可清空导致命中不稳定。只复用高频、构造成本明显、大小受控的临时对象。

六、Goroutine、Channel、锁、死锁与泄漏排查

40. Goroutine 数量持续上涨如何定位？

按时间抓 Goroutine Profile Diff，查看增长 Stack 和状态；对照 Queue、连接、超时和发布；找到创建点和退出条件。重点状态是 chan send/receive、select、semacquire、IO wait。

41. 大量 Goroutine 处于 IO Wait 正常吗？

长连接服务可能正常，但若随请求积压增长且无 Deadline，则是风险。结合活跃连接、请求数、连接池、最老等待和回落情况判断。

42. Channel Send 阻塞怎样处理？

确认 Receiver 是否退出、消费变慢、Buffer 是否满和 Context 是否可取消。止血降低生产、扩消费前保护下游；根治有界队列、Select ctx、Owner 关闭和队列指标。

43. Channel Receive 阻塞怎样处理？

确认 Sender 生命周期、是否忘记关闭、是否发送到另一个实例/Channel、nil Channel 是否被启用。Pipeline 每级必须有取消和关闭协议。

44. 如何排查死锁？

Goroutine Dump 看等待图，Mutex/Block Profile 找热点；检查锁顺序、锁内 Channel/IO、回调重入和 WaitGroup。全局死锁 Runtime 可能 Fatal，部分业务死锁则只表现接口挂起。

45. Timer/Ticker 泄漏如何识别？

高频循环创建 `time.After`、长期 Ticker 未 Stop、AfterFunc Closure 持有对象。看 Heap 分配路径、Goroutine/Timer 指标和代码；复用 Timer 并明确停止/Reset。

46. 如何证明修复了 Goroutine 泄漏？

构造原始触发场景长稳压测，观察请求结束后 Goroutine 回到稳定基线；Profile 中增长 Stack 消失；队列和内存回落；加可重复回归测试和泄漏阈值。

47. Race Detector 没报错能否证明并发正确？

不能，它只检测执行过的冲突内存访问；死锁、逻辑竞态和未覆盖路径仍存在。结合同步设计、状态机测试、压力调度和代码 Review。

七、HTTP/RPC、连接池、DNS 与网络故障

48. 接口 P99 上升如何拆阶段？

入口排队 -> Handler -> Pool Wait -> DNS -> Connect -> TLS -> Server Processing -> TTFB -> Body Read。Trace/httptrace 和客户端指标定位，不把全部归为“下游慢”。

49. context deadline exceeded 如何排查？

检查入口原始 Deadline、当前剩余预算、在哪层创建子 Context、客户端/服务端 Span、下游是否收到取消、Pool Wait 和重试。不要直接调大超时。

50. HTTP 连接池耗尽有什么表现？

请求等待可用连接、P99 上升但下游 QPS未增、Goroutine 卡在 Transport 获取连接。原因包括 Body 未关闭、MaxConns 过低、下游慢和每请求建 Client。

51. 数据库连接池耗尽如何处理？

看 InUse、Idle、WaitCount/Duration、慢 SQL 和事务时长。短期限流重请求/回滚；根治关闭 Rows、缩短事务、优化 SQL、合理池预算。盲目加连接可能压垮数据库。

52. 大量 TIME_WAIT 是什么原因？

频繁短连接或未复用 Keep-Alive。检查 Client/Transport 生命周期、服务端 Connection Header、NAT/LB 和端口范围。TIME_WAIT 是 TCP 正常机制，不直接在系统层粗暴消除。

53. DNS 故障怎样表现？

偶发 Dial Timeout、解析耗时高、部分实例使用旧 IP。检查 Resolver、TTL、连接复用、搜索域和网络；保存分阶段 Trace。服务发现变更后要让旧连接逐步轮换。

54. TLS 握手慢或失败如何排查？

证书过期/链、SNI、时钟、Cipher、握手频率、CPU 和网络。复用连接降低握手；监控 Handshake Duration/Error；证书轮换前验证客户端信任。

55. Connection Reset、EOF 和 Broken Pipe 如何理解？

可能对端主动关闭、LB Idle Timeout、服务发布、网络中断或本方写已关闭连接。结合请求是否幂等、服务器日志和连接生命周期分类；不能一律重试写请求。

56. 如何排查 FD 耗尽？

看进程 FD 数/上限和类型，检查连接/文件/Body 泄漏、长连接增长和日志文件。止血限流和回收异常实例；修复 Close、池、超时并设置 FD 告警，调高上限不是根治。

八、依赖变慢、队列积压、过载与级联故障

57. 如何判断是 Go 服务慢还是下游慢？

Trace 分解本地处理、Pool Wait 和依赖 Span；服务 CPU/Queue 与下游 P99 交叉验证。客户端超时不代表下游没执行，写操作还要查询最终状态。

58. Redis 变慢时如何保护数据库？

入口限流、短时本地缓存/过期数据、热点请求合并、关闭非核心查询；限制回源并发，防穿透。恢复后渐进回流和预热，不瞬间让所有 Key 回源。

59. Kafka 积压如何计算恢复时间？

净消化速率 = 当前消费速率 - 新增速率
恢复时间 = backlog / 净消化速率

若净值不为正，增加分区/消费者或降低生产；扩消费前确认 DB/ES 容量。看最老消息年龄，不只 Lag 数。

60. Poison Message 如何影响 Go Consumer？

同一消息快速失败重试，占满 Worker/Partition。分类永久错误，有限重试后隔离到 DLQ，记录业务 ID 和 Cause；不能跳过后假装业务完整，需人工/补偿流程。

61. 重试风暴如何识别？

入口 QPS 未变但下游调用和 attempt 增长，错误与延迟同步恶化。按 attempt 指标和 Trace 识别多层重试；止血关闭/降低重试，启用熔断和 Retry Budget。

62. 如何识别过载而不是单纯慢代码？

到达率超过服务率，Queue/Pool Wait 和在途持续增长，P99 先升后错误；CPU/下游可能饱和。过载需要 Admission Control 和 Backpressure，仅优化一个函数不够。

63. 级联故障的止血顺序是什么？

控制入口 -> 关闭重试放大 -> 熔断慢依赖 -> 降级非核心 -> 释放卡住资源 -> 保护事实存储 -> 按底层到上层渐进恢复。保留核心写与数据正确性优先。

64. 第三方 API 全面异常怎么办？

按平台熔断并隔离，任务持久排队而非占 Goroutine；用户看到“处理中/平台异常”；暂停快速重试；监控配额和恢复；平台恢复后按账号限速回放，防二次限流。

九、容量、发布、恢复、复盘与稳定性建设

65. Go 服务实例容量如何估算？

单实例安全 QPS 来自真实混合压测并受 CPU、内存、连接和下游限制；实例数按峰值/安全 QPS 乘冗余，并验证 N-1。Little's Law 用 $\text{并发} \approx \text{QPS} \times \text{延迟}$ 估算连接和 Goroutine。

66. 如何做长稳压测？

持续数小时/天保持代表性流量，观察 GC 后 Heap、RSS、Goroutine、FD、连接、Queue 和 P99 是否稳定；周期任务和缓存淘汰必须覆盖。短峰值压测发现不了泄漏。

67. 灰度发布监控哪些指标？

按 version 比较 QPS、错误码、P99、CPU、RSS、GC、Goroutine、池等待、依赖调用倍数和业务转化；设置自动停止阈值。灰度样本量不足时延长而非盲目全量。

68. 发布后内存缓慢上涨如何处理？

冻结放量，比较新旧 Heap/Goroutine Diff 和代码变更；若接近 Limit 先回滚保存 Profile；判断缓存预热还是持续增长。回滚后确认对象和实例是否恢复，不只看错误率。

69. 服务恢复后为什么不能立即全量放流？

缓存冷、连接未建、积压待消化、GC/IO 仍高。小比例探测，预热，限制积压回放和重试，逐级放量；监控恢复斜率和业务一致性。

70. 一份有效 Postmortem 包含什么？

影响、时间线、检测、止血、根因、触发条件、放大机制、恢复验证和改进。区分“代码 Bug”与为何测试/灰度/限流未挡住；改进有 Owner、期限和可验证指标。

71. 如何建立 Go 生产运行手册？

每类告警写确认命令、Dashboard、Profile 采集、止血选项、风险、升级联系人、恢复和数据校验。命令定期演练，链接和权限可用；不把危险操作作为第一步。

72. 稳定性建设优先级怎样排？

先核心 SLO和业务不变量，再 Deadline/有界资源/幂等，之后监控与灰度，最后自动扩缩和高级调优。没有基础限流和回滚时先上复杂自愈通常风险更大。

十、真实项目、优化案例与资深表达

73. 素材同步服务内存突增如何回答？

背景是全量补偿一次加载大量素材并每条启动 Goroutine。先限制作业和回滚；Heap/Goroutine Profile 证明大 Slice 与在途任务；改为游标分页、有界 Worker、按字节 Bulk、任务状态落库；用峰值 RSS、GC CPU、P99 和完成时间对比。

74. 外部 SDK 造成 CPU 和 Race 如何回答？

发布后 CPU 高且偶发并发异常。Race 定位共享 Map，CPU Profile 显示锁/重试或序列化；Adapter 隔离 SDK，账号级实例/锁，限制重试，推动升级；回归运行 Race 和负载测试，并按平台监控。

75. Kafka Consumer 积压如何回答？

拆分 Poll、业务处理、DB/ES Bulk 和 Offset；发现无限并发导致 DB Pool Wait，吞吐反而下降。按 Partition/Key 建有界 Worker，批量写，限制 DB 并发，毒消息隔离；计算净消化速率，渐进恢复并验证幂等。

76. 如何表达一次 CPU 优化？

必须给证据链：业务 P99/QPS -> CPU Profile 热点 -> 假设 -> Benchmark -> 改动 -> 压测/灰度。
例：减少重复 JSON 序列化和无用字段后，说明 CPU、B/op、P99 和实例数变化，而不是只说“用了 sync.Pool”。

77. 如何表达一次内存优化？

区分累计分配和存活；Heap Diff 找到大 Slice/缓存/Goroutine持有；通过分页、Clone 小片段、缓存上限和取消修复；展示 GC 后 Heap、RSS、Goroutine、GC CPU 和 OOM 余量。

78. 如何用两分钟回答 Go 线上排障？

我先确认业务影响、时间线和发布/流量变化，同时保存 Trace、日志、Runtime 和依赖证据；根据风险限流、降级或回滚，避免盲目重启和重试。接口慢先用 Trace 拆本地、池等待和下游，再按现象选工具：CPU Profile 查计算，Heap 的 alloc/inuse 区分分配与滞留，Goroutine/Block/Mutex 查泄漏和竞争，Runtime Trace 查调度、IO 和 GC。内存还要区分 Heap、Runtime保留、CGO 与 RSS；连接问题看 DNS、建连、TLS、Body Close 和 Pool Wait。修复后长稳和灰度验证 P99、资源、队列和业务不变量，渐进恢复积压。最后复盘触发与放大机制，把改进落到容量、门禁、监控、预案和故障演练。

02-00 | 核心数据结构与存储引擎索引对比

技术 | 核心数据结构与存储引擎索引对比

这份文档用于承接 Redis、MySQL、MongoDB、Elasticsearch、Kafka 等组件背后的通用数据结构问题。

面试时不要孤立背“某组件用了什么结构”，而要从访问模式、存储介质、读写比例、范围查询、空间成本和工程复杂度解释为什么。

一、先建立全局视角

1. 数据结构选型看什么？

结论先行：

数据结构不是越高级越好，而是要匹配访问模式。内存结构更关注 CPU、指针、实现复杂度；磁盘结构更关注 IO 次数、页大小、顺序读写；搜索引擎更关注 term 到文档集合的倒排关系；日志系统更关注顺序追加和快速定位。

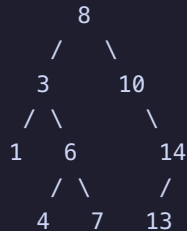
问题	典型结构
快速点查	Hash、B+Tree、LSM + Bloom
有序范围查询	B+Tree、SkipList、红黑树
排名/排行榜	SkipList、堆、排序数组
全文检索	倒排索引、FST、Posting List
高写入吞吐	LSM Tree
顺序日志定位	稀疏索引
布尔状态压缩	Bitmap
近似去重统计	HyperLogLog、Bloom Filter

二、二叉搜索树和平衡树

2. 二叉搜索树 BST 是什么？

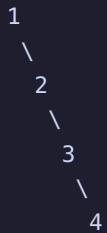
定义：

二叉搜索树满足左子树小于当前节点，右子树大于当前节点。



特点：

- 平衡时查找、插入、删除平均 $O(\log N)$ 。
- 如果按有序数据插入，可能退化成链表，最坏 $O(N)$ 。



所以工程里通常不用普通 BST，而用平衡树。

3. 平衡树、AVL 树、红黑树是一回事吗？

结论先行：

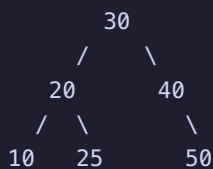
不是一回事。平衡树是大类，AVL 树和红黑树都是平衡二叉搜索树的具体实现。

```
平衡二叉搜索树
├── AVL 树
├── 红黑树
├── Treap
├── Splay Tree
└── 其他变种
```

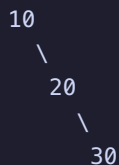
4. AVL 树是什么？

定义：

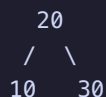
AVL 树要求任意节点左右子树高度差不超过 1，是一种严格平衡的二叉搜索树。



插入 10, 20, 30 时，普通 BST 会倾斜：



AVL 会通过旋转变成：



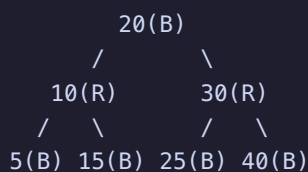
特点：

- 查询稳定，树高更低。
- 插入删除可能旋转频繁。
- 适合读多写少、对查询延迟敏感的场景。

5. 红黑树是什么？

定义：

红黑树也是二叉搜索树，但它通过红/黑颜色规则维持近似平衡，不像 AVL 那么严格。



核心规则可以简化管理：

- 根节点是黑色。
- 红色节点不能连续。
- 任意节点到叶子路径上的黑色节点数量一致。
- 通过变色和旋转保持近似平衡。

对比	AVL 树	红黑树
平衡程度	更严格	相对宽松

对比	AVL 树	红黑树
查询性能	略好	稍弱
插入删除	旋转更多	调整更少
常见场景	读多写少	读写都多
工程使用	相对少	Java TreeMap、C++ map/set 等

一句话：

AVL 更偏查询性能，红黑树更偏综合读写性能。

三、跳表 SkipList

6. 跳表是什么？

定义：

跳表是多层有序链表。底层保存完整数据，上层是稀疏索引。

```

Level 3: 1 ----- 30
Level 2: 1 ----- 15 ----- 30
Level 1: 1 ----- 8 ----- 15 ----- 22 --- 30
Level 0: 1 -> 3 -> 5 -> 8 -> 10 -> 15 -> 18 -> 22 -> 25 -> 30

```

查找 22：

```

从最高层开始
能往右就往右
右边太大就往下
最后在底层定位到目标

```

特点：

- 查找、插入、删除平均 $O(\log N)$ 。
- 范围遍历很自然，找到起点后沿底层链表扫。
- 实现比红黑树简单，不需要旋转和染色。
- Redis ZSet、LevelDB MemTable 都能看到跳表思想。

7. 跳表和红黑树怎么对比？

对比	跳表	红黑树
查找	平均 $O(\log N)$	$O(\log N)$
插入删除	调整多层指针	旋转 + 变色

对比	跳表	红黑树
范围查询	找到起点后顺序扫	中序遍历或额外迭代器
排名 rank	可维护 span	要维护 subtree size
实现复杂度	较低	较高
最坏情况	理论可能退化，概率极低	严格平衡

Redis ZSet 用 `dict + skiplist` :

```
dict: member -> score
skiplist: 按 score/member 有序
```

这是空间换时间：dict 负责按 member 点查，skiplist 负责排序、范围和排名。

8. Redis zskiplist 的查找、插入、update 和 span 如何工作？

结论先行：

Redis 的经典 ZSet 跳表按 `score + member` 排序。查找从最高层向右推进，下一节点超过目标就下降一层；插入时用 `update[]` 保存每层前驱，再一次性连接随机高度的新节点；`span` 记录跨越的底层节点数，用于 Rank 计算。

结构示意：

```
Level 3: head ----- cat(9)
Level 2: head ----- banana(8) --- cat(9)
Level 1: head -- apple(7) - banana(8) --- cat(9)
Level 0: head -> apple(7) -> banana(8) -> cat(9)
```

查找 `banana(8)` :

1. 从当前最高层和 Head 开始。
2. 若下一节点的 score 小于 8，或 score 相同但 member 字典序更小，就向右。
3. 下一节点已经大于等于目标时下降一层。
4. 到 Level 0 后检查下一节点是否等于目标。

查找区间 `[7,8]` 时，先用同样过程定位第一个 `score >= 7` 的节点，再沿 Level 0 顺序遍历，直到 score 大于 8。跳表的优势不是把区间每个元素都“跳着取”，而是快速找到区间起点。

插入过程：

```
从高层到低层查找插入位置
-> update[i] 保存第 i 层最后一个小于新节点的前驱
-> rank[i] 保存到该前驱跨越的底层节点数量
-> zslRandomLevel 生成随机高度
-> 每层修改 update[i].forward 与 new.forward
```

```
-> 根据 rank 修正各层 span
-> 更新 backward、tail、length 和最高层级
```

随机层数让高层节点指数减少，平均查找、插入和删除复杂度为 $O(\log N)$ ，空间为 $O(N)$ 。这是概率平衡，不保证每次操作的严格最坏 $O(\log N)$ 。

span 为什么重要？

如果 Level 2 的一条 Forward 跨过 4 个 Level 0 节点，其 span 就记录 4。查询排名时沿高层向右的同时累加 span，无需在底层从头计数，因此 Rank 平均也是 $O(\log N)$ 。

同分元素如何排序？

score 相同时继续比较 member，形成稳定全序。这保证插入、查找和删除能定位具体成员，而不是只定位分数。

为什么 Redis 选择跳表而不是红黑树？

- 范围扫描定位起点后沿底层链表顺序访问很自然。
- 通过 span 支持 Rank。
- 插入删除只修改多层指针，不需要旋转和染色，实现与调试相对直观。
- Redis 同时使用 Dict 提供 member 到 score 的 $O(1)$ 点查，跳表负责顺序和范围，两种结构各司其职。

版本边界： Redis 内部实现会演进，面试应先说明 ZSet 对点查、排序、范围和 Rank 的需求，再结合对应版本源码，不把具体字段布局当永久接口。

四、B 树、B-树、B+ 树

9. B 树和 B-树是一回事吗？

结论先行：

是。B-树 不是“B 减树”，它就是 B Tree 的中文写法，也叫 B 树。

```
B 树 = B-树 = B Tree
B+ 树 = B+ Tree
```

10. B 树是什么？

定义：

B 树是一种多路平衡搜索树。一个节点可以存多个 key，并拥有多个子节点。

```
      [20 | 40]
      /   |   \
[5 | 10 | 15] [25 | 30] [45 | 50 | 60]
```

为什么要这样设计？

二叉树一个节点只有一个 key，树高较高。
B 树一个节点放多个 key，树更矮，磁盘 IO 更少。

特点：

- 内部节点和叶子节点都可以存数据。
- 查询可能在中间节点命中。
- 树高低，适合磁盘页。
- 范围扫描不如 B+ 树自然。

11. B+ 树是什么？

定义：

B+ 树是 B 树的变种。非叶子节点只存索引，真实数据只存叶子节点；叶子节点之间通过链表相连。

```
      [20 | 40]
     /   |   \
    [10] [20|30] [40|50]
     |     |     |
     v     v     v
叶子: [1,5,10] -> [20,25,30] -> [40,45,50]
```

特点：

- 非叶子节点只做索引，一个页能放更多 key。
- 所有查询最终都到叶子节点，路径更稳定。
- 叶子节点有序链表，范围查询非常强。
- MySQL InnoDB 索引典型使用 B+ 树。

12. B 树和 B+ 树区别

对比	B 树 / B-树	B+ 树
数据存储	内部节点和叶子节点都可存数据	数据只在叶子节点
非叶子节点	索引 + 数据	只存索引
查询路径	可能中途命中	一定到叶子
范围查询	相对麻烦	叶子链表顺序扫描
单节点索引数	较少，因为可能存数据	更多，因为只存 key 和指针
树高	相对高一点	更矮
数据库索引	有使用，但较少	MySQL InnoDB 典型使用

13. 为什么 MySQL InnoDB 用 B+ 树?

结论先行:

MySQL 索引主要面向磁盘页和范围查询, B+ 树能减少 IO、支持排序和范围扫描, 所以非常适合 OLTP 场景。

```
where id = 100
where age between 20 and 30
order by created_at
limit 100
```

B+ 树适合这些查询:

- 点查: 从根节点一路定位到叶子。
- 范围: 找到起点后沿叶子链表扫描。
- 排序: 索引天然有序。
- 分页: 顺序读取叶子节点。

为什么不用红黑树?

- 红黑树节点通常只有一个 key, 树高更高。
- 指针跳转多, 不适合磁盘页。
- 磁盘 IO 成本远大于内存指针调整成本。

五、LSM Tree

14. LSM 是什么?

定义:

LSM 全称 **Log-Structured Merge Tree**, 核心思想是把随机写尽量变成顺序写, 以提升写入吞吐。

```
写入
|
v
MemTable 内存有序表
|
| 满了刷盘
v
Level 0: SSTable_1 SSTable_2 SSTable_3
|
| Compaction 合并
v
Level 1: SSTable_A
|
v
Level 2: SSTable_B
```

LSM 通常由几部分组成:

结构	作用
WAL	先写日志，防止宕机丢数据
MemTable	内存有序表，承接写入
Immutable MemTable	等待刷盘的只读 MemTable
SSTable	磁盘有序不可变文件
Bloom Filter	判断 key 一定不存在，减少无效读
Sparse Index	快速定位 SSTable 内部数据块
Compaction	后台合并、清理旧版本和删除标记

15. LSM 和 B+ 树怎么对比？

对比	B+ 树	LSM Tree
写入	可能随机写、页分裂	顺序写为主
读取	路径稳定	可能查多层 SSTable
范围查询	强	也支持，但受多层文件影响
空间	较紧凑	Compaction 前可能有重复数据
删除	直接改索引/记录	写 tombstone，后续合并清理
适合	读多、范围查询、事务型 OLTP	写多、大吞吐、KV/日志型存储

一句话：

B+ 树偏读和范围查询稳定，LSM 偏写入吞吐。

六、倒排索引、FST、BKD Tree

16. 倒排索引是什么？

定义：

倒排索引是从词到文档列表的映射。

```
term -> docID list

"redis" -> [1, 5, 9]
"mysql" -> [2, 5, 8]
"广告" -> [3, 9, 10]
```

搜索 `redis AND mysql` :

```
[1,5,9] n [2,5,8] = [5]
```

它适合全文检索，因为全文搜索的核心问题是：

某个词出现在哪些文档里？
多个词怎么求交集？
相关性怎么排序？

17. ES/Lucene 常见结构

结构	用途
倒排索引	term 到 docID 列表
Term Dictionary	词典，记录有哪些 term
FST	压缩 term dictionary，支持快速查词
Posting List	某个 term 对应的文档列表
BKD Tree	数值、日期、地理位置范围查询
Doc Values	排序、聚合、脚本访问
Segment	不可变索引段，便于顺序写和合并

为什么 ES 不用 B+ 树做全文检索？

B+ 树适合按字段值比较和范围扫描；全文检索需要从 term 快速找到文档集合，并做交集、并集、相关性排序，所以倒排索引更合适。

七、Kafka 稀疏索引

18. Kafka 为什么用稀疏索引？

Kafka 是顺序日志系统，不是通用数据库。每个 partition 被切成多个 segment：

```
00000000000000000000.log  
00000000000000000000.index  
00000000000000000000.timeindex
```

`.index` 保存 offset 到物理位置的稀疏映射：

```
offset -> position
```

```
1000 -> 0
1500 -> 4096
2000 -> 8192
```

查找 offset 1800:

1. 在 index 中找到 ≤ 1800 的最近 offset, 例如 1500
2. 跳到 position 4096
3. 顺序扫描到 offset 1800

为什么不用 B+ 树?

- Kafka 主要顺序追加写。
- segment 文件不可变或近似追加。
- 稀疏索引空间小, 写入成本低。
- 定位到附近后顺序扫描一点点很快。

八、组件索引结构对比

19. MySQL、MongoDB、ES、Kafka、Redis 都用什么?

组件	核心索引/类似索引	为什么
MySQL InnoDB	B+ 树	磁盘页友好、树高低、范围查询强、排序分页友好
MongoDB WiredTiger	B-Tree 类结构	文档字段索引、复合索引、范围查询和排序
Elasticsearch / Lucene	倒排索引、FST、BKD Tree、Doc Values	全文检索、term 查询、数值/地理范围、聚合排序
Kafka	稀疏 offset index、time index	顺序日志, 低成本定位 segment 位置
Redis ZSet	Hash + SkipList	内存有序集合, 点查、排名、范围遍历综合均衡
RocksDB / LevelDB	LSM Tree	高写入吞吐、顺序刷盘、后台合并
PostgreSQL	B-Tree、Hash、GIN、GiST、BRIN	根据查询类型提供多种索引

九、限流算法结构：固定窗口、滑动窗口、令牌桶与漏桶

20. 限流算法为什么也属于数据结构选型?

结论先行:

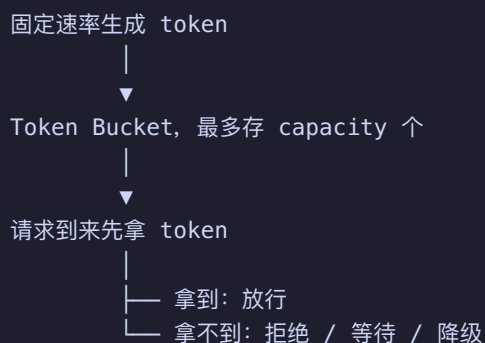
限流本质是在“时间维度上管理请求状态”。不同限流算法背后依赖不同数据结构: 固定窗口是计数器,

滑动窗口是时间有序集合，令牌桶是状态机，漏桶是队列或调度时间。面试时不要只背算法名，要说明它解决什么问题、用什么状态表示、适合什么工程场景。

算法	核心数据结构/状态	典型实现	优点	缺点	适合场景
固定窗口	计数器	Redis <code>INCR + EXPIRE</code> 、内存 map	简单、快、内存少	窗口边界可能突刺	粗粒度接口保护
滑动窗口	时间有序集合 / 时间队列	Redis <code>ZSet + Lua</code> 、内存 deque	精确统计最近 N 秒	高 QPS 下内存成本较高	外部 API 精确配额
令牌桶	<code>tokens + lastRefillTime</code> 状态机	Go <code>x/time/rate</code> 、Guava、Redis Lua	允许突发，长期平均受控	需要理解突发容量	网关、RPC、第三方 API
漏桶	队列 / 下一次可执行时间	Nginx <code>limit_req</code> 、任务队列、Redis Lua	匀速流出，削峰明显	增加排队延迟	慢下游、短信、邮件、批任务
并发数限制	semaphore / 计数器	Go channel、 <code>errgroup.SetLimit</code> 、Redis running key	控制同时执行数量	不控制 QPS	DB 连接、RPC in-flight 保护

21. 令牌桶和漏桶怎么对比？

令牌桶：允许突发，控制长期平均速率

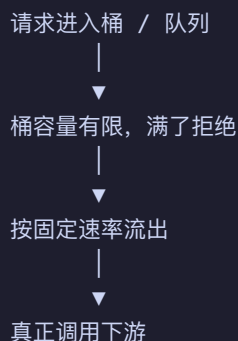


核心参数：

- `rate`：每秒生成多少 token，决定长期平均速率。
- `capacity`：桶容量，决定最大突发能力。
- `requested`：每次请求消耗多少 token。

例子：`rate=100/s, capacity=500`，表示长期平均 100 QPS，但低峰积攒 token 后，瞬间最多可以承接约 500 个突发请求。

漏桶：稳定流出，保护慢下游



核心参数：

- `capacity`：最大排队数量。
- `leakRate` / `interval`：固定流出速度。
- `maxDelay`：最大可接受等待时间。

例子：`capacity=1000, leakRate=100/s`，表示最多排队 1000 个请求，下游每秒稳定处理 100 个。它不追求瞬间放行，而是把突发流量抹平。

22. 主流框架和组件一般用什么限流思路？

框架/组件	常见限流思路	适合解决的问题
Go <code>golang.org/x/time/rate</code>	令牌桶	单实例 QPS 控制、客户端 SDK 限速
Go channel / semaphore	并发数限制	DB 连接、RPC in-flight、批任务并发
Nginx <code>limit_req</code>	漏桶思想，支持 burst	接入层 IP/URI 粗粒度限流
Envoy / API Gateway	本地 token bucket + 全局限流服务	微服务入口统一治理
Redis	固定窗口、滑动窗口、令牌桶、漏桶都可实现	多实例共享额度、账号/API/租户限流
Sentinel	QPS、并发、热点参数、熔断降级	Java 服务治理和热点保护

框架/组件	常见限流思路	适合解决的问题
Guava RateLimiter	令牌桶 / 平滑预热	Java 客户端本地限速
Resilience4j	RateLimiter、Bulkhead	限流、隔离、熔断组合治理
MQ / 任务队列	漏桶 / 队列削峰	把突发写入变成稳定消费
数据库条件更新	原子扣减额度	强一致配额、库存、资金类扣减

工程选型口诀：

- 单实例保护：Go 本地令牌桶 + semaphore。
- 多实例共享额度：Redis 滑动窗口 / 令牌桶 Lua。
- 入口粗粒度防刷：Nginx、Envoy、API Gateway。
- 慢下游削峰：漏桶、队列、MQ。
- 强一致额度：数据库条件更新或事务，不要用普通缓存计数替代。

十、面试总结模板

23. 如何回答“为什么不同组件索引结构不一样？”

可以这样答：

数据结构选型要看访问模式和存储介质。MySQL 是磁盘型 OLTP，核心瓶颈是 IO，所以用 B+ 树降低树高，并通过叶子链表支持范围查询和排序。Redis ZSet 是内存有序集合，需要排名、范围查询和插入删除，跳表实现简单、范围遍历自然。Elasticsearch 是全文检索，核心是 term 到 docID 的倒排索引，数值范围再用 BKD Tree。Kafka 是顺序日志，只需要稀疏 offset/time index 快速定位 segment。LSM 则适合高写入，把随机写转成顺序写。

24. 速记口诀

BST 左小右大，但会退化；
 AVL 严格平衡，查找强；
 红黑树 近似平衡，工程常用；
 跳表 多层链表，范围和排名友好；
 B树/B-树 多 key 多子节点；
 B+树 数据在叶子，叶子有链表；
 LSM 内存写入，磁盘有序文件，后台合并；
 倒排索引 词到文档；
 Kafka 稀疏索引 offset 到 position；
 限流看突发和平滑，令牌桶允许突发，漏桶稳定流出；
 固定窗口便宜，滑动窗口精准；
 组件选型看访问模式和存储介质。

技术 | Redis 缓存面试题

面向 9 年以上 Golang Web 后端业务岗。全文按照“定位与边界 → 请求执行 → 数据结构 → 缓存架构 → 并发原子性 → 持久化高可用 → 性能容量 → 治理排障 → 项目实战”展开。

核心回答采用：结论先行 → 核心机制 → 完整流程 → 工程取舍 → 生产实践 → 面试追问。

一、Redis 学习地图与资深回答框架

1. Redis 的知识图谱主线是什么？

结论先行： Redis 是内存数据结构服务。面试不能只背命令，要从数据模型、单线程执行、缓存一致性、高可用和容量治理串成完整体系。

场景与边界

- 客户端 / RESP / 事件循环 / 命令执行
- 数据类型 / 对象编码 / 复杂度
- Cache Aside / 一致性 / 三大缓存问题
- Lua / 事务 / 锁 / 限流
- RDB / AOF / 复制 / Sentinel / Cluster
- 性能 / 内存 / 容量
- 治理 / 安全 / 排障
- 业务架构与项目表达

2. 资深后端如何回答 Redis 题？

题型	回答顺序
数据结构	结构与编码 → 复杂度 → 场景 → 大 Key/阻塞风险
缓存设计	权威数据源 → 读写路径 → 一致性窗口 → 防穿透击穿 → 降级
原子性	并发目标 → 原子边界 → Lua/事务/锁 → 超时与幂等
高可用	故障模型 → 复制/选举/分片 → 数据丢失窗口 → 客户端恢复
排障	影响面 → 客户端/网络/服务端证据 → 止血 → 根因 → 复盘

高级岗位要能说明：为什么选 Redis、失效后谁兜底、容量怎么算、故障如何恢复，而不是只说“查得快”。

关联资料与原有定位

面向 BAT、字节、小红书等大厂后端面试的 Redis 题库与结构化答案。

当前文档按“三大主线”重排：Redis 当前能力、核心能力原理分析、实战设计与线上治理。

答案统一采用“结论先行 -> 分层展开 -> 面试追问”的金字塔表达方式，方便在面试中先给判断，再给原理、场景和权衡。

相关补充文档：

- 网络 IO 深挖：[05-02 | 计算机网络与HTTP工程.md]
- 通用数据结构与索引对比：[02-00 | 核心数据结构与存储引擎索引对比.md]
- Redis 跳表图解与实现细节：[02-00 | 核心数据结构与存储引擎索引对比.md]

补充模块：Redis 面试方法论、速记清单与回答模板

本模块统一收纳所有“怎么答、怎么想、怎么背”的内容。前面五章负责知识本体，这一章负责面试表达和临场组织。

补充方法：复习地图

第一层：Redis 当前能力

命令、数据类型、底层编码、持久化、过期/淘汰、主从、哨兵、Cluster、Pipeline、key 设计

第二层：核心能力原理分析

为什么快、网络 IO、ZSet 跳表、缓存一致性、分布式锁、Lua/事务、fork、脑裂

第三层：实战与线上治理

商品缓存、排行榜、限流、延迟队列、点赞、预热、本地缓存、大 key、热 key、慢查询、CPU、超时、故障保护 DB、项目表达

补充方法：Redis 题目怎么答才像资深后端？

结论先行：

Redis 面试不要只背命令，要围绕“为什么快、怎么保证一致性、怎么扛高并发、怎么处理故障、怎么做容量与成本权衡”来回答。

分层展开：

基础层：数据结构与命令

- String、Hash、List、Set、ZSet、Bitmap、HyperLogLog、Geo、Stream。
- 底层编码、时间复杂度、适用场景。

性能层：单线程、高性能 IO 与内存模型

- 单线程执行命令，避免锁竞争。

- IO 多路复用处理大量连接。
- 内存操作、紧凑编码、渐进式 rehash、惰性删除。

可靠性层：持久化、复制、高可用

- RDB 适合快照和恢复。
- AOF 适合更高数据安全。
- 主从复制、哨兵、Cluster 解决可用性和扩展性。

工程层：缓存架构与一致性

- 缓存穿透、击穿、雪崩。
- Cache Aside、延迟双删、异步补偿、消息队列。
- 分布式锁、限流、排行榜、延迟队列、会话存储。

治理层：容量、监控、故障演练

- 热 key、大 key、慢查询、内存碎片、淘汰策略。
- 指标监控、压测、熔断降级、预案。

面试追问：

如果面试官问“你线上怎么用 Redis”，不要泛泛说缓存，要讲清楚业务读写路径、缓存粒度、过期策略、一致性边界、失败兜底和监控指标。

补充方法：Redis 题通用决策框架

Redis 面试不要一上来背命令。先判断面试官问的是哪类问题，再套对应的回答模板。

第一反应：先问 6 个问题

1. 问的是数据结构选型，还是缓存架构？
2. 问的是读性能，还是写一致性？
3. 问的是单机能力，还是高可用/分片扩展？
4. 问的是业务设计，还是线上故障排查？
5. 问的是原子性/并发控制，还是容量/成本治理？
6. 这个 Redis 方案失败时，数据库和业务怎么兜底？

对应模板：

问题类型	优先想到	回答重点
为什么快	内存 + 单线程命令 + IO 多路复用 + 数据结构	别只说内存，要讲主线程、socket、Pipeline、慢命令边界
数据结构选型	String/Has/ZSet/Bitmap/HLL/Bloom/Stream	先讲业务需求，再讲结构能力和风险
缓存一致性	Cache Aside	先更新 DB，再删缓存，删除失败要重试/补偿
	穿透/击穿/雪崩	空值/Bloom、互斥重建、随机 TTL/预热

问题类型	优先想到	回答重点
缓存三大问题		
并发控制	分布式锁/Lua/事务	原子性、锁误删、过期时间、业务幂等
限流	固定窗口/滑动窗口/令牌桶/ZSET+Lua	单机 vs 全局，精度 vs 成本，降级和监控
高可用	主从/Sentinel/Cluster	可用性、故障转移、分片扩容、客户端复杂度
持久化	RDB/AOF/混合持久化	恢复速度、数据丢失窗口、fork 和磁盘 IO 风险
性能排查	慢命令/大 key/热 key/连接/网络/fork	先影响面，再指标，再止血，再复盘
架构设计	容量/QPS/热点/一致性/降级	先约束量级，再选架构，不要直接堆中间件

Redis 题通用解题步骤

Step 1：先定业务场景
 Step 2：判断读多还是写多
 Step 3：选择数据结构或缓存模式
 Step 4：说明一致性和原子性边界
 Step 5：说明高并发风险：热 key、大 key、击穿、雪崩
 Step 6：说明故障兜底：降级、限流、本地缓存、DB 保护
 Step 7：补监控指标：命中率、延迟、慢查询、内存、连接数、错误率

面试回答不要停在“我用了 Redis 缓存”，要讲成：

业务为什么需要 Redis
 key/value 怎么设计
 读写路径是什么
 一致性怎么保证
 异常失败怎么补偿
 高并发下有什么风险
 线上怎么监控和止血

Redis 思想分类

思想	代表方案	解决问题	关键词	典型题
数据结构选型思想	String/Hash/ZSet/Stream 等	用合适结构表达业务	对象、排行、去重、计数、消息	用户缓存、排行榜、延迟队列
旁路缓存思想	Cache Aside	缓存与 DB 最终一致	先读缓存、miss 查 DB、写 DB 后删缓存	缓存一致性
热点保护思想	互斥重建/逻辑过期/本地缓存	防止热点 key 打爆 DB	击穿、热 key、预热	商品详情、播放页

思想	代表方案	解决问题	关键词	典型题
防穿透思想	空值缓存/Bloom Filter	拦截不存在请求	非法 ID、不存在数据	商品/用户不存在
原子执行思想	Lua/事务/SET NX EX	多步操作不可被插入	限流、锁释放、扣减	分布式锁、滑动窗口
时间窗口思想	ZSet/计数器/令牌桶	控制请求频率	QPS、窗口、突发	媒体 API 限流
高可用思想	主从/Sentinel/Cluster	故障转移和容量扩展	副本、分片、slot	Redis 架构选型
异步补偿思想	MQ/binlog/重试任务	删除失败、同步失败可恢复	最终一致、补偿、重试	缓存删除、ES 同步
治理排障思想	slowlog/bigkeys/hotkeys/监控	定位线上问题	延迟、CPU、连接、内存	Redis 超时/CPU 高

快速决策树

题目问“存什么结构”？

对象整体读写 -> String + JSON/ProtoBuf
 字段级读写 -> Hash
 排名/范围/延迟 -> ZSet
 去重集合/交并差 -> Set
 海量 0/1 状态 -> Bitmap
 近似 UV -> HyperLogLog
 防穿透/可能存在 -> Bloom Filter
 轻量消息流 -> Stream

题目问“缓存和 DB 一致性”？

常规场景 -> Cache Aside：读缓存，miss 查 DB 回填；写 DB 后删缓存
 删除失败 -> MQ/binlog/重试表补偿
 极短不一致不能接受 -> 版本号/强校验/不走缓存

题目问“高并发缓存风险”？

查不存在 -> 穿透：空值/Bloom
 热点 key 过期 -> 击穿：互斥锁/逻辑过期/热点不过期
 大量 key 同时过期 -> 雪崩：随机 TTL/分批预热/降级

题目问“并发原子性”？

单 key 加锁 -> SET NX EX + token + Lua 删除
 多步读改写 -> Lua
 批量发送减少 RTT -> Pipeline，但不保证原子性
 事务排队执行 -> MULTI/EXEC，但不是数据库事务

题目问“限流”？

单机简单限制 -> 本地滑动窗口/令牌桶
 多实例全局限制 -> Redis ZSET + Lua

允许突发 -> 令牌桶
精确窗口统计 -> 滑动窗口

题目问“架构选型”？

数据量小、可接受短暂不可用 -> 单机/主从
需要自动故障转移 -> Sentinel
单实例容量或吞吐不够 -> Cluster
极热点读 -> 本地缓存 + Redis + DB 多级缓存

题目问“线上变慢/超时”？

先看影响面和指标
再查 slowlog、大 key、热 key、连接数、CPU、内存、fork、网络、客户端连接池
先止血：限流、降级、摘流量、禁用重命令、扩容
后复盘：key 设计、容量、监控、预案

常见 Redis 业务题模板组合表

业务题	模板组合
商品/剧集详情缓存	Cache Aside + 随机 TTL + 热点互斥重建 + 降级
用户/账号基础信息	String 或 Hash + TTL + 删除缓存重试
排行榜	ZSet + ZINCRBY/ZREVRANGE/ZREVRANK + 大 key 治理
媒体 API 限流	Redis ZSET + Lua + 账号/API 维度 key + 破频降级
分布式锁	SET NX EX + token + Lua 删除 + watchdog + 业务幂等
延迟队列	ZSet score=执行时间 + Lua 取删 + 重试/死信
点赞系统	Set/Bitmap 去重 + 计数 + 异步落库
防缓存穿透	Bloom Filter + 空值缓存 + 参数校验
缓存预热	分批扫描 + 限速 + 随机 TTL + 可回滚
Redis 故障保护 DB	本地缓存兜底 + 限流降级 + 熔断 + 读 DB 保护
Redis CPU 高	慢命令 + 大 key + 热 key + Lua + 连接风暴排查
Redis 内存高	大 key + 过期策略 + 淘汰策略 + 碎片率 + 容量评估

面试前速记口诀

先问场景，再选结构；
读多看缓存，写多看一致性；
高并发先防穿透、击穿、雪崩；
多步原子用 Lua，批量 RTT 用 Pipeline；
锁只防重复，正确性靠幂等；
限流先分单机和全局；
容量不够上 Cluster，可用性不够看 Sentinel；

线上慢先看慢命令、大 key、热 key、连接、fork、网络；
任何 Redis 方案都要说 DB 兜底、降级和监控。

二、Redis 基础与数据结构

补充方法：Redis 高频题速记清单

1. Redis 快：内存、单线程、IO 多路复用、高效结构、后台线程。
2. 单线程：命令执行串行，网络连接并发。
3. String：SDS，二进制安全，O(1) 获取长度。
4. Hash：适合字段级读写，但要防大 key。
5. ZSet：dict + skiplist，适合排行榜和范围查询。
6. 跳表：实现简单、范围查询友好、平均 $O(\log N)$ 。
7. 缓存一致性：先写 DB，再删缓存，失败重试，TTL 兜底。
8. 穿透：空值缓存、Bloom Filter、参数校验。
9. 击穿：互斥锁、逻辑过期、热点不过期。
10. 雪崩：TTL 随机、多级缓存、限流降级、高可用。
11. 分布式锁：`SET NX EX` + 唯一值 + Lua 解锁。
12. RDB：快照，恢复快，可能丢数据。
13. AOF：命令日志，安全性更好，文件更大。
14. 主从复制：全量 + 增量，异步复制有延迟。
15. Sentinel：自动故障转移，不解决分片。
16. Cluster：16384 slot，支持水平扩容。
17. 大 key：阻塞、网络抖动、迁移慢、内存倾斜。
18. 热 key：单节点打满，本地缓存、拆分副本、限流。
19. 淘汰策略：LRU/LFU/random/TTL/noeviction。
20. 慢查询：命令复杂度、大 key、Lua、网络、持久化、客户端都要看。

补充方法：面试回答模板

补充模板：原理类题

结论：

先用一句话回答核心判断。

原因：

从数据结构、复杂度、线程模型、IO 模型解释。

场景：

说明适合什么业务，不适合什么业务。

风险：

指出线上坑，例如大 key、热 key、一致性、阻塞。

治理：

给出工程方案和监控指标。

补充模板：场景设计题**目标：**

明确 QPS、延迟、一致性、容量和可用性目标。

读链路：

本地缓存 -> Redis -> DB。

写链路：

更新 DB -> 删除缓存 -> 异步补偿 -> TTL 兜底。

异常链路：

Redis 故障、DB 故障、缓存雪崩、热点 key、大 key。

监控：

命中率、P99、慢查询、DB QPS、Redis CPU/内存/网络、主从延迟。

补充模板：项目深挖题

二、使用场景、定位与能力边界

3. Redis 适合解决什么问题？

结论先行： Redis 适合低延迟高频访问、共享临时状态和数据结构计算，常用于缓存、计数、排行榜、限流、会话、锁和轻量消息流。

典型能力：

- 缓存热点数据，降低数据库和外部接口压力。
- 原子计数、集合运算、排行榜和时间窗口。
- 多实例共享 Session、验证码、幂等标记和任务状态。
- 通过 Lua 组合多步原子操作。
- 用 Stream 承载中小规模、Redis 内部的消费组场景。

代价： 内存成本高；大 Key、热 Key 和慢命令会阻塞；异步复制存在数据丢失窗口；缓存与数据库通常只能最终一致。

4. 什么场景不适合使用 Redis?

- 需要复杂关联、任意查询、强约束和长期权威存储。
- 数据规模远超可承受内存，且访问并不热点。
- 资金、库存等正确性只依赖 Redis 锁或持久化。
- 精确、可靠、长期积压的大规模消息系统。
- 单条操作复杂度高或需要长时间计算。

面试追问：Redis 可以作为数据库吗？

技术上可以持久化，但是否适合作为权威源取决于一致性、查询、成本和恢复目标。多数业务把数据库作为权威源，Redis 作为可重建的加速层或临时状态层。

5. Redis 与本地缓存如何选？

维度	本地缓存	Redis
延迟	最低，无网络 RTT	多一次网络访问
一致性	多实例失效困难	共享状态相对统一
容量	占用每个实例内存	集中管理和扩展
故障	进程重启丢失	可复制、持久化和故障转移

生产中常用“本地缓存 + Redis + DB”三级结构，但必须设计版本号或广播失效，避免本地缓存长期脏读。

三、请求执行模型、网络协议与命令

6. 一条 Redis 命令如何执行？

结论先行： 客户端编码 RESP 请求，经连接发送；Redis 事件循环监听 Socket，读取解析命令，在主执行线程完成数据操作，再编码响应并写回客户端。

```
Go Client
-> 连接池获取连接
-> RESP 编码
-> TCP/TLS 发送
-> IO 多路复用发现可读
-> 读取并解析命令
-> 命令执行
-> RESP 编码响应
-> Socket 写回
```

生产判断： 端到端耗时包含连接池等待、DNS/网络、服务端排队、命令执行和响应传输。`SLOWLOG` 只记录服务端命令执行阶段，不能证明网络或客户端没有问题。

7. RESP 协议有什么特点？

RESP 是 Redis 的请求响应协议，可编码简单字符串、错误、整数、批量字符串、数组以及新版扩展类型。它实现简单、解析快，也支持 Pipeline 和推送式能力。

边界： 协议轻量不代表消息体可以无限大；大 Value 会同时放大序列化、网络、内存分配和主线程处理时间。

8. Redis 为什么常被描述为单线程？

结论先行： 核心命令通常串行执行，因此单条命令具备天然原子执行边界；网络读写和后台任务可使用其他线程，不能把 Redis 简化成“整个进程只有一个线程”。

为什么快：

- 内存访问。
- IO 多路复用管理大量连接。
- 命令执行避免复杂线程锁竞争。
- SDS、dict、listpack、quicklist、skiplist 等高效结构。
- 后台线程或子进程承担部分释放、持久化等工作。

风险： `KEYS`、大集合运算、超长 Lua、大 Key 删除等慢操作会占住执行线程，阻塞其他请求。

9. Pipeline、事务和 Lua 有什么区别？

能力	Pipeline	MULTI/EXEC	Lua
目标	减少 RTT	命令队列连续执行	服务端原子组合逻辑
原子性	无	执行期间不被其他命令插入，但不提供数据库式回滚	脚本执行期间原子
条件逻辑	客户端处理	WATCH 可做乐观控制	脚本内判断
风险	批次过大占内存	错误和回滚语义有限	长脚本阻塞主线程

Cluster 下 Pipeline 应按 Slot/节点分组；多 Key 原子操作要求 Key 位于同一 Slot，可用 Hash Tag 控制。

10. Key 如何设计？

推荐形式： `环境:业务域:实体:ID:维度`，例如 `prod:ad:campaign:123:detail`。

- 可读、可检索、可明确 Owner。
- 控制长度，避免把完整查询条件直接拼成超长 Key。

- 缓存默认有 TTL，永久 Key 必须有理由。
- Cluster 多 Key 操作使用 `{entity-id}` Hash Tag。
- Value 拆分要平衡读取次数与大 Key 风险。
- 禁止用 `KEYS` 做线上业务扫描，使用 `SCAN` 并限速。

补充模块：Redis 当前能力全景：命令、类型、编码与架构模式

这一章回答“Redis 有什么能力”。先知道命令、数据类型、底层编码、持久化、过期淘汰、复制、哨兵、Cluster、Pipeline 和 key 设计。

深入：Redis 命令作用速查

这一节把 Redis 基础数据类型相关命令集中放在一起。面试时先说“命令做什么”，再讲数据结构、复杂度、业务场景和风险。下面覆盖核心数据类型的常用命令；模块命令如 Bloom Filter 需确认线上是否部署 RedisBloom。

Key 通用命令

命令	作用	面试提醒
<code>EXISTS key</code>	判断 key 是否存在	可用于兜底判断，但不要代替业务状态校验
<code>TYPE key</code>	查看 key 的数据类型	排查线上 key 类型误用
<code>DEL key</code>	同步删除 key	大 key 删除可能阻塞，优先考虑 <code>UNLINK</code>
<code>UNLINK key</code>	异步删除 key	适合删除大 key，降低主线程阻塞
<code>EXPIRE key seconds</code> / <code>PEXPIRE key ms</code>	设置过期时间	缓存必须有 TTL 兜底
<code>TTL key</code> / <code>PTTL key</code>	查看剩余过期时间	排查缓存是否会集中失效
<code>PERSIST key</code>	移除过期时间	小心把缓存变成永久 key
<code>RENAME key newkey</code> / <code>RENAMENX key newkey</code>	重命名 key	<code>RENAMENX</code> 仅在新 key 不存在时成功
<code>SCAN cursor MATCH pattern COUNT n</code>	渐进式扫描 key	线上替代 <code>KEYS</code>
<code>KEYS pattern</code>	全量匹配 key	生产禁用或极慎用，可能阻塞主线程

String 命令

命令	作用	示例含义
<code>SET key val</code>	设置 key 的值为 val；不存在则创建，存在则覆盖	<code>SET name tom</code> ：创建/更新 <code>name=tom</code>
<code>SET key val NX EX seconds</code>	key 不存在才设置，并设置秒级 TTL	<code>SET lock:order:1 requestId NX EX 10</code> ：加 10 秒过期锁
<code>SETNX key val</code>	key 不存在才设置	旧式锁基础命令，通常更推荐 <code>SET NX EX</code>
<code>SETEX key seconds val</code> / <code>PSETEX key ms val</code>	设置值并同时设置过期时间	缓存写入时原子设置 TTL
<code>GET key</code>	获取 key 对应 value	<code>GET user:1</code> ：读取缓存值
<code>MGET key1 key2</code>	批量获取多个 key	减少网络往返，Cluster 下注意 slot
<code>MSET key val ...</code> / <code>MSETNX key val ...</code>	批量设置多个 key	<code>MSETNX</code> 要求所有 key 不存在
<code>GETSET key val</code>	返回旧值并设置新值	老命令，很多场景可用 <code>GETEX/GETDEL</code> 替代
<code>GETEX key EX seconds</code>	获取值并修改过期时间	读取同时续期
<code>GETDEL key</code>	获取值并删除 key	一次性 token/验证码场景
<code>INCR key</code> / <code>DECR key</code>	整数自增/自减 1	计数器、固定窗口限流
<code>INCRBY key n</code> / <code>DECRBY key n</code>	整数增加/减少指定值	批量计数
<code>INCRBYFLOAT key f</code>	浮点数增加指定值	金额类不建议只靠 Redis 做权威数据
<code>APPEND key val</code>	追加字符串	日志片段类场景慎用，容易形成大 key
<code>STRLEN key</code>	获取字符串长度	排查大 value
<code>GETRANGE key start end</code>	获取子串	字节范围读取
<code>SETRANGE key offset val</code>	从 offset 开始覆写部分内容	底层仍是 String，offset 太大可能撑大 value

Hash 命令

命令	作用	示例含义
<code>HSET key field val</code>	设置 field 的值	

命令	作用	示例含义
		<code>HSET user:1 name Tom</code> : 设置 name
<code>HSETNX key field val</code>	field 不存在才设置	防止覆盖已有字段
<code>HGET key field</code>	获取指定 field	<code>HGET user:1 name</code> : 读取 name
<code>HMGET key field1 field2</code>	批量获取多个 field	只取需要字段, 避免全量
<code>HGETALL key</code>	获取全部 field-value	大 Hash 慎用, 可能阻塞和打满网络
<code>HKEYS key</code> / <code>HVALS key</code>	获取全部 field / value	大 Hash 慎用
<code>HLEN key</code>	获取 field 数量	判断 Hash 规模
<code>HEXISTS key field</code>	判断 field 是否存在	字段级存在性判断
<code>HDEL key field</code>	删除 field	<code>HDEL cart:user:1 sku:2</code> : 删购物车商品
<code>HINCRBY key field n</code>	整数字段增加 n	<code>HINCRBY product:1 stock -1</code> : 库存字段 -1
<code>HINCRBYFLOAT key field f</code>	浮点字段增加 f	统计值场景
<code>HSTRLEN key field</code>	获取 field 对应 value 长度	排查字段大 value
<code>HRANDFIELD key count WITHVALUES</code>	随机返回 field, 可带 value	抽样检查 Hash
<code>HSCAN key cursor MATCH pattern COUNT n</code>	渐进式扫描 field	大 Hash 遍历替代 <code>HGETALL</code>

List 命令

命令	作用	示例含义
<code>LPUSH key val</code> / <code>RPUSH key val</code>	从左/右侧插入元素	队列、栈、最新列表
<code>LPUSHX key val</code> / <code>RPUSHX key val</code>	key 存在时才插入	避免意外创建队列
<code>LPOP key</code> / <code>RPOP key</code>	从左/右侧弹出元素	消费队列元素
<code>BLPOP key timeout</code> / <code>BRPOP key timeout</code>	阻塞式弹出元素	简单阻塞队列

命令	作用	示例含义
<code>LLEN key</code>	获取列表长度	判断队列堆积
<code>LINDEX key index</code>	按下标读取元素	随机访问不是 List 强项
<code>LSET key index val</code>	按下标设置元素	下标越界会失败
<code>LRANGE key start stop</code>	获取范围元素	<code>LRANGE list 0 -1</code> 取全量，大 List 慎用
<code>LTRIM key start stop</code>	只保留指定范围	维护最新 N 条
<code>LINSERT key BEFORE/AFTER pivot val</code>	在指定元素前/后插入	需要查找 pivot
<code>LREM key count val</code>	删除指定值元素	count 控制方向和数量
<code>LPOS key val</code>	查找元素位置	Redis 6.0+ 常见
<code>LMOVE source dest LEFT RIGHT</code>	从 source 弹出并推入 dest	队列迁移/可靠队列雏形
<code>BLMOVE source dest LEFT RIGHT timeout</code>	阻塞版 <code>LMOVE</code>	等待 source 有数据
<code>RPOPLUSH source dest</code> / <code>BRPOPLUSH source dest timeout</code>	右弹左推	旧式可靠队列方案，逐渐被 <code>LMOVE/BLMOVE</code> 替代

Set 命令

命令	作用	示例含义
<code>SADD key member</code>	添加元素，天然去重	标签、关注、黑名单
<code>SREM key member</code>	删除元素	删除标签/取消关注
<code>SISMEMBER key member</code>	判断元素是否存在	判断用户是否点赞/关注
<code>SMISMEMBER key m1 m2</code>	批量判断多个元素是否存在	批量判断关系
<code>SMEMBERS key</code>	获取全部元素	大 Set 慎用
<code>SCARD key</code>	获取元素数量	统计集合规模
<code>SPOP key count</code>	随机弹出元素	抽奖/随机消费

命令	作用	示例含义
<code>SRANDMEMBER key count</code>	随机返回元素但不删除	随机展示
<code>SMOVE source dest member</code>	将元素从 source 移到 dest	状态集合迁移
<code>SINTER key1 key2</code>	求交集	共同好友、共同标签
<code>SINTERCARD numkeys key...</code>	只返回交集数量	不返回大结果，适合计数
<code>SUNION key1 key2</code>	求并集	合并集合
<code>SDIFF key1 key2</code>	求差集	A 有 B 没有
<code>SINTERSTORE dest key... / SUNIONSTORE dest key... / SDIFFSTORE dest key...</code>	结果写入目标 Set	离线/缓存集合结果
<code>SSCAN key cursor MATCH pattern COUNT n</code>	渐进式扫描 Set	大 Set 遍历替代 <code>SMEMBERS</code>

ZSet 命令

命令	作用	示例含义
<code>ZADD key score member</code>	添加/更新 member 分数	<code>ZADD rank 100 user:1</code>
<code>ZINCRBY key increment member</code>	增加 member 分数	排行榜分数累加
<code>ZSCORE key member / ZMSCORE key m1 m2</code>	查询一个/多个 member 分数	查个人分数
<code>ZCARD key</code>	获取元素数量	排行榜规模
<code>ZCOUNT key min max</code>	统计 score 范围内数量	区间计数
<code>ZLEXCOUNT key min max</code>	按字典序范围计数	相同 score 下字典序场景
<code>ZRANGE key start stop WITHSCORES</code>	按排名/score/字典序范围查询	新版 <code>ZRANGE</code> 可覆盖多类范围查询
<code>ZREVRANGE key start stop WITHSCORES</code>	按分数倒序查询排名范围	TopN
<code>ZRANGEBYSCORE key min max</code>	按 score 范围查询	延迟队列取到期任务
<code>ZRANGEBYLEX key min max</code>	按字典序范围查询	特定字典序场景

命令	作用	示例含义
<code>ZRANK key member</code> / <code>ZREVRANK key member</code>	查询正序/倒序排名	个人排名
<code>ZREM key member</code>	删除 member	删除已处理任务
<code>ZREMRANGEBYSCORE key min max</code>	删除 score 范围	滑动窗口清理过期请求
<code>ZREMRANGEBYRANK key start stop</code>	删除排名范围	保留 TopN
<code>ZREMRANGEBYLEX key min max</code>	删除字典序范围	字典序清理
<code>ZPOPMIN key count</code> / <code>ZPOPMAX key count</code>	弹出最小/最大分数元素	任务/排行榜裁剪
<code>BZPOPMIN key timeout</code> / <code>BZPOPMAX key timeout</code>	阻塞弹出最小/最大元素	阻塞任务队列
<code>ZMPOP</code> / <code>BZMPOP</code>	从多个 ZSet 弹出元素	多队列场景
<code>ZUNION</code> / <code>ZINTER</code> / <code>ZDIFF</code>	多个 ZSet 并/交/差计算	多榜单合并
<code>ZUNIONSTORE</code> / <code>ZINTERSTORE</code> / <code>ZDIFFSTORE</code>	结果写入目标 ZSet	缓存榜单结果
<code>ZRANGESTORE dest src ...</code>	范围结果写入目标 ZSet	保存范围查询结果
<code>ZRANDMEMBER key count WITHSCORES</code>	随机返回 member	抽样
<code>ZSCAN key cursor MATCH pattern COUNT n</code>	渐进式扫描 ZSet	大 ZSet 遍历

Bitmap 命令

命令	作用	示例含义
<code>SETBIT key offset value</code>	设置某个 bit 位为 0/1	签到、是否已读
<code>GETBIT key offset</code>	获取某个 bit 位	查某天是否签到
<code>BITCOUNT key</code>	统计 bit 为 1 的数量	统计签到天数
<code>BITPOS key bit</code>	查找第一个 0/1 的位置	找第一个未完成位置
<code>BITOP AND/OR/XOR/NOT dest key...</code>	多个 bitmap 做位运算	多日活跃交并集
<code>BITFIELD key ...</code>	按位段读写整数	多 bit 状态压缩
<code>BITFIELD_R0 key ...</code>	只读 bit field	只读场景更安全

HyperLogLog 命令

命令	作用	示例含义
<code>PFADD key element</code>	添加元素参与基数估算	记录 UV
<code>PFCOUNT key</code>	估算基数	估算独立用户数
<code>PFMERGE dest key...</code>	合并多个 HLL	合并多天 UV

Geo 命令

命令	作用	示例含义
<code>GEOADD key lon lat member</code>	添加地理位置	添加门店坐标
<code>GEOPOS key member</code>	查询成员坐标	查门店位置
<code>GEODIST key m1 m2 unit</code>	计算两点距离	计算用户到门店距离
<code>GEOHASH key member</code>	返回 geohash 字符串	地理编码调试
<code>GEOSEARCH key ...</code>	按半径/矩形搜索附近成员	附近的人/店
<code>GEOSEARCHSTORE dest key ...</code>	搜索结果写入目标 key	缓存附近结果
<code>GEORADIUS</code> / <code>GEORADIUSBYMEMBER</code>	旧版附近搜索命令	新项目优先使用 <code>GEOSEARCH</code>

Stream 命令

命令	作用	示例含义
<code>XADD stream * field val</code>	追加消息	写入事件流
<code>XLEN stream</code>	查看消息数量	判断 Stream 规模
<code>XRANGE stream start end</code> / <code>XREVRANGE stream end start</code>	正序/倒序范围读取	消息回溯
<code>XTRIM stream MAXLEN n</code>	裁剪 Stream 长度	控制消息保留量
<code>XDEL stream id</code>	删除消息	删除指定消息
<code>XREAD STREAMS stream id</code>	普通读取 Stream	非消费者组读取
<code>XGROUP CREATE stream group id MKSTREAM</code>	创建消费者组	建立消费组
<code>XGROUP SETID stream group id</code>	修改消费者组游标	调整消费起点

命令	作用	示例含义
<code>XGROUP DESTROY stream group</code>	删除消费者组	清理消费组
<code>XGROUP DELCONSUMER stream group consumer</code>	删除消费者	清理下线消费者
<code>XREADGROUP GROUP group consumer STREAMS stream ></code>	消费者组读取新消息	组内分摊消费
<code>XACK stream group id</code>	确认消息已处理	从 PEL 移除
<code>XPENDING stream group</code>	查看未确认消息	排查堆积/失败消费
<code>XCLAIM stream group consumer min-idle id</code>	转移 pending 消息给其他消费者	消费者故障接管
<code>XAUTOCLAIM stream group consumer min-idle start</code>	自动转移 pending 消息	批量接管超时消息
<code>XINFO STREAM/GROUPS/CONSUMERS</code>	查看 Stream/组/消费者信息	故障和监控

Bloom Filter 命令, RedisBloom 模块

命令	作用	示例含义
<code>BF.RESERVE key error_rate capacity</code>	创建 Bloom Filter 并设置误判率/容量	提前规划容量
<code>BF.ADD key item</code>	添加元素	写入合法商品 ID
<code>BF.MADD key item...</code>	批量添加元素	初始化布隆
<code>BF.EXISTS key item</code>	判断元素是否可能存在	防缓存穿透
<code>BF.MEXISTS key item...</code>	批量判断	批量防穿透
<code>BF.INSERT key ...</code>	带参数批量插入	可指定容量/误判策略
<code>BF.INFO key</code>	查看 Bloom 信息	排查容量和误判配置

深入：Redis Pipeline 是什么？和事务有什么区别？

结论先行：

Pipeline 是客户端批量发送命令以减少网络往返，不能保证原子性；事务是 Redis 服务端按顺序执行命令队列，强调隔离执行。

分层展开：

Pipeline

- 多个命令一次发送。
- Redis 依次执行后批量返回。
- 主要优化网络 RTT 和客户端、服务端的 read/write 系统调用次数。
- 不保证原子性，也不会让 Redis 并行执行命令。

普通请求/响应

```
write cmd1 -> read resp1
write cmd2 -> read resp2
write cmd3 -> read resp3
```

每条命令都要等待一次网络往返，请求量大时大量时间消耗在 RTT 和系统调用上。

Pipeline 请求/响应

```
write cmd1+cmd2+cmd3 -> read resp1+resp2+resp3
```

客户端不用等上一条响应就继续发送后续命令，最后统一读取响应。严格来说，不一定永远只有一次 `read/write`，因为还受 TCP 缓冲区、命令大小、客户端库实现影响，但它会把多次网络往返和系统调用压缩成少数几次。

事务

- `MULTI/EXEC`。
- 保证事务队列执行期间不被插入其他命令。
- 不等于数据库事务。

使用建议

- 批量读写用 Pipeline。
- 多步状态变更需要原子性用 Lua 或事务。
- Pipeline 批量不能无限大，否则会占用内存和网络缓冲。

面试追问：Pipeline 在 Cluster 下有什么注意点？

不同 key 可能落到不同节点，客户端需要按 slot 拆分 pipeline 并发送到对应节点。多 key 原子操作仍要求同 slot。

面试追问：Pipeline 和 IO 多路复用是什么关系？

IO 多路复用是 Redis 服务端高效管理大量 socket 连接的网络模型；Pipeline 是客户端批量发送命令、批量读取响应的使用方式。前者解决“一个线程怎么管理很多连接”，后者解决“一个连接上多条命令怎么减少 RTT 和系统调用”。

深入：Redis 为什么不建议使用 KEYS?

结论先行：

KEYS 会遍历整个 keyspace，是 $O(N)$ 命令，在生产大实例上可能阻塞主线程，导致请求延迟飙升。

分层展开：

问题

- 全量扫描所有 key。
- 单线程执行期间其他请求排队。
- 大实例上耗时不可控。

替代方案

- 用 **SCAN** 渐进式扫描。
- 业务侧维护索引集合。
- 按前缀和业务维度规划 key。

SCAN 注意点

- **SCAN** 不是一次完整快照。
- 可能返回重复 key。
- 扫描期间新增或删除 key 会影响结果。

面试追问：SCAN 会阻塞吗？

单次 SCAN 只扫描一小部分，阻塞风险远小于 KEYS，但如果 count 过大、频率过高或 value 后续操作很重，仍会影响线上。

深入：Redis key 如何设计？

结论先行：

好的 key 设计要做到可读、可控、可分片、可治理，避免过长、过大、无 TTL 和热点集中。

分层展开：

命名规范

- 使用业务前缀： **user:profile:{uid}**。
- 层次清晰： **biz:module:type:id**。
- 避免含糊和冲突。

长度控制

- key 太长会浪费内存和网络。
- key 太短又不利于排查。
- 在可读性和成本之间平衡。

过期策略

- 缓存 key 必须有 TTL。
- TTL 加随机值。
- 永不过期 key 要有明确理由。

Cluster 友好

- 需要同 slot 的 key 使用 hash tag。
- 避免滥用 hash tag 造成倾斜。

治理能力

- 可按前缀统计。
- 可定位 owner。
- 可批量清理。

面试追问：key 越短越好吗？

不是。key 太短排查困难、容易冲突。大厂更看重可治理性，通常会牺牲少量内存换取可读性和可维护性。

深入：Redis 为什么这么快？

结论先行：

Redis 快的核心原因是“纯内存 + 单线程命令执行 + IO 多路复用 + 高效数据结构 + 工程级优化”。

分层展开：

纯内存访问

- 大部分操作在内存中完成，避免磁盘随机 IO。
- 常见命令复杂度为 $O(1)$ 或 $O(\log N)$ 。

单线程执行命令

- Redis 主要用单线程执行用户命令，避免多线程锁竞争、上下文切换和复杂并发控制。
- 单线程不是没有并发，而是网络连接很多，命令执行串行。

IO 多路复用

- 通过 `epoll/kqueue/select` 等机制，一个线程可以监听大量 socket。
- 连接就绪后再读取和处理，避免一个连接阻塞整个服务。

高效数据结构

- SDS、dict、ziplist/listpack、quicklist、skiplist、intset 等针对场景做了内存和性能优化。

后台线程辅助

- Redis 6 之后引入多线程 IO，主要用于网络读写，命令执行仍以单线程为主。
- lazy free、AOF fsync、关闭文件等任务可后台处理，减少主线程阻塞。

面试追问：Redis 单线程为什么还能支持高并发？

因为 Redis 的瓶颈通常不是 CPU，而是网络和内存访问。单线程执行命令很快，配合 IO 多路复用可以高效处理大量连接。但如果命令本身很重，例如大 key、复杂 Lua、全量扫描，仍会阻塞主线程。

深入：Redis 网络 IO 与 Pipeline 应该怎么讲？

结论先行：

Redis 的网络模型可以简化成：主线程通过 IO 多路复用监听大量 socket，就绪后读取请求、执行命

令、写响应；Pipeline 通过批量发送命令和批量读取响应减少 RTT 和系统调用次数，但不会让 Redis 并行执行命令。

面试回答主线：

1. Redis 命令执行主要是单线程，避免锁竞争。
2. 网络连接很多，但不是一个连接一个线程，而是用 epoll/kqueue/select 这类 IO 多路复用机制管理。
3. `GET/SET` 的读写是 Redis 数据语义；socket 可读/可写是网络事件；`read/write` 是系统调用，不要混在一起。
4. Pipeline 不是事务，也不是并行，只是把多条命令批量发送，减少网络往返。
5. Pipeline 批次不能无限大，否则会造成客户端/服务端缓冲区压力和延迟抖动。

详细拆解位置：

网络 IO、socket 可读/可写、`read/write` 系统调用、Pipeline 完整链路已经拆到：
[05-02 | 计算机网络与HTTP工程.md]

面试追问：Pipeline 是否保证原子性？

不保证。Pipeline 只减少网络往返，Redis 仍按命令顺序逐条执行。如果中间有其他客户端命令插入，Pipeline 本身不提供事务隔离；需要原子性时用 Lua 或事务。

四、数据类型、对象编码与底层结构

11. Redis 数据类型如何选？

类型	底层核心	典型场景	主要风险
String	SDS, int/embstr/raw 等编码	对象缓存、计数、Token、锁	大 Value、整体序列化
Hash	listpack/hashtable	对象字段读写	大 Hash、无 Field 级 TTL
List	quicklist + listpack	两端队列、最新列表	缺少完善 ACK，深层遍历
Set	intset/listpack/hashtable	标签、去重、交并差	大集合运算阻塞
ZSet	listpack 或 dict + skiplist	排行榜、延迟任务、滑动窗口	高基数占内存、大范围删除
Stream	radix tree 等结构与消费组元数据	Redis 内部消息流	Pending、裁剪和积压治理

12. SDS 为什么优于 C 字符串？

SDS 保存长度和容量，获取长度 $O(1)$ ，二进制安全，并通过预分配减少扩容。短字符串可用连续内存编码减少分配，长字符串转为普通动态结构。

面试追问：用 String 存 JSON 有什么问题？

适合整体读写，但修改单字段需要完整反序列化和回写；大 JSON 会增加网络与内存成本。高频字段更新可考虑 Hash，权威复杂查询仍放数据库。

13. Hash Table 如何扩容？

Redis 字典通常通过渐进式 Rehash，把旧表中的桶逐步迁移到新表，避免一次迁移造成长阻塞；正常访问也可顺带推进迁移。

工程边界： 渐进不等于零成本。超大字典扩缩容仍会增加 CPU 和内存峰值，容量规划需要保留双表并存空间。

14. Listpack 和 Quicklist 解决什么问题？

Listpack 使用紧凑连续内存降低指针开销，适合小对象；Quicklist 用双向链表连接多个紧凑块，在两端操作和内存效率之间折中。

编码会随元素数量和大小变化，不能把“小 Hash 一定是某编码”当成永久事实；应结合版本和配置理解转换阈值。

15. ZSet 为什么使用跳表？

跳表结构简单，支持 $O(\log N)$ 插入、删除、按分数定位和范围遍历，范围查询实现自然；配合字典可快速按 Member 查 Score。

对比红黑树： 两者复杂度相近，但跳表的区间遍历、实现和并发演进更直观。Redis 的选择是工程权衡，不是红黑树不能实现。

16. Bitmap、HyperLogLog 和 Bloom Filter 怎么选？

- Bitmap：精确布尔状态，适合连续 ID 的签到、在线标记；稀疏大 ID 会浪费空间。
- HyperLogLog：近似基数统计，内存小，不能列出元素。
- Bloom Filter：判断“可能存在/一定不存在”，可防穿透；有误判且删除困难，需要按容量和误判率初始化。

17. Stream 与 List 队列有什么区别？

Stream 支持消息 ID、消费组、Pending、确认和故障认领，可靠消费能力强于简单 List；但仍要治理积压、裁剪、死信和消费者状态。

大规模跨系统事件流、长期重放和独立运维边界通常选择专业消息平台，而不是只因已有 Redis 就使用 Stream。

深入：Redis 底层数据结构速记

这张表用来背“类型 -> 编码 -> 为什么这么设计”。面试时不需要一上来报源码细节，但要能讲出结构和取舍。

Redis 类型	常见底层编码/结构	速记	适合场景	主要风险
String	SDS；对象编码可能是 <code>int</code> 、 <code>embstr</code> 、 <code>raw</code>	<code>len + alloc + buf</code> ，二进制安全，O(1) 取长度	缓存对象、计数、锁、限流计数	大 value、整体序列化成本
Hash	小 Hash 用 listpack；大 Hash 用 hashtable	小对象紧凑，大对象查找快	用户字段、商品属性、购物车	字段太多形成大 key，无 field TTL
List	quicklist，节点内部是 listpack	双向链表 + 紧凑块，兼顾两端操作和内存	简单队列、最新列表	大 List、缺 ACK
Set	intset / listpack / hashtable	小整数省内存，普通集合靠哈希去重	标签、关注、黑名单、共同好友	大集合全量/交并差阻塞
ZSet	小 ZSet 用 listpack；大 ZSet 用 dict + skiplist	dict 查 member，skiplist 做排序和范围	排行榜、延迟队列、滑动窗口	大 ZSet、范围操作成本
Bitmap	本质是 String/SDS 的 bit 操作	一个 bit 表示一个布尔状态	签到、在线、是否完成动作	offset 稀疏会浪费空间
HyperLogLog	稀疏/密集概率结构，Redis 以字符串对象承载	用误差换极低内存	UV、独立 IP、独立设备	有误差，不能取具体元素
Geo	底层使用 ZSet，score 存 geohash 编码	地理位置转成可排序 score	附近的人/店	精度、半径查询成本
Stream	radix tree/rax + listpack；消费者组维护 PEL	append-only log + 消费组确认	轻量消息流、异步任务	不等同 Kafka，需治理 PEL
Bloom Filter	bitmap + 多个 hash 函数，RedisBloom 模块提供	说不存在一定不存在，说存在只是可能存在	缓存穿透、黑名单	有误判、删除困难、需容量规划

底层结构一句话背法：

String 是 SDS；
Hash 小用 listpack，大用 hashtable；
List 是 quicklist；
Set 小整数用 intset，普通用 hashtable；
ZSet 小用 listpack，大用 dict + skiplist；
Bitmap 是 String 的 bit；
HLL 是概率计数；
Geo 是 ZSet 的地理编码；

Stream 是 rax + listpack + PEL ;
Bloom 是 bitmap + 多 hash。

深入：Redis 常见数据结构如何总览？

结论先行：

Redis 更准确地说是一个内存数据结构服务器，不只是简单 key-value。面试时不要只背命令，要讲清楚“结构能力、底层编码、适用场景、风险边界”。

数据结构	核心能力	典型场景	主要风险
String	二进制安全 value、原子计数	缓存对象、计数器、锁、限流	大 value、整体序列化成本
Hash	field-value 对象	用户信息、商品属性、购物车	字段过多形成大 key、无 field TTL
List	有序、可重复、两端操作	简单队列、栈、最新列表	缺少 ACK、大 List 风险
Set	无序、唯一、集合运算	标签、关注、共同好友、去重	大集合交并差阻塞
ZSet	唯一、score 排序	排行榜、延迟队列、滑动限流	大 ZSet、范围操作成本
Bitmap	bit 状态	签到、在线状态、是否完成动作	只支持 0/1、稀疏 offset 浪费
HyperLogLog	近似基数统计	UV、独立 IP、独立设备数	有误差、不能取元素
Bloom Filter	判断可能存在	缓存穿透、黑名单、已读判断	有误判、删除困难、需容量规划
Stream	消息流、消费组、ACK	轻量消息队列、事件流	不能替代专业 MQ 的所有能力

一句话记忆：

String 适合整体值；
Hash 适合局部字段；
List 适合简单队列；
Set 适合去重集合；
ZSet 适合排序范围；
Bitmap 适合海量布尔状态；
HyperLogLog 适合近似去重计数；
Bloom Filter 适合防穿透；
Stream 适合轻量可靠消息流。

深入：Redis 的 String 底层是什么？适合哪些场景？

结论先行：

String 是 Redis 最基础的数据类型，底层不是直接使用 C 字符串，而是使用 SDS，也就是 Simple Dynamic String。它适合缓存对象、计数器、分布式锁、限流计数、Session、验证码、Token、简单状态值和二进制数据。

为什么不用 C 字符串？

C 字符串有几个问题：

1. 获取长度需要 $O(N)$ ，因为要从头扫到 `'\0'`
2. 不二进制安全，遇到 `'\0'` 会被认为字符串结束
3. 拼接时容易缓冲区溢出
4. 扩容和内存管理不方便

SDS 可以理解成：

```
SDS = header + buf[]
```

len	alloc/free	buf[]
已用长度	可用容量	实际字节内容

SDS 的核心优点

- $O(1)$ 获取长度：直接读 header 里的 len，不需要像 `strlen` 一样扫描。
- 二进制安全：不依赖 `'\0'` 判断结束，可以存 JSON、ProtoBuf、MessagePack、图片字节、压缩后的字节。
- 减少频繁扩容：内部保存容量信息，空间够就直接追加，空间不够再扩容，思路类似 Go slice 的 `len/cap`。

String 的对象编码

```
String 对象
├─ int      -> 适合整数值，比如 "100"
├─ embstr   -> 适合短字符串，Redis object 和 SDS 连续分配，减少内存分配次数
└─ raw      -> 普通长字符串
```

常见命令和复杂度

```
SET user:1 '{"id":1,"name":"Tom"}'
GET user:1

INCR article:100:views
DECR stock:sku:1
```

```
SET lock:order:1 requestId NX EX 10
```

- `GET` / `SET` 通常是 $O(1)$ 。
- `INCR` / `DECR` 是原子自增自减，适合计数器。
- `SET key value NX EX seconds` 是分布式锁的基础命令。

典型场景

缓存对象：

```
SET user:1001 '{"id":1001,"name":"Tom","age":18}' EX 3600  
GET user:1001
```

适合对象整体读写、字段不需要频繁单独修改、跨语言序列化方便的场景。

计数器：

```
INCR article:1001:views  
INCR user:1001:likes
```

`INCR` 是 Redis 原子操作，多个客户端并发执行也不会丢更新。

分布式锁：

```
SET lock:order:1001 requestId NX EX 10
```

`NX` 表示 key 不存在才设置，`EX` 表示过期时间，`requestId` 是锁持有者唯一标识。释放锁时不能简单 `DEL`，要先判断 value 是否是自己的 requestId，再删除，否则可能误删别人的锁。

固定窗口限流：

```
INCR rate:user:1001:2026-07-10-10:00  
EXPIRE rate:user:1001:2026-07-10-10:00 60
```

固定窗口实现简单，但有边界突刺问题，比如 10:00:59 和 10:01:00 各请求很多次，短时间内可能突破真实限流。

风险

- value 太大形成大 key。
- 整体更新需要反序列化和重新序列化。
- 局部字段更新成本高。
- 热点 key 访问压力大。
- 分布式锁实现不当会有误删风险。

面试追问：String 的 SDS 和 C 字符串有什么区别？

C 字符串以 `\0` 结尾，获取长度需要遍历，不是二进制安全的。SDS 自己保存长度和容量，所以获取长度是 $O(1)$ ，并且二进制安全，可以存任意字节内容。SDS 还可以通过容量信息减少频繁扩容。

深入：Redis 的 Hash 适合存对象吗？

结论先行：

Hash 适合存储字段较少、结构扁平、需要局部读写的对象，例如用户基础信息、商品基础属性、购物车、配置项集合、计数字段集合。

Hash 的结构

```
key = user:1001
```

field	value
name	Tom
age	18
city	Beijing
level	3

命令示例：

```
HSET user:1001 name Tom age 18 city Beijing
HGET user:1001 name
HGETALL user:1001
HINCRBY user:1001 level 1
```

图示：

Redis key: user:1001

|
v
Hash

field	value
name	Tom
age	18
city	Beijing

Hash 底层编码

Hash

- 小对象：listpack，紧凑、省内存
- 大对象：hashtable，查询和更新效率更好

listpack 可以理解成一块连续紧凑内存：

listpack:

name	Tom	age	18	city	BJ
------	-----	-----	----	------	----

优点是省内存、指针少、缓存友好；缺点是字段多时查找可能需要遍历，中间插入/删除成本相对更高。

当 Hash 变大后，会转成 hashtable：

field hash -> bucket -> field/value

hashtable 字段查找快、字段更新快，适合字段数量较多的对象，但比 listpack 更耗内存，也有哈希表扩容成本。

典型场景

用户基础信息：

```
HSET user:1001 name Tom age 18 city Beijing
HGET user:1001 name
HSET user:1001 age 19
```

商品库存或属性：

```
HSET product:1001 name phone price 3999 stock 100
HINCRBY product:1001 stock -1
```

购物车：

```
HSET cart:user:1001 sku:1 2
HINCRBY cart:user:1001 sku:1 1
HDEL cart:user:1001 sku:2
```

结构：

```
cart:user:1001
├─ sku:1 -> 3
├─ sku:2 -> 1
└─ sku:3 -> 5
```

String 和 Hash 怎么选？

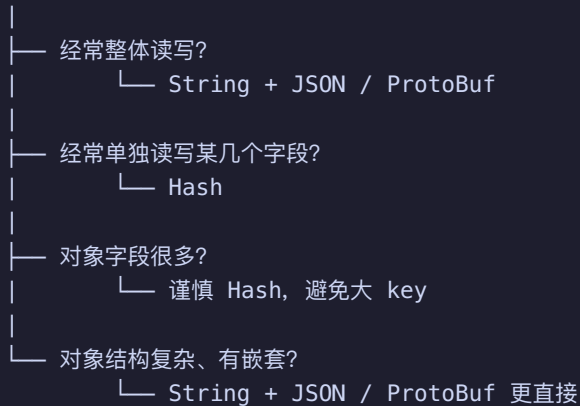
String + JSON / ProtoBuf：
适合整体读写、结构复杂、跨语言序列化。

Hash:

适合字段扁平、字段级读写频繁、字段数量可控。

流程图:

缓存一个对象



风险

- 字段过多会形成大 key。
- **HGETALL** 大 Hash 会阻塞主线程。
- 删除、过期、迁移、复制大 Hash 都会变重。
- Redis 不支持 field 级 TTL，只能给整个 key 设置过期。
- 嵌套复杂对象不适合直接拆成 Hash。

面试追问：Hash 扩容会不会阻塞？

Redis 的字典扩容通常采用渐进式 rehash，不会一次性迁移全部数据，而是在后续操作中逐步迁移，减少单次阻塞风险。但如果 Hash 本身是超大 key，集中访问、删除、迁移、**HGETALL** 仍然可能造成延迟抖动。

深入：List、Set、ZSet 分别适合什么场景？

结论先行：

List 是有序可重复列表，适合队列和最新列表；Set 是无序去重集合，适合标签、关注和集合运算；ZSet 是带 score 的有序集合，适合排行榜、延迟队列、滑动窗口限流和范围查询。

总体对比

List：有序，可重复，适合队列和最新列表
Set：无序，唯一，适合去重和集合运算
ZSet：唯一，有 score，适合排序、排行榜、范围查询

List：有序、可重复、两端操作

List 支持两端 push/pop：

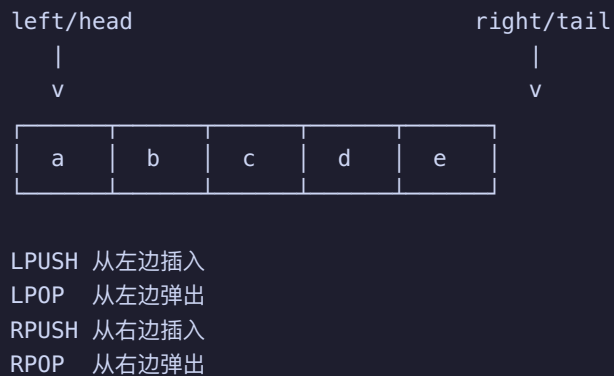

```

LPUSH queue task1
RPUSH queue task2

LPOP queue
RPOP queue

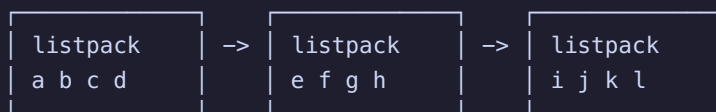
```

图示：



现代 Redis List 底层常见是 quicklist：

quicklist = 双向链表 + listpack



如果只用链表，两端插入删除方便，但每个节点都有指针开销；如果只用连续数组，省内存但中间插入删除成本高。quicklist 的折中是：每个节点内部用 listpack 紧凑存储，节点之间用双向链表连接，兼顾内存和两端操作效率。

List 做简单队列：

```

LPUSH queue task1
LPUSH queue task2
BRPOP queue 0

```

风险：

- 缺少完善 ACK 机制。
- 消费者取出后宕机，消息可能丢。
- 不适合复杂消息追踪。
- 大 List 做 `LRANGE 0 -1` 可能很重。
- 队列堆积会形成大 key。

Set：无序、唯一、集合运算

Set 适合标签集合、关注集合、粉丝集合、抽奖去重、共同好友、黑名单、权限集合。

```
SADD user:1:tags go redis backend
SISMEMBER user:1:tags redis
SREM user:1:tags go
SMEMBERS user:1:tags

SINTER user:1:follows user:2:follows
SUNION set1 set2
SDIFF set1 set2
```

Set 底层编码常见有：

intset	小整数集合
listpack	Redis 7.2+ 小集合可能使用
hashtable	普通集合

共同好友示例：

```
SADD user:1:follows u2 u3 u4
SADD user:2:follows u3 u4 u5
SINTER user:1:follows user:2:follows
```

结果：

```
user:1:follows = {u2, u3, u4}
user:2:follows = {u3, u4, u5}

交集 = {u3, u4}
```

风险：

- 大 Set 做 **SMEMBERS** 会很重。
- 大 Set 做 **SINTER** / **SUNION** / **SDIFF** 可能阻塞。
- 集合元素太多会形成大 key。
- Set 无序，不适合需要排序的场景。

ZSet：唯一、score 排序、范围查询

ZSet 特点：

```
member 唯一
每个 member 有一个 score
按 score 排序
支持排名和范围查询
```

适合排行榜、延迟队列、按时间排序的 Feed、滑动窗口限流、权重排序、范围检索、优先级队列。

常见命令：

```
ZADD rank 100 user:1
ZADD rank 200 user:2
ZADD rank 150 user:3

ZRANGE rank 0 -1 WITHSCORES
ZREVRANGE rank 0 9 WITHSCORES
ZSCORE rank user:2
ZREVRANK rank user:2
ZREM rank user:1
```

ZSet 常见两种编码：

```
小 ZSet    -> listpack
普通 ZSet  -> skiplist + dict
```

普通 ZSet 可以理解成两套结构：

```
ZSet
├── dict
│   └── member -> score
└── skiplist
    └── 按 score 排序的 member
```

图示：

```
dict:
user:1 -> 100
user:2 -> 200
user:3 -> 150

skiplist:
score 100 -> user:1
score 150 -> user:3
score 200 -> user:2
```

为什么要 dict + skiplist 两套结构？

- **dict** 负责快速按 member 查 score，例如 `ZSCORE rank user:2`。
- **skiplist** 负责按 score 排序和范围扫描，例如 `ZRANGE`、`ZRANGEBYSCORE`。
- 这是典型的空间换时间。如果只有 dict，不能高效按 score 排序；如果只有 skiplist，根据 member 查 score 不够直接。

排行榜：

```
ZINCRBY game:rank 10 user:1001
ZREVRANGE game:rank 0 9 WITHSCORES
ZREVRANK game:rank user:1001
```

延迟队列：

```
ZADD delay:queue 1720000000 task:1
ZADD delay:queue 1720000010 task:2
ZRANGEBYSCORE delay:queue -inf now LIMIT 0 10
ZREM delay:queue task:1
```

这条查询命令可以读成：从 `delay:queue` 里取出 score 小于等于当前时间的前 10 个任务。

片段	是否动态	含义
<code>ZRANGEBYSCORE</code>	固定	按 ZSet score 范围查询
<code>delay:queue</code>	动态	延迟队列 key，可以按业务拆成 <code>delay:queue:{biz}</code>
<code>-inf</code>	通常固定	score 下界，表示从最早任务开始查
<code>now</code>	动态	当前时间戳；实际执行时要替换成具体时间，例如 <code>1720000050</code>
<code>LIMIT</code>	固定	限制返回数量
<code>0</code>	通常固定	从结果第 0 条开始取
<code>10</code>	动态	本批最多取 10 条，避免一次拉太多

多消费者并发时，要避免重复消费，可以用 Lua 脚本保证“取出 + 删除”的原子性。

滑动窗口限流：

ZSet 做滑动窗口限流的核心是：

```
key    = 限流维度
score  = 请求发生时间戳
member = 请求唯一 ID
```

推荐的核心命令顺序：

```
ZREMRANGEBYSCORE rate:user:1001 -inf now-60
ZCARD rate:user:1001
ZADD rate:user:1001 now requestId
EXPIRE rate:user:1001 60
```

逐条理解：

命令	作用	动态部分
<code>ZREMRANGEBYSCORE rate:user:1001 -inf now-60</code>	删除 60 秒窗口外的旧请求	<code>rate:user:1001</code> 、 <code>now-60</code>
<code>ZCARD rate:user:1001</code>	统计当前窗口内请求数	<code>rate:user:1001</code>
<code>ZADD rate:user:1001 now requestId</code>	未超时写入当前请求	<code>rate:user:1001</code> 、 <code>now</code> 、 <code>requestId</code>
<code>EXPIRE rate:user:1001 60</code>	给限流 key 设置过期时间，避免冷 key 常驻	<code>rate:user:1001</code> 、 <code>60</code>

实际工程中，判断逻辑通常是：

1. 删除窗口外请求
2. 统计窗口内请求数
3. 如果 `count >= limit`，拒绝
4. 如果 `count < limit`，写入当前请求并放行
5. 设置 TTL

这几步要用 Lua 包起来，保证“清理 + 统计 + 判断 + 写入 + 设置过期”原子执行，否则并发请求可能同时看到未超限并一起放行。

工程示例：调用 Facebook 获取 campaign 列表接口，多层限流：

接口：GET /facebook/campaigns
 限制：
 每秒最多 60 次
 每 5 分钟最多 3000 次
 每天最多 10000 次

可以按平台、账号、接口、窗口拆 key：

```
rate:facebook:{account_id}:campaign_list:1s
rate:facebook:{account_id}:campaign_list:5m
rate:facebook:{account_id}:campaign_list:1d
```

每次请求同时检查三个窗口：

窗口 1: 1 秒, limit = 60
 窗口 2: 300 秒, limit = 3000
 窗口 3: 86400 秒, limit = 10000

伪流程：

```
for each rule in [1s/60, 5m/3000, 1d/10000]:
  key = rate:facebook:{account_id}:campaign_list:{window}
```

```
ZREMRANGEBYSCORE key -inf now-window
count = ZCARD key
if count >= limit:
    reject

for each rule:
    ZADD key now requestId
    EXPIRE key window + buffer
```

面试表达：

ZSet 滑动窗口限流用 score 存请求时间，用 member 存请求唯一 ID。每次请求先清理窗口外数据，再统计当前窗口请求数，未超阈值才写入当前请求。像 Facebook campaign 列表这种外部 API，可以按平台、账号、接口维度做多层窗口，例如 1 秒 60 次、5 分钟 3000 次、1 天 1 万次。多个窗口必须同时通过才放行，并且用 Lua 保证原子性。

它比 String 固定窗口更平滑、更精确，但存储和计算成本更高，因此要控制 key 粒度、窗口数量、TTL 和单次清理规模。

面试追问：排行榜为什么用 ZSet?

因为 ZSet 同时支持 member 唯一、score 排序、按排名范围查询、按分数范围查询、更新分数、删除元素等操作。排行榜需要频繁更新分数、查 TopN、查个人排名，这些都和 ZSet 的能力高度匹配。

深入：Bitmap、HyperLogLog、Bloom Filter 分别解决什么问题？

结论先行：

Bitmap 解决海量布尔状态，HyperLogLog 解决近似去重计数，Bloom Filter 解决高效判断“可能存在或一定不存在”。

```
Bitmap:
    海量布尔状态

HyperLogLog:
    近似去重计数

Bloom Filter:
    判断元素是否可能存在
```

一句话区别：

```
Bitmap 关心某个位置是 0 还是 1;
HyperLogLog 关心大概有多少个不同元素;
Bloom Filter 关心某个元素是否可能存在。
```

Bitmap：海量布尔状态

Bitmap 本质上不是一种独立的底层对象，而是基于 String 的 bit 操作。一个 bit 表示一个状态：

0 = 否
1 = 是

用户签到:

```
SETBIT sign:user:1001 0 1
SETBIT sign:user:1001 1 0
GETBIT sign:user:1001 0
BITCOUNT sign:user:1001
```

图示:

user:1001 本月签到情况:

day: 1 2 3 4 5 6 7 8
bit: 1 1 0 1 0 0 1 1

sign:user:1001

1	1	0	1	0	0	1	1
---	---	---	---	---	---	---	---

1日 2日 3日 4日 5日 6日 7日 8日

优点是极省内存，适合海量用户布尔状态；缺点是只能表达 0/1，offset 过大可能导致底层 String 被撑大，不适合非常稀疏且 ID 极大的场景，通常需要对 ID 做映射或分片。

HyperLogLog: 近似去重计数

HyperLogLog 适合做 UV、独立 IP、独立设备数、独立搜索关键词数。

```
PFADD uv:2026-07-10 user1
PFADD uv:2026-07-10 user2
PFADD uv:2026-07-10 user1

PFCOUNT uv:2026-07-10
```

即使 user1 添加两次，估算基数也会按去重后计算。它的优势是内存占用很小，适合允许少量误差的统计；限制是只能估算数量，不能列出元素，有误差，不能判断某个元素是否存在。

Bloom Filter: 判断是否可能存在

Bloom Filter 用很少内存判断一个元素是否可能存在。它的结论只有两种:

说不存在:
一定不存在

说存在:
可能存在

原理是多个 hash 函数把一个元素映射到多个 bit 位：

```
添加 item:
item -> hash1 -> bit 3 = 1
      -> hash2 -> bit 8 = 1
      -> hash3 -> bit 15 = 1
```

```
查询 item:
如果对应 bit 位有一个是 0:
    一定不存在
```

```
如果对应 bit 位全是 1:
    可能存在
```

缓存穿透流程：

```
请求 productId
  |
  v
Bloom Filter
  |
  ├── 不存在：直接返回空
  |
  └── 可能存在
      |
      v
      查 Redis 缓存
      |
      ├── 命中：返回
      └── 未命中：查 DB
```

风险：

- 有误判，少量不存在请求仍可能打到后端。
- 普通 Bloom Filter 删除困难，删除可能影响其他元素。
- 容量和误判率要提前规划，否则误判率会升高。
- 数据初始化和增量更新要做好。
- Bloom Filter 本身也要考虑可用性。

面试追问：Bloom Filter 为什么说不存在一定不存在，存在只是可能存在？

查询时，如果对应 bit 位有任何一个是 0，说明这个元素一定没被添加过，所以一定不存在。如果对应 bit 位全是 1，可能是这个元素设置的，也可能是其他元素碰撞设置的，所以只是可能存在。

深入：Redis Stream 和 List 做消息队列有什么区别？

结论先行：

List 可以做简单队列，Stream 更像轻量消息队列，支持消息 ID、消费者组、ACK、Pending Entries List 和消息回溯。

List 做队列

```
LPUSH queue task1
BRPOP queue 0
```

流程：

```
生产者
|
| LPUSH
v
List 队列
|
| BRPOP
v
消费者
```

优点是简单、高效、容易理解，适合轻量异步任务。缺点是没有内置消息 ID、没有消费者组、没有天然 ACK，消费者取出后宕机可能丢消息，也不方便追踪消息状态。可以用 `BRPOPLPUSH` 或 `BLMOVE` 做可靠队列增强，但复杂度会上升。

Stream 做消息队列

```
XADD mystream * user 1001 action login
XGROUP CREATE mystream group1 0 MKSTREAM
XREADGROUP GROUP group1 consumer1 COUNT 10 STREAMS mystream >
XACK mystream group1 messageId
```

Stream 消息结构：

```
stream key: mystream

messageId      fields
-----
1720000000000-0    user=1001 action=login
1720000000001-0    user=1002 action=pay
```

消费者组可以让多个消费者分摊消息：

```
Stream
|
v
Consumer Group: group1
├─ consumer1
├─ consumer2
└─ consumer3
```

消费者读到消息后，如果还没 ACK，这条消息会进入 PEL：

PEL = Pending Entries List

XREADGROUP 读取消息



消息进入 PEL



- ├── 处理成功: XACK, 从 PEL 删除
- └── 消费者宕机: 消息仍在 PEL, 可重试或转移

选型建议

本地异步、小规模缓冲:

List

需要 ACK、消费者组、消息回溯:

Stream

强一致、高吞吐、复杂消息治理:

Kafka / RocketMQ / Pulsar

面试追问: Stream 能替代 Kafka 吗?

不能简单替代。Stream 适合 Redis 生态内的轻量异步任务和中小规模消息流。Kafka 更适合大吞吐、分区扩展、持久日志、跨系统数据管道和复杂消息治理。选型上, 如果只是业务内部轻量队列, Stream 可以; 如果是核心消息中间件、大规模日志流或跨系统事件总线, 应该选 Kafka、RocketMQ、Pulsar。

补充模块: 核心能力原理分析: 为什么这样设计

这一章回答“Redis 为什么这样做”。面试时从机制、复杂度、风险边界讲, 避免只背结论。

深入: ZSet 为什么用跳表而不是红黑树?

结论先行:

Redis ZSet 选择跳表, 是因为跳表实现简单、范围查询自然、性能足够稳定, 配合 dict 可以很好满足 ZSet 的需求。

跳表是什么?

跳表可以理解成多层有序链表, 上层是索引, 底层保存完整数据:

```
Level 3: 1 -----> 9
Level 2: 1 -----> 5 -----> 9
Level 1: 1 -> 3 -> 5 -> 7 -> 9
```

查找 7 时, 从高层开始跳, 跳不过去就下降一层, 最后在底层找到目标。

平均复杂度:

```
查找  $O(\log N)$ 
插入  $O(\log N)$ 
删除  $O(\log N)$ 
范围遍历  $O(\log N + M)$ 
```

其中 M 是返回元素数量。

为什么不用红黑树？

范围查询更自然：

```
ZRANGE rank 0 9
ZRANGEBYSCORE rank 100 200
ZREVRANGE rank 0 9
```

跳表找到起点后，可以沿底层链表顺序往后扫：

```
score: 100 -> 120 -> 150 -> 180 -> 220
          ↑-----↑
          范围查询结果
```

红黑树也可以做范围查询，但实现上通常要做中序遍历，节点关系维护更复杂。

实现更简单：

- 红黑树需要旋转、染色和复杂平衡逻辑。
- 跳表通过随机层高维持概率平衡，代码更容易维护。

性能足够稳定：

- 跳表理论上有退化概率，但概率极低。
- Redis 通过随机层数、最大层数限制和合理概率参数，让跳表在工程上保持接近 $O(\log N)$ 。

面试追问：ZSet 为什么还要配 dict？

因为跳表不擅长通过 member 直接定位 score。比如 `ZSCORE rank user:1001`，用 dict 做 `member -> score` 很快；skiplist 负责 score 排序和范围查询。两者配合是空间换时间。

五、缓存模型、一致性与高并发保护

18. Cache Aside 的标准读写路径是什么？

```
读: GET Cache
    -> Hit: 返回
    -> Miss: 查 DB
    -> 回填 Cache + TTL

写: 更新 DB
```

-> 删除 Cache
-> 删除失败进入重试/CDC/补偿

结论先行： 常规业务优先“更新数据库后删除缓存”，因为数据库是权威源，删除比更新缓存更少引入双写覆盖问题。

19. 为什么更新 DB 后删缓存仍可能不一致？

并发读可能在写事务前读到旧值，并在写线程删除缓存之后回填旧值。该窗口通常很短，可通过合理 TTL、版本号、延迟补删、CDC 失效或强一致场景读主库降低风险。

工程判断： 先定义允许的不一致时间。不要把延迟双删包装成强一致；核心资产数据不能只靠缓存正确性。

20. 缓存删除失败如何兜底？

- 同事务写 Outbox，由 Publisher 重试删除事件。
- 订阅 Binlog/CDC，按数据库变更失效缓存。
- 删除动作天然幂等，可指数退避重试。
- TTL 是最后兜底，不是唯一方案。
- 监控删除失败、缓存版本落后和 DB/缓存差异。

21. 穿透、击穿和雪崩分别是什么？

问题	本质	方案
穿透	大量请求不存在数据	参数校验、空值缓存、Bloom Filter、限流
击穿	单个热点 Key 失效并发回源	互斥重建、逻辑过期、预热、本地缓存
雪崩	大量 Key 同时失效或 Redis 故障	TTL 随机、分批预热、多级缓存、限流降级

所有方案都要限制回源并发，否则 Redis 故障会把数据库一并拖垮。

22. 热点缓存如何重建？

发现逻辑过期

- > 一个请求获得重建资格
- > 其他请求返回可接受旧值或等待短时间
- > 查权威源
- > 带版本回填
- > 释放资格

强一致数据不能随意返回旧值；可用性优先的详情、模板等场景可以接受短暂旧值。重建锁只是控制并发，业务仍要防止锁过期后的并发覆盖。

23. 多级缓存如何保证失效？

- DB 更新后发布带实体 ID 和版本号的失效事件。
- 各实例删除本地缓存和 Redis，或者本地条目读取时校验版本指纹。
- 本地缓存设置较短 TTL 和容量上限。
- 消费失效事件必须幂等，失败可重放。
- 监控本地命中、Redis 命中、回源 QPS 和版本落后。

深入：缓存和数据库如何保证一致性？

结论先行：

缓存与数据库在常规高并发架构下很难低成本做到绝对强一致。工程上通常把数据库作为唯一事实源，采用 Cache Aside，通过“先更新数据库，再删除缓存”，配合删除失败重试、MQ/CDC 异步补偿、延迟二次删除、版本号校验和 TTL 兜底，实现最终一致或有界不一致。

缓存与数据库一致性

- ├ 一致性目标分级：强一致 / 最终一致 / 有界不一致
- ├ Cache Aside 读写流程
- ├ 为什么先更新数据库，再删除缓存
- ├ 删除缓存失败如何补偿
- ├ 并发读回填旧值问题
 - └ 延迟二次删除
- ├ 本地缓存 + Redis + DB 的多级一致性
- └ 高并发下的缓存重建与性能权衡

完整“缓存一致性与多级缓存实战”上位方法论已沉淀到 [04-02 | 分布式与高并发架构.md](#) 的专题模块；本题重点保留 Redis 缓存落地视角。

一、为什么会有一致性问题？

没有缓存时，数据只有数据库一份，不存在缓存一致性问题；引入 Redis 后，数据变成：

Redis：派生副本
DB：真实数据源

如果再引入本地缓存，同一份数据可能同时存在于：

应用实例 A 的本地缓存
应用实例 B 的本地缓存
Redis 集群
MySQL / MongoDB

只要数据存在多个副本，就必然要回答：先更新谁、失败怎么办、并发读写怎么办、消息丢失怎么办、副本延迟怎么办。所以一致性问题的本质不是 Redis 独有，而是多副本系统在性能、可用性和一致性之间做权衡。

二、一致性目标要先分级

强一致性

- 写入完成后，后续读取必须立即看到最新值。
- 适合余额、支付状态、订单状态流转、库存最终扣减、权限安全校验。
- 通常依赖数据库事务、条件更新、锁、版本号、状态机和关键读取绕过缓存。
- 普通 Redis 缓存方案不应承担最终正确性。

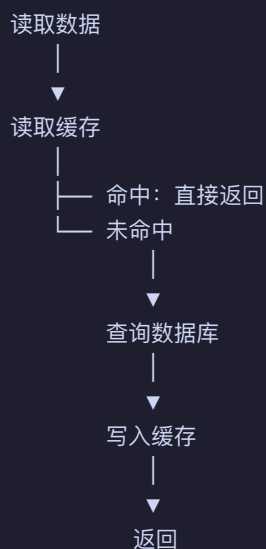
最终一致性

- 短时间允许读到旧值，但经过重试、过期或异步同步后最终一致。
- 适合用户资料、商品介绍、文章内容、普通配置展示、非核心统计。
- Cache Aside 主要解决这类场景。

有界不一致

- 允许短时间旧数据，但必须限制在业务可接受窗口内。
- 例如最多旧 1 秒、5 秒，或旧到本地缓存短 TTL 到期。
- 多级缓存常见目标不是瞬时强一致，而是把不一致窗口控制住。

三、Cache Aside 读路径



```
func GetUser(ctx context.Context, id int64) (*User, error) {
    key := fmt.Sprintf("user:%d", id)

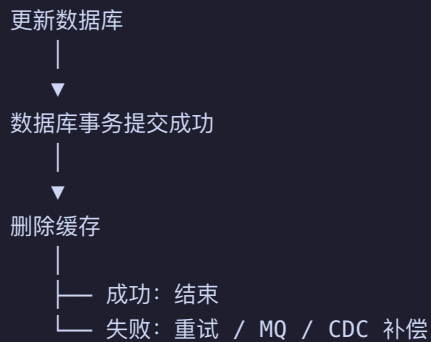
    user, err := redisGetUser(ctx, key)
    if err == nil {
        return user, nil
    }

    user, err = queryUserFromDB(ctx, id)
    if err != nil {
        return nil, err
    }
}
```

```
// 写缓存失败通常不影响本次读请求返回，但要记录日志和指标。
_ = redisSetUser(ctx, key, user, 30*time.Minute)
return user, nil
}
```

四、Cache Aside 写路径

推荐流程：



```
func UpdateUser(ctx context.Context, user *User) error {
    if err := updateUserInDB(ctx, user); err != nil {
        return err
    }

    key := fmt.Sprintf("user:%d", user.ID)
    if err := redisDelete(ctx, key); err != nil {
        enqueueCacheInvalidation(key)
    }

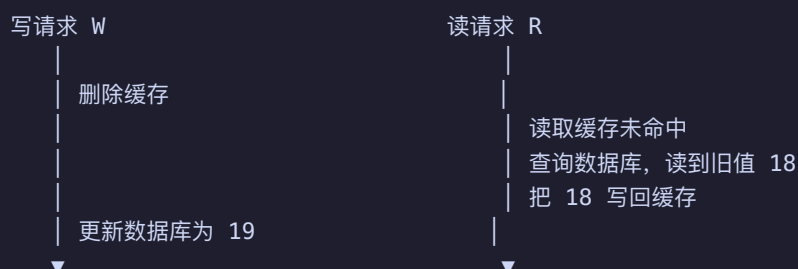
    return nil
}
```

五、为什么不是先删除缓存，再更新数据库？

假设初始状态：

```
数据库：age = 18
缓存：  age = 18
```

如果先删缓存再更新数据库，可能出现：



最终：

```
数据库：19
缓存： 18
```

先删除缓存会制造一个缓存空窗期，并发读可能把数据库旧值重新填回缓存。

六、为什么不是更新数据库后直接更新缓存？

并发写可能乱序覆盖：

```
W1 更新数据库为 1
W2 更新数据库为 2
W2 更新缓存为 2
W1 因网络较慢，最后更新缓存为 1
```

最终：

```
数据库：2
缓存： 1
```

此外，缓存可能是复杂聚合结果，一次数据库写可能影响多个缓存 key，直接更新缓存的维护成本高。删除缓存是幂等动作，`DEL key` 执行一次或多次最终效果相同，更适合重试和异步补偿。

七、先更新数据库，再删除缓存是否绝对安全？

不是。它仍然存在并发读回填旧值的竞态。关键不是写请求顺序错了，而是读请求的“查 DB”和“写缓存”不是原子操作，可能横跨写请求的更新和删除过程。

假设缓存已经过期：

```
数据库：旧值 18
缓存：不存在
```

可能出现：

读请求 R	写请求 W
1. 读 Redis，缓存未命中	
2. 查询数据库 此时 DB 还是 18，R 读到旧值 18	
	3. 更新数据库为 19
	4. 删除 Redis 此时 Redis 本来就不存在
5. R 的 DB 查询返回较慢	

| 在 W 删除缓存之后，才把 18 写入 Redis |
▼ ▼

最终：

数据库： 19
缓存： 18

问题关键是：旧值不是写请求删除缓存前就已经存在于 Redis，而是读请求先从 DB 读到了旧值，随后在写请求删除缓存之后，才把旧值回填进 Redis。

八、延迟二次删除怎么理解？

延迟二次删除不是独立的一致性方案，而是 Cache Aside 的并发补偿手段。更推荐的工程流程是：

```
更新数据库
|
▼
立即删除缓存
|
▼
发送延迟消息
|
▼
等待一段时间
|
▼
再次删除缓存
```

第一次删除解决正常缓存失效；第二次删除负责清理并发竞态中被旧读请求重新回填的旧缓存。

不建议在主请求里直接 `sleep`：

```
updateDB()
deleteCache()
time.Sleep(500 * time.Millisecond)
deleteCache()
```

原因：

- 增加接口响应时间。
- 占用 goroutine 和连接资源。
- sleep 时间难以准确设置。
- 服务重启后第二次删除可能丢失。
- 失败后没有可靠重试。

更推荐使用延迟 MQ、任务表、Redis Stream 或 CDC 消费链路做第二次删除。

延迟多久？

不能机械背 500ms 或 1s，应该覆盖可能产生旧回填的最长链路：

数据库查询 P99
+ 缓存回填耗时
+ 网络抖动
+ 读从库复制延迟
+ 安全余量

如果系统读从库，还要考虑主从延迟导致“缓存未命中后读到从库旧值再回填”。这时可以结合写后短时间读主库、读写粘滞、版本号校验和复制延迟监控。

如果算出来要延迟 5 到 12 秒，延迟双删还有意义吗？

有意义，但它的定位会变窄：它不再是一个优雅的主一致性方案，而更像兜底补偿手段。

它的价值是把错误缓存的存活时间从 TTL 级别缩短到秒级补偿窗口：

没有延迟二次删除：
旧缓存可能一直存在到 TTL 到期，例如 10 分钟、30 分钟

有延迟二次删除：
旧缓存大概率在 5 到 12 秒后被清掉

如果业务能接受几秒级旧数据，Cache Aside + 延迟二次删除 + TTL 仍然是性价比较高的方案。

如果业务不能接受 5 到 12 秒旧数据，就不能主要依赖延迟双删，而要升级一致性方案：

- 写后立刻读场景：写后短时间读主库、读写粘滞或绕过缓存。
- 高并发热点数据：缓存 value 带版本号，配合 Lua 防止旧版本覆盖新版本。
- 多级缓存场景：MQ/CDC 广播失效本地缓存，并设置较短 L1 TTL。
- 更高一致性场景：用数据库事务、条件更新、状态机、幂等号兜住最终正确性，缓存只做展示或加速。

面试表达可以这样说：

如果延迟时间计算出来要 5 到 12 秒，我会认为延迟双删仍然有兜底价值，因为它能把旧缓存从 TTL 级别缩短到秒级清理。但这也说明读链路 P99、网络抖动或主从复制延迟偏长，不能只依赖延迟双删。如果业务对写后读一致性敏感，我会配合写后读主、版本号校验、Lua 防旧值覆盖或 CDC 失效；强一致场景则不把缓存作为最终判断依据。

延迟双删能保证强一致吗？

不能。第二次删除也可能失败，延迟消息可能丢失，删除后仍可能有更晚的旧请求回填，数据库副本延迟也可能超过预估。它只能降低旧值回填概率、缩短不一致窗口，不能提供严格强一致。

九、删除缓存失败怎么办？

删除失败是 Cache Aside 最关键的异常场景：

数据库：新值
缓存：旧值

不能只打印日志或只等 TTL，应该把删除缓存做成可靠、可重试的动作。

方案 1：同步有限重试

- 删除失败后指数退避重试 2 到 3 次。
- 仍失败则进入异步补偿。
- 不能无限同步重试，否则拖垮写接口。

方案 2：MQ 异步重试

- 数据库更新成功。
- 删除缓存失败或统一发送缓存失效消息。
- 消费者删除缓存，失败则重试或进入死信队列。
- 消息包含缓存 key、业务类型、数据 ID、版本号、事件 ID、重试次数。

方案 3：事务 Outbox

解决“数据库提交成功，但应用在发送 MQ 前崩溃”的问题：

```
BEGIN;

UPDATE user
SET name = 'new_name',
    version = version + 1
WHERE id = 1001;

INSERT INTO outbox_event(
    event_type,
    aggregate_id,
    payload,
    status
) VALUES (
    'USER_CACHE_INVALIDATE',
    1001,
    '{"key": "user:1001"}',
    'PENDING'
);

COMMIT;
```

业务数据和失效事件在同一个数据库事务中提交，后台 Publisher 再投递 MQ。

方案 4：binlog / Change Stream / CDC

- MySQL 可订阅 binlog。
- MongoDB 可通过 Change Stream 感知变更。
- CDC 消费程序统一删除 Redis 或通知本地缓存失效。
- 优点是业务代码解耦；缺点是链路更长、存在传播延迟、要处理重复事件和消费积压。

方案 5: TTL 最终兜底

所有缓存都应设置合理 TTL。但 TTL 只能做最后兜底，不能作为唯一一致性方案。否则 TTL 是 30 分钟，就可能出现长达 30 分钟的旧数据窗口。

十、多级缓存如何保证一致性？

典型多级缓存：

```
请求 -> L1 本地缓存 -> L2 Redis -> DB
```

读路径：

```
读 L1
├── 命中：返回
└── 未命中：读 Redis
    ├── 命中：写 L1, 返回
    └── 未命中：读 DB, 写 Redis, 写 L1, 返回
```

写路径：

```
更新数据库并提交
┆
┆
┆
▼
删除 Redis
┆
┆
┆
▼
发布缓存失效事件
├── 实例 A 删除 L1
├── 实例 B 删除 L1
└── 实例 C 删除 L1
```

本地缓存分散在多个应用进程中，不能像 Redis 一样删除一个共享 key 就结束。需要通过 MQ、Redis Pub/Sub、Redis Stream、配置中心事件或 CDC 通知所有实例失效。

本地缓存实践建议

- 容量小。
- TTL 短。
- 只缓存超热点数据。
- 主动失效 + TTL 兜底。
- 关键请求可绕过 L1。

Redis Pub/Sub 可以做本地缓存失效，但通常不保证离线消息和可靠重试；重要业务更适合可靠 MQ、Redis Stream 或 CDC。

十一、高并发下还要解决缓存重建

删除缓存后，大量请求可能同时未命中并打到 DB，这就是缓存击穿。一致性方案如果不兼顾重建性能，可能把数据库打垮。

单机 singleflight

- 同一应用实例内，相同 key 只允许一个请求查 DB 并重建缓存。
- 其他请求等待结果。

分布式互斥重建

- 缓存未命中后尝试获取 Redis 锁。
- 获取成功者查 DB、写缓存、释放锁。
- 获取失败者短暂等待后重试缓存。
- 锁必须有过期时间，释放锁要校验唯一 token。

逻辑过期

超热点 key 可以不让物理 key 立即过期，而是在 value 中保存逻辑过期时间：

```
{
  "expireAt": 1720000000,
  "data": {
    "id": 1001,
    "name": "Tom"
  }
}
```

数据已逻辑过期时，先返回旧值，再后台异步重建。优点是接口延迟稳定、保护 DB；代价是允许短时间返回旧数据。

十二、版本号如何降低旧值覆盖？

对于一致性要求更高的缓存，可以给数据增加版本：

```
value = { version: 105, data: ... }
```

数据库每次更新：

```
UPDATE user
SET name = ?,
    version = version + 1
WHERE id = ?;
```

缓存写入时只允许新版本覆盖旧版本：

```
准备写入 version=104
Redis 已存在 version=105
拒绝旧版本写入
```

这个判断通常需要 Lua 脚本在 Redis 内原子完成。版本号比单纯延迟双删更严谨，但实现和维护成本更高。

十三、不同业务怎么选？

普通商品详情、用户资料

- Cache Aside。
- 先更新数据库，再删除 Redis。
- 删除失败重试。
- 设置 TTL。

高并发热点数据

- 延迟二次删除。
- singleflight 或分布式锁重建。
- TTL 随机抖动。
- 版本号。
- MQ / CDC 补偿。

本地缓存 + Redis + DB

- 数据库为事实源。
- Redis 使用 Cache Aside。
- MQ 广播 L1 缓存失效。
- 本地缓存短 TTL。
- 失效消息幂等处理。

余额、支付、库存最终结果

- 不能单纯依赖缓存一致性方案。
- 使用数据库事务、乐观锁/悲观锁、条件更新、幂等、账本和状态机。
- 缓存只做查询加速。

面试 1 分钟回答：

缓存和数据库在常规高并发架构下很难低成本做到绝对强一致，通常采用 Cache Aside 实现最终一致，并把数据库作为唯一可信数据源。读请求先查缓存，未命中查数据库并回填；写请求通常先更新数据库，事务提交成功后删除缓存，而不是直接更新缓存。删除失败要通过同步重试、MQ、事务 Outbox、binlog 或 Mongo Change Stream 做异步补偿，TTL 只作为最后兜底。即使先更新数据库再删除缓存，也可能出现并发旧读回填旧缓存，可以通过异步延迟二次删除、版本号校验降低风险。如果还有本地缓存，需要通过 MQ 或广播通知所有应用实例失效，并给本地缓存设置较短 TTL。高并发缓存失效时，还要用 singleflight、分布式锁或逻辑过期防止大量请求同时打到数据库。

面试追问：缓存和数据库能做到绝对一致吗？

普通 Redis + 数据库双写很难低成本做到绝对强一致，因为二者不在同一个本地事务中。工程上通常选择数据库作为事实源，通过缓存失效、重试、MQ、CDC 和 TTL 实现最终一致。真正要求强一致的余额、支付、库存等数据，应以数据库事务、锁和条件更新为准，缓存只做查询加速。

面试追问：只设置 TTL 可以吗？

不建议。TTL 只能保证最终恢复，不能限制正常情况下的不一致窗口。如果 TTL 是 30 分钟，删除失败就可能持续返回 30 分钟旧值。正确方式是主动删除、失败重试、异步补偿，TTL 只做最后兜底。

深入：缓存穿透、击穿、雪崩分别是什么？怎么解决？

结论先行：

穿透是查不存在数据，击穿是热点 key 过期，雪崩是大量 key 同时失效或 Redis 故障。三者本质都是请求绕过缓存打到后端，需要分别用空值缓存、互斥重建、过期随机化和降级限流解决。

分层展开：

缓存穿透

- 现象：请求的数据缓存没有，数据库也没有。
- 风险：恶意或异常请求持续打 DB。
- 方案：缓存空值、Bloom Filter、参数校验、黑名单限流。

缓存击穿

- 现象：一个热点 key 过期，大量请求同时打 DB。
- 风险：瞬时 DB 压力暴涨。
- 方案：互斥锁重建、逻辑过期、热点 key 不过期、后台刷新。

缓存雪崩

- 现象：大量 key 同时失效，或者 Redis 集群不可用。
- 风险：后端服务和数据库被流量压垮。
- 方案：TTL 加随机值、多级缓存、限流降级、熔断、Redis 高可用。

面试追问：热点 key 你会用互斥锁还是逻辑过期？

强一致要求更高、可接受少量等待时用互斥锁；更关注可用性和低延迟时用逻辑过期，先返回旧值，后台异步刷新。

深入：如何设计 Redis 缓存预热？

结论先行：

缓存预热是把预计会被高频访问的数据提前加载到 Redis，核心是识别热点、控制节奏、校验结果和提供回滚。

分层展开：

热点识别

- 历史访问 TopN。
- 大促活动商品。
- 推荐位、首页、榜单。

加载方式

- 发布前批量加载。
- 定时任务刷新。
- MQ 事件触发。
- 灰度预热。

节奏控制

- 分批写入，避免 Redis 和 DB 被打满。
- 控制并发和 QPS。
- 设置合理 TTL 和随机过期。

验证和回滚

- 校验 key 数量、命中率、数据版本。
- 预热失败时保留旧缓存或降级。

面试追问：预热会不会造成缓存污染？

会。如果预热了大量实际不访问的数据，会浪费内存并挤掉真正热点。要根据访问数据动态调整预热范围。

深入：Redis 和本地缓存怎么配合？

结论先行：

本地缓存解决极致低延迟和热 key 压力，Redis 解决分布式共享缓存，两者结合是多级缓存，但要额外处理本地缓存一致性。

分层展开：

架构

- 请求先查本地缓存。
- 未命中查 Redis。
- Redis 未命中查 DB。

优点

- 降低 Redis QPS。
- 降低网络开销。
- 提升热点数据访问性能。

一致性问题

- 每个应用实例都有本地副本。
- 数据变更时需要广播失效或设置短 TTL。
- 本地缓存不适合强一致数据。

典型场景

- 配置项。
- 商品详情中的低频变化字段。
- 热点榜单。
- 字典数据。

面试追问：本地缓存怎么失效？

常见做法是短 TTL、MQ 广播、配置中心推送、Redis Pub/Sub 或版本号校验。核心是接受短暂不一致，并把窗口控制在业务可接受范围内。

深入：缓存数据要不要存 null？

结论先行：

可以缓存 null，但要设置较短 TTL，用于防止缓存穿透；不能长期缓存 null，否则会掩盖后续新增数据。

分层展开：

适用场景

- 查询不存在的用户、商品、订单。
- 防止相同不存在 key 反复打 DB。

注意事项

- TTL 要短，例如几十秒到几分钟。
- 数据新增时要删除 null 缓存。
- 区分“空值”和“系统异常”。

替代方案

- Bloom Filter。
- 参数校验。
- 黑名单和风控。

面试追问：数据库异常时能不能缓存 null？

不能。数据库异常和数据不存在是两回事。异常时缓存 null 会把临时故障变成错误缓存，导致恢复后仍返回不存在。

深入：Redis 适合做数据库吗？

结论先行：

Redis 可以存数据，但不适合替代通用关系型数据库。它更适合缓存、计数、排行榜、会话、队列、限流等高性能内存场景。

分层展开：

优势

- 高性能。
- 丰富数据结构。
- 原子操作。
- TTL 和过期机制。

短板

- 内存成本高。
- 查询能力弱。
- 事务能力有限。
- 持久化和一致性不如数据库。

适合场景

- 可重建缓存。
- 中间状态。
- 低延迟访问。
- 允许最终一致的数据。

不适合场景

- 强事务。
- 复杂查询。
- 大规模低价值冷数据。
- 需要严格审计的数据。

面试追问：Redis 持久化开了就能当数据库吗？

不能。持久化只解决部分恢复问题，不等于完整事务、强一致、复杂查询和数据治理能力。

六、原子性、Lua、事务、锁与限流

24. Redis 的原子性边界是什么？

单条命令和单个 Lua 脚本在服务端执行期间通常不会被其他命令穿插，但这不等于跨 Redis、数据库和外部接口的业务事务。

复杂业务应把 Redis 原子操作视为并发控制工具，最终正确性由业务幂等、数据库约束、状态机和补偿共同保证。

25. 分布式锁如何正确实现？

```
加锁: SET lock:key unique-token NX PX ttl  
解锁: Lua 判断 value == unique-token 后 DEL  
续期: 确认仍是自己的 token 后延长 TTL
```

必须处理：

- TTL 太短导致业务未完成锁已失效。
- 进程暂停或网络分区后旧持有者继续操作。
- 主从异步复制和故障转移导致锁状态丢失。
- 重入、续期、最大持锁时间和异常退出。

关键写可增加 Fencing Token，让下游只接受更大令牌，从资源侧拒绝过期持有者。锁不能替代业务幂等。

26. Redlock 是否能解决所有锁问题？

不能。它试图通过多个独立节点降低单点状态丢失，但仍涉及时钟、网络延迟、进程暂停和下游资源是否识别旧持有者等假设。

选择前先问：是否真的需要分布式互斥；数据库唯一约束、条件更新、租约服务或幂等状态机是否更直接。正确性极高时要引入 Fencing Token 和权威资源校验。

27. Lua 脚本有哪些优势和风险？

优势是减少 RTT，并把多步读改写放在服务端原子执行；适合锁释放、滑动窗口、库存预占等短逻辑。

风险是长脚本阻塞主线程、脚本复杂难治理、Cluster 多 Key 必须同 Slot。脚本应短小、有最大输入规模、可观测，并在上线前压测最坏数据量。

28. 如何实现限流？

算法	优点	缺点
固定窗口计数	简单、成本低	边界突刺
滑动窗口 ZSet + Lua	精度高、可审计时间点	内存和 $O(\log N)$ 成本
令牌桶	允许受控突发	状态计算较复杂
漏桶	输出平滑	突发请求可能排队/丢弃

限流 Key 应按租户、账号、接口分层；Redis 异常时明确 Fail-open 还是 Fail-closed，并监控拒绝率和剩余额度。

29. Redis 延迟任务如何设计？

用 ZSet Score 保存执行时间，多个消费者通过 Lua 原子认领到期任务，再进入处理中状态；失败要重试、死信、超时回收和幂等。

边界： ZSet 只解决时间排序，不自动提供可靠 ACK、任务所有权和灾难恢复。核心大规模调度优先专用系统。

深入：Redis 分布式锁如何设计？生产中会遇到哪些问题？

结论先行：

Redis 分布式锁本质上是一个“带 TTL 的互斥租约”，适合跨服务实例做短时间互斥，例如缓存重建、定时任务抢占、防重复提交和非核心资源串行化。生产级 Redis 锁不能只会 `SET NX EX`，还要处理唯一 owner token、安全释放、TTL、续期、抢锁失败退避、故障切换、旧持锁者继续写、fencing token、业务幂等和监控告警。

Redis 分布式锁生产实践

- 锁的本质：所有权、互斥、访问顺序
- 基础实现：SET NX PX + 唯一 owner token
- 安全释放：Lua 校验 token 后删除，Redis 8.4+ 可用 DELEX IFEQ
- TTL 与续期：覆盖 P99，但必须有最大持锁时间
- 抢锁失败：退避、抖动、最大等待时间、context 取消
- 生产问题：锁过期、主从切换、旧持锁者、热点锁、多锁死锁
- fencing token：防止旧持锁者恢复后继续写
- Redlock 评价：提升容错，但不是强一致银弹
- 场景选型：什么时候用 Redis 锁，什么时候不用

一、锁的横向视角摘要

锁不是业务正确性的魔法开关。锁本质上是所有权与访问顺序协议，解决“谁能进入临界区”，但不能单独解决重复执行、锁过期后旧客户端继续写、消息重复、网络超时或数据库部分提交。

生产级正确性通常是：

锁 + 数据库原子操作 + 幂等 + 唯一约束 + 版本号 + fencing token + 补偿

横向对比可以记住：

- Go 锁解决单进程 goroutine 共享内存。
- MySQL/MongoDB 更适合用事务、条件更新、唯一约束、状态机保护最终数据。
- Redis 锁适合跨实例短时互斥。
- Kafka 更像按 key 分区串行化处理，不是通用锁服务。
- ES 用 `seq_no + primary_term` 做乐观并发，不适合当强一致事实源。
- etcd/ZooKeeper 更适合强协调、选主和严格分布式锁。

完整“锁机制与并发控制体系”已沉淀到 [04-02 | 分布式与高并发架构.md](#) 的专题模块；本题只展开 Redis 锁生产实践。

二、Redis 锁基础实现

加锁：

```
SET lock:order:1001 randomToken NX PX 10000
```

含义：

- `NX`：锁不存在时才能成功。
- `PX/EX`：设置过期时间，防止客户端宕机后死锁。
- `randomToken`：本次锁请求的唯一所有者标识。

Go 中不建议使用“线程 ID”。Go 没有稳定暴露给业务代码的 goroutine ID。实际可以使用：

```
128 位以上安全随机数
或 UUID
+ 可选 instance_id / request_id, 便于排查
```

真正参与所有权判断的是随机 owner token，不是机器名、线程 ID 或 goroutine ID。

三、加锁结果必须区分三种情况

```
Redis 返回成功: 可以进入临界区
Redis 返回锁已存在: 没有获得锁
Redis 请求超时: 结果未知
```

请求超时，不能默认认为没加上，也不能默认认为加上了。更稳妥的策略是：

```
加锁结果不确定
├── 不进入临界区
└── 使用相同 token 尝试安全清理或等待 TTL 过期
```

四、安全释放锁

不能直接：

```
DEL lock:order:1001
```

因为可能出现：

```
A 获得锁
A 执行时间过长，锁过期
B 获得同一个锁
A 执行结束
A 直接 DEL，把 B 的锁删除
```

正确方案是比较 token 后删除。

Redis 8.4+ 可以使用：

```
DELEX lock:order:1001 IFEQ randomToken
```

多数生产环境仍可能使用 Redis 6/7/8.0，通用做法是 Lua：

```
if redis.call("GET", KEYS[1]) == ARGV[1] then
    return redis.call("DEL", KEYS[1])
end
return 0
```

五、续期怎么做？

续期也不能直接：

```
PEXPIRE lockKey 10000
```

否则可能给别人的锁续期。必须原子校验 owner token：

```
if redis.call("GET", KEYS[1]) == ARGV[1] then
    return redis.call("PEXPIRE", KEYS[1], ARGV[2])
end
return 0
```

生产中常见模式：

```
TTL = 9 秒
每 3 秒尝试续期
最大持锁时间 = 业务硬上限
```

这只是示例，核心原则是：

- 续期间隔明显小于 TTL，例如 TTL 的 1/3。
- 续期失败时尽快停止业务或进入补偿逻辑。
- 限制最大续期时长，不能无限续期。
- 业务结束立即取消续期 goroutine。
- 续期必须校验 owner token。

看门狗只能降低锁过期概率，不能保证绝对安全。进程长时间 STW、机器卡顿、网络分区时，续期线程也可能无法正常执行。

六、锁的 TTL 怎么设置？

不要简单背“统一 30 秒”。应该根据真实链路：

```
TTL > 业务执行 P99
+ 最慢下游调用超时
+ 允许的重试时间
+ 调度 / GC / 网络抖动余量
```

例如：

```
正常 P99 : 800ms
数据库超时: 1s
没有内部重试
抖动预留: 1s
```

初始 TTL 可以考虑 3 到 5 秒，而不是机械设置 30 秒。最终必须根据监控调整。

如果任务耗时无法明确上界，例如第三方生成大文件、长时间 AI 任务、批量数据迁移、人工审批，就不应该只依赖固定 TTL，而应改成：

任务状态机 + worker owner + heartbeat + lease + fencing token + 可恢复执行

七、抢锁失败怎么处理？

不能无脑死循环：

```
for {
    tryLock()
}
```

这会造成 Redis 热点和重试风暴。应该使用：

最大等待时间 + 指数退避 + 随机抖动 jitter + context 取消

例如：

第一次等待 20 到 40ms
第二次等待 40 到 80ms
第三次等待 80 到 160ms
超过总等待时间则失败

如果业务不是必须等待锁，建议快速失败或降级，避免所有请求一起阻塞。

八、Redis 锁必须面对的生产问题

问题 1：锁过期，但业务没有完成

A 获得锁
A 执行慢 SQL / RPC 卡顿 / 进程暂停
锁过期
B 获得锁
A 和 B 同时执行

解决不是单选，而是组合：

合理 TTL + 有上限自动续期 + 业务幂等 + 数据库条件更新 + fencing token

问题 2：Redis 主从故障切换导致锁丢失

Redis 主从复制是异步的，可能出现：

A 在主节点加锁成功
锁还没有复制到从节点
主节点宕机

```
从节点提升为主  
B 再次获得同一个锁
```

因此单 Redis 锁更适合：

- 重复执行只产生额外成本。
- 业务有幂等兜底。
- 极少数故障下允许重复。
- 锁用于缓存重建、防抖、非核心任务。

问题 3：旧持锁者恢复后继续写

```
A token=100 获得锁  
A 暂停  
锁过期  
B token=101 获得锁并完成写入  
A 恢复，使用旧结果覆盖 B
```

随机 UUID 只能防止 A 删除 B 的锁，不能防止 A 继续写数据库、文件或外部系统。这时需要 fencing token。

问题 4：锁热点

```
lock:global  
lock:all_orders  
lock:all_products
```

所有请求竞争同一把锁，系统最终退化成单线程。应尽量按业务资源拆分：

```
lock:order:{orderId}  
lock:product:{productId}  
lock:user:{userId}
```

但锁粒度过细会增加多锁管理和死锁风险，所以要围绕业务不变量设计。

问题 5：多把锁死锁

```
请求 A：先锁用户，再锁订单  
请求 B：先锁订单，再锁用户
```

实践建议：

- 尽量避免嵌套分布式锁。
- 多资源锁按固定顺序获取。
- 每次获取都有超时。
- 获取部分失败后立即释放已获得锁。

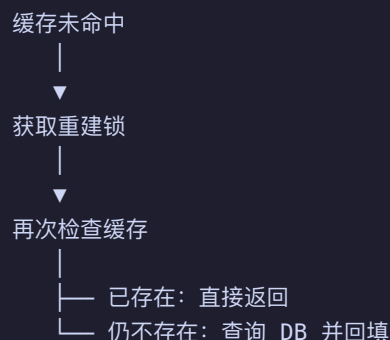
- 优先重新设计数据模型，使用单一聚合根或数据库事务。

问题 6：Redis 锁不公平

SET NX 不保证先到先得。高竞争时可能有请求长期抢不到锁。对公平性要求高时，可考虑队列串行化、ZooKeeper 顺序节点、Redis Stream 排队或 Kafka partition 排队，但公平通常会增加复杂度和延迟。

问题 7：拿锁后不做二次检查

缓存重建场景中，获得锁后必须再次检查缓存：



否则可能出现 A 重建完缓存释放锁，B 随后获得锁，又重复查 DB 重建一次。

九、什么是 fencing token?

fencing token 是每次成功获得锁时生成的单调递增序号：

```
第一次持锁：fence = 100  
第二次持锁：fence = 101  
第三次持锁：fence = 102
```

下游资源记录已经接受的最大 token：

```
当前最大 fence = 101  
收到 fence=100：拒绝  
收到 fence=102：接受
```

数据库可以这样防止旧持锁者写入：

```
UPDATE resource  
SET value = ?,  
    fence_token = ?  
WHERE id = ?  
    AND fence_token < ?;
```

要区分两个 token：

```
owner token : UUID, 用于判断“锁是不是我的”  
fencing token : 单调递增数字, 用于判断“我是不是最新持锁者”
```

如果业务真的依赖 fencing token 保证严格正确性, 递增 token 最好来自能提供可靠顺序的数据库或共识系统, 而不是普通 Redis 主从实例。

十、Redlock 怎么评价?

Redlock 使用多个相互独立的 Redis master, 例如 5 个节点。客户端尝试加锁:

```
成功节点数 >= 3  
并且加锁耗时 < TTL
```

才认为锁成功。失败则清理所有可能已经获得的锁。

我的工程评价是: 不建议把 Redlock 作为普通业务默认方案。

原因:

- 需要维护多个相互独立的 Redis master。
- 加锁链路延迟更高。
- 故障恢复、持久化和时间漂移模型复杂。
- 最终仍然建议加入 fencing token。
- 如果业务真的要求严格一致, etcd、ZooKeeper 或数据库事务通常更直接。
- 如果业务允许极低概率重复, 单 Redis 锁 + 幂等通常已经足够。

选型建议:

缓存重建、接口防抖:

单 Redis 锁

普通定时任务, 允许偶发重复:

Redis 锁 + 幂等

严格单 Leader、集群协调:

etcd / ZooKeeper

库存、余额、订单状态:

数据库事务 / 条件更新 / 唯一约束

跨系统长任务:

租约 + fencing token + 幂等状态机

十一、生产监控要看什么?

- 锁获取成功率。
- 锁获取失败率。
- 获取锁等待时间。

- 锁持有时间 / TTL 比例。
- 续期成功和失败次数。
- 解锁 token 不匹配次数。
- 锁过期后业务仍在运行次数。
- 热点锁 TopN。
- 单 key 抢锁 QPS。

这些指标能帮助判断：锁粒度是否过大、TTL 是否过短、是否出现旧持锁者、是否存在重试风暴。

十二、生产中怎么选，而不是哪里都加 Redis 锁？

缓存重建

推荐：

Redis 锁 + 获得锁后二次检查缓存 + 较短 TTL + singleflight + DB 限流

重复重建通常只增加 DB 压力，不破坏核心数据，因此 Redis 锁很合适。

支付回调

不推荐 Redis 锁作为核心保障。推荐：

业务流水号唯一索引 + 状态条件更新 + 事务 + 幂等返回

Redis 锁最多用于减轻瞬时重复请求，不能成为最终防线。

库存扣减

推荐：

```
UPDATE product
SET stock = stock - 1
WHERE id = ?
  AND stock > 0;
```

或使用事务行锁。不要因为拿到 Redis 锁，就认为库存一定不会超卖。

同一订单事件按顺序处理

推荐：

Kafka 按 orderId 分区 + 单分区串行消费 + 数据库状态机 + 幂等消费

通常比每条消息都抢 Redis 锁更自然。

ES 文档并发更新

推荐：

读取 seq_no + primary_term
条件更新
冲突后重新读取并合并

而不是围绕 ES 文档再设计 Redis 锁。

集群只允许一个调度器执行

允许偶发重复: Redis lease + 任务幂等
不允许双 Leader: etcd / ZooKeeper leader election + fencing token
任务结果存 MySQL: 也可以通过数据库抢占任务记录

面试 1 分钟回答:

Redis 分布式锁本质是一个带 TTL 的互斥租约, 基础实现是 `SET key token NX PX ttl`, token 必须唯一, 释放和续期都要校验 token, 老版本用 Lua, Redis 8.4+ 可以用 `DELEX key IFEQ token` 做安全删除。TTL 要覆盖业务 P99 和下游抖动, 但不能无限长; 复杂任务要有续期、最大持锁时间和状态机。生产里还要处理加锁超时结果未知、抢锁失败退避、主从切换锁丢失、锁过期后旧持锁者继续写、热点锁和多锁死锁。Redis 锁适合缓存重建、任务抢占、防重复提交这类短时互斥, 不能替代数据库事务、幂等和条件更新。强一致场景要用数据落点的原子规则兜底, 必要时用 etcd/ZooKeeper 或 fencing token。

面试追问: Redis 锁一定安全吗?

不绝对安全。单 Redis 锁在主从异步复制、故障切换、GC 停顿、网络分区、锁过期等情况下都可能失效。生产上不能只靠锁保证正确性, 要结合业务幂等、数据库条件更新、版本号、fencing token 和补偿机制。

面试追问: 锁过期业务还没执行完怎么办?

会出现并发执行风险。解决思路是合理 TTL、有上限续期、任务拆分、状态机、业务幂等、数据库条件更新和 fencing token。不能只靠看门狗兜住所有一致性问题。

深入: Redis 事务和数据库事务有什么区别?

结论先行:

Redis 事务通过 `MULTI/EXEC` 保证一组命令在服务端按顺序连续执行, 但它不是 MySQL/MongoDB 那种完整数据库事务, 不支持运行时错误自动回滚, 也不负责跨系统一致性。面试里要把 Redis、MySQL、MongoDB 的事务目标区分清楚: Redis 解决“多条 Redis 命令连续执行”, MySQL 解决“关系型数据强一致变更”, MongoDB 解决“文档模型下少量跨文档/跨集合一致变更”。

分层展开:

1. Redis 事务解决什么问题

- 把多条 Redis 命令放入队列, 最后由 `EXEC` 一次性按顺序执行。
- 执行期间不会被其他客户端命令插入, 具备“连续执行”的隔离效果。
- 配合 `WATCH` 可以做乐观锁: 如果被监控 key 在 `EXEC` 前被别人改过, 本次事务失败。
- 适合简单的缓存状态更新、计数、队列元数据变更等 Redis 内部原子组合。

2. Redis 事务如何使用

```
WATCH stock:sku:1001
GET stock:sku:1001
MULTI
DECR stock:sku:1001
LPUSH order:queue order_123
EXEC
```

- **WATCH**：监控 key，提供乐观锁语义。
- **MULTI** 开启事务。
- 普通命令进入队列，不会立即执行。
- **EXEC** 提交队列。
- **DISCARD** 放弃事务队列。
- **UNWATCH** 取消监控。

3. Redis 事务的关键限制

- 命令运行时报错，不会回滚前面已经执行的命令。
- 命令入队时的语法错误会导致事务整体不执行。
- 不支持复杂条件分支，复杂原子逻辑通常更适合 Lua。
- 不保证跨 Redis、DB、MQ 的一致性。
- 不等于关系型数据库的 ACID 事务。

4. Redis、MySQL、MongoDB 事务对比

组件	事务设计目标	原子性范围	隔离/并发控制	回滚能力	典型使用场景
Redis	多条 Redis 命令连续执行，减少并发插入	单实例内一组命令	单线程顺序执行 + WATCH 乐观锁	不支持运行时错误自动回滚	简单计数、队列元数据、库存预扣、缓存状态切换
MySQL/InnoDB	关系型数据强一致变更	一个本地事务内多行多表	MVCC、行锁、next-key lock、隔离级别	支持 rollback，依赖 undo log	订单、库存、账务、权限、工单状态
MongoDB	文档模型下补充跨文档一致性	单文档天然原子，多文档事务可跨集合	WiredTiger MVCC、snapshot、write conflict retry	支持事务 abort	配置发布、任务状态、少量跨集合一致变更

5. 和 MySQL 事务的核心差异

- MySQL 事务追求完整 ACID，Redis 事务更像“命令队列 + 连续执行”。
- MySQL 有 undo log 支持回滚，Redis 执行中某条命令失败不会自动撤销前面命令。

- MySQL 事务可以通过隔离级别控制并发读写现象，Redis 事务主要依赖单线程执行和 `WATCH` 乐观锁。
- MySQL 是事实数据源时，余额、订单、库存最终正确性应落在 MySQL 事务、唯一约束、条件更新和状态机上。

6. 和 MongoDB 事务的核心差异

- MongoDB 单文档更新天然原子，很多场景通过嵌入式建模避免事务。
- MongoDB 多文档事务适合少量跨文档/跨集合一致变更，但成本比单文档写高。
- Redis 事务通常只管理缓存/内存状态，MongoDB 事务管理持久化文档数据。
- MongoDB 事务仍要处理 `TransientTransactionError`、`UnknownTransactionCommitResult` 等重试场景。

使用建议

- Redis 内部简单组合：可以用 `MULTI/EXEC`。
- 需要判断、循环、校验后删除：优先 Lua，例如分布式锁释放、滑动窗口限流。
- 需要最终数据正确：优先 MySQL/MongoDB 的条件更新、事务、唯一约束和状态机。
- 跨系统一致性不要依赖 Redis 事务。
- 写 DB + 删缓存：不能用 Redis 事务解决，应该用 Cache Aside、重试、MQ、Outbox、binlog/Change Stream 补偿。

工程面试表达：

Redis 事务不是为了替代数据库事务，而是为了让 Redis 内部多条命令连续执行。真正的数据一致性要看数据落点：关系型强一致用 MySQL 事务，文档型聚合根优先用 Mongo 单文档原子更新，少量跨文档再用 Mongo 事务；缓存只做加速，不能当事实源。

面试追问：为什么 Redis 事务不支持回滚？

Redis 命令执行失败通常是编程错误或类型错误，官方设计上更强调简单和性能。如果需要复杂事务语义，应该由业务或数据库保证。

面试追问：Redis 事务、Lua、Pipeline 怎么选？

- 只想减少网络往返：Pipeline。
- 要多条命令连续执行，但逻辑简单：`MULTI/EXEC`。
- 要校验 token、判断库存、滑动窗口等复杂原子逻辑：Lua。
- 要保证订单、库存、资金等最终正确：数据库事务/条件更新，而不是 Redis 事务。

深入：Lua 脚本有什么优势和风险？

结论先行：

Lua 脚本的优势是把多步操作放到 Redis 内原子执行，减少网络往返；风险是脚本执行时间过长会阻塞主线程。

分层展开：

优势

- 原子执行。

- 减少多次网络请求。
- 适合限流、锁释放、库存扣减、状态机转换。

风险

- 长脚本阻塞 Redis。
- 访问大 key 或循环过多会造成延迟。
- 脚本调试和版本管理更复杂。

工程规范

- 控制脚本复杂度。
- 入参显式传入，不在脚本中扫大量 key。
- 使用 SHA 缓存脚本。
- 设置脚本超时治理预案。

面试追问：Lua 脚本执行中能被打断吗？

Redis 保证脚本原子执行。长时间脚本会阻塞其他命令，必要时可通过 `SCRIPT KILL` 处理未写入脚本；如果脚本已执行写操作，处理会更复杂。

深入：如何用 Redis 做限流？

结论先行：

Redis 可以实现固定窗口、滑动窗口、令牌桶、漏桶和并发数限制。简单低成本场景用 `INCR + EXPIRE`，精确窗口用 `ZSet + Lua`，允许突发用令牌桶，要求匀速排队用漏桶。工程上要先确定限流维度、时间窗口、失败策略和兜底方案，不是所有限流都必须用 Redis；单机限流、网关限流、Nginx/Envoy、Sentinel、令牌桶库、MQ 削峰都可以做限流。

分层展开：

1. 限流先明确四个问题

- 限谁：用户、IP、账号、租户、广告账户、接口、平台、全局。
- 限什么：QPS、并发数、每日总量、外部 API 配额、任务消费速度。
- 超限怎么办：直接拒绝、排队等待、降级返回、异步重试、进入死信。
- 放在哪里：本机、网关、Redis 全局、服务端队列、MQ、第三方 SDK。

2. 固定窗口： `INCR + EXPIRE`

适合：简单接口限流、低成本粗粒度保护，例如每个用户每分钟最多 100 次。

```
INCR rate:user:1001:202607121430
EXPIRE rate:user:1001:202607121430 70
```

- key 按限流维度和窗口时间命名。
- 第一次 `INCR` 后设置过期时间。
- 查询计数是否超过阈值。
- 优点：实现简单，性能好，内存少。

- 缺点：窗口边界有突刺，比如 10:00:59 和 10:01:00 各打满一次，瞬时流量可能翻倍。

更严谨的固定窗口 Lua 思路：

```
local current = redis.call("INCR", KEYS[1])
if current == 1 then
    redis.call("EXPIRE", KEYS[1], ARGV[2])
end
if current > tonumber(ARGV[1]) then
    return 0
end
return 1
```

- **KEYS[1]**：限流 key。
- **ARGV[1]**：窗口内最大请求数。
- **ARGV[2]**：key 过期秒数。

3. 滑动窗口：ZSet + Lua

适合：精确控制最近 N 秒/N 分钟请求数，尤其适合外部媒体 API 配额。

- 用 ZSet 存请求时间戳。
- 删除窗口外数据。
- 统计窗口内请求数。
- 判断是否放行。

核心命令：

```
ZREMRANGEBYSCORE rate:facebook:act_1001:campaign_list:1s -inf now-1000
ZCARD rate:facebook:act_1001:campaign_list:1s
ZADD rate:facebook:act_1001:campaign_list:1s now requestId
EXPIRE rate:facebook:act_1001:campaign_list:1s 2
```

- **ZREMRANGEBYSCORE**：清理窗口外请求。
- **ZCARD**：统计当前窗口内请求量。
- **ZADD**：未超限才写入当前请求。
- **EXPIRE**：避免冷 key 常驻。
- 多条命令必须用 Lua 包起来，否则并发下可能多个请求同时看到未超限而全部放行。

单窗口 Lua 思路：

```
redis.call("ZREMRANGEBYSCORE", KEYS[1], "-inf", ARGV[1] - ARGV[2])
local count = redis.call("ZCARD", KEYS[1])
if count >= tonumber(ARGV[3]) then
    return 0
end
redis.call("ZADD", KEYS[1], ARGV[1], ARGV[4])
```



```
redis.call("PEXPIRE", KEYS[1], ARGV[2] + 1000)
return 1
```

- `ARGV[1]` : 当前毫秒时间戳。
- `ARGV[2]` : 窗口大小, 单位毫秒。
- `ARGV[3]` : 窗口最大请求数。
- `ARGV[4]` : 本次请求唯一 ID。

4. 多层滑动窗口: 外部 API 工程示例

场景: 调用 Facebook 获取 campaign 列表, 平台限制:

- 每秒最多 60 次。
- 每 5 分钟最多 3000 次。
- 每天最多 10000 次。

限流 key 设计:

```
rate:facebook:{account_id}:campaign_list:1s
rate:facebook:{account_id}:campaign_list:5m
rate:facebook:{account_id}:campaign_list:1d
```

工程执行逻辑:

- 请求进入时生成 `requestId`。
- 同一个 Lua 脚本检查三个窗口。
- 任一窗口超限, 返回拒绝和剩余等待时间。
- 三个窗口都通过, 才把本次请求写入三个 ZSet。
- 调用 Facebook 成功/失败都算一次消耗, 避免失败重试击穿平台配额。
- 平台返回限流错误时, 动态降低该账号/接口的阈值, 进入冷却窗口。

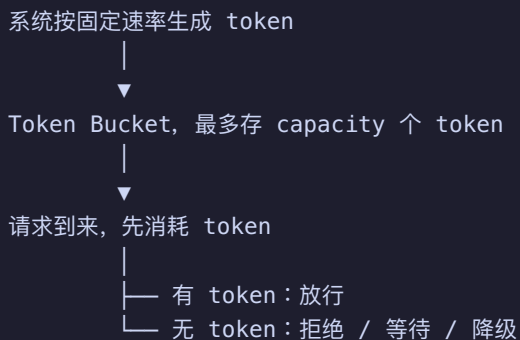
伪代码:

```
allowed, retryAfter, err := limiter.Allow(ctx, []Window{
    {Key: "rate:facebook:act_1001:campaign_list:1s", Limit: 60, Window: time.Second},
    {Key: "rate:facebook:act_1001:campaign_list:5m", Limit: 3000, Window: 5 *
time.Minute},
    {Key: "rate:facebook:act_1001:campaign_list:1d", Limit: 10000, Window: 24 *
time.Hour},
})
if err != nil {
    // Redis 异常时按业务重要性选择 fail open 或 fail closed
    return err
}
if !allowed {
    return ErrRateLimited{RetryAfter: retryAfter}
}
return facebookClient.ListCampaigns(ctx, accountID)
```

5. 令牌桶：允许突发，长期平均受控

适合：接口允许短时间突发，但长期 QPS 不能超过平均速率，例如每秒补充 100 个 token，桶容量 500。

设计思想：



核心参数：

- **rate**：每秒生成多少 token，决定长期平均速率。
- **capacity**：桶最多存多少 token，决定最大突发能力。
- **requested**：每次请求消耗多少 token，普通请求一般是 1。

例如 **rate=100/s**, **capacity=500**：

- 长期平均每秒最多 100 个请求。
- 低峰期可以积攒 token。
- 突发时最多可以瞬间放行约 500 个请求。
 - 按固定速率生成令牌。
 - 请求消耗令牌。
 - 允许一定突发流量。
 - 桶容量决定最大突发。
 - Redis 中通常存 **tokens** 和 **lastRefillTime**，用 Lua 原子计算补充和扣减。

适合场景：

- 用户接口 QPS 控制。
- 允许瞬时峰值的网关限流。
- 外部 API 在短期突发不敏感、只看平均速率时。

Go 本地令牌桶实现：

```
package main

import (
    "context"
    "fmt"
    "time"
```

```

    "golang.org/x/time/rate"
)

func main() {
    // 每秒生成 5 个 token, 桶容量为 10, 表示平均 QPS=5, 最大突发=10。
    limiter := rate.NewLimiter(rate.Limit(5), 10)

    for i := 0; i < 20; i++ {
        if limiter.Allow() {
            fmt.Println("allowed", i)
        } else {
            fmt.Println("limited", i)
        }
        time.Sleep(100 * time.Millisecond)
    }

    // Wait 会等待 token, 适合愿意排队的调用; ctx 控制最多等待多久。
    ctx, cancel := context.WithTimeout(context.Background(), 200*time.Millisecond)
    defer cancel()

    if err := limiter.Wait(ctx); err != nil {
        fmt.Println("wait token failed:", err)
        return
    }
    fmt.Println("allowed after wait")
}

```

Redis 全局令牌桶 Lua:

```

-- KEYS[1]: 令牌桶 key
-- ARGV[1]: 当前毫秒时间 now
-- ARGV[2]: 生成速率 rate, 每秒生成多少 token
-- ARGV[3]: 桶容量 capacity
-- ARGV[4]: 本次请求消耗 token 数, 一般是 1
-- ARGV[5]: key 过期时间, 秒

local bucket = redis.call("HMGET", KEYS[1], "tokens", "last_refill")

local tokens = tonumber(bucket[1])
local last_refill = tonumber(bucket[2])

local now = tonumber(ARGV[1])
local rate = tonumber(ARGV[2])
local capacity = tonumber(ARGV[3])
local requested = tonumber(ARGV[4])
local ttl = tonumber(ARGV[5])

if tokens == nil then
    tokens = capacity
end

if last_refill == nil then
    last_refill = now
end

```

```

local delta = math.max(0, now - last_refill)
local refill = delta / 1000 * rate

tokens = math.min(capacity, tokens + refill)
last_refill = now

local allowed = 0

if tokens >= requested then
    tokens = tokens - requested
    allowed = 1
end

redis.call("HMSET", KEYS[1], "tokens", tokens, "last_refill", last_refill)
redis.call("EXPIRE", KEYS[1], ttl)

return {allowed, tokens}

```

Go 调用参数示例：

```

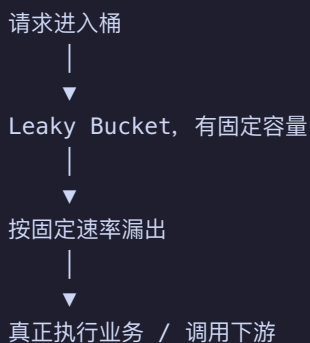
// 每秒补充 60 个 token, 桶容量 120, 适合允许短时突发的外部 API 调用。
allowed, tokens := evalTokenBucket(
    ctx,
    "rate:token:facebook:act_1001:campaign_list",
    time.Now().UnixMilli(),
    60,
    120,
    1,
    300,
)

```

6. 漏桶：匀速流出，削峰排队

适合：要求下游处理速度稳定，宁愿排队也不想突发打爆下游。

设计思想：



核心参数：

- `capacity`：桶容量，也就是最大排队数量。
- `leakRate` 或 `interval`：漏出速率，决定下游看到的处理速度。
- `maxDelay`：最大可接受排队等待时间，超过就拒绝。

例如 `capacity=1000, leakRate=100/s` :

- 最多排队 1000 个请求。
- 下游每秒稳定处理 100 个请求。
- 超过桶容量或最大等待时间就拒绝，避免排队无限增长。
 - 请求进入桶，按固定速率流出。
 - 桶满则拒绝或丢弃。
 - 更像队列削峰，不强调突发能力。
 - Redis 可以存队列和调度时间，但高可靠排队更建议 MQ。

适合场景：

- 短信/邮件发送。
- 外部 API 慢速消费。
- 批量任务稳定出队。

Go 本地漏桶实现：

```
package main

import (
    "fmt"
    "time"
)

type LeakyBucket struct {
    queue chan int
}

func NewLeakyBucket(capacity int, interval time.Duration) *LeakyBucket {
    b := &LeakyBucket{
        queue: make(chan int, capacity),
    }

    go func() {
        ticker := time.NewTicker(interval)
        defer ticker.Stop()

        for range ticker.C {
            select {
            case req := <-b.queue:
                fmt.Println("process request:", req)
            default:
            }
        }
    }()

    return b
}

func (b *LeakyBucket) Allow(req int) bool {
    select {
```

```

    case b.queue <- req:
        return true
    default:
        return false
    }
}

func main() {
    // 桶容量 5, 每 200ms 漏出 1 个请求, 即每秒处理 5 个。
    bucket := NewLeakyBucket(5, 200*time.Millisecond)

    for i := 0; i < 20; i++ {
        if bucket.Allow(i) {
            fmt.Println("enqueue", i)
        } else {
            fmt.Println("reject", i)
        }
        time.Sleep(50 * time.Millisecond)
    }

    time.Sleep(2 * time.Second)
}

```

Redis 全局漏桶 Lua:

```

-- KEYS[1]: 漏桶 key
-- ARGV[1]: 当前毫秒时间 now
-- ARGV[2]: 漏出间隔 intervalMs, 例如每 100ms 处理 1 个
-- ARGV[3]: 最大排队等待时间 maxDelayMs
-- ARGV[4]: key 过期时间, 秒

local now = tonumber(ARGV[1])
local interval = tonumber(ARGV[2])
local max_delay = tonumber(ARGV[3])
local ttl = tonumber(ARGV[4])

local last_time = tonumber(redis.call("GET", KEYS[1]))

if last_time == nil then
    last_time = now
end

local scheduled_time = math.max(now, last_time) + interval
local delay = scheduled_time - now

if delay > max_delay then
    return {0, delay}
end

redis.call("SET", KEYS[1], scheduled_time, "EX", ttl)

return {1, delay}

```

Go 调用参数示例:

```

allowed, delay := evalLeakyBucket(
    ctx,
    "rate:leaky:facebook:act_1001:campaign_list",
    time.Now().UnixMilli(),
    100, // 每 100ms 漏出 1 个，即每秒 10 个
    1000, // 最多允许排队 1 秒
    300,
)

if !allowed {
    return ErrRateLimited
}

if delay > 0 {
    time.Sleep(time.Duration(delay) * time.Millisecond)
}

return callFacebookAPI()

```

7. 并发数限制：计数器或信号量

适合：限制“同时执行中的任务数”，不是限制 QPS。

```

INCR running:facebook:campaign_list
EXPIRE running:facebook:campaign_list 60
DECR running:facebook:campaign_list

```

- 进入时 **INCR**，超过并发上限则拒绝并 **DECR**。
- 退出时 **DECR**。
- 必须设置 TTL，防止进程异常导致计数不释放。
- 更严谨可以用 ZSet 存执行中的 requestId 和过期时间，定期清理超时请求。

工程建议

- 限流逻辑用 Lua 保证原子。
- key 设置 TTL，避免无限增长。
- 高 QPS 全局限流不要让 Redis 成为瓶颈，可结合本地限流。
- key 建议包含平台、账号、接口和窗口，例如 **rate:facebook:{account_id}:campaign_list:1s**，方便隔离不同媒体账号和接口。
- 限流失败策略要明确：核心保护下游时 fail closed，用户体验优先时可 fail open + 本地限流兜底。
- 限流命中要打点：限流 key、接口、账号、窗口、阈值、当前计数、拒绝次数。
- Redis 集群下同一个限流维度要落到同一 hash slot，可使用 hash tag，例如 **rate:{facebook:act_1001:campaign_list}:1s**。
- 大规模滑动窗口会消耗较多内存，窗口越大、请求越多，ZSet 元素越多；高 QPS 可用令牌桶或本地预分配 token 降低 Redis 压力。

高并发下如何保护数据库和下游 RPC

限流不是只在 Redis 做一个 key，而是分层保护：

```
入口网关粗限流
↓
服务本地令牌桶 / semaphore
↓
Redis 全局账号/API 限流
↓
连接池 / worker pool 有界并发
↓
DB 条件更新 / MQ 削峰 / 下游降级
```

保护 MySQL 回源：

- Redis 缓存 miss 后，不要让所有请求直接查 DB。
- 本地令牌桶限制回源 QPS。
- semaphore 限制同时查 DB 的 goroutine 数。
- 热点 key 用 singleflight 合并请求。
- DB 侧用连接池上限、慢 SQL 告警和条件更新兜底。

保护下游 RPC / 外部 API：

- 单实例用本地 limiter 控制当前进程调用速率。
- 多实例共享额度用 Redis 滑动窗口或令牌桶。
- 账号/API/租户维度拆 key，避免互相影响。
- 下游返回限流错误时，动态降速并进入冷却窗口。
- 超限请求可以拒绝、延迟重试、投递 MQ 或降级返回。

只有 Redis 可以做限流吗？不是。

方案	典型实现	优点	缺点	适合场景
Go 本地限流	golang.org/x/time/rate 令牌桶、 <code>semaphore</code> 并发控制	极快，无网络开销，延迟最低	单机生效，多实例不共享额度	单实例保护、客户端 SDK、Redis 故障兜底
Redis 全局限流	<code>INCR/EXPIRE</code> 、 <code>ZSet + Lua</code> 、令牌桶 Lua	多实例共享额度，灵活按用户/账号/API 限制	有网络开销，Redis 可能成瓶颈	多实例服务、外部 API 账号级配额
Nginx 限流	<code>limit_req</code> 、 <code>limit_conn</code>	接入层简单高效	业务维度有限，动态规则弱	IP、URI 粗粒度入口保护
Envoy/API Gateway	local/global rate limit service	统一入口治理，规则集中	需要网关体系和配置治理	微服务统一入口、租户/API 配额
	QPS、并发、熔断、热点参数			

方案	典型实现	优点	缺点	适合场景
Sentinel/ Hystrix 类库		与服务治理结合，降级能力强	语言/框架绑定，跨实例全局能力取决于后端	Java 微服务、热点接口保护
MQ/队列削峰	Kafka/RocketMQ/任务队列	平滑流量，保护下游，可靠重试	不是实时拒绝，延迟增加	批量任务、异步创建、报表拉取
数据库条件更新	<code>UPDATE quota SET used=used+1 WHERE used<limit</code>	强一致，和业务数据同事务	性能低，不适合高 QPS 入口	资金、库存、强配额扣减
CDN/WAF/云网关	云厂商限流、Bot 防护	抗大流量，离用户近	业务细粒度弱，成本和配置依赖平台	公网入口、防刷、防攻击

性能和成本粗略判断

- 本地限流最快：内存操作，通常纳秒到微秒级，但只能保护单实例。
- Nginx/网关限流很快：适合入口粗粒度限流。
- Redis 限流多一次网络往返：通常毫秒内，但热 key、高 QPS Lua、ZSet 大窗口会带来 CPU 和内存压力。
- DB 限流最重：有锁、事务和磁盘成本，不适合普通接口入口限流，只适合强一致额度扣减。
- MQ 削峰不追求立即返回，核心价值是把突发流量变成可控消费速度。

面试 1 分钟回答：

Redis 限流常见有固定窗口、滑动窗口、令牌桶、漏桶和并发数限制。固定窗口用 `INCR + EXPIRE`，简单高效但有边界突刺；精确滑动窗口用 `ZSet + Lua`，按时间戳清理窗口外请求并统计窗口内数量；令牌桶适合允许突发但控制长期平均 QPS；漏桶适合匀速排队保护下游。像 Facebook campaign 列表这种外部 API，我会按平台、账号、接口做多层窗口，例如 `1s/60`、`5m/3000`、`1d/10000`，用 Lua 保证多个窗口检查和写入原子性。限流不只有 Redis，本地令牌桶、Nginx/网关、Sentinel、MQ 削峰、数据库条件更新都能做，选择时看是否需要全局共享额度、精度、延迟、成本和失败兜底。

面试追问：为什么限流要用 Lua？

因为限流通常包含读计数、判断、写计数、设置过期等多个步骤。Lua 能保证这些步骤在 Redis 中原子执行，避免并发穿透。

面试追问：Redis 限流失败时应该放行还是拒绝？

看业务目标。如果限流是为了保护第三方 API 或数据库，Redis 异常时倾向 fail closed 或本地小额度放行，避免下游被打爆；如果是非核心用户体验接口，可以 fail open，同时打告警并启用本地限流兜底。不能默认所有场景都放行。

面试追问：高 QPS 下 Redis 限流会不会成为瓶颈？

会。热点限流 key、复杂 Lua、大 ZSet 窗口都会增加 Redis CPU、内存和网络压力。优化方式包括本

地预限流、令牌预取、分维度拆 key、缩短窗口、聚合上报、网关粗限流 + Redis 精限流，以及给 Redis 做分片和监控。

深入：如何用 Redis 实现延迟队列？

结论先行：

延迟队列可以用 ZSet 实现，score 存执行时间戳，消费者轮询取出到期任务，再用原子操作删除并处理。

分层展开：

写入任务

- `ZADD delay_queue executeAt taskId`。
- task 详情可以存 String/Hash 或数据库。

消费任务

- 查询 `score <=` 当前时间的任务。
- 获取少量任务后尝试删除。
- 删除成功才执行业务，防止多个消费者重复处理。

可靠性

- 任务处理要幂等。
- 失败任务要重试或进入死信。
- 消费和删除最好用 Lua 或事务控制。

适用边界

- 适合中小规模延迟任务。
- 大规模、高可靠延迟消息建议 RocketMQ、Kafka 延迟方案或专门调度系统。

面试追问：消费者宕机导致任务丢失怎么办？

不要先删除再处理，或者需要处理中的队列和超时回收机制。更常见是删除成功后业务处理失败，通过幂等和重试表补偿。

深入：如何用 Redis 实现点赞系统？

结论先行：

点赞系统要拆成“是否点赞、点赞数、点赞列表”三个问题，分别用 Set/String/ZSet 或数据库事件流组合实现。

分层展开：

是否点赞

- `SADD like:post:{id} userId`。
- `SISMEMBER` 判断用户是否点赞。

点赞数

- 小规模可用 `SCARD`。
- 高并发建议独立计数器 `INCR/DECR`。

点赞列表

- 需要按时间排序时用 ZSet, score 为时间戳。
- 只需要去重集合时用 Set。

落库策略

- Redis 承接高频写。
- 异步批量落库。
- 用户重复点击要做幂等。

面试追问：点赞取消和重复点击如何保证正确？

使用 `SADD` / `SREM` 的返回值判断状态是否真的变化，再决定是否增减计数。计数更新和集合更新可用 Lua 保证原子。

七、持久化、复制、Sentinel 与 Cluster

30. RDB、AOF 和混合持久化如何选？

机制	优点	风险
RDB	文件紧凑、恢复快	两次快照间可能丢数据，Fork 有成本
AOF	丢失窗口更小、可审计写命令	文件和 Fsync 成本、Rewrite 压力
混合持久化	兼顾加载速度和增量安全	运维与版本兼容更复杂

缓存可以接受重建时可弱化持久化；重要临时状态常用 AOF Everysec + RDB，但金融权威数据不应只依赖 Redis。

31. Fork 和写时复制为什么会造成抖动？

生成 RDB 或 AOF Rewrite 时创建子进程。Fork 需要复制页表，内存越大耗时越明显；期间主进程继续写会触发 Copy-on-Write，额外复制内存页，并与子进程竞争磁盘和内存带宽。

监控 Fork 耗时、RSS、脏页率、磁盘延迟和延迟尖峰；容量必须预留 COW 峰值，避免接近内存上限时执行持久化。

32. Redis 主从复制如何工作？

完整同步：

```
Replica 建立连接并认证
-> 交换 Replication ID 和 Offset
```

```
-> 无法部分同步时执行 Full Sync
-> Master 生成/发送 RDB
-> 同步期间写命令进入复制缓冲
-> Replica 加载 RDB 后追增量命令
```

断线重连时，**PSYNC** 根据 Replication ID、复制 Offset 和 Replication Backlog 判断能否部分同步；Backlog 覆盖缺失区间即可补发，否则重新全量同步。

33. Sentinel 如何完成故障转移？

Sentinel 主观下线判断

```
-> 达到 Quorum 形成客观下线
-> Sentinel Leader 选举
-> 按优先级、复制进度等选择 Replica
-> 提升为新 Master
-> 其他 Replica 改为复制新 Master
-> 通知客户端更新地址
```

异步复制下，旧 Master 未复制的数据可能丢失；网络分区还可能形成双主写入。客户端必须支持发现、重连和幂等重试。

34. Redis Cluster 如何分片和路由？

Cluster 把 Key 映射到 16384 个 Hash Slot，Slot 分配给不同 Master。节点通过 Cluster Bus 和 Gossip 交换状态并参与故障判断。

- 客户端访问错误节点时收到 **MOVED**，更新长期路由。
- Slot 迁移期间可能收到 **ASK**，临时访问目标节点。
- 多 Key 操作要求同 Slot，可使用 **{tag}**。
- 扩缩容本质是迁移 Slot 和 Key，需限速并监控延迟、网络 and 内存。

35. Cluster 脑裂和故障转移有什么风险？

少数派节点可能仍接受旧连接写入，最终因无法获得多数派而停止服务；故障转移后，未复制到新 Master 的写可能丢失。

治理包括合理节点与副本跨故障域部署、配置节点超时、限制复制落后时继续写、客户端快速刷新拓扑，以及故障演练和业务幂等补偿。

深入：RDB 和 AOF 有什么区别？

结论先行：

RDB 是某个时间点的内存快照，恢复快、文件小，但可能丢数据；AOF 是写命令日志，数据安全性更高，但文件更大、恢复相对慢。

分层展开：

RDB

- 定期生成快照文件。
- 适合备份、灾备、快速恢复。
- 如果宕机，可能丢失上次快照后的数据。

AOF

- 追加记录写命令。
- `appendfsync always/everysec/no` 控制刷盘策略。
- `everysec` 常用，通常最多丢 1 秒数据。

混合持久化

- AOF rewrite 后文件前半部分可以是 RDB 格式，后半部分是增量 AOF。
- 兼顾恢复速度和数据安全。

选择建议

- 缓存场景可只开 RDB 或不开持久化。
- 重要数据建议 AOF everysec + RDB。
- 金融级数据不应只依赖 Redis 持久化。

面试追问：AOF 重写会不会阻塞主线程？

AOF rewrite 由子进程完成，主线程继续处理请求。但 fork、写时复制、磁盘 IO 和 rewrite buffer 仍可能带来延迟和内存压力。

深入：Redis 主从复制是怎么工作的？

结论先行：

Redis 主从复制分为全量复制和增量复制，从节点通过复制主节点数据实现读扩展和高可用基础，但复制是异步的，存在延迟和数据丢失风险。

分层展开：

全量复制

- 从节点第一次连接主节点。
- 主节点生成 RDB 快照并发送给从节点。
- 同步期间新增写命令写入复制缓冲区，再发送给从节点。

增量复制

- 主从短暂断开后，从节点携带复制偏移量重新连接。
- 如果主节点复制积压缓冲区还保留缺失数据，就只补发增量命令。

异步复制风险

- 主节点写成功后不等待从节点确认。
- 主节点宕机时，未复制的数据可能丢失。

工程建议

- 读从库要接受短暂旧数据。

- 关键写读一致场景读主库。
- 配置合理的复制 backlog，减少全量复制。

面试追问：主从复制能提高写性能吗？

不能直接提高单 key 写性能。写仍走主节点，从节点主要用于读扩展、备份和高可用切换。

深入：Redis Sentinel 哨兵解决什么问题？

结论先行：

Sentinel 主要解决主从架构下的自动故障发现、主节点选举、故障转移和客户端发现新主节点的问题。

分层展开：

监控

- 哨兵持续 ping 主从节点。
- 判断主观下线和客观下线。

选主

- 主节点客观下线后，哨兵从从节点中选一个新主。
- 选择依据包括优先级、复制偏移量、runid 等。

故障转移

- 将选出的从节点提升为主。
- 让其他从节点复制新主。
- 通知客户端新的主节点地址。

局限

- 不解决水平扩容。
- 异步复制仍可能丢数据。
- 故障切换期间存在短暂不可用。

面试追问：哨兵至少几个节点？

生产通常至少 3 个哨兵节点，避免单点和误判，并通过多数派完成客观下线判断和 leader 选举。

深入：Redis Cluster 如何实现分片？

结论先行：

Redis Cluster 通过 16384 个 hash slot 实现数据分片，每个 key 根据 CRC16 计算所属 slot，再映射到具体 master 节点。

分层展开：

slot 分片

- 全集群共有 16384 个 slot。
- 每个 master 负责一部分 slot。
- key 通过 $\text{CRC16}(\text{key}) \% 16384$ 定位 slot。

请求路由

- 客户端访问错误节点时，Redis 返回 MOVED 或 ASK。
- 智能客户端会缓存 slot 到节点的映射。

高可用

- 每个 master 可以有 replica。
- master 故障后，replica 被提升为新 master。

扩容缩容

- 本质是迁移 slot。
- 迁移期间客户端可能遇到 ASK 重定向。

面试追问：Cluster 支持多 key 操作吗？

只支持同一个 slot 内的多 key 操作。可以用 hash tag，例如 `{user:1}:profile` 和 `{user:1}:orders` 会落到同一 slot，但不要滥用，否则会导致数据倾斜。

深入：Redis fork 子进程时有什么风险？

结论先行：

Redis fork 子进程用于 RDB、AOF rewrite 等操作，风险主要是 fork 耗时、写时复制导致内存放大、磁盘 IO 影响性能。

分层展开：

fork 耗时

- Redis 内存越大，页表越大，fork 越慢。
- fork 期间主线程可能短暂阻塞。

写时复制

- 子进程生成快照时，父进程继续写数据。
- 被修改的内存页会复制一份，导致内存短时上涨。

磁盘 IO

- RDB 保存和 AOF rewrite 都会产生大量磁盘写。
- 如果磁盘慢，会影响 fsync 和整体延迟。

优化建议

- 控制单实例内存大小。
- 避免高峰期做重写和全量备份。
- 监控 `latest_fork_usec`、内存使用和磁盘 IO。

面试追问：为什么 Redis 实例不建议特别大？

实例太大会让 fork、迁移、恢复、主从全量同步、故障切换都变慢。大厂通常会做分片和容量上限治理。

深入：Redis Cluster 脑裂是什么？怎么避免？

结论先行：

脑裂是旧主节点与新主节点同时对外提供写服务，导致数据分叉。避免脑裂要依赖多数派、合理超时、从节点复制限制和客户端写入保护。

分层展开：

产生原因

- 网络分区导致主节点与集群多数节点失联。
- 集群另一侧提升从节点为新主。
- 旧主如果仍被客户端访问并接受写入，就产生脑裂。

Redis 的保护

- Cluster 依赖多数 master 判断故障。
- 主节点失去多数派后应停止服务。
- 配置 `min-replicas-to-write` 和 `min-replicas-max-lag` 可以降低主从复制异常时继续写的风险。

业务侧治理

- 客户端及时刷新拓扑。
- 写请求走集群客户端，不绕过路由。
- 对关键写入加版本号或幂等校验。

面试追问：脑裂一定能完全避免吗？

分布式系统里不能只靠某个配置绝对避免所有异常。工程上要降低概率，并让业务写入具备幂等、版本保护和可补偿能力。

八、性能、内存与容量规划

36. Redis 容量如何估算？

```
logical memory = key count * average bytes per key
estimated memory = logical memory * encoding/allocator overhead factor
cluster memory = estimated memory * replica factor * headroom factor
```

平均字节必须实测 Key、Value、对象头、过期字典、哈希表、碎片和客户端缓冲；Headroom 还要覆盖 Rehash、Fork COW、复制和故障转移。

示例： 5000 万 Key，每个连同结构开销实测平均 600B，逻辑约 30GB；加碎片和增长后不能按 30GB 配实例，还要乘副本并保证单节点故障时剩余节点不过载。

37. QPS、带宽和节点数如何估算？

```
network throughput ~= QPS * average request+response bytes
base shards >= max(
    peak QPS / measured safe QPS per shard,
    required memory / safe memory per shard,
    peak bandwidth / safe bandwidth per shard
)
```

压测必须使用真实命令分布、Value 大小、Pipeline、TLS、热 Key 和持久化配置；节点数还要满足一个主节点故障后的降容量。

38. Big Key 和 Hot Key 有什么区别？

- Big Key：单个 Value/集合元素过多，造成网络大包、删除阻塞、迁移慢、内存倾斜和复制压力。
- Hot Key：访问频率集中，造成单节点 CPU、带宽和连接压力。

Big Key 通过拆分、渐进扫描、异步删除和限制元素数治理；Hot Key 通过本地缓存、只读副本、Key 分散、请求合并和业务限流治理。

39. 内存碎片和淘汰策略如何治理？

碎片率高可能来自分配器、对象频繁变化和内存页无法归还。先区分实际数据增长与 RSS 碎片，再评估主动整理、重启迁移或数据模型调整。

缓存常用 `allkeys-lfu/lru`，只淘汰有 TTL 数据可用 `volatile-*`；`noeviction` 内存满后拒绝写。策略选择必须匹配“哪些数据可丢”，不能在同实例混放不可淘汰状态与普通缓存。

深入：Redis 过期删除策略是什么？

结论先行：

Redis 采用惰性删除 + 定期删除结合的策略，兼顾 CPU 开销和内存回收效率。

分层展开：

惰性删除

- 访问 key 时检查是否过期。
- 如果过期就删除。
- 优点是省 CPU，缺点是没人访问的过期 key 可能滞留内存。

定期删除

- Redis 周期性抽样检查带过期时间的 key。
- 删除其中已过期的 key。
- 避免过期 key 长期占用内存。

内存淘汰

- 如果内存达到 `maxmemory`，触发淘汰策略。
- 淘汰和过期不是一回事：过期是 TTL 到期，淘汰是内存不够。

面试追问：为什么不定时精确删除？

给每个 key 设置定时器会消耗大量 CPU 和内存，海量 key 场景下不可接受。

深入：Redis 有哪些内存淘汰策略？

结论先行：

Redis 淘汰策略围绕“从哪些 key 中淘汰”和“按什么规则淘汰”两条线展开，生产缓存场景常用 `allkeys-lru`、`volatile-lru` 或 LFU 类策略。

分层展开：

不淘汰

- `noeviction`：内存满后写入报错。
- 适合不能丢数据的场景。

在所有 key 中淘汰

- `allkeys-lru`：淘汰最近最少使用。
- `allkeys-lfu`：淘汰使用频率低的。
- `allkeys-random`：随机淘汰。

只在设置 TTL 的 key 中淘汰

- `volatile-lru`。
- `volatile-lfu`。
- `volatile-random`。
- `volatile-ttl`：优先淘汰快过期的。

选择建议

- 纯缓存系统常用 `allkeys-lru` 或 `allkeys-lfu`。
- 混合存储重要数据和缓存时要谨慎，避免重要 key 被淘汰。
- 最好不同用途拆实例。

面试追问：Redis 的 LRU 是精确的吗？

不是完全精确 LRU，而是近似 LRU。Redis 通过采样若干 key 后淘汰其中最久未访问的，平衡性能和效果。

深入：什么是大 key？有什么危害？

结论先行：

大 key 是 value 很大或集合元素很多的 key，会造成阻塞、网络抖动、迁移困难、内存不均和故障恢复变慢。

分层展开：

常见形态

- String value 几 MB 甚至几十 MB。
- Hash/List/Set/ZSet 元素几十万甚至更多。
- 单个 key 下存储了过多业务对象。

主要危害

- 读取和删除耗时长，阻塞主线程。
- 网络传输大包导致延迟抖动。
- Cluster 中 slot 数据倾斜。
- RDB/AOF、主从复制、迁移成本变高。

治理方案

- 拆 key，例如按用户、业务、时间片分片。
- 删除用 `UNLINK` 或分批删除。
- 集合分页读取，避免一次拉全量。
- 定期扫描识别大 key。

面试追问：怎么发现大 key？

可以用 `redis-cli --bigkeys`、`MEMORY USAGE`、离线分析 RDB、监控慢查询和网络大包。线上执行扫描要控制频率，避免影响主库。

深入：什么是热 key？怎么处理？

结论先行：

热 key 是被超高频访问的 key，会导致单节点 CPU、网络或带宽打满。处理思路是发现、拆分、复制、本地缓存和限流降级。

分层展开：

发现方式

- 业务埋点统计 key 访问频率。
- Redis hotkeys 采样。
- 监控单节点 QPS、带宽、CPU 异常。
- 代理层统计。

处理方式

- 本地缓存热点数据，降低 Redis 压力。
- 热 key 拆分成多个副本 key，随机读。
- 读写分离，增加副本承担读流量。
- 对热点请求限流和降级。

注意事项

- 拆分副本会增加一致性复杂度。
- 本地缓存需要失效通知或短 TTL。
- 对强一致字段不要轻易复制缓存。

面试追问：热点 key 能靠 Cluster 自动解决吗？

不能。Cluster 解决的是数据分片，不会把单个 key 拆到多个节点。单 key 热点仍会集中在一个节点。

深入：Redis 内存碎片是什么？怎么治理？

结论先行：

内存碎片是 Redis 释放的内存没有完全归还操作系统或无法被高效复用，表现为 `RSS` 明显高于 `used_memory`。

分层展开：

产生原因

- key 频繁创建和删除。
- value 大小差异大。
- jemalloc 分配器按不同 size class 管理内存。
- 大量过期和淘汰导致内存空洞。

观察指标

- `mem_fragmentation_ratio`。
- `used_memory`。
- `used_memory_rss`。

治理方案

- 开启 active defrag。
- 控制 value 大小。
- 避免频繁写入删除大对象。
- 必要时主从切换或重启释放内存。

面试追问：mem_fragmentation_ratio 越低越好吗？

不是。过低可能意味着使用了 swap 或 RSS 小于逻辑内存，反而危险。要结合 `used_memory`、`RSS`、系统内存和延迟综合判断。

深入：如何避免 Redis 缓存污染？

结论先行：

缓存污染是大量低价值或一次性数据进入缓存，挤掉高价值热点数据。治理核心是准入控制、TTL 分级和淘汰策略。

分层展开：

产生原因

- 爬虫或批处理扫描大量冷数据。
- 搜索结果页把一次性数据都写缓存。
- 预热范围过大。
- TTL 设置过长。

治理方案

- 只缓存高频访问数据。
- 对冷数据设置短 TTL。
- 对批量任务禁用缓存回填。
- 使用 TinyLFU、本地缓存准入或业务白名单。

监控指标

- key 新增速度。
- 命中率变化。
- evicted_keys。
- 热点 key 占比。

面试追问：查询 DB 后一定要回填缓存吗？

不一定。低频数据、批量扫描、后台任务、分页深翻数据不一定要回填，否则会污染缓存。

深入：如何做 Redis 容量评估？

结论先行：

容量评估要从 key 数量、value 大小、数据结构开销、TTL、增长速度、复制副本、碎片率和峰值写入综合估算。

分层展开：

数据规模

- 预计 key 数量。
- 平均 value 大小。
- 最大 value 大小。
- 集合元素数量。

额外开销

- key 本身占内存。
- Redis 对象头、dict、编码结构开销。
- 内存碎片。
- 复制 backlog、client buffer、AOF rewrite buffer。

副本和高可用

- 主从至少双份内存。
- Cluster 多分片要预留倾斜空间。
- fork 时要预留写时复制内存。

安全水位

- 不建议长期超过 70% 到 80%。
- 大促或批处理前预留更多空间。
- 设置 maxmemory 和告警阈值。

面试追问：如何估算一个 key 的真实内存？

可以用 `MEMORY USAGE key` 抽样，再结合线上 key 数量、类型分布和业务增长率估算。精确容量需要压测和 RDB 离线分析。

九、生产治理、安全与故障排查

40. Redis 监控体系包括什么？

层次	指标
客户端	连接池等待、超时、错误、重试、命令延迟
服务端	CPU、内存/RSS、碎片、连接数、阻塞客户端、命令统计
数据	命中率、Key 数、过期、淘汰、Big/Hot Key
持久化复制	Fork、AOF Fsync、复制延迟、Backlog、主从状态
Cluster	Slot、节点状态、重定向、故障转移和迁移进度
业务	DB 回源、限流拒绝、锁失败、缓存版本落后

41. Redis 超时如何排查？

```
客户端连接池等待？
-> 网络 RTT/TLS/DNS？
-> Redis 执行排队或慢命令？
-> Big Key/Hot Key/Lua？
-> Fork/AOF/磁盘抖动？
-> Cluster 重定向/迁移/故障转移？
```

先限流、降级和隔离重任务，再对齐客户端耗时、`SLOWLOG`、Latency、CPU、网络、Fork 和复制时间线。不要只因为超时就扩大连接池。

42. CPU 飙高和连接数打满怎么处理？

CPU 高优先排查慢命令、热 Key、大集合、长 Lua、过期风暴和连接风暴。连接数高要区分业务流量、连接泄漏、短连接和故障重连。

止血可限制问题接口、暂停扫描、启用本地缓存、降低重试并发；根治包括命令复杂度治理、连接池复用、热 Key 拆分和容量扩展。

43. Redis 故障时如何保护数据库？

- 网关和服务双层限流，禁止所有请求直接回源。
- 本地缓存返回允许的旧数据。

- 非核心功能降级或快速失败。
- 热点回源使用 Singleflight/互斥重建。
- DB 连接池和并发设硬上限。
- Redis 恢复后分批预热，避免瞬时回填和二次雪崩。

44. Redis 安全和多租户如何治理？

- TLS 保护传输，ACL 按应用授予最小命令和 Key Pattern 权限。
- 禁止公网暴露，使用网络隔离、认证和凭证轮换。
- 限制或重命名高风险管理命令不能替代 ACL 和网络控制。
- 不同环境、数据等级和高风险业务使用独立实例/集群。
- 配置连接、内存、吞吐和 Key 数配额，明确 Owner 与 TTL 策略。
- 审计配置变更、Flush、故障转移和大规模删除。

补充模块：线上治理与故障排查：发现问题、定位问题、解决问题

这一章回答“Redis 出问题怎么办”。按现象、指标、止血、根因治理来背，面试时会显得更像做过线上系统。

深入：线上 Redis 生产故障总手册：从告警到恢复

这是一张总流程图。第 41~46 题分别展开慢查询、CPU、连接数、超时、命中率和 Redis 故障保护 DB；实际排障时先按本节确定故障分支，再进入对应小节。

先明确故障目标：保护什么、影响谁

收到 Redis 告警后，先用 1~3 分钟回答四个问题：

1. **影响面**：是单个接口、单个业务、单个 Redis 节点，还是所有依赖 Redis 的服务？
2. **业务后果**：是延迟升高、请求失败、缓存 miss、数据暂时不可见，还是写入/扣减等核心操作失败？
3. **资源位置**：是应用连接池、网络、Redis 主线程、后台持久化，还是数据库被回源流量打高？
4. **时间线**：故障开始前是否发生发布、扩容、配置变更、批处理、缓存清理、主从切换或流量活动？

不能看到 **CPU 高** 就直接扩容，也不能看到 **Redis 超时** 就直接重启。先把应用、Redis 和 DB 的时间线对齐，才能避免错误止血。

第一阶段：快速确认与止血

应用侧先看：

- Redis 调用 QPS、成功率、P50/P95/P99、超时率和错误类型。
- 连接池获取等待时间、活跃连接、空闲连接和重试次数。
- 哪些接口、服务实例、租户或 key 前缀异常。
- Redis 失败后是否大量回源 DB，DB QPS、CPU、连接池等待是否同步上涨。

Redis 侧先看：

```
INFO server
INFO clients
INFO memory
INFO stats
INFO commandstats
INFO persistence
INFO replication
SLOWLOG GET 20
LATENCY LATEST
```

止血动作必须和现象对应：

现象	优先止血动作	不要直接做的事
新版本发布后延迟或错误突增	暂停发布、关闭 feature flag、回滚版本	先加 Redis 连接数或盲目扩容
Redis 故障导致 DB 回源上涨	限制回源并发、接口限流、非核心功能降级、本地旧值兜底	让所有请求继续重试和回源
某个调用方重试风暴	按应用/接口摘流、限制重试次数、增加回避	继续放大重试次数
单节点 CPU 或带宽打满	热 key 分流、暂停扫描/批处理、拆大 key 或限制大请求	只把 <code>maxclients</code> 调大
连接数接近上限	找异常客户端并摘流，收紧连接池上限	直接无限提高 <code>maxclients</code>
主从延迟或副本异常	从读节点摘流；强一致读取临时走主节点	继续把实时读发给落后副本
大 key/慢命令阻塞	停止问题命令来源，改分页、拆 key，删除用 <code>UNLINK</code>	在线执行 <code>KEYS</code> 、 <code>HGETALL</code> 或大范围集合运算
持久化或磁盘抖动	暂停低优先级批量写，检查磁盘和 fork 压力	未确认数据安全前直接删除 AOF/RDB

止血的判断原则：

- 限流发生在网关、服务入口、消费者或 Redis 调用封装层，目标是减少进入 Redis 的请求量。
- 降级只关闭非核心能力，例如推荐、报表、预热、非实时统计；不能把扣库存、投放状态等核心写操作静默丢弃。
- 本地缓存适合短时间兜底可重建数据；旧值要标记允许短暂过期，不能把错误数据伪装成最新数据。
- 主从切换或读主只解决读副本不可用/延迟问题，不能解决主节点本身 CPU、内存或网络已打满的问题。
- 任何重试都必须有次数上限、退避和熔断，否则 Redis 故障会被重试流量放大。

第二阶段：按现象进入排查分支

Redis 请求变慢或失败

```
|
+-- 连接池等待高? -> 查连接泄漏、池配置、实例数、阻塞连接
|
+-- slowlog/commandstats 高? -> 查慢命令、Lua、返回结果大小
|
+-- CPU 高但慢命令不明显? -> 查热 key、过期淘汰、扫描、持久化
|
+-- 网络 RTT/重传/带宽异常? -> 查跨机房、丢包、大 value、大响应
|
+-- 命中率下降且 DB 被打高? -> 查 key 规则、TTL、淘汰、回填和流量结构
|
+-- 副本延迟/故障? -> 查复制链路、主从状态、读路由和故障转移
|
+-- 内存接近上限? -> 查大 key、碎片、增长速度、淘汰和容量规划
```

第三阶段：最小证据集

排查不能只凭截图或单点指标。至少保留以下证据，并记录采集时间：

维度	需要保留的证据	用来回答的问题
请求端	接口 QPS、P99、错误码、Redis span、pool wait、重试数	请求在哪里变慢或失败？
命令端	<code>SLOWLOG</code> 、 <code>INFO commandstats</code> 、命令调用量和平均耗时	哪类命令变重或变多？
数据端	key 前缀、 <code>TYPE</code> 、 <code>MEMORY USAGE</code> 、集合规模、TTL	是大 key、热 key、过期还是数据模型问题？
实例端	CPU、内存、RSS、碎片率、连接、带宽、blocked clients	Redis 自身哪个资源到瓶颈？

维度	需要保留的证据	用来回答的问题
持久化	RDB/AOF rewrite、fork、fsync、磁盘延迟	是否是后台任务造成抖动?
高可用	主从 offset、复制延迟、角色、故障转移日志	是否是复制或路由问题?
变更	发布、配置、运维命令、批任务、活动流量	故障是否由近期变化触发?

常用的安全检查命令：

```

TYPE some:key
TTL some:key
MEMORY USAGE some:key
OBJECT ENCODING some:key
INFO keyspace
CLIENT LIST
ROLE

```

生产环境使用这些命令时，要避免对未知大 key 执行全量读取；`MEMORY USAGE`、`TYPE`、`TTL` 也要控制采样范围，不能把排查命令变成新的压力来源。

第四阶段：根因定位后的修复选择

根因	短期修复	长期治理
错误命令或错误参数	回滚、关闭开关、限制请求规模	SDK 封装禁用危险命令，增加参数上限和代码扫描
大 key	分批读取/删除， <code>UNLINK</code> ，暂停生产者	按用户、时间或业务维度拆 key，限制集合规模
热 key	本地缓存、副本 key、限流	热点识别、分层缓存、热点数据预案
缓存雪崩/命中率下降	保护 DB、分批预热、延长核心数据 TTL	TTL 抖动、singleflight、缓存准入和回填监控
连接池过大或泄漏	摘异常实例、收紧池上限、重启泄漏实例	按实例数计算连接预算，连接池指标和压测校准
内存/碎片/淘汰	清理低价值数据、扩容、降低写入	容量水位、内存模型、active defrag 和拆实例
主从延迟	摘除落后副本，核心读走主	复制链路监控、读路由策略、故障演练
单实例能力不足	分流、扩容、迁移 slot	Cluster 分片、业务隔离、容量和增长规划

恢复阶段：避免“恢复即二次故障”

1. 先确认 Redis 延迟、错误率、连接数和 CPU 已恢复，而不是只看进程存活。
2. 先恢复核心读写，再逐步恢复非核心接口、批任务和预热任务。
3. 缓存为空时按热点、租户、接口分批预热，限制回源并发和写入速率。
4. 主从恢复后确认复制 offset、延迟和数据追平，再把副本加入读流量。
5. 观察至少一个完整业务高峰或 TTL 周期，确认命中率、DB 压力和淘汰没有再次异常。

复盘必须回答的五个问题

- 为什么监控能发现 Redis 异常，却不能快速定位到具体应用、命令或 key 前缀？
- 为什么故障没有被限流、熔断、隔离或降级挡住？
- 是否存在危险命令、大 key、热 key、连接池无上限或重试无上限？
- Redis 恢复后是否会导致缓存击穿、DB 雪崩或流量瞬时回灌？
- 是否需要补充容量水位、命令审计、key 采样、演练和自动化止血能力？

面试中的完整回答模板：

我会先确认影响范围和时间线，同时看应用 Redis 调用耗时、连接池等待、错误类型、Redis `slowlog/commandstats/latency` 以及 DB 回源压力。根据现象做定向止血：新发布先回滚，异常调用方限流并关闭重试，非核心功能降级，Redis 故障时限制回源并保护 DB，副本延迟则摘除副本。业务稳定后，再从请求端、命令端、数据端、实例资源、持久化和主从复制六个方向定位，最后按根因选择改命令、拆 key、治理热 key、调整 TTL、修复连接池、扩容分片或调整架构。恢复时分批放量并验证命中率、DB 压力和复制延迟，最后补监控、限额和故障演练。

深入：Redis 慢查询怎么排查？

结论先行：

Redis 慢查询排查不能只看 `SLOWLOG`。生产上要先确认“慢”发生在哪里：客户端排队、网络传输、Redis 执行命令、返回大结果、还是 fork/fsync 等系统抖动。排查路径是先止血，再按请求端、Redis 自身、数据层和系统层逐层定位。

分层展开：

常见现象

- 接口 P95/P99 延迟突然升高，Redis 调用耗时从几毫秒变成几十/几百毫秒。
- 客户端出现 `read timeout`、`context deadline exceeded`。
- DB QPS 可能被带高，因为 Redis 慢导致缓存读失败或降级回源。
- 监控上 Redis `instantaneous_ops_per_sec`、CPU、带宽、`connected_clients`、慢查询数异常。

一般怎么发现

- APM/链路追踪看到 Redis span 耗时升高。

- 客户端埋点看到 Redis 命令耗时、连接池等待时间升高。
- Redis `SLOWLOG GET` 出现慢命令。
- Redis latency monitor 出现 command、fork、aof、expire-cycle 等事件。
- 业务报警：接口超时率、错误率、DB 回源量、缓存命中率异常。

第一步：先判断是不是 Redis 执行慢

```
SLOWLOG GET 20
SLOWLOG LEN
INFO commandstats
LATENCY LATEST
LATENCY DOCTOR
```

- 如果客户端耗时高，但 `SLOWLOG` 没有记录，优先怀疑连接池等待、网络、大 value 传输、客户端 GC 或请求排队。
- 如果 `SLOWLOG` 里有慢命令，继续看命令类型、key、执行耗时和出现时间。
- `SLOWLOG` 只记录 Redis 执行命令的时间，不包含客户端排队、网络传输和结果读取时间。

第二步：从命令层排查

- 是否出现 `KEYS`、`FLUSHALL`、`SORT`、`SUNION`、`SINTER`、`HGETALL`、`SMEMBERS`、大范围 `ZRANGE/ZREVRANGE`。
- 是否有 Lua 脚本执行过久，脚本里循环处理大量 key 或大集合。
- 是否有批量 `MGET/MSET` 一次带太多 key。
- 是否一次返回大量数据，服务端执行不慢，但网络传输和客户端反序列化很慢。

治理动作：

- `KEYS` 改为 `SCAN`，并控制 `COUNT` 和执行频率。
- 大集合全量读取改成分页读取，例如 `HSCAN/SSCAN/ZSCAN` 或 `ZRANGE start stop`。
- 大 Lua 拆小，避免在脚本中处理不可控数据量。
- 批量命令控制 batch size，例如每批 100~500 个 key，根据 value 大小压测调整。

第三步：从数据层排查

- 是否存在大 key：`redis-cli --bigkeys`、`MEMORY USAGE key`、离线分析 RDB。
- 是否存在热 key：业务埋点、代理层统计、`redis-cli --hotkeys`。
- 是否 key 集中过期，导致 expire cycle 抖动。
- 是否 value 变大，例如缓存从摘要字段变成完整对象列表。

治理动作：

- 大 key 拆分：按用户、时间片、业务维度分片。
- 热 key 加本地缓存、副本 key、读写分离或限流。
- 集中过期加随机 TTL，避免同一时间大量过期。
- 删除大 key 用 `UNLINK`，不要直接 `DEL`。

第四步：从请求端排查

- 连接池是否耗尽，是否有获取连接等待时间。
- 是否有重试风暴，超时后立即重试导致 Redis 压力翻倍。
- 客户端超时时间是否过短，导致正常抖动被放大成失败。
- Go 服务是否发生 GC 停顿、goroutine 堆积、worker pool 阻塞。

治理动作：

- 连接池设置最大连接数、最大等待时间、空闲连接回收。
- 重试必须加退避和最大次数，Redis 超时时不要无脑重试。
- 对非核心缓存读允许降级，本地缓存兜底。
- 对热点接口加本地 limiter，避免异常流量继续打 Redis。

第五步：从 Redis 自身和系统层排查

- CPU 是否接近单核打满。
- 网络入/出带宽是否打满。
- 是否正在 RDB、AOF rewrite、fork。
- 是否 AOF `appendfsync always/everysec` 遇到磁盘抖动。
- 是否内存接近上限、发生频繁淘汰、swap。

```
INFO stats
INFO cpu
INFO memory
INFO persistence
INFO clients
```

治理动作：

- fork/AOF 抖动明显时，评估持久化策略、磁盘性能和实例内存规模。
- 网络打满时，减少大 value 返回、拆分热点、扩容分片。
- CPU 打满时，治理慢命令、热 key、Lua，必要时拆分实例。

生产止血顺序

1. 先限流或降级异常接口，阻止 Redis 压力继续扩大。
2. 如果是明确慢命令，快速回滚发布或关闭对应任务。
3. 如果是热 key，临时开启本地缓存或副本 key。
4. 如果是大 key 删除/读取，暂停任务，改为分批或异步删除。
5. 如果 Redis 实例资源打满，临时扩容读副本、拆分流量或迁移热点。

面试追问：慢查询日志记录网络耗时吗？

不记录。slowlog 主要记录 Redis 执行命令的耗时，不包含客户端排队、网络传输和响应读取时间。

深入：线上 Redis CPU 飙高怎么办？

结论先行：

Redis CPU 飙高要先判断是“流量变多”还是“单次命令变重”。生产排查要把 CPU 飙高和 QPS、慢查询、命令分布、热 key、大 key、过期淘汰、Lua、持久化事件放在同一时间线里看。

分层展开：

常见现象

- Redis 实例 CPU 使用率持续高位，单核接近 100%。
- 请求延迟升高，慢查询增加。
- 某个 Cluster 节点明显比其他节点忙。
- 带宽、QPS、连接数可能同步升高，也可能 QPS 没升但 CPU 升高。

一般怎么发现

- Redis 监控 CPU 报警。
- APM 看到 Redis 调用延迟升高。
- `INFO commandstats` 某类命令调用次数或耗时异常。
- `SLOWLOG` 出现高复杂度命令或 Lua 脚本。
- 业务发布、定时任务、活动流量开始后 CPU 升高。

第一步：确认 CPU 飙高形态

```
INFO cpu
INFO stats
INFO commandstats
SLOWLOG GET 20
```

判断重点：

- QPS 是否升高：如果 QPS 升高，大概率是流量、热 key 或连接风暴。
- QPS 没升但 CPU 升高：大概率是命令变重、大 key、Lua、过期淘汰、持久化事件。
- 是否只有单个节点高：大概率是热 key、slot 倾斜或某类业务 key 集中。

第二步：从请求端看流量来源

- 哪个服务、接口、定时任务调用 Redis 激增。
- 是否刚发布新版本，新增了缓存回填、批量扫描或大范围读取。
- 是否有重试风暴，超时后多实例同时重试。
- 是否有后台任务和在线流量抢同一个 Redis。

处置：

- 对异常接口限流、降级或回滚。
- 暂停批处理、预热、扫描类任务。
- 重试增加退避，必要时关闭 Redis 失败重试。

第三步：从命令层定位高 CPU 命令

- `INFO commandstats` 看 `cmdstat_xxx` 的 `calls`、`usec`、`usec_per_call`。
- `SLOWLOG GET` 看是否有 `HGETALL`、`SMEMBERS`、`ZRANGE` 大范围、`SUNION`、Lua。
- 查看是否有大量 `SCAN`，虽然 `SCAN` 非阻塞，但高频扫描也会消耗 CPU。

处置：

- 高复杂度命令改分页、拆 key 或异步任务。
- Lua 设置输入规模上限，禁止脚本里做大循环。
- 批量操作限制 batch size。

第四步：从数据层看热 key、大 key、过期淘汰

- 热 key 会让单个节点 CPU 和带宽打满。
- 大 key 会让单条命令消耗大量 CPU 和网络。
- 大量 key 同时过期会让过期删除周期占用 CPU。
- maxmemory 触发频繁淘汰时，也会增加 CPU 消耗。

```
INFO keyspace
INFO memory
INFO stats
```

重点看：

- `expired_keys` 是否突增。
- `evicted_keys` 是否突增。
- `used_memory` 是否接近 `maxmemory`。
- 某些 key 前缀是否访问异常。

处置：

- 热 key 本地缓存、副本 key、读副本、限流。
- 大 key 拆分，删除用 `UNLINK`。
- TTL 增加随机抖动。
- 容量不足时扩容或调整淘汰策略。

第五步：从 Redis 自身事件看 fork、AOF、后台线程

- RDB/AOF rewrite fork 期间可能带来 CPU 和内存压力。
- AOF rewrite、lazy free、active defrag 也会消耗 CPU。
- Redis 6+ IO 线程、后台线程使进程总 CPU 可能超过 100%。

```
INFO persistence
INFO memory
LATENCY LATEST
```

处置：

- 业务高峰避开 rewrite、bgsave。
- 大实例拆小，降低 fork 成本。
- active defrag 根据碎片率和 CPU 情况谨慎开启或调参。

生产止血顺序

1. 限流/熔断异常调用方，暂停批量任务。
2. 快速查看 `commandstats + slowlog`，定位是否有明显问题命令。
3. 如果是热 key，启用本地缓存或读副本分流。
4. 如果是大 key/大范围命令，回滚或改分页。
5. 如果单节点长期高，做数据拆分、slot 迁移或扩容分片。

面试追问：Redis 是单线程，CPU 为什么可能超过 100%？

Redis 还有后台线程和子进程，例如 AOF、lazy free、IO 线程、RDB/AOF rewrite 子进程。多核环境下进程整体 CPU 可能超过单核 100%。

深入：Redis 连接数打满怎么办？

结论先行：

Redis 连接数打满通常不是单纯把 `maxclients` 调大就结束，而是要排查调用方连接池、连接泄漏、慢请求堆积、短连接风暴和异常重试。生产上先保护 Redis 和核心链路，再定位是哪批客户端占满连接。

分层展开：

现象

- 新请求连接 Redis 失败。
- 客户端超时增多。
- Redis `connected_clients` 接近 `maxclients`。
- 业务日志出现 `ERR max number of clients reached`、`connection refused`、`connection pool timeout`。
- Redis 端 `blocked_clients`、客户端连接池等待时间可能升高。

一般怎么发现

- Redis `INFO clients` 看到 `connected_clients` 接近上限。
- 业务侧连接池等待、获取连接超时报警。
- 实例连接数曲线突然上升，常和发布、扩容、重试风暴同时发生。
- `CLIENT LIST` 看到某些来源 IP 连接数异常。

第一步：确认连接数和客户端来源

```
INFO clients
CONFIG GET maxclients
CLIENT LIST
```


重点看：

- `connected_clients` 是否接近 `maxclients`。
- `blocked_clients` 是否异常。
- `CLIENT LIST` 中哪个 `addr`、`name`、`cmd` 占连接多。
- 是否大量连接处于 idle 很久，说明可能连接泄漏或池未回收。

第二步：从请求端排查连接池

- 是否每个请求都新建 Redis client，没有复用连接池。
- 服务实例数是否扩容后，每个实例连接池太大，连接总数超过 Redis 承载。
- 连接池是否没有设置最大连接数、最大空闲数、最大等待时间。
- pipeline、pubsub、阻塞命令是否长期占用专用连接。
- Go 代码是否忘记关闭 `Rows` 类似资源；Redis 场景一般是 pipeline/pubsub/lock watch 等对象未正确释放。

估算方式：

Redis 总连接数 $\approx$ 服务实例数 $\times$ 单实例连接池上限 + 其他后台任务连接

如果服务 100 个实例，每个实例 Redis pool 100，理论最大就是 10000 条连接，可能直接打满 Redis。

第三步：从 Redis 命令和数据层排查连接堆积

- 慢命令导致连接被长时间占用。
- 大 key 返回慢，客户端读响应慢，连接释放慢。
- `BLPOP/BRPOP/XREAD BLOCK` 等阻塞命令占住连接。
- Lua 脚本或大范围查询阻塞主线程，其他连接排队。

处置：

- 治理慢命令、大 key、热 key。
- 阻塞命令使用独立连接池，避免占满普通请求连接。
- 读写流量拆分，后台任务和在线请求隔离 Redis 实例或连接池。

第四步：从系统层确认上限是否真的够

- Redis `maxclients`。
- 操作系统 `ulimit -n`。
- Redis 进程可用 fd。
- 每条连接的内存开销。

临时提高 `maxclients` 前必须确认 fd 和内存足够，否则会“从连接数问题”变成“系统资源问题”。

生产止血顺序

1. 用 `CLIENT LIST` 找出异常来源 IP 或应用名。
2. 对异常服务限流、摘流量、重启异常实例或回滚发布。

3. 必要时临时提高 `maxclients`，同时确认 `ulimit -n` 和内存。
4. 暂停批处理、扫描、阻塞消费等非核心任务。
5. 事后调整连接池上限、等待超时、空闲回收和实例扩容策略。

面试追问：连接池越大越好吗？

不是。连接池太大会增加 Redis 上下文管理、客户端资源和突发并发压力。连接池大小要按服务实例数、QPS、命令耗时和 Redis 承载能力计算。

深入：Redis 出现超时，可能是什么原因？

结论先行：

Redis 超时不等于 Redis 服务端一定慢。生产排查要把一次 Redis 调用拆成“获取连接 → 发送命令 → Redis 排队执行 → 网络返回 → 客户端反序列化”，逐段确认耗时在哪里。

分层展开：

常见现象

- 客户端出现 `i/o timeout`、`read timeout`、`context deadline exceeded`。
- 接口错误率升高，P99 抖动明显。
- Redis slowlog 可能有记录，也可能完全没有。
- 连接池等待时间、网络 RTT、CPU、带宽、Redis 延迟事件可能有一个或多个异常。

一般怎么发现

- 链路追踪看到 Redis span 超过超时阈值。
- 客户端 Redis SDK 指标显示 pool wait、dial、read、write 耗时。
- Redis 监控出现 latency、CPU、connected_clients、blocked_clients 异常。
- 网络监控看到丢包、重传、带宽打满。

第一步：先分清是哪一种超时

获取连接超时：连接池满、连接泄漏、慢请求占连接。
建立连接超时：网络、防火墙、Redis 连接数满、跨机房抖动。
写超时：客户端到 Redis 网络异常或服务端收包慢。
读超时：Redis 执行慢、返回大包、网络慢、客户端读慢。
context 超时：上游链路预算不足或排队太久。

第二步：从请求端排查

- 连接池等待时间是否升高。
- 单实例并发是否突然上升。
- 是否有重试风暴，失败后立即重试。
- Go 服务是否 GC、goroutine 堆积、CPU 被打满。
- context 超时是否设置过短，比如业务总超时 100ms，却给 Redis 留了 5ms。

处置：

- 调整连接池上限和等待时间，但不能盲目放大。

- Redis 调用失败重试要加退避、限次和熔断。
- 非核心缓存读失败时降级，不要继续放大请求。
- 将 Redis 超时预算纳入整条链路预算。

第三步：从 Redis 自身排查

```
SLOWLOG GET 20
LATENCY LATEST
INFO commandstats
INFO clients
INFO memory
INFO persistence
INFO stats
```

重点看：

- 是否有慢命令、大 key、Lua。
- CPU 是否打满。
- `connected_clients`、`blocked_clients` 是否异常。
- 是否发生 fork、AOF fsync、rewrite。
- 是否内存不足、swap、淘汰频繁。

处置：

- 慢命令分页、拆 key、回滚问题发布。
- 大 key 读取/删除改分批或 `UNLINK`。
- 持久化抖动评估磁盘、fork 成本和大实例拆分。

第四步：从网络层排查

- 应用和 Redis 是否跨机房、跨可用区。
- 是否有丢包、重传、网络 RTT 抖动。
- Redis 出口或入口带宽是否打满。
- 是否返回大 value，导致网络传输耗时长。

处置：

- 服务和 Redis 尽量同机房/同可用区。
- 大 value 拆分或压缩，避免一次返回巨大对象。
- 热点读走本地缓存或读副本。

第五步：从业务层排查

- 是否缓存雪崩，大量 key 同时过期。
- 是否热点 key 集中。
- 是否新发布导致 key 命名变化，命中率下降。
- 是否后台任务扫描 Redis 或批量回填。

处置：

- TTL 增加随机值。
- 热点 key 本地缓存。
- 后台任务限速，和在线 Redis 隔离。
- 缓存失败时保护 DB，避免双重故障。

生产止血顺序

1. 先看客户端错误类型，区分 pool timeout、dial timeout、read timeout。
2. 同时看 `slowlog/latency/commandstats`，判断 Redis 是否真的慢。
3. 如果 Redis 不慢，重点查连接池、网络、客户端 GC 和大包传输。
4. 如果 Redis 慢，按慢查询/CPU/大 key/热 key 路径处理。
5. 加限流、熔断、降级，避免超时重试把故障放大。

面试追问：如何证明是 Redis 服务端慢还是客户端慢？

对比客户端耗时、Redis slowlog、Redis latency monitor、网络 RTT、服务端 CPU 和连接池等待时间。如果客户端耗时高但 slowlog 没记录，往往是网络、排队或连接池问题。

深入：如果 Redis 命中率突然下降，你怎么排查？

结论先行：

Redis 命中率突然下降时，核心不是先怪 Redis，而是先确认“哪些接口、哪些 key 前缀、从什么时候开始下降”。生产排查要把命中率、DB 回源、key 过期/淘汰、发布变更、流量结构和缓存回填链路串起来看。

分层展开：

常见现象

- 缓存命中率从 90%+ 降到 60%、40%。
- DB QPS、慢 SQL、连接池等待时间上升。
- 接口延迟升高，Redis QPS 可能升高，也可能 miss 后回源变多。
- `evicted_keys`、`expired_keys`、缓存写失败、key 数量变化可能异常。

一般怎么发现

- 缓存命中率监控报警。
- DB QPS/CPU/慢查询报警。
- APM 看到大量请求走 DB 回源链路。
- 业务指标异常，例如首页、广告计划列表、素材列表加载变慢。

第一步：先定位影响范围

是全局命中率下降？
还是某个服务/接口下降？
是某类 key 前缀下降？
是某个 Redis 节点/slot 下降？
是读请求变了，还是缓存数据没了？

需要拉齐时间线：

- 命中率从什么时候开始降。
- 同一时间是否有发布、扩容、活动、批处理、缓存清理、Redis 故障切换。
- DB QPS 是否同步上升。

第二步：从请求端看 key 规则和读写路径

- 代码发布后 key 前缀、版本号、参数拼接是否变化。
- 读缓存和写缓存是否使用同一个 key。
- 是否新增了查询参数，导致 key 离散度变高。
- 是否灰度期间新老版本 key 不兼容。
- 是否缓存回填失败但没有报警。

典型事故：

```
旧版本读写 key : campaign:{accountId}:list
新版本读 key : campaign:v2:{accountId}:list
写入仍然写旧 key
```

结果：读永远 miss，DB 回源升高。

处置：

- 回滚 key 规则变更，或同时兼容新老 key。
- 给缓存写入失败加报警。
- key 生成逻辑收敛到统一方法，避免读写两套拼接。

第三步：从 Redis 数据层看过期、淘汰和容量

```
INFO stats
INFO memory
INFO keyspace
CONFIG GET maxmemory
CONFIG GET maxmemory-policy
```

重点看：

- `expired_keys` 是否突增：可能 TTL 过短或集中过期。
- `evicted_keys` 是否突增：可能内存打满触发淘汰。
- `used_memory` 是否接近 `maxmemory`。
- keyspace 中 key 数量是否明显下降。
- 淘汰策略是否误伤热点，例如 `allkeys-lru` 下大量冷数据污染缓存。

处置：

- TTL 加随机抖动，避免同一时刻大量过期。
- 对核心热点 key 设置更合理 TTL 或预热策略。

- 清理低价值缓存，避免污染。
- 扩容 Redis 或拆分实例。

第四步：从业务流量看是不是访问模式变了

- 活动、新用户、新广告账户、新地区流量进入，冷数据访问增加。
- 爬虫、批处理、后台任务扫描大量冷数据。
- 搜索、分页深翻、报表类请求把大量低频数据写入缓存。
- 某些接口参数过于离散，导致缓存 key 数量暴涨但复用率低。

处置：

- 冷数据不回填缓存，或设置短 TTL。
- 后台任务禁用缓存回填，避免污染在线缓存。
- 搜索/分页类缓存做准入控制，只缓存高频页或核心条件。
- 对异常来源限流。

第五步：从缓存回填链路看是否写不进去

- DB 查询成功后是否真的写 Redis。
- Redis 写是否超时、被熔断、被降级。
- 序列化失败、value 过大、pipeline 部分失败是否被吞掉。
- 写缓存 TTL 是否设置错误，例如从 1 小时变成 1 秒。

处置：

- 回填失败必须打日志和指标。
- pipeline 批量写要检查每个结果。
- TTL 统一配置，避免魔法数字散落。
- 对核心 key 做主动预热和定时巡检。

第六步：从 Redis 架构事件看

- Cluster 是否发生故障切换，导致短期读到空节点或客户端路由异常。
- 主从延迟是否导致读副本读不到刚写入的数据。
- 是否误执行批量删除、flush、迁移脚本。
- 是否某些 slot 迁移期间客户端处理 MOVED/ASK 异常。

处置：

- 检查 Redis 变更记录和运维操作。
- 客户端 SDK 使用成熟 Cluster 模式。
- 读副本场景关注复制延迟，强一致读走主库或降级策略。

生产止血顺序

1. 先确认 DB 是否被打高，必要时立即限流和降级，保护数据库。
2. 定位下降范围：全局、接口、key 前缀、节点。

3. 查同时间发布和配置变更，尤其是 key 规则、TTL、缓存回填。
4. 查 Redis `expired_keys/evicted_keys/used_memory/keyspace`。
5. 对热点数据临时预热，对冷流量和批任务限流。
6. 修复根因后分批预热，避免恢复瞬间再次打爆 DB。

面试追问：命中率一定低是坏事吗？

不一定。新用户、新商品、冷启动、活动流量变化都可能导致短期命中率低。要结合 DB 压力、接口延迟和业务场景判断。

深入：Redis 故障时怎么保护数据库？

结论先行：

Redis 故障时最怕流量直接打爆数据库，要通过熔断、限流、本地兜底、降级和隔离保护 DB。

分层展开：

入口限流

- 对高 QPS 接口限流。
- 对非核心请求快速失败。

本地兜底

- 返回本地缓存旧值。
- 返回默认值或静态数据。
- 只保核心字段。

回源保护

- 控制 DB 并发。
- 请求合并。
- 热点 key 互斥回源。

故障恢复

- Redis 恢复后分批预热。
- 避免瞬间全量回填。
- 复盘命中率和容量。

面试追问：Redis 恢复后为什么还要预热？

如果缓存是空的，所有请求仍会回源 DB。恢复后要分批加载热点数据，让命中率逐步恢复，避免恢复阶段二次雪崩。

深入：Redis 如何做灰度和多环境隔离？

结论先行：

Redis 灰度隔离主要通过 key 前缀、独立实例、逻辑库、权限账号和流量路由实现，生产核心业务更推荐实例或集群级隔离。

分层展开：

key 前缀隔离

- 简单、成本低。
- 适合小规模灰度。
- 风险是误删和资源竞争。

逻辑库隔离

- 单机 Redis 支持多个 DB。
- Cluster 不支持多 DB。
- 生产大规模不推荐依赖逻辑库。

实例隔离

- 不同环境或业务使用不同 Redis。
- 资源和故障边界清晰。
- 成本更高。

权限隔离

- Redis ACL 控制命令和 key pattern。
- 降低误操作风险。

面试追问：为什么大厂强调资源隔离？

因为共享 Redis 很容易出现某个业务的大 key、热 key、慢命令或内存打满影响其他业务。隔离可以把故障限制在业务边界内。

十、业务架构与项目实战

45. 高并发商品详情缓存如何设计？

```
本地缓存 -> Redis -> 商品 DB
                -> 价格/库存独立强一致链路
```

- 详情使用 Cache Aside，TTL 加随机值。
- 热点提前预热，逻辑过期配合互斥重建。
- 价格库存与低频描述拆开，避免大对象和错误一致性假设。
- DB 更新后通过 Outbox/CDC 删除 Redis 和本地缓存。
- Redis 故障时限回源并返回可接受旧详情，交易前再校验价格库存。
- 监控命中率、P99、回源 QPS、重建次数和版本落后。

46. 排行榜如何设计？

使用 ZSet：Member 为业务实体，Score 为排序值。并列规则可组合时间或二级排序字段；按周期拆 Key 并设置 TTL，历史榜单异步落库。

大榜单要限制 `ZRANGE` 范围，避免全量读取；高频更新可聚合后批量写，最终结果通过数据库或日志对账。

47. 广告 API 滑动窗口限流如何设计？

- Key 按平台、账号、接口和窗口维度组织。
- ZSet Score 为毫秒时间，Member 为唯一请求 ID。
- Lua 原子执行清理窗口、计数、判断、写入和设置 TTL。
- 同时支持秒、分钟、日多级窗口。
- Redis 异常时根据外部平台风险选择保守拒绝或本地降级。
- 记录拒绝原因、剩余额度、脚本耗时和 Key 基数。

48. 广告投放项目中的分布式锁如何表达？

我们用锁避免同一广告任务被多个实例同时发布。锁使用任务 ID 作为 Key、实例请求 ID 作为 Token，`SET NX PX` 加锁，Lua 比较 Token 后解锁；任务状态还通过数据库/Mongo 条件更新保证只有合法状态能进入 publishing。调用媒体 API 成功但本地更新失败时进入 unknown 并反查平台，不能依赖锁重试。也就是说，Redis 锁降低并发冲突，状态机、幂等 Key 和平台反查才保证业务正确性。

49. 模板配置缓存如何避免长期脏读？

- 数据库为权威源，Redis 缓存按端、版本、场景组合后的模板。
- 配置发布生成递增版本，缓存 Key 或 Value 带版本指纹。
- 发布事件使 Redis 和本地缓存失效；读取发现版本落后主动回源。
- TTL 兜底，灰度环境与生产隔离。
- 模板允许短暂最终一致，但发布回滚和客户端兼容必须保留旧版本。

50. 如何用两分钟回答 Redis 面试题？

我先明确 Redis 在该场景是缓存、共享状态还是并发控制，以及数据库是否为权威源。缓存场景采用 Cache Aside，写库后可删除，TTL 和版本号控制不一致窗口；热点通过预热、本地缓存和互斥重建保护，穿透通过参数校验、空值和 Bloom Filter 处理。数据结构按访问模式选择并限制 Big Key，原子组合用短 Lua。架构按 QPS、内存和带宽压测选择主从、Sentinel 或 Cluster，并预留单节点故障余量。线上监控客户端延迟、命中率、CPU、内存、热 Key、慢命令和复制状态；Redis 故障时限回源、返回可接受旧值并分批恢复。管理经验只体现风险识别和协作推进，我的定位仍是负责业务架构与交付的资深后端 IC。

补充模块：Redis 实战设计：会用、用好、知道边界

这一章回答“怎么用 Redis 解决业务问题”。重点是数据结构选择、key 设计、TTL、一致性、原子性、失败兜底。

深入：如何设计一个高并发商品详情缓存？

结论先行：

商品详情缓存要按“多级缓存 + 热点保护 + 最终一致 + 降级兜底”设计，不能只写一个 `get/set`。

分层展开：

读链路

- 本地缓存 Caffeine/Guava。
- Redis 分布式缓存。
- 数据库或商品服务。
- 本地缓存 TTL 短，Redis TTL 稍长。

写链路

- 商品变更先落库。
- 发 MQ 或 binlog 事件。
- 异步删除 Redis 和本地缓存。
- 删除失败重试。

热点保护

- 热点商品预热。
- 逻辑过期或互斥锁重建。
- 大促前固定热点 key 不设置短 TTL。
- 对 DB 回源做并发限制。

降级策略

- Redis 故障时返回本地旧值。
- 非核心字段可降级为空。
- 保护数据库连接池和线程池。

监控指标

- 缓存命中率。
- 回源 QPS。
- 热 key 分布。
- Redis 延迟和慢查询。
- DB 查询耗时。

面试追问：商品价格库存能不能和详情一起缓存？

一般不建议混在一个大缓存里。价格、库存变化频率和一致性要求更高，应该拆分缓存或直接走专门服务，避免一个字段频繁变化导致整个详情缓存频繁失效。

深入：如何用 Redis 设计排行榜？

结论先行：

排行榜优先用 ZSet，score 表示排序分数，member 表示用户或对象 ID，配合定时归档和冷热拆分解决容量问题。

分层展开：

基础操作

- 更新分数：`ZADD rank score userId`。
- 增加分数：`ZINCRBY rank delta userId`。
- 查前 N 名：`ZREVRANGE rank 0 N WITHSCORES`。
- 查个人排名：`ZREVRANK rank userId`。

时间维度

- 日榜：`rank:daily:2026-07-08`。
- 周榜：`rank:weekly:2026-W28`。
- 总榜：`rank:total`。

容量治理

- 只保留 top N 或活跃用户。
- 历史榜单落库归档。
- 大榜单分业务、地区、时间拆分。

一致性

- 分数变更可以先写 Redis，再异步落库。
- 或写业务事件，由消费者更新 Redis 和 DB。
- 要保证事件幂等。

面试追问：排行榜同分怎么排序？

Redis ZSet score 相同时按 member 字典序排序。如果需要按时间先后排序，可以把 score 设计成复合分数，或在 member/额外结构中记录时间并由业务层处理。

深入：你负责一个高并发系统，Redis 架构怎么设计？

结论先行：

我会从业务读写模型出发，设计“多级缓存 + Redis Cluster + 主从高可用 + 一致性补偿 + 热点治理 + 监控预案”的完整体系。

分层展开：

业务分层

- 核心强一致数据不放或少放 Redis。
- 高频读、可重建数据放 Redis。
- 热点数据加本地缓存。

部署架构

- Cluster 分片扩展容量。
- 每个 master 配 replica。
- 跨机器、跨机架部署。
- 客户端使用成熟 Cluster SDK。

一致性设计

- Cache Aside。
- 写 DB 后删缓存。
- MQ/binlog 异步补偿。
- TTL 兜底。

稳定性设计

- 热 key 识别和本地缓存。
- 大 key 拆分。
- 限流、熔断、降级。
- Redis 故障时保护 DB。

可观测性

- 命中率。
- QPS、延迟、慢查询。
- CPU、内存、碎片率。
- 主从延迟。
- keyspace 和大 key 扫描。

面试追问：这个方案最大的风险是什么？

最大风险通常不是 Redis 本身，而是缓存一致性、热点流量和故障时流量回源。架构上要默认 Redis 会慢、会挂、会丢部分缓存，然后让系统仍能保护核心链路。

补充模块：项目实战表达：把 Redis 讲成项目能力

这一章用于简历和项目深挖。回答时一定从业务问题开始，落到数据模型、链路、风险、监控和收益。

深入：枫叶互动项目里 Redis 用在哪些核心场景？

可以这样答：

在枫叶互动广告投放平台里，Redis 不是单纯做缓存，主要有三类核心场景：

1. **分布式锁**：广告数据拉取、素材同步、自动投放、报表预警等 Cron 任务需要保证多实例下单实例执行，避免重复创建广告、重复写入和重复通知。
2. **媒体 API 限流**：Facebook/TikTok/Snapchat Marketing API 有账号级、应用级、接口级限流，所以使用 Redis ZSET + Lua 做全局滑动窗口限流。
3. **缓存版本指纹**：广告账号下 Campaign/AdGroup/Ad 配置频繁变更，通过账号级版本号生成缓存指纹，实现多账号查询缓存精准失效。

面试官追问：为什么这几个场景都适合 Redis？

- 分布式锁需要跨实例共享状态，Redis 的 `SET NX EX` 和 Lua 能比较轻量地解决。
- 限流需要低延迟计数和过期窗口，ZSET 天然适合按时间窗口清理。
- 缓存版本需要高频读写、小数据量、低延迟，Redis 比 DB 更适合。

深入：如何讲清楚 Redis 分布式锁的项目细节？

业务背景：

广告数据同步、素材同步、自动投放策略执行通常由定时任务触发。如果服务多副本部署，多个实例同时执行会导致重复调用媒体 API、重复写 MongoDB、重复发送通知，甚至重复创建广告。

方案：

- 使用 Redis 分布式锁控制任务单实例执行。
- 加锁时设置 TTL，避免实例宕机导致死锁。
- 长任务开启 watchdog，按锁过期时间的一定比例自动续期。
- 加锁失败时做随机抖动重试，避免多个实例同时竞争。
- 任务退出时释放锁，panic 场景也要 recover 并释放资源。

风险和改进：

- 释放锁必须校验 token，避免误删别人的锁。
- watchdog 只能续自己的锁，不能无脑 `EXPIRE`。
- 任务要有业务幂等，因为锁是稳定性手段，不是业务正确性的唯一保障。

一句话总结：

我会把 Redis 锁定位成“降低并发重复执行概率的工程手段”，最终还要靠任务状态机、唯一键、幂等消费和补偿机制兜底。

深入：如何讲 Redis ZSET + Lua 滑动窗口限流？

业务背景：

广告创建和报表拉取会调用媒体平台 API。平台一旦判定破频，可能会返回限流错误、进入冷却窗口，影响运营创建广告和数据回流。

核心设计：

- key 按平台、账号、接口维度拆分，例如 `rate:meta:account:{id}:create_ad`。

- ZSET 的 score 使用请求时间戳。
- 每次请求先删除窗口外数据，再统计窗口内请求数。
- 未超过阈值则写入当前请求并放行。
- 全部操作通过 Lua 保证原子性。

重要细节：

ZSET member 不能只用秒级时间戳。因为同一秒内多个请求 member 相同会被覆盖，导致统计值偏小。可以使用微秒时间戳，或者 `timestamp:random/requestID` 作为 member。

降级策略：

- 触发平台限流后，临时降低账号 QPS。
- 对写接口更保守，对读接口分级处理。
- 任务状态保留为可重试，不直接丢弃。
- 告警中展示平台错误码、账号、接口、请求量和限流命中数。

深入：缓存版本指纹怎么讲？

业务背景：

广告账号下 Campaign、AdGroup、Ad 配置经常由同步任务、广告创建任务、状态拉取任务更新。如果每次变更都直接删除大范围缓存，容易误删、成本高，也不利于多账号组合查询。

方案：

- 每个账号维护一个配置版本号，例如 `cfgver:{channel}:{account_id}`。
- 账号下关键配置变化时自增版本。
- 查询多账号数据时，批量读取账号版本号。
- 将 `accountID=version` 排序拼接后生成 hash，作为缓存 key 的一部分。
- 任一账号版本变化，组合查询缓存自然失效。

为什么用 pipeline GET，不直接 MGET？

在 Redis Cluster 下，如果多个 key 不在同一个 slot，`MGET` 可能受限；pipeline GET 更容易兼容跨 slot 场景，也能减少网络往返。

面试表达亮点：

这类方案比“更新就删缓存”更精细，适合多账号、多条件组合查询，能降低缓存误删和缓存雪崩风险。

深入：如何把知乎动态模板项目里的 Redis 讲出来？

业务背景：

知乎动态化模板平台需要按端类型、App 版本、场景、内容类型、AB 实验等条件匹配模板。模板更新后，客户端不能长期读取旧模板。

方案：

- 模板数据以 DB 为准，Redis 缓存热点模板和匹配结果。
- 更新模板后先删缓存，再更新 DB，再延迟删除一次。

- 缓存设置合理过期时间，避免删除失败导致长期脏读。
- 客户端侧有本地预置和业务接口兜底，服务端缓存异常时不影响基础体验。

追问：延迟双删一定可靠吗？

不绝对可靠。它解决的是常见并发读写下的脏缓存窗口，不能保证强一致。如果要求更强，可以用版本号、binlog 订阅、消息通知或读请求校验版本。模板配置这类场景通常允许短时间最终一致，所以延迟双删 + TTL 是性价比较高的方案。

背景：

业务量、痛点、原方案问题。

方案：

数据结构、key 设计、TTL、一致性、降级。

难点：

热点、大 key、并发重建、删除失败、故障恢复。

结果：

用数字说话：命中率、QPS、延迟、成本、稳定性。

反思：

方案边界和后续优化。

深入：你在项目里如何体现 Redis 使用的工程深度？

结论先行：

不要只说“用了 Redis 做缓存”，要讲清楚业务问题、数据模型、读写链路、一致性策略、异常处理、监控指标和最终收益。

分层展开：

业务背景

- 哪个接口 QPS 高。
- 原来瓶颈在哪里。
- 为什么数据库扛不住。

技术方案

- 选用什么数据结构。
- key 如何设计。
- TTL 如何设置。
- 是否多级缓存。

一致性

- 写 DB 后删缓存。
- MQ 或 binlog 补偿。
- 删除失败如何重试。
- 可接受的不一致窗口是多少。

稳定性

- 如何防穿透、击穿、雪崩。
- 如何处理热 key、大 key。
- Redis 故障如何降级。

结果量化

- 命中率提升。
- DB QPS 降低。
- P99 延迟降低。
- 故障次数下降。

面试追问：能不能举一个简短项目表达？

可以这样说：我在商品详情页引入 Redis + 本地缓存，详情主数据采用 Cache Aside，写库后通过 MQ 删除 Redis 和本地缓存；热点商品提前预热，TTL 加随机值，价格库存拆成独立缓存；上线后 Redis 命中率稳定在 95% 以上，DB 查询 QPS 降低约 80%，大促期间通过逻辑过期和回源互斥避免了热点击穿。

技术 | MySQL 数据库面试题

面向 9 年以上 Golang Web 后端业务岗。全文按照“定位与建模 → SQL 执行 → InnoDB 与索引 → 事务并发 → 日志恢复 → 复制高可用 → 性能容量 → 治理与 Go 工程 → 项目实战”展开。

核心回答采用：结论先行 → 核心机制 → 完整流程 → 工程取舍 → 生产实践 → 面试追问。

一、MySQL 学习地图与资深回答框架

1. MySQL 的知识图谱主线是什么？

业务场景与数据模型

- Client / Connector / Parser / Optimizer / Executor
- InnoDB Page / Buffer Pool / B+Tree
- Transaction / MVCC / Lock
- Undo / Redo / Binlog / Crash Recovery
- Replication / HA / Backup / PITR
- SQL 性能 / 容量 / 分片迁移
- 治理 / 安全 / Go database/sql
- 业务架构与项目表达

结论先行： MySQL 面试的主线是“数据如何正确地写入、并发地访问、可靠地恢复和持续地扩展”，索引只是其中一环。

2. 资深后端如何回答 MySQL 题？

题型	回答顺序
建模	业务实体 → 查询/写入模式 → 约束 → 生命周期 → 扩展
SQL	执行链路 → 扫描量 → 索引/回表/排序 → 实测
并发	事务边界 → 隔离/MVCC/锁 → 幂等 → 重试补偿
高可用	故障模型 → 复制一致性 → 切换 → RPO/RTO → 对账
排障	影响面 → SQL/锁/连接/IO/复制证据 → 止血 → 根因

高级岗位要从正确性开始，再谈性能；能量化容量、解释异常路径并给出回滚和恢复证据。

关联资料与原有定位

面向 9 年+ Golang Web 后端业务岗。

回答重点：数据建模、索引、事务、锁、SQL 优化、主从复制、分库分表、线上治理和项目表达。

补充模块：MySQL 面试思想方法论篇：先建立结构化回答框架

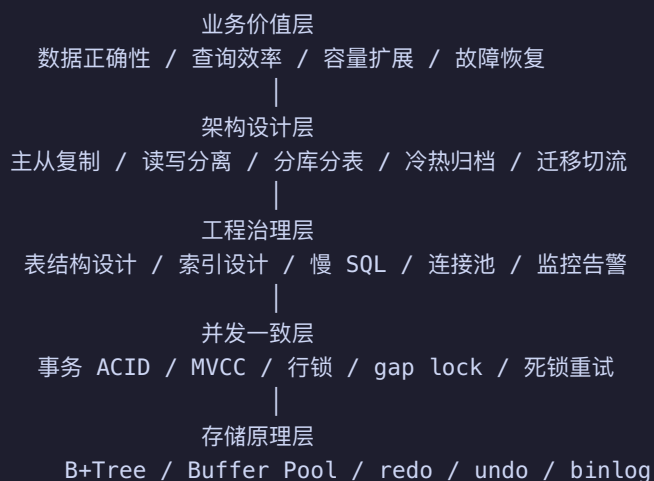
本模块回答“怎么学、怎么答、怎么临场组织”。MySQL 面试不要散点背诵，要围绕数据正确性、查询性能、并发控制、容量扩展和故障恢复建立金字塔。

补充方法：复习地图

面试方法论 -> 当前能力全景 -> 核心原理 -> 实战设计 -> 线上治理 -> 项目表达

补充方法：资深后端 MySQL 金字塔学习框架

顶层先回答“为什么选 MySQL、如何保证正确性、如何扛住增长”；底层再落到索引、事务、锁、日志和执行计划。



资深后端必须掌握的 8 个问题域：

问题域	必须会什么	面试高频追问	项目表达落点
表结构设计	字段类型、主键、唯一键、范式/反范式、冷热字段拆分	为什么不用 UUID 做主键？金额怎么存？JSON 字段能不能用？	投放工单、广告任务、配额流水、审计日志
索引设计	B+Tree、聚簇/二级索引、联合索引、覆盖索引、索引失效	联合索引顺序怎么定？为什么回表慢？	列表查询、状态筛选、报表查询、深分页

问题域	必须会什么	面试高频追问	项目表达落点
SQL 执行	解析、优化器、执行计划、回表、排序、临时表	EXPLAIN 怎么看？Using filesort 一定坏吗？	慢 SQL 排查和上线 SQL 审核
事务一致性	ACID、隔离级别、MVCC、undo/redo、两阶段提交	RR 怎么避免幻读？binlog/redo 为什么两阶段提交？	工单分配、库存扣减、状态流转
锁与并发	行锁、gap lock、next-key lock、死锁、锁等待	为什么没走索引会锁很多行？死锁怎么治理？	并发更新、任务抢占、热点行治理
高可用复制	binlog、relay log、主从延迟、半同步、读写分离	写完读不到怎么办？主从延迟怎么处理？	读扩展、报表读库、关键链路读主
容量扩展	分库分表、分片键、全局 ID、迁移扩容、归档	什么时候分表？分表后怎么查？	广告日志、事件流水、历史报表
线上治理	慢 SQL、连接池、Online DDL、备份恢复、监控告警	连接池满了怎么办？大表加字段怎么做？	事故排查、性能优化、灰度变更

金字塔表达模板：

结论：我会先保证数据正确性，再优化性能和扩展性。

第一层：数据模型是否合理，主键、唯一键、字段类型和事务边界是否清晰。

第二层：核心查询是否有合适索引，执行计划扫描量、回表、排序是否可控。

第三层：并发写是否通过事务、条件更新、锁、幂等和重试保证正确。

第四层：容量增长后通过读写分离、冷热拆分、分库分表、归档和迁移治理。

第五层：线上通过慢日志、连接池指标、锁等待、主从延迟、DDL 风险和告警闭环。

一张总图：MySQL 在后端系统中的位置

```

flowchart TD
    A["业务请求"] --> B["服务层：参数校验/幂等/事务边界"]
    B --> C["MySQL：事实数据源"]
    C --> D["事务：ACID/MVCC/锁"]
    C --> E["索引：B+Tree/执行计划"]
    C --> F["日志：redo/undo/binlog"]
    F --> G["主从复制/备份恢复/CDC"]
    C --> H["衍生读模型：Redis/ES/数仓"]
    B --> I["MQ/Outbox：最终一致补偿"]
  
```

补充方法：资深后端如何回答 MySQL 问题？

结论先行：

MySQL 面试要围绕“数据正确性、查询性能、并发控制、容量扩展、故障恢复”展开，而不是只背 SQL。

分层展开：

基础层

- InnoDB、B+Tree、聚簇索引、二级索引。
- 事务、MVCC、锁、隔离级别。

性能层

- SQL 执行计划。
- 索引设计。
- 慢 SQL 治理。
- 分页、排序、回表、覆盖索引。

架构层

- 主从复制、读写分离。
- 分库分表。
- 冷热数据拆分。
- 数据迁移和扩容。

工程层

- 连接池、超时、重试、幂等。
- 数据一致性。
- 监控告警和故障预案。

面试追问：数据库问题最重要的意识是什么？

数据正确性优先于性能优化。性能可以扩容和优化，数据错了往往只能靠补偿、修复和审计。

补充方法：MySQL 题目通用决策框架

结论先行：

遇到 MySQL 题，先判断它属于哪类问题：建模、查询、并发、扩展、恢复、治理。不要一上来只回答“加索引”。

题目问表设计？

实体关系 -> 字段类型 -> 主键/唯一键 -> 查询入口 -> 生命周期

题目问查询慢？

慢日志 -> EXPLAIN -> 扫描行数 -> 回表/排序/临时表 -> 索引/SQL/架构优化

题目问并发写？

事务边界 -> 条件更新 -> 锁范围 -> 幂等 -> 死锁重试 -> 补偿

题目问容量增长？

读写分离 -> 冷热拆分 -> 分库分表 -> 全局 ID -> 迁移校验

题目问线上事故？

先止血 -> 定位瓶颈 -> 修复 -> 复盘制度化

补充方法：MySQL 速记清单与回答模板

本模块用于临考前快速复盘，帮助把零散知识压缩成可输出的回答。

模板 1：原理类题

结论先行：先说这个机制解决什么问题。
核心结构：再说依赖哪些结构，例如 B+Tree、ReadView、undo/redo、锁。
执行过程：按步骤讲一遍链路。
适用边界：说明它解决不了什么。
工程案例：最后落到慢 SQL、库存扣减、状态流转、主从延迟等场景。

模板 2：场景设计题

业务目标 -> 数据模型 -> 事务边界 -> 索引设计 -> 并发控制 -> 容量扩展 -> 监控补偿

模板 3：线上排障题

先止血：限流、降级、暂停任务、切读主、摘除异常从库。
再定位：慢日志、EXPLAIN、锁等待、连接池、主从延迟、CPU/IO。
再修复：索引、SQL 改写、事务缩短、批量拆分、DDL 灰度。
再复盘：SQL 审核、压测、监控、告警、容量规划。

高频速记清单

方向	必背关键词
表设计	主键短稳递增、唯一约束兜底、金额整数、冷热字段拆分
索引	B+Tree、聚簇索引、二级索引、回表、覆盖索引、最左前缀
SQL	EXPLAIN、rows、filtered、Using index、filesort、temporary
事务	ACID、undo、redo、MVCC、ReadView、隔离级别
锁	record lock、gap lock、next-key lock、死锁、锁等待
日志	redo 崩溃恢复、undo 回滚/MVCC、binlog 复制/恢复、两阶段提交
架构	主从复制、读写分离、主从延迟、分库分表、全局 ID
治理	慢 SQL、连接池、Online DDL、备份恢复、监控告警

二、使用场景、数据建模与能力边界

3. 什么数据适合放 MySQL?

结论先行： 结构稳定、关系明确、需要事务、约束和多条件查询的核心业务数据适合 MySQL，如订单、支付单、工单、账户、权限和审计流水。

不适合直接承载海量原始日志、频繁变化的任意文档结构、全文搜索和低价值超高频计数；这些可以落到日志、文档、搜索或分析系统，MySQL 保留权威状态和索引关系。

4. 表结构设计从哪些维度开始?

业务对象

- > 主键与业务唯一键
- > 查询过滤、排序、分页、Join
- > 并发更新和事务边界
- > 字段类型、Null、默认值
- > 索引和约束
- > 冷热生命周期、归档和分片

- 金额使用定点小数或最小货币单位整数，避免浮点误差。
- 状态使用明确枚举和值域约束，配合状态流水审计。
- 幂等号、外部流水号使用唯一索引兜底。
- 高频查询字段避免藏在 JSON；JSON 适合低频扩展和原始快照。
- 大字段和低频字段可垂直拆分，减少热页和 Buffer Pool 污染。

5. 主键如何选择?

InnoDB 主键决定聚簇索引组织，也会存入每个二级索引叶子。主键应短、稳定、非空、尽量递增且无业务可变语义。

- 自增 ID：局部性好，单库简单；分布式生成和暴露趋势要考虑。
- 雪花 ID：跨库生成快，但较长且需处理时钟回拨。
- 号段：数据库批量分配，性能好，需高可用发号服务。
- 随机 UUID：写入离散、索引大、页分裂多；若必须使用可考虑更紧凑、有序表示。

6. 范式和反范式如何取舍?

范式减少冗余和更新异常，适合权威关系；反范式通过冗余减少 Join、稳定查询，但增加同步和修复成本。

资深设计先保留一个权威字段，再为高频读建立可重建冗余，并明确更新事件、版本、补偿和对账，不做无责任人的双写。

7. 状态和流水为什么要分开？

主表保存当前状态，流水表保存不可变变化事实，兼顾快速查询、审计、补偿和问题回放。

```
order.current_status = paid
order_status_log: created -> paying -> paid
```

状态更新与关键流水应在同一本地事务；非关键通知可通过 Outbox 异步派生。

补充模块：MySQL 当前能力全景：数据建模、索引、SQL、复制、扩展与连接池

本模块回答“MySQL 能做什么、怎么正确使用”。重点是中介总览：先知道能力边界，再进入底层原理。

深入：数据建模与表结构设计怎么做？

结论先行：

表结构设计要从业务实体、查询模式、并发写入、一致性边界和数据生命周期出发。资深后端不会只看“字段够不够”，还会考虑主键、唯一约束、索引、状态机、审计、冷热拆分和未来扩展。

设计步骤：

业务对象 -> 事务边界 -> 查询入口 -> 字段类型 -> 约束索引 -> 生命周期 -> 扩展治理

这五个维度具体指什么？

表结构设计不是先想字段，而是先回答五个问题：

数据是什么 -> 怎么被查 -> 怎么被写 -> 哪些必须一起正确 -> 这批数据会活多久

维度	核心问题	主要影响	例子
业务实体	这张表描述哪个业务对象？对象之间是什么关系？	字段、主键、外键/逻辑关联、主表/明细表	广告账户、Campaign、AdGroup、素材、投放报表
查询模式	线上最常怎么查？按什么过滤、排序、分页、聚合？	联合索引、覆盖索引、冗余字段、汇总表	按 <code>account_id + status</code> 查计划列表，并按 <code>updated_at</code> 倒序
并发写入	多个请求会不会同时改同一行？写入 QPS 多高？有没有热点行？	行锁、乐观锁、条件更新、流水表、MQ 聚合、分片	并发修改预算、状态；高频更新点击数、消耗
一致性边界	哪些数据必须在一个事务里一起成功？哪些允许最终一致？	事务范围、唯一约束、幂等表、补偿任务、Outbox/MQ	预算变更和操作日志要一致；报表统计可延迟

维度	核心问题	主要影响	例子
数据 生命 周期	数据什么时候产生？多久高频访问？何时变冷、归档、删除？	分区、分表、冷热拆分、归档、TTL、历史表	近 7 天报表热查，1 年前报表归档

它们会交叉，但关注点不同：

- 业务实体决定“表长什么样”。
- 查询模式决定“索引怎么建”。
- 并发写入决定“怎么避免锁冲突和热点行”。
- 一致性边界决定“事务、约束和补偿怎么设计”。
- 数据生命周期决定“分区、归档和清理怎么做”。

完整例子：广告计划 Campaign 表

业务实体：

Campaign 是广告投放的计划层级，属于某个广告账户，有名称、状态、预算、起止时间。

查询模式：

投手经常按 `account_id + status` 查列表，并按 `updated_at` 倒序分页。

并发写入：

预算、状态可能被多人或自动规则同时修改，需要避免并发覆盖。

一致性边界：

修改预算时要同时写预算变更日志；状态流转要保证不能从已删除状态回到运行中。

数据生命周期：

运行中计划高频访问；结束/删除很久的计划可以归档或减少索引维护。

表结构可以这样推导：

```
create table campaign (
  id bigint primary key,
  account_id bigint not null,
  name varchar(128) not null,
  status tinyint not null,
  budget_cent bigint not null,
  version int not null default 0,
  start_time datetime,
  end_time datetime,
  created_at datetime not null,
  updated_at datetime not null,
  key idx_account_status_updated(account_id, status, updated_at)
);
```

设计解释：

- `id` 来自业务实体，作为短、稳、趋势递增的聚簇主键。
- `idx_account_status_updated` 来自查询模式，支撑账户下按状态筛选和更新时间排序。

- `version` 来自并发写入，用于乐观锁或并发覆盖检测。
- 预算变更日志表来自一致性边界，预算修改和日志写入应放在同一事务。
- 历史归档策略来自数据生命周期，避免历史计划长期拖累热表。

1. 先找业务实体和聚合根

- 订单系统：订单主表、订单明细、支付单、履约单、状态流转。
- 广告投放：广告任务、广告账户、campaign/adgroup/ad、素材、创建日志。
- 工单分配：工单主表、组统计、轮次状态、历史配对、分配日志。

2. 字段类型要保守

- 金额用整数分，不用 float/double。
- 状态用 tinyint/smallint 或 varchar 枚举，但要统一规范。
- 时间字段统一 `created_at`、`updated_at`，必要时加 `deleted_at`。
- 大文本、JSON、扩展字段不要污染高频列表查询，可拆详情表。
- 字符集、排序规则统一，避免关联时隐式转换。

3. 主键设计

- InnoDB 主键就是聚簇索引，叶子节点存整行。
- 主键尽量短、稳定、趋势递增。
- 不建议 UUID 直接做主键：长、随机、二级索引膨胀、页分裂更多。
- 分库分表场景可用雪花 ID 或号段 ID。

4. 约束比代码判断更可靠

- 幂等号、业务流水号、外部平台 ID 要用唯一索引兜底。
- 状态流转用条件更新兜底，例如 `where id=? and status='INIT'`。
- 软删除表的唯一约束要考虑 `deleted_at` 或业务唯一键设计。

5. 宽表和拆表取舍

方案	优点	缺点	适合场景
单表	查询简单，事务边界清晰	宽字段影响缓存命中，DDL 成本高	中小规模、字段稳定
主表 + 详情表	高频字段轻，列表查询快	查询详情多一次 join/查询	大文本、JSON、低频字段
主表 + 流水表	当前状态和历史轨迹分离	写入链路多一步	状态机、审计、补偿
冷热拆分	热表小，性能稳定	归档和查询复杂	历史数据很大

示例：广告创建任务表

```
create table ad_create_task (
  id bigint primary key,
```

```
request_id varchar(64) not null,
account_id varchar(64) not null,
platform varchar(32) not null,
status tinyint not null,
retry_count int not null default 0,
created_at datetime not null,
updated_at datetime not null,
unique key uk_request_id(request_id),
key idx_account_status_time(account_id, status, created_at)
);
```

为什么这样设计：

- `id` 做聚簇主键，稳定且短。
- `request_id` 唯一索引防重复提交。
- `account_id + status + created_at` 支撑运营列表和定时任务扫描。
- 状态流转通过条件更新保证并发安全。

面试追问：JSON 字段能不能用？

可以用，但要克制。JSON 适合低频扩展属性、配置快照、第三方原始 payload；不适合作为高频过滤、排序、关联字段。核心查询条件应拆成普通列并建索引。

三、SQL 执行链路、优化器与执行计划

8. 一条 SQL 如何从客户端执行到 InnoDB？

```
Go database/sql
-> 连接/认证/会话
-> Parser：词法、语法和语义检查
-> Optimizer：统计信息、索引、Join 顺序和成本
-> Executor：驱动执行计划
-> InnoDB：Buffer Pool、索引、MVCC、锁和日志
-> 返回结果
```

工程判断： SQL 慢可能发生在连接池等待、锁等待、优化器选错计划、存储 IO、结果传输或客户端处理，不能只看语句执行时间。

9. 优化器如何选择执行计划？

优化器根据统计信息、基数、选择性、访问成本、Join 顺序、排序和临时表成本选择计划。它做成本估算，不保证每次都选到真实最优方案。

数据分布突变、统计信息失真、参数差异和大量回表都可能导致计划变化。修复应先用真实参数和 `EXPLAIN ANALYZE` / 执行证据验证，再考虑更新统计、索引重构、SQL 改写或谨慎 Hint。

10. EXPLAIN 应该怎么看？

顺序建议：表和 Join 顺序 → `type` → `key/key_len` → `rows/filtered` → `Extra`。

信号	判断
<code>const/eq_ref/ref/range</code>	通常比全扫描好，但仍看返回比例
<code>index</code>	可能扫描整棵索引，不等于高效
<code>ALL</code>	大表危险，小表未必有问题
<code>Using index</code>	覆盖索引，减少回表
<code>Using filesort</code>	额外排序，不一定落盘但需看规模
<code>Using temporary</code>	中间结果，聚合/排序可能放大成本

估算行数必须结合实际执行时间、扫描行、返回行、锁等待和数据分布。

11. 为什么深分页慢？

`LIMIT offset,size` 通常需要扫描并丢弃前面记录，Offset 越大浪费越多。可使用 Seek 分页：以上一页最后的稳定排序键继续查询。

```
SELECT id, status, created_at
FROM orders
WHERE tenant_id = ?
  AND (created_at, id) < (?, ?)
ORDER BY created_at DESC, id DESC
LIMIT 50;
```

排序字段要稳定且有匹配联合索引；后台任意跳页和用户连续翻页是不同需求。

12. Join、子查询和大结果集如何优化？

- 先缩小驱动集，让被驱动表 Join Key 有索引。
- 避免循环查询形成 N+1，使用批量查询或合理 Join。
- 大结果流式读取并设置超时，不一次加载到 Go 内存。
- 聚合报表可预聚合、异步化或使用分析系统。
- 关注临时表、排序、回表和 network 传输，而非机械拒绝 Join。

深入：一条 SQL 从客户端到 InnoDB 怎么执行？

面试速记

连接器负责建立连接、认证、权限和会话管理；解析器负责词法分析、语法分析和语义检查，判断 SQL 写得是否正确；优化器根据统计信息和成本模型选择索引、JOIN 顺序和执行计划；执行器按照执行计划驱动存储引擎执行，并负责过滤、排序、聚合和结果返回；InnoDB 负责底层数据、索引、事务、锁和日志。

结论先行：

SQL 执行不是“发给数据库就结束”，它会经过连接层、解析器、优化器、执行器、存储引擎，最后落到 Buffer Pool、索引页和日志系统。

```
sequenceDiagram
    participant AppNode as Go 服务
    participant ConnNode as MySQL 连接层
    participant ParserNode as 解析器
    participant OptimizerNode as 优化器
    participant ExecutorNode as 执行器
    participant InnoDBNode as InnoDB
    participant BufferPoolNode as Buffer Pool
    participant LogNode as redo/undo/binlog

    AppNode->>ConnNode: SQL + 参数
    ConnNode->>ParserNode: 词法/语法解析
    ParserNode->>OptimizerNode: 生成候选执行计划
    OptimizerNode->>ExecutorNode: 选择索引和 join 顺序
    ExecutorNode->>InnoDBNode: 按执行计划读取/修改
    InnoDBNode->>BufferPoolNode: 访问数据页/索引页
    InnoDBNode->>LogNode: 写 undo/redo
    ExecutorNode->>LogNode: 提交阶段写 binlog
    ExecutorNode-->>AppNode: 返回结果
```

查询链路重点：

- 优化器根据统计信息、索引、条件选择执行计划。
- **EXPLAIN** 看到的是优化器选择的计划，不是业务“以为”的计划。
- 回表、排序、临时表、扫描行数决定查询是否稳定。

更新链路重点：

- 先写 undo，支持回滚和 MVCC。
- 修改 Buffer Pool 中的数据页。
- 写 redo，保证崩溃恢复。
- 提交时协调 redo 和 binlog，保证复制和恢复一致。

面试追问：为什么同一条 SQL 有时突然变慢？

可能是执行计划变化、统计信息不准、数据分布倾斜、Buffer Pool 未命中、锁等待、主从延迟、连接池排队、磁盘 IO 抖动或临时表变大。资深排查不能只盯索引。

四、InnoDB 存储、Buffer Pool 与索引

13. InnoDB 如何组织数据？

InnoDB 以 Page 为基本读写单位，页组成区和表空间；记录按聚簇 B+Tree 组织。内存通过 Buffer Pool 缓存数据页、索引页和辅助结构。

读取先查 Buffer Pool，未命中再从磁盘加载；更新先修改内存页并生成 Redo，脏页由后台刷盘。这是性能与崩溃恢复的基础。

14. Buffer Pool 如何工作？

- 缓存数据页和索引页，降低随机 IO。
- 使用 LRU 类策略并区分新老区域，降低全表扫描污染。
- 脏页异步刷盘，Checkpoint 推进 Redo 可回收位置。
- Change Buffer 等机制可减少部分非唯一二级索引随机写。

Buffer Pool 不能占满机器内存，还要为连接、排序、OS、备份和监控留空间。命中率下降要结合数据增长、扫描、脏页和磁盘延迟分析。

15. 为什么 InnoDB 使用 B+Tree？

B+Tree 分支多、树高低，节点与数据库页匹配，叶子有序链表适合范围查询；相比二叉树可用更少磁盘随机 IO 定位更多 Key。

哈希适合等值但不自然支持范围和排序；红黑树分支少、树高更高。选择 B+Tree 是磁盘页、范围查询和综合负载的工程权衡。

16. 聚簇索引和二级索引有什么区别？

- 聚簇索引叶子保存完整行，一张表只有一种物理聚簇组织。
- 二级索引叶子保存索引列和主键。
- 二级索引查非覆盖字段需要回主键树，称为回表。
- 主键越长，每个二级索引越大。

覆盖索引减少回表，但索引越宽写放大和空间成本越高，不能为每条 SQL 都堆覆盖索引。

17. 联合索引如何设计？

根据等值过滤、范围、排序和返回列设计，而不是简单按“区分度最高放最左”。

```
等值租户/账号
-> 等值状态
-> 范围时间
-> 稳定排序 ID
-> 必要覆盖列
```

索引 `(tenant_id,status,created_at,id)` 可服务租户状态列表与 Seek 分页。范围列之后的列通常难以继续缩小扫描范围，但仍可能用于索引下推或覆盖。

18. 索引为什么会失效或不被选择？

- 对列做函数、表达式或隐式类型转换。

- 前导模糊匹配。
- 跳过联合索引最左部分。
- **OR** 一侧无法有效索引。
- 选择性差、返回比例高，优化器认为全扫描更便宜。
- 数据分布和统计信息与实际不符。

最终以执行计划和真实耗时为准，“用了索引”不等于快。

19. 页分裂和索引写放大如何产生？

随机主键或索引中间插入可能让目标页空间不足，发生页分裂和更多随机 IO；每次写还需维护所有相关二级索引并生成日志。

治理包括使用有序短主键、减少低价值索引、批量写、控制事务规模和监控页/索引增长。删除大量数据后文件不一定自动缩小，空间回收需单独规划。

深入：为什么 MySQL InnoDB 用 B+Tree 做索引？

结论先行：

InnoDB 使用 B+Tree 是因为它适合磁盘 IO 场景，树高低、范围查询友好、节点可存多个 key，综合性能稳定。

分层展开：

磁盘友好

- B+Tree 每个节点可存多个 key。
- 树高通常较低。
- 一次页读取能加载多个索引项。

范围查询友好

- 叶子节点按 key 有序。
- 叶子节点之间有链表。
- 范围扫描比哈希索引更自然。

稳定性

- 查询、插入、删除复杂度稳定。
- 适合 OLTP 常见等值和范围查询。

面试追问：为什么不用红黑树？

红黑树每个节点通常只存一个 key，树高更高，磁盘随机 IO 更多，不适合数据库页存储模型。

深入：聚簇索引和二级索引有什么区别？

结论先行：

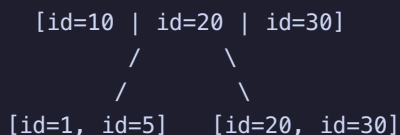
聚簇索引的叶子节点存整行数据，二级索引的叶子节点存主键值；通过二级索引查非覆盖字段通常需要回表。

分层展开：

为什么 InnoDB 主键索引又叫聚簇索引？

因为 InnoDB 的主键索引叶子节点存的不是“数据地址”，而是整行数据本身。也就是说，索引和数据行聚在同一棵 B+Tree 上，所以叫聚簇索引。

InnoDB 主键索引 B+Tree



叶子节点：

[id=1 -> 整行数据]

[id=5 -> 整行数据]

[id=20 -> 整行数据]

[id=30 -> 整行数据]

所以可以记住一句话：

主键索引的叶子节点 = 数据页

通过主键查询时：

```
select * from user where id = 100;
```

执行路径是：

沿主键 B+Tree 查到 id=100 的叶子节点 -> 直接拿到整行数据

二级索引不一样：

二级索引 idx_name

叶子节点：

[name='Tom' -> 主键 id=100]

[name='Jerry' -> 主键 id=200]

如果执行：

```
select * from user where name = 'Tom';
```

执行路径是：

```
先查 idx_name 找到 id=100
再用 id=100 回到主键索引查整行数据
```

第二步就是回表。

聚簇索引

- InnoDB 表数据按主键组织。
- 主键索引叶子节点就是完整行。
- 一张表只能有一个聚簇索引。

二级索引

- 叶子节点保存索引列和主键。
- 查到主键后再回聚簇索引查整行。
- 覆盖索引可以避免回表。

工程建议

- 主键尽量短、稳定、递增。
- 二级索引不要过多。
- 高价值查询尽量设计覆盖索引。

面试加分点

- 一张 InnoDB 表只能有一个聚簇索引，因为数据行只能按照一种顺序组织。
- 如果表有主键，InnoDB 用主键作为聚簇索引。
- 如果没有主键，InnoDB 会选择第一个非空唯一索引。
- 如果连非空唯一索引也没有，InnoDB 会生成隐藏的 `row_id` 作为聚簇索引。

面试追问：为什么不建议用很长字符串做主键？

主键会出现在所有二级索引叶子节点中，太长会放大索引空间、降低缓存命中率和查询性能。

深入：什么是最左前缀原则？

结论先行：

联合索引按字段顺序组织，查询必须从最左列开始连续匹配，才能充分利用索引。

分层展开：

基本规则

- 索引 `(a,b,c)` 可用于 `a`、`a,b`、`a,b,c`。
- 如果缺少 `a`，通常无法有效使用该联合索引。
- 范围查询后的列通常不能继续用于有序定位。

设计原则

- 区分度高、过滤能力强的字段靠前。
- 等值条件字段通常放在范围字段前。
- 结合排序和覆盖索引设计。

常见误区

- 不是字段出现在 where 就一定用索引。
- 函数、隐式转换、前导模糊匹配可能导致索引失效。

面试追问：联合索引顺序怎么定？

要结合查询模式、过滤性、排序需求和覆盖需求。没有脱离业务 SQL 的固定答案。

深入：什么情况下索引会失效？

结论先行：

索引失效通常是因为查询条件破坏了索引有序性、类型不匹配、选择性太差或优化器认为全表扫描更划算。

分层展开：

常见原因

- 对索引列使用函数或表达式。
- 隐式类型转换。
- `like '%abc'` 前导模糊。
- 联合索引不满足最左前缀。
- `or` 条件部分字段无索引。
- 选择性太低。

排查方式

- `EXPLAIN` 看 type、key、rows、Extra。
- 对比实际扫描行数。
- 分析索引基数和数据分布。

治理建议

- 改写 SQL。
- 增加合适联合索引。
- 避免隐式转换。
- 大表避免函数包列，改为冗余字段或计算列。

面试追问：用了索引一定快吗？

不一定。低选择性索引、大量回表、排序临时表、随机 IO 都可能让索引查询很慢。

深入：IN、NOT IN、AND、OR 都能使用索引吗？

结论先行：

这些运算符都可能使用索引，但没有“出现某个运算符就一定走索引”的规则。最终要结合索引结构、条件选择性、数据分布和 `EXPLAIN` 判断。

`IN` 通常可以使用索引

```
select *
from ad_task
where status in (1, 2, 3);
```

有 `index(status)` 时，MySQL 通常会把它看成多个等值查找或多个范围查找，常见执行类型是 `range`。

对于联合索引：

```
index(account_id, status, created_at)
```

下面的条件通常可以使用 `account_id + status`：

```
where account_id = ?
      and status in (1, 2, 3)
```

但 `IN` 可能产生多个索引范围。例如：

```
where account_id = ?
      and status in (1, 2, 3)
      and created_at >= ?
order by created_at desc
```

可以理解为：

```
(account_id=?, status=1, created_at>=?)
(account_id=?, status=2, created_at>=?)
(account_id=?, status=3, created_at>=?)
```

这些分支分别有序，但合并后不一定满足 `created_at` 的全局倒序，因此仍可能出现 `Using filesort`。如果 `status` 只有这三个值且查询包含全部状态，`status` 的过滤价值也很低，常需要评估 `(account_id, created_at)` 是否更贴合查询目标。

`NOT IN` 可能使用索引，但通常不如 `IN`

```
where status not in (1, 2)
```

它可以被理解为多个范围：

```
status < 1 or status > 2
```

但 `NOT IN` 往往匹配大量数据，选择性较差，优化器可能认为全表扫描成本更低。因此不能简单说 `NOT IN` 一定不走索引，必须看实际执行计划。

还要注意 `NULL` 语义：

```
status not in (1, 2)
```

不会把 `status is null` 的记录当作满足条件的数据。如果业务要求包含 `NULL`，需要显式写出：

```
where status not in (1, 2)
      or status is null
```

AND 最适合通过联合索引优化

```
where account_id = ?
      and status = ?
      and created_at >= ?
```

索引：

```
index(account_id, status, created_at)
```

可以沿着下面的路径查找：

```
account_id 等值
  -> status 等值
    -> created_at 范围
```

但 `AND` 中的每个字段不一定都能完整使用索引。例如索引仍为 `(account_id, status, created_at)`，查询改成：

```
where account_id = ?
      and created_at >= ?
```

由于跳过了中间列 `status`，通常只能充分利用 `account_id`；`created_at` 可能通过索引下推或回表后过滤，不能把 `AND` 理解成“每个字段都会独立命中一个索引”。

OR 有可能使用索引，但更容易退化

同一个字段的条件：

```
where status = 1 or status = 2
```

通常可以改写为：

```
where status in (1, 2)
```

不同字段的条件：

```
where account_id = ?  
    or campaign_id = ?
```

如果两个字段各自都有索引，MySQL 可能使用 `index_merge`，分别扫描两个索引后再合并结果：

```
扫描 idx_account_id  
扫描 idx_campaign_id  
合并索引结果  
回表读取数据
```

但 `index_merge` 不一定比联合索引或查询改写更快。如果 `OR` 的某一侧无法有效使用索引，例如：

```
where account_id = ?  
    or name like '%abc'
```

为了保证结果完整，MySQL 可能直接选择全表扫描。

OR 拆成 UNION ALL 后能否使用多个索引？

可以。`UNION ALL` 的每个子查询会独立优化，因此不同分支可以使用不同索引：

```
select *  
from user  
where user_id = ?  
  
union all  
  
select *  
from user  
where phone = ?;
```

可能的执行路径是：

```
第一个分支 -> idx_user_id  
第二个分支 -> uk_phone
```

但 `UNION ALL` 不会自动去重。如果一条记录同时满足两个分支，结果会返回两次。可以改用 `UNION` 去重，但会引入额外的去重成本；或者在第二个分支增加排除条件。

如果最终还要全局排序和分页：

```
select ... where account_id = ?  
union all  
select ... where campaign_id = ?  
order by created_at desc  
limit 20;
```

两个分支即使都走索引，合并后仍可能需要全局排序。是否改写，要通过 `EXPLAIN` 和实际耗时验证，不能机械认为 `UNION ALL` 一定更快。

一个 SQL 一次最多只能命中一个索引吗？

不准确。应该区分“同一张表的一次访问”和“整条 SQL”：

场景	是否可能使用多个索引	说明
单表普通查询	通常一个主要访问索引	多个条件通常通过一个联合索引解决
多表 <code>JOIN</code>	可以	每张表可以使用自己的索引
<code>UNION ALL</code>	可以	每个分支可以使用不同索引
<code>OR</code> / <code>AND</code>	可能	可能出现 <code>index_merge</code> ，如 <code>Using union</code> 、 <code>Using intersect</code>
子查询、派生表	可以	不同查询阶段可以选择不同索引

例如：

```
select *
from orders o
join users u on o.user_id = u.id
where o.account_id = ?
      and u.status = 1;
```

可能是：

```
orders 表 -> idx_account_id
users 表 -> idx_status 或 PRIMARY
```

而下面的普通单表查询：

```
where account_id = ?
      and status = ?
      and created_at >= ?
```

通常更希望使用一个合理的联合索引：

```
index(account_id, status, created_at)
```

而不是分别依赖三个单列索引。最终以 `EXPLAIN` 中的 `type`、`key`、`rows`、`Extra`，以及真实压测和线上耗时为准。

深入：EXPLAIN 执行计划必须怎么看？

结论先行：

EXPLAIN 不是背字段，而是判断 SQL 是否走对索引、扫描量是否可控、是否发生大量回表、额外排序和临时表。资深后端看 EXPLAIN 的顺序是：先看访问类型 `type`，再看命中的索引 `key/key_len`，再看扫描量 `rows/filtered`，最后看 `Extra` 里的回表、排序、临时表和索引下推。

核心字段：

- `type`：访问类型，常见从好到差是 `const`、`eq_ref`、`ref`、`range`、`index`、`ALL`。
- `possible_keys`：优化器认为可能使用的索引。
- `key`：最终使用的索引。
- `key_len`：实际使用索引长度，可辅助判断联合索引用到几列。
- `rows`：预估扫描行数。
- `filtered`：过滤后剩余比例。
- `Extra`：额外执行信息，最能暴露风险。

type 不同取值怎么理解

type	含义	常见 SQL 场景	性能判断
<code>system</code>	表只有一行，是 <code>const</code> 的特殊情况	系统表或极小表	极少见，性能最好
<code>const</code>	通过主键或唯一索引等值查询，最多命中一行	<code>where id = ?</code> 、 <code>where uk_order_no = ?</code>	很好，通常是单行定位
<code>eq_ref</code>	join 中，被驱动表通过主键/唯一索引等值匹配，每次最多一行	订单表 join 用户表 <code>on user.id = order.user_id</code>	很好，join 中最理想之一
<code>ref</code>	使用非唯一索引等值查询，可能匹配多行	<code>where account_id = ?</code> 、 <code>where status = ?</code>	通常不错，但要看 rows
<code>range</code>	使用索引范围扫描	<code>where created_at >= ?</code> 、 <code>between</code> 、 <code>in</code>	可接受，但范围越大越慢
<code>index</code>	扫描整棵索引树	覆盖索引全扫描、无 where 但只读索引列	比全表小，但仍可能很重
<code>ALL</code>	全表扫描	无索引、索引失效、优化器认为全表更划算	大表危险，小表不一定有问题

容易误解的点

- `type=ref` 不一定快，如果 `rows` 很大，仍然可能扫描大量数据。
- `type=index` 不是“用了好索引”，它可能是在扫整个索引。
- `type=ALL` 不一定永远错，小表或返回大比例数据时全表扫描可能合理。
- 判断好坏不能只看 `type`，还要看 `rows`、`filtered`、`Extra` 和实际业务 QPS。

Extra 高频判断：

- `Using index`：覆盖索引，不需要回表，通常较好。
- `Using where`：server 层仍需过滤。
- `Using filesort`：需要额外排序，不一定落盘，但大数据量危险。
- `Using temporary`：使用临时表，常见于 group by/order by，需重点关注。
- `Using index condition`：索引下推 ICP，先在存储引擎层过滤部分条件。

经典案例 1：主键/唯一键查询，`type=const`

SQL：

```
explain
select id, order_no, status
from orders
where order_no = 'N0202607130001';
```

索引：

```
unique key uk_order_no(order_no)
```

典型 EXPLAIN：

type	key	rows	Extra
const	uk_order_no	1	

分析：

- `order_no` 是唯一索引，等值查询最多返回一行。
- `type=const` 表示优化器可以把这条记录当成常量处理。
- 这种查询通常没有优化空间，重点是保证字段类型一致、索引存在。

优化建议：

- 业务幂等号、订单号、支付流水号适合唯一索引。
- 不要对唯一索引列做函数或隐式转换，否则可能从 `const` 退化。

经典案例 2：普通二级索引等值查询，`type=ref`，但 `rows` 很大

SQL：

```
explain
select id, account_id, status, created_at
from ad_task
where status = 1;
```

索引:

```
key idx_status(status)
```

典型 EXPLAIN:

type	key	rows	filtered	Extra
ref	idx_status	500000	100.00	Using where

分析:

- **type=ref** 说明用了普通二级索引等值查询。
- 但 **status** 区分度低, **rows** 预估 50 万, 扫描量仍然很大。
- 如果还要回表读取多个字段, 随机 IO 成本会很高。

优化建议:

- 不要单独迷信低区分度字段索引。
- 结合业务查询增加联合索引, 例如:

```
key idx_account_status_time(account_id, status, created_at)
```

- 如果是后台扫描任务, 增加时间范围、分页游标或分批处理。

经典案例 3: 联合索引只用到一部分, **key_len** 暴露问题

SQL:

```
explain
select id, status, created_at
from ad_task
where account_id = ?
      and created_at >= ?
order by created_at desc
limit 20;
```

已有索引:

```
key idx_account_status_time(account_id, status, created_at)
```

典型 EXPLAIN:

type	key	key_len	rows	Extra
ref	idx_account_status_time	account_id长度	10000	Using where; Using filesort

分析：

- 联合索引是 `(account_id, status, created_at)`。
- SQL 缺少 `status` 条件，所以 `created_at` 不能充分接上索引顺序。
- `key_len` 只能看到使用到了 `account_id` 这一段。
- `order by created_at` 可能无法直接利用索引顺序，出现 `Using filesort`。

优化建议：

- 如果这个查询高频，可以新增更贴合的索引：

```
key idx_account_time(account_id, created_at)
```

- 如果业务允许，补充 `status` 条件，让原联合索引完整发挥作用。
- 一个联合索引服务一类核心查询，不要幻想一个索引覆盖所有查询。

变体： `status in (1, 2, 3)` 是否等于补齐了中间列？

如果业务里 `status` 只有 `1、2、3` 三种取值，SQL 改成：

```
explain
select id, status, created_at
from ad_task
where account_id = ?
      and status in (1, 2, 3)
      and created_at >= ?
order by created_at desc
limit 20;
```

索引仍然是：

```
key idx_account_status_time(account_id, status, created_at)
```

典型 EXPLAIN 可能变成：

type	key	key_len	rows	Extra
range	idx_account_status_time	account_id + status + created_at 长度	取决于数 据量	Using where; Using index; Using filesort

分析：

- `status in (1,2,3)` 在业务上等于“所有状态”，但 MySQL 仍会把它当作多个范围条件。
- 访问路径类似拆成三段：

```
(account_id=?, status=1, created_at>=?)  
(account_id=?, status=2, created_at>=?)  
(account_id=?, status=3, created_at>=?)
```

- 因此 `key_len` 可能显示用到了 `account_id + status + created_at`。
- 但索引顺序是 `(account_id, status, created_at)`，结果天然是先按 `status` 分组，再在每个 `status` 里按 `created_at` 有序。
- 如果业务要的是所有状态混在一起后按 `created_at desc` 取最新 20 条，MySQL 通常仍需要 `Using filesort` 做全局排序。

记忆点：

IN 补齐中间列，不等于能消除排序。
多值 IN 会让结果分成多段有序，但整体不一定按 order by 字段全局有序。

如果这个查询是高频核心查询，更匹配的索引仍然是：

```
key idx_account_time(account_id, created_at)
```

因为它可以直接服务：

account_id = ? 后，按 created_at desc 扫描最新数据

经典案例 4：范围查询后排序，可能出现 `Using filesort`

SQL：

```
explain  
select id, account_id, status, created_at  
from ad_task  
where account_id = ?  
      and status in (1, 2, 3)  
      and created_at >= '2026-07-01'  
order by updated_at desc  
limit 20;
```

已有索引：

```
key idx_account_status_created(account_id, status, created_at)
```

典型 EXPLAIN：

type	key	rows	Extra
range	idx_account_status_created	30000	Using index condition; Using filesort

分析：

- where 能用到部分联合索引，`type=range`。
- 但排序字段是 `updated_at`，不在当前索引顺序里。
- MySQL 需要把符合条件的数据再额外排序，所以出现 `Using filesort`。

优化建议：

- 如果排序固定是 `updated_at desc`，考虑新建符合查询和排序的联合索引：

```
key idx_account_status_updated(account_id, status, updated_at)
```

- 如果范围条件返回数据很多，优先减少候选集，再考虑排序索引。
- `Using filesort` 不一定错，但大 rows + filesort 是风险组合。

经典案例 5：隐式转换或函数包列导致索引失效，`type=ALL`

SQL：

```
explain
select id, phone
from user
where phone = 13800138000;
```

字段和索引：

```
phone varchar(20)
key idx_phone(phone)
```

典型 EXPLAIN：

type	key	rows	Extra
ALL	NULL	1000000	Using where

分析：

- `phone` 是字符串，但 SQL 传入数字。
- MySQL 可能发生隐式类型转换，导致无法按字符串索引有序查找。
- 类似地，`where date(created_at) = '2026-07-13'` 这种函数包列也可能导致索引失效。

优化建议：

```
where phone = '13800138000'
```

或把函数条件改写成范围：

```
where created_at >= '2026-07-13 00:00:00'
and created_at < '2026-07-14 00:00:00'
```

经典案例 6：覆盖索引优化，**Extra=Using index**

SQL：

```
explain
select id, status, created_at
from ad_task
where account_id = ?
and status = ?
order by created_at desc
limit 20;
```

索引：

```
key idx_account_status_time(account_id, status, created_at, id)
```

典型 EXPLAIN：

type	key	rows	Extra
ref	idx_account_status_time	200	Using where; Using index

分析：

- where 和 order by 都能利用联合索引。
- 查询返回字段都在索引里，**Using index** 表示覆盖索引。
- 不需要回表读取完整行，列表页性能更稳定。

优化建议：

- 高频列表查询可设计覆盖索引，但不要无限把字段塞进索引。
- 覆盖索引适合返回字段少、查询频率高、延迟敏感的场景。

经典案例 7：深分页，type 看起来不差但仍然慢

SQL：

```
explain
select id, title, created_at
from article
where account_id = ?
order by created_at desc
limit 100000, 20;
```

典型 EXPLAIN：

type	key	rows	Extra
ref	idx_account_time	100020	Using where

分析：

- `type=ref` 并不差。
- 但 `limit 100000, 20` 需要扫描并丢弃前 100000 行。
- 如果查询字段不覆盖索引，还可能产生大量回表。

优化建议：

- 改成游标分页/seek pagination：

```
select id, title, created_at
from article
where account_id = ?
    and created_at < ?
order by created_at desc
limit 20;
```

- 索引设计：

```
key idx_account_time(account_id, created_at, id)
```

EXPLAIN 分析模板

1. 看 `type`：是 `const/ref/range`，还是 `index/ALL`？
2. 看 `key`：是否命中预期索引？
3. 看 `key_len`：联合索引用到几列？
4. 看 `rows + filtered`：扫描量是否可控？
5. 看 `Extra`：是否有 `Using filesort`、`Using temporary`、大量回表？
6. 结合业务：SQL 频率、表数据量、返回比例、写入成本和是否需要新索引。

面试表达：

我看执行计划会先看 `type` 和 `key` 判断是否命中预期索引，再看 `key_len` 判断联合索引用到几列，再看 `rows` 和 `filtered` 判断扫描代价，最后看 `Extra` 是否有 `filesort`、`temporary`、回表和索引下推。优化不是看到 `ALL` 就机械加索引，也不是看到 `ref` 就认为没问题，而是结合数据分布、查询频率、返回行数和写入成本一起判断。

深入：联合索引高级设计怎么答？

核心原则：

- 等值条件优先，范围条件靠后。
- 高频过滤字段优先于低频字段。
- 排序字段尽量接在等值条件之后，减少 `filesort`。

- 需要返回的少量字段可放入联合索引形成覆盖索引。
- 一个索引尽量服务一类核心查询，不要幻想一个超长索引覆盖所有场景。

典型例子：

查询：

```
select id, status, created_at
from ad_task
where account_id = ?
      and status = ?
      and created_at >= ?
order by created_at desc
limit 20;
```

可考虑索引：

```
idx_account_status_time(account_id, status, created_at)
```

原因：

- `account_id`、`status` 是等值过滤。
- `created_at` 是范围和排序字段。
- 返回字段少时可以补充覆盖索引字段，减少回表。

面试追问：低区分度字段能不能建索引？

单独建通常价值不高，例如 `status` 只有几个值。但它放在联合索引中，配合租户、账户、时间等字段，仍可能很有价值。

补充模块：核心能力原理分析：InnoDB、事务、MVCC、锁、日志与复制一致性

本模块回答“为什么 MySQL 能保证事务、并发和恢复”。复杂原理要结合图例、执行链路和生产问题理解。

深入：InnoDB 存储结构与 Buffer Pool 怎么理解？

结论先行：

InnoDB 不是每次都直接读写磁盘记录，而是以页为单位管理数据，通过 Buffer Pool 缓存数据页和索引页；更新时先改内存页并写 redo log，再由后台线程把脏页刷盘。这是 MySQL 性能和崩溃恢复的基础。

分层展开：

1. 存储层级

```
表空间 tablespace
├── 段 segment
│   ├── 区 extent
│   │   ├── 页 page, 默认 16KB
│   │   └── 行 record
```

需要掌握的概念：

- 数据页：存放真实行记录。
- 索引页：存放 B+Tree 节点。
- Buffer Pool：InnoDB 的核心内存缓存。
- 脏页：内存中被修改但还没刷到磁盘的页。
- Free List：空闲页链表。
- Flush List：需要刷盘的脏页链表。
- LRU List：缓存淘汰链表，避免内存无限增长。

2. 为什么修改数据不是立即改磁盘

读取数据页到 Buffer Pool



修改内存中的数据页



数据页变成脏页



先写 redo log



事务提交



后台线程择机把脏页刷盘

核心思想是 WAL，也就是 Write-Ahead Logging：先顺序写日志保证可恢复，再异步写随机数据页。这样既提升性能，又能在宕机后通过 redo log 恢复已提交事务。

3. Buffer Pool 面试必然会讲什么

- Buffer Pool 命中率越高，越少访问磁盘。
- 大查询和全表扫描可能把热点页挤出去，造成缓存污染。
- 脏页太多时刷盘压力上升，可能导致延迟抖动。
- Buffer Pool 不是越大越好，不能占满机器内存，还要给连接、线程、OS page cache、备份和监控留空间。
- redo log 写满或 checkpoint 推进慢时，会触发更激烈的刷脏页。

面试追问：为什么数据库偶发抖动不一定是 SQL 变慢？

可能是 Buffer Pool 命中率下降、脏页集中刷盘、redo checkpoint 压力、磁盘 IO 抖动、备份任务或大查询缓存污染导致。资深排查要把内存页、日志和磁盘 IO 一起看。

事务主题学习顺序：先理解事务边界和 ACID，再理解并发异常与隔离级别；随后学习 MVCC 和锁如何实现隔离，最后学习 undo/redo/binlog、两阶段提交和崩溃恢复如何实现原子性与持久性。

五、事务、MVCC、隔离级别与锁

20. ACID 如何落到 InnoDB？

属性	主要机制
原子性	Undo、事务状态和回滚
一致性	约束 + 事务 + 业务规则共同保证
隔离性	MVCC、锁和隔离级别
持久性	Redo、WAL、刷盘策略和崩溃恢复

数据库机制不能自动保证业务不超卖；还需正确事务边界、条件更新、唯一约束和幂等。

21. 隔离级别解决哪些并发问题？

隔离级别	主要表现
Read Uncommitted	可能读未提交数据
Read Committed	每次快照读生成新视图，可能不可重复读
Repeatable Read	事务内快照通常稳定，结合 Next-Key Lock 处理当前读范围
Serializable	并发最低，读写更强串行化

隔离越强并发成本通常越高。先根据业务正确性选择，再处理锁等待和重试。

22. MVCC、ReadView 和 Undo 版本链如何工作？

记录变更通过 Undo 形成历史版本链；快照读根据事务 ID、ReadView 和可见性规则选择可见版本，从而让读写并发而不总是互相阻塞。

RC 通常每个语句生成新 ReadView；RR 通常在事务首次一致性读时形成并复用。当前读如 **FOR UPDATE** 读取最新可见版本并加锁，不是普通快照读。

23. Record、Gap 和 Next-Key Lock 是什么？

- Record Lock：锁索引记录。
- Gap Lock：锁索引间隙，抑制范围内插入。
- Next-Key Lock：记录锁与前间隙组合。
- Insert Intention：多个插入者表达插入意图，可在不同位置并发。

锁落在索引上。更新条件不走合适索引会扩大扫描和锁范围；范围更新、长事务和热点行更容易放大阻塞。

24. 死锁如何产生和治理？

多个事务以不同顺序持有并等待资源形成环路。InnoDB 检测后选择一个事务回滚，应用需把死锁视为可预期错误并做有限、带退避的幂等重试。

根治方向：统一加锁顺序、缩短事务、命中索引、拆批、减少热点和避免事务内 RPC。死锁日志要还原涉及 SQL、索引和资源顺序。

25. 乐观锁、悲观锁和条件更新怎么选？

- 条件更新：`UPDATE ... WHERE status=?`，简单高效，优先用于状态推进和库存扣减。
- 乐观锁：Version 比较，冲突低且允许重试。
- 悲观锁：`FOR UPDATE`，冲突高、必须读取后决策且事务可保持很短。

不能在持有数据库锁时调用慢外部接口；对外部操作使用状态机、任务表和补偿。

深入：事务模块：ACID、并发与工程边界

MySQL 事务 ACID 怎么理解？

结论先行：

ACID 分别是原子性、一致性、隔离性、持久性，InnoDB 通过 undo log、redo log、锁和 MVCC 等机制实现。

分层展开：

原子性

- 事务内操作要么都成功，要么都回滚。
- 主要依赖 undo log。

一致性

- 事务前后数据满足约束和业务规则。
- 需要数据库机制和业务逻辑共同保证。

隔离性

- 并发事务互不干扰。
- 通过锁和 MVCC 实现。

持久性

- 提交后的数据即使宕机也能恢复。
- 主要依赖 redo log。

面试追问：一致性完全由数据库保证吗？

不是。数据库保证约束层面的一致性，业务一致性还需要代码、事务边界、幂等和补偿机制。

事务到底在解决什么问题？

没有事务时，多条 SQL 彼此独立，容易出现“部分成功”：

```
账户 A 扣款成功
账户 B 入账失败
系统异常
```

事务把一组必须共同成功的 SQL 绑定成一个执行单元：

```
开始事务
-> 扣款
-> 入账
-> 全部成功：COMMIT
-> 任意失败：ROLLBACK
```

它主要解决四类问题：

1. 多条 SQL 部分成功或部分失败。
2. 并发读写时的数据可见性和隔离问题。
3. 并发写入时的覆盖、丢失更新和资源竞争。
4. 数据库宕机后，已提交数据无法恢复或未提交数据错误残留。

事务不能保证所有业务天然正确，它只能在明确的事务边界内提供数据库级保证。

SQL 事务的基本使用：

```
start transaction;

update account
set balance = balance - 100
where id = 1
  and balance >= 100;

update account
set balance = balance + 100
where id = 2;
```

```
commit;
```

失败时：

```
rollback;
```

局部回滚可以使用保存点：

```
savepoint before_update;  
rollback to savepoint before_update;  
release savepoint before_update;
```

MySQL 默认开启自动提交：

```
select @@autocommit;
```

在 `autocommit=1` 时，一条独立的 `INSERT`、`UPDATE` 或 `DELETE` 通常就是一个独立事务；多条 SQL 需要原子执行时，要显式开启事务。

并发事务会遇到哪些问题？

要按读-读、读-写、写-读、写-写四种组合分析：

并发组合	主要问题
读-读	通常没有数据覆盖，但可能存在快照版本差异、资源竞争、主从延迟和元数据锁等待
读-写	脏读、不可重复读、幻读、当前读阻塞
写-读	未提交写入的可见性、快照读与当前读差异、读写锁等待
写-写	丢失更新、覆盖更新、锁等待、死锁、锁范围扩大

四个并发异常：

脏读：读取了其他事务尚未提交的数据
脏写：覆盖其他事务尚未提交的修改
不可重复读：同一事务两次读取同一行，字段值发生变化
幻读：同一事务两次执行范围查询，符合条件的记录集合发生变化

脏读示例：

```
A：把 balance 从 1000 改成 900，但未提交  
B：读到 balance=900  
A：回滚，balance 恢复为 1000
```

脏写示例：

A：写入 stock=9，但未提交
B：覆盖写入 stock=8
A：回滚，可能把 B 的修改一起覆盖

不可重复读示例：

A：第一次读取 balance=1000
B：修改 balance=900 并提交
A：第二次读取 balance=900

幻读示例：

A：第一次查询 age >= 18，返回 10 条
B：插入一条 age=20 的记录并提交
A：第二次查询，返回 11 条

可以这样区分：

不可重复读：原来的那一行变了
幻读：原来的查询范围里多了或少了行

MySQL InnoDB 如何实现 ACID？

特性	InnoDB 主要手段	解决的问题
原子性	undo log、事务回滚	多条 SQL 全部成功或全部撤销
一致性	约束、事务边界、条件更新、状态机、幂等	数据满足结构和业务规则
隔离性	MVCC、Read View、行锁、间隙锁、Next-Key Lock、隔离级别	控制并发事务之间的可见性和冲突
持久性	redo log、WAL、binlog 两阶段提交、崩溃恢复	宕机后恢复已提交数据

一次写事务的大致流程：

按需加载数据页和索引页到 Buffer Pool
-> 获取必要的记录锁
-> 写 undo log，保存旧版本或回滚信息
-> 修改 Buffer Pool 中的数据页和索引页
-> 生成 redo log，写入 Log Buffer
-> 提交时 redo prepare
-> 写 binlog 并按配置持久化

```
-> redo commit
-> 返回提交成功
-> 后台线程异步刷脏页
```

因此：

事务提交成功 != 数据页已经写入数据文件

提交成功但脏页尚未刷盘时，可能是：

```
Buffer Pool：新值
磁盘数据页：旧值
redo log：已满足持久化条件
```

宕机后，InnoDB 读取旧数据页并重放 redo log，恢复已提交修改；未完成提交的事务则根据 undo log 回滚。

四种标准隔离级别：

隔离级别	脏读	不可重复读	幻读	并发性能
READ UNCOMMITTED	可能	可能	可能	最高
READ COMMITTED	避免	可能	可能	较高
REPEATABLE READ	避免	避免普通快照读的不一致	InnoDB 通过 MVCC 和锁处理	较好
SERIALIZABLE	避免	避免	避免	最低

MySQL InnoDB 默认使用 **REPEATABLE READ**。但必须区分：

```
普通 SELECT：快照读，主要依赖 MVCC 和 Read View
SELECT ... FOR UPDATE：当前读，读取最新版本并加锁
UPDATE/DELETE：当前读，需要锁控制并发
```

在 RR 下，普通快照读通过 MVCC 和 undo log 读取一致版本；范围当前读则可能通过记录锁、间隙锁和 Next-Key Lock 防止其他事务插入符合条件的新记录。

事务解决不了什么？

- 不能把多个微服务的操作自动纳入同一个本地事务。
- 不能自动回滚 Redis、MQ、第三方支付、HTTP、短信和文件上传等外部副作用。
- 不能自动保证缓存与数据库一致，需要删除缓存、消息、补偿和对账。
- 不能保证业务逻辑正确，例如没有 `balance >= 100` 条件时，事务也可能提交负余额。
- 不能彻底消除死锁，只能通过固定加锁顺序、缩短事务、命中索引和重试降低影响。
- 不能解决数据库过载、连接池耗尽和主从延迟，需要限流、降级、缓存、MQ、读写分离和分片。

工程事务边界的判断：

必须强一致的本地操作：放入同一个 MySQL 事务
允许最终一致的操作：事务提交后通过 MQ/Outbox 异步处理
外部副作用：不要长时间占用数据库事务，使用状态机、幂等和补偿

生产原则：

- 事务尽量短，不在事务中调用外部 RPC。
- 更新条件命中索引，减少锁范围。
- 多资源加锁保持固定顺序，降低死锁概率。
- 使用唯一索引、条件更新和版本号保证幂等与防并发覆盖。
- 捕获死锁和锁等待超时，区分重试、回滚和业务补偿。
- 大批量操作拆成小批次，避免长事务、undo 膨胀和主从延迟。

和 Redis、MongoDB 的区别

组件	事务定位	更适合解决的问题	不适合解决的问题
MySQL/ InnoDB	完整本地 ACID 事务	多表多行强一致写入、状态流转、唯一约束、资金/库存/订单	跨服务长流程强事务、缓存一致性直接绑定
Redis	命令队列连续执行	Redis 内部简单原子组合、计数、缓存状态切换	数据库级回滚、复杂业务一致性、跨系统事务
MongoDB	单文档原子 + 多文档事务补充	文档聚合根内原子更新、少量跨文档一致变更	高频大范围跨集合事务、复杂关系约束

MySQL 事务底层分别靠什么保证

- 原子性：靠 undo log，失败时按旧版本回滚。
- 持久性：靠 redo log，提交后宕机可重放恢复。
- 隔离性：靠 MVCC + 锁，快照读减少读写冲突，当前读通过行锁/间隙锁控制并发。
- 一致性：靠主键/唯一键/外键/检查约束、事务边界、业务规则共同保证。
- 复制一致性：靠 redo log 和 binlog 的两阶段提交，避免本机提交状态和复制日志不一致。

工程里如何使用 MySQL 事务

```
START TRANSACTION;

UPDATE account
SET balance = balance - 100
WHERE user_id = 1001 AND balance >= 100;

INSERT INTO account_flow(flow_no, user_id, amount, type)
VALUES ('flow_001', 1001, -100, 'deduct');
```

```
COMMIT;
```

- 事务边界要短，不要在事务里调用外部 HTTP/RPC。
- 关键更新用条件更新兜底，例如 `WHERE status = 'INIT'`、`WHERE stock >= n`。
- 幂等用唯一索引兜底，例如订单号、流水号、请求号。
- 失败要能重试，死锁和锁等待超时不能直接当业务成功。
- 写 DB + 发 MQ 不在一个本地事务里，常用 Outbox/本地消息表补偿。

面试追问：MySQL 事务能不能解决所有一致性问题？

不能。MySQL 事务只能保证同一个数据库本地事务内的一致性。跨 Redis 缓存、MQ、第三方 API、多个微服务时，仍然要靠幂等、Outbox、事务消息、补偿任务、对账和状态机。

MVCC 是什么？

结论先行：

MVCC 是多版本并发控制，通过 undo log 和 ReadView 让读写并发时读请求看到一致版本，减少锁冲突。

分层展开：

核心机制

- 每行记录有隐藏事务 ID 和回滚指针。
- undo log 保存历史版本。
- ReadView 判断哪个版本对当前事务可见。

解决问题

- 快照读不阻塞写。
- 写不阻塞快照读。
- 提升并发性能。

隔离级别差异

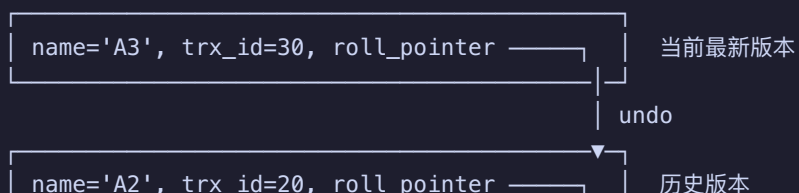
- RC 每次快照读生成新 ReadView。
- RR 事务内第一次快照读生成 ReadView，后续复用。

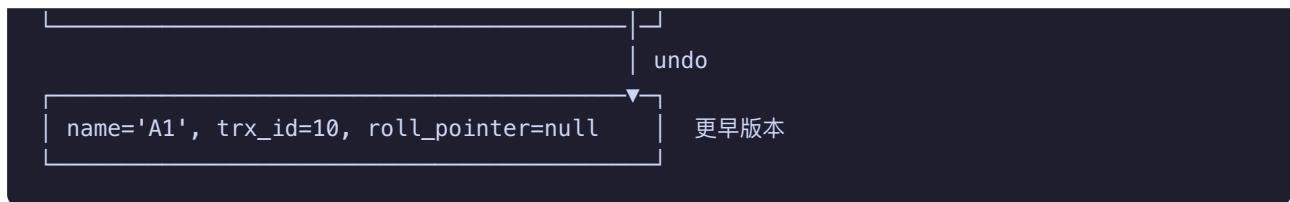
面试追问：MVCC 能解决幻读吗？

快照读在 RR 下通过一致性视图避免幻读感知；当前读需要 next-key lock 防止幻读。

图例：一行数据的版本链

聚簇索引记录 id=1001





ReadView 怎么判断可见：

- 当前事务能看到哪些已提交事务版本。
- 如果最新版本不可见，就沿着 undo 版本链往前找。
- RC：每次快照读生成新 ReadView，所以能读到别人新提交的数据。
- RR：事务第一次快照读生成 ReadView，后续复用，所以同一事务内多次读结果一致。

案例：为什么 RR 下两次普通 select 结果一样？

```
T1 开启事务并第一次 select, 生成 ReadView。  
T2 更新同一行并提交。  
T1 再次普通 select, 仍复用旧 ReadView, 所以看不到 T2 的新版本。  
T1 如果执行 select ... for update, 这是当前读, 会读最新版本并加锁。
```

面试表达：

MVCC 解决的是快照读的一致性和读写并发问题，不是让所有读都不加锁。普通 `select` 多数是快照读，`update/delete/select for update` 是当前读，需要通过锁保证并发正确。

可重复读和读已提交有什么区别？

结论先行：

读已提交每次读都能看到其他事务已提交的新数据；可重复读保证同一事务内多次快照读结果一致。

分层展开：

读已提交 RC

- 每次 select 创建新的 ReadView。
- 可以避免脏读。
- 可能出现不可重复读。

可重复读 RR

- 事务内复用 ReadView。
- 避免不可重复读。
- InnoDB 结合 next-key lock 处理当前读幻读。

工程选择

- MySQL 默认 RR。
- 需要更接近实时读取已提交数据可用 RC。
- 核心业务要明确读是快照读还是当前读。

面试追问：普通 select 和 select for update 一样吗？

不一样。普通 select 是快照读，`select for update` 是当前读，会加锁并读取最新已提交版本。

深入：锁模块：MySQL 和 InnoDB 的锁如何分类？

结论先行：

不要把“行锁、排他锁、间隙锁”当成同一层级的概念。MySQL 的锁可以从作用域、锁模式、锁定对象、生命周期和事务关系五个维度分类。InnoDB 主要通过索引记录锁、范围锁和意向锁控制事务并发；MySQL Server 层还存在元数据锁，业务层还可以使用命名锁。

锁由哪一层实现？

InnoDB 存储引擎层：
Record Lock、Gap Lock、Next-Key Lock
Insert Intention Lock、IS/IX 意向锁、AUTO-INC 锁

MySQL Server 层：
MDL 元数据锁、显式表锁、GET_LOCK 命名锁

InnoDB 锁主要保护事务中的索引记录和索引范围；MySQL Server 层的 MDL 保护表结构，显式表锁保护整张表，命名锁保护一个逻辑名称。不要把它们混成同一种锁。

Record/Gap/Next-Key：锁定对象
S/X：锁模式
IS/IX：表级意向状态
MDL：表结构元数据
GET_LOCK：连接级业务互斥

补充说明：

- InnoDB 行锁本质是索引记录锁，聚簇索引和二级索引都可能被锁定。
- 通过二级索引更新时，通常还要回到聚簇索引锁定对应主键记录。
- RR 下范围当前读可能在聚簇索引或二级索引上形成 Gap Lock / Next-Key Lock。
- GET_LOCK 不保护业务行数据，也不随普通事务 COMMIT/ROLLBACK 自动回滚，通常与连接生命周期相关。

一、按作用域分类

作用域	典型锁	锁住的对象	主要作用
实例/业务范围	命名锁 <code>GET_LOCK</code>	一个字符串名称	协调同一实例内的业务任务
表级	表锁、意向锁、AUTO-INC 锁	整张表或表级状态	协调表级操作和行锁
行/索引级	记录锁、间隙锁、Next-Key Lock	索引记录或索引范围	控制事务并发读写
元数据级	MDL	表结构元数据	防止查询和 DDL 同时破坏表定义

二、按锁模式分类

锁模式	含义	典型来源
S (Shared) 共享锁	多个事务可以共享读取，但通常阻止写入	FOR SHARE
X (Exclusive) 排他锁	当前事务独占修改，阻止其他 S/X 锁	FOR UPDATE、UPDATE、DELETE
IS (Intention Shared)	表示事务准备在行上加 S 锁	共享锁前的表级意向锁
IX (Intention Exclusive)	表示事务准备在行上加 X 锁	排他锁前的表级意向锁
AUTO-INC	保护自增值分配过程	自增主键插入

S/X 描述的是“允许什么操作”，记录锁/间隙锁描述的是“锁住哪里”，两者可以组合：

```
id=100 的索引记录 + X 锁
(10,20) 的索引间隙 + GAP 锁
(10,20] 的索引范围 + Next-Key Lock
```

三、按锁定对象分类

- 记录锁 (Record Lock)**：锁住 B+Tree 中的一条索引记录。InnoDB 的行锁本质上是索引记录锁，不是直接给物理数据行加一个 `locked=true` 标记。主键记录位于聚簇索引中，锁住主键索引记录通常等价于锁住整行。
- 间隙锁 (Gap Lock)**：锁住两条索引记录之间不存在真实行的范围，主要阻止其他事务向该范围插入记录。
- Next-Key Lock**：记录锁 + 间隙锁，例如锁住 `(10,20]`，既限制范围内插入，也限制记录 `20` 的修改。
- 插入意向锁 (Insert Intention Lock)**：事务准备在某个间隙插入记录时产生的意向。多个事务在不同位置插入通常可以并发，但会与阻止插入的 Gap Lock 冲突。
- 表级意向锁 (IS/IX)**：不是把整张表锁住，而是向其他表级锁请求者声明“本表中存在行级 S/X 锁”，用于快速判断表锁是否冲突。

四、按是否属于事务锁分类

类型	是否跟随事务提交/回滚	典型用途
InnoDB 记录锁、间隙锁、意向锁	是，通常持有到事务结束	行级并发控制
显式表锁 LOCK TABLES	由显式解锁或会话/事务状态控制	特殊批处理或兼容场景
MDL	与语句或事务持有的表访问状态相关	防止 DDL 与查询/写入冲突

类型	是否跟随事务提交/回滚	典型用途
命名锁 <code>GET_LOCK</code>	通常绑定连接，不随事务回滚自动释放	任务互斥、单实例选主

五、锁的兼容关系

简化管理如下：

已持有 / 请求	S	X
S	通常兼容	冲突
X	冲突	冲突

因此：

- 普通快照读通常不申请行 S 锁，不会因为 X 锁而等待。
- `FOR SHARE` 申请 S 锁，多个共享锁通常可以共存。
- `FOR UPDATE`、`UPDATE`、`DELETE` 申请 X 锁，会阻塞其他冲突锁请求。

六、最容易混淆的三种锁

行锁：锁住哪里？一条索引记录
 排他锁：锁的模式是什么？X，独占访问
 间隙锁：锁住哪里？索引记录之间的范围

例如：

```
select *
from user
where id = 100
for update;
```

可以理解为：

表 user：先有 IX 意向排他锁
 主键索引 id=100：再加 X 记录锁

范围当前读：

```
select *
from user
where age between 10 and 20
for update;
```

在 RR 下可能形成：

age=10、age=20：记录锁
(10,20)：间隙锁
整体范围：Next-Key Lock

七、锁对其他操作的影响

普通 SELECT
-> 通常是快照读，不申请记录 S 锁
-> 通常不被行 X 锁阻塞

SELECT ... FOR SHARE
-> 当前读，申请 S 锁
-> 与 UPDATE/DELETE 的 X 锁冲突

SELECT ... FOR UPDATE
-> 当前读，申请 X 锁
-> 阻塞其他修改和冲突的当前读

UPDATE/DELETE
-> 当前读并申请 X 锁
-> 锁通常持有到 COMMIT/ROLLBACK

八、为什么没走索引会放大锁范围？

InnoDB 的锁依赖索引访问路径。条件没有合适索引时：

全表或大范围扫描
-> 判断哪些记录满足条件
-> 访问和锁定更多索引记录
-> 其他事务等待范围扩大

因此锁排查必须同时看：

SQL 条件
执行计划
实际命中的索引
锁定索引
锁模式和锁范围
事务持有时间

分层展开：

行锁

- 锁住索引记录。
- 命中索引时粒度较小。

间隙锁

- 锁住索引记录之间的间隙。
- 防止其他事务插入导致幻读。

next-key lock

- 行锁 + 间隙锁。
- RR 下当前读常见。

意向锁

- 表级锁。
- 表示事务接下来要加行锁。
- 用于协调表锁和行锁。

面试追问：行锁一定只锁一行吗？

不一定。如果查询没走索引，可能扫描大量记录并加锁，甚至接近锁表效果。

图例：next-key lock 锁住什么

假设索引 `idx_age` 中已有记录：

```
age index:
(-∞, 10] (10, 20] (20, 30] (30, +∞)
      10      20      30
```

执行：

```
select * from user where age between 10 and 20 for update;
```

在 RR 隔离级别下，InnoDB 可能不仅锁住 `10`、`20` 这些记录，还会锁住相关间隙，防止其他事务插入 `age=15` 造成当前读幻读。

```
锁记录：age=10、age=20
锁间隙：(10,20)
效果：其他事务不能插入 age=15
```

当前读和快照读的区别：

类型	典型 SQL	是否读最新版本	是否加锁	用途
快照读	<code>select * from t where id=1</code>	按 ReadView 判断	通常不加锁	普通查询
当前读	<code>select ... for update</code>	是	加锁	先查后改
当前读	<code>update/delete</code>	是	加锁	修改数据

工程提醒：

库存扣减、任务抢占、状态流转不能只靠先 `select` 再 `update`。要么用 `select for update` 放进事务，要么更推荐条件更新：

```
update task
set status = 'RUNNING'
where id = ? and status = 'INIT';
```

根据 affected rows 判断是否抢占成功。

record lock、gap lock、next-key lock 和 insert intention lock 怎么讲？

结论先行：

InnoDB 行锁是加在索引上的。RR 隔离级别下，为了防止当前读幻读，范围查询可能加 next-key lock，也就是记录锁 + 间隙锁。

四类锁的直观理解：

- record lock：锁住一条索引记录。
- gap lock：锁住两条索引记录之间的间隙，不锁已有记录本身。
- next-key lock：record lock + gap lock，锁住记录以及它前面的间隙。
- insert intention lock：插入意向锁，多个事务插入不同位置时可并发，但会和 gap lock 冲突。

典型场景：

```
select * from user where age between 10 and 20 for update;
```

如果 age 上有索引，RR 下不只是锁住已有的 age=10 到 20 的记录，还可能锁住相关间隙，防止其他事务插入新的 age=15 记录造成幻读。

高危点：

- 条件没走索引，锁范围可能扩大。
- 范围更新比等值主键更新更容易锁住间隙。
- 长事务持锁时间长，容易放大阻塞。
- RC 下 gap lock 使用更少，但并不代表没有锁冲突。

面试追问：为什么一个 update 没命中索引会锁很多行？

因为 InnoDB 需要扫描记录判断哪些行满足条件，扫描过程中会对相关记录加锁。条件没有索引时，扫描范围变大，锁范围也会随之扩大。

死锁怎么产生？怎么处理？

结论先行：

死锁是多个事务互相持有对方需要的锁，形成循环等待。处理方式是统一加锁顺序、缩短事务、命中索引和失败重试。

分层展开：

产生原因

- 多事务更新资源顺序不一致。
- 范围更新锁住间隙。
- SQL 未命中索引导致锁范围扩大。
- 长事务持锁时间过长。

处理机制

- InnoDB 可以检测死锁。
- 回滚其中一个事务。
- 应用层需要识别错误并重试。

治理方案

- 固定资源访问顺序。
- 事务尽量短。
- 更新条件走索引。
- 拆分大事务。

面试追问：死锁能完全避免吗？

复杂系统里很难完全避免，关键是降低概率，并让业务具备安全重试和幂等能力。

六、Redo、Undo、Binlog、提交与崩溃恢复

26. Redo、Undo 和 Binlog 分别做什么？

日志	层次	作用
Redo	InnoDB	WAL、崩溃后重做已提交页修改
Undo	InnoDB	回滚未提交修改、提供 MVCC 历史版本
Binlog	Server	逻辑变更、复制、CDC 和时间点恢复

Redo 不是业务审计，Binlog 也不能代替完整备份。

27. 一次更新和提交经历什么？

定位/加载数据页

- > 申请必要索引锁
- > 写 Undo
- > 修改 Buffer Pool 脏页
- > 生成 Redo 到 Log Buffer
- > Redo Prepare
- > 写 Binlog
- > Redo Commit

-> 返回提交成功
-> 后台刷脏页

提交成功不要求数据页已经落盘，而要求提交所需日志达到配置定义的持久化条件。

28. Redo 与 Binlog 为什么需要两阶段提交？

它协调 InnoDB 恢复状态与 Server Binlog，避免一个已提交、另一个缺失造成主库恢复与复制不一致。

- Redo Prepare 且存在对应 Binlog：恢复时提交。
- Redo Prepare 但无 Binlog：恢复时回滚。
- Redo Commit：事务已提交。

这不是跨业务系统的分布式两阶段提交。

29. Doublewrite 和 Checkpoint 解决什么问题？

Doublewrite 降低数据页部分写造成页损坏的风险；Checkpoint 推进已安全刷盘位置，让旧 Redo 空间可复用。

脏页过多、Checkpoint 压力和磁盘抖动会造成延迟尖峰。应监控脏页比例、Redo 使用、Fsync 和磁盘延迟，而不是只看 CPU。

深入：事务持久化模块：redo、undo、binlog 与恢复

redo log、undo log、binlog 分别做什么？

结论先行：

redo log 保证崩溃恢复，undo log 支持回滚和 MVCC，binlog 用于复制、归档和数据恢复。

分层展开：

redo log

- InnoDB 引擎层日志。
- 记录物理页修改。
- 保证持久性。

undo log

- 保存修改前版本。
- 支持事务回滚。
- 支持 MVCC 历史版本读取。

binlog

- MySQL Server 层日志。
- 记录逻辑变更。
- 用于主从复制和恢复。

面试追问：为什么需要两阶段提交？

为了保证 redo log 和 binlog 一致，避免事务提交后一个日志有、另一个没有，导致崩溃恢复或主从复制不一致。

redo/binlog 两阶段提交和崩溃恢复怎么答？

核心链路：

```
执行 INSERT/UPDATE/DELETE
-> 按需加载数据页和索引页到 Buffer Pool
-> 加记录锁或其他必要的索引锁
-> 写 undo log, 保存旧版本或回滚信息
-> 修改 Buffer Pool 中的数据页和索引页
-> 生成 redo log, 写入 Log Buffer
-> 事务提交时, redo log 写 prepare
-> MySQL Server 写 binlog 并按配置持久化
-> redo log 写 commit
-> 返回提交成功
-> 后台线程异步刷脏页
```

一次写操作具体会加哪些锁？

写操作的锁不是固定的一把锁，而是由以下因素共同决定：

```
INSERT / UPDATE / DELETE 类型
WHERE 条件和访问路径
使用的是主键、唯一二级索引还是普通二级索引
是否存在范围条件
事务隔离级别
是否需要检查唯一键、外键或自增值
```

可以用下面的通用顺序理解：

1. 对表建立 IX 等意向锁，表示事务准备加行级排他锁
2. 按执行计划扫描聚簇索引或二级索引
3. 对需要修改或检查的索引记录申请 X 记录锁
4. RR 下范围访问可能同时申请 Gap Lock / Next-Key Lock
5. 通过二级索引定位数据时，回到聚簇索引锁定对应主键记录
6. 修改 Buffer Pool 中的数据页和索引页
7. 提交或回滚时统一释放事务持有的锁

UPDATE / DELETE 的典型锁过程：

```
update user
set name = 'Tom'
where age = 20;
```

如果 age 上有普通二级索引：

表 user: IX 意向排他锁
二级索引 idx_age: 锁定符合条件的索引记录
聚簇索引 PRIMARY: 锁定对应主键记录

如果查询是范围条件:

```
update user
set name = 'Tom'
where age between 10 and 20;
```

RR 下可能形成:

二级索引中的记录锁
二级索引中的间隙锁
二级索引上的 Next-Key Lock
对应聚簇索引记录的 X 锁

如果 age 没有索引, InnoDB 只能扫描聚簇索引, 无法只定位 age=10~20 的索引范围, 可能锁住大量聚簇索引记录和间隙, 效果接近大范围锁定。因此“写条件是否命中索引”不仅影响查询速度, 也直接影响锁冲突范围。

INSERT 的典型锁过程:

```
insert into user(id, age, name)
values (100, 20, 'Tom');
```

大致会经历:

表 user: IX 意向排他锁
唯一键/主键检查: 读取并检查相关索引记录
插入位置: 申请 Insert Intention Lock
聚簇索引: 写入新主键记录并形成 X 记录锁
二级索引: 写入 age、name 等索引项并加相应记录锁
自增主键: 必要时参与 AUTO-INC 锁或自增值分配保护

如果插入位置的间隙被其他事务的 Gap Lock 或 Next-Key Lock 占用, INSERT 可能等待; 如果主键或唯一键冲突, 插入会失败或进入 ON DUPLICATE KEY UPDATE 的更新路径。

锁的持有时间:

显式事务: 通常持有到 COMMIT 或 ROLLBACK
autocommit=1: 语句完成后自动提交并释放事务锁

因此下面两种写法的锁影响范围不同:

```
-- 独立语句：执行完成后通常立即提交
update user set name = 'Tom' where id = 1;
```

```
-- 显式事务：锁会持续到事务结束
start transaction;
update user set name = 'Tom' where id = 1;
-- 此时其他事务可能等待
commit;
```

需要强调：普通快照读通常不申请行级 S 锁，因此可以读取其他事务提交前的历史版本；但 SELECT ... FOR UPDATE、SELECT ... FOR SHARE、UPDATE、DELETE 都可能与写锁产生等待。

事务执行过程中的几个关键状态：

事务未提交：
Buffer Pool 中可能已经是新值
磁盘数据页可能仍是旧值
undo 保存旧版本
行锁仍由当前事务持有

事务提交成功但尚未刷脏页：
Buffer Pool 是新值
磁盘数据页可能仍是旧值
redo 和 binlog 已满足持久化条件

后台刷脏完成：
Buffer Pool 是新值
磁盘数据页也是新值

因此，Buffer Pool 和数据文件不要求在任意时刻都完全一致。常见的正常状态是：Buffer Pool 中保存最新值，数据文件中暂时保留旧值；只要 redo log 已经按照提交要求持久化，宕机后就可以通过 redo 重放恢复数据页。

事务回滚：

如果执行过程中发生 SQL 异常、应用主动执行 **ROLLBACK**，或者崩溃恢复发现事务没有完成提交，InnoDB 会根据 undo log 撤销本次修改：

```
读取 undo log
-> 恢复数据页和索引页的旧版本
-> 清理事务版本信息
-> 释放行锁
-> 事务回滚完成
```

undo log 不仅用于回滚，也用于 MVCC。其他事务在事务 A 尚未提交时，不能直接读取事务 A 的未提交修改，而是根据隔离级别通过 undo 版本链读取可见的历史版本。

并发写入中可能遇到的情况：

- 两个事务更新同一行时，后来的事务需要等待前一个事务释放行锁。
- 等待超过 `innodb_lock_wait_timeout`，可能返回锁等待超时错误。
- 多个事务以不同顺序持有多把锁时，可能形成死锁；InnoDB 会选择回滚其中一个事务，应用层应捕获死锁错误并进行有限次数重试。
- 条件没有命中合适索引时，扫描范围和锁范围都可能扩大，放大阻塞风险。

不同崩溃时机的处理：

崩溃位置	典型日志状态	恢复结果
redo prepare 之前	没有完整提交状态	未完成事务回滚
redo prepare 后、binlog 前	redo 为 prepare, binlog 不存在	回滚事务
binlog 已写入、redo commit 前	redo 为 prepare, binlog 存在	继续提交 redo
redo commit 后、数据页刷盘前	redo 已提交, 数据页可能是旧页	重放 redo, 恢复已提交修改

两阶段提交依赖的判断原则是：

```
redo prepare + 找到对应 binlog -> 事务应提交
redo prepare + 找不到对应 binlog -> 事务应回滚
redo commit          -> 事务已提交
```

磁盘与 Buffer Pool 一致性的保证手段：

1. **WAL (Write-Ahead Logging)**：先保证 redo log 持久化，再允许数据页延迟刷盘。这样可以用顺序日志写入替代每次更新都进行随机数据页写入。
2. **redo log**：保证已提交但尚未刷盘的数据页在崩溃后可以重做恢复。
3. **undo log**：保证未提交事务可以撤销，并为 MVCC 提供历史版本。
4. **redo/binlog 两阶段提交**：协调 InnoDB 的 redo 和 Server 层的 binlog，避免主库恢复状态和从库复制状态不一致。
5. **checkpoint 和后台刷脏**：持续将脏页写回数据文件，并推进 redo 空间回收，避免脏页和日志无限增长。
6. **doublewrite buffer**：降低数据页只写入一部分导致页断裂的风险。刷脏时先写 doublewrite 区域，再写真实数据文件；崩溃恢复时可用完整页修复损坏页。

需要区分：

```
事务提交成功 != 数据页已经写入数据文件
事务提交成功 = 提交所需日志已经满足持久化条件
```

为什么需要两阶段提交：

- redo log 是 InnoDB 的崩溃恢复依据。
- binlog 是主从复制和逻辑恢复依据。
- 如果二者不一致，可能出现主库恢复后有数据、从库没同步，或者 binlog 有记录但引擎没提交。

崩溃恢复判断：

- redo prepare，有 binlog：认为事务已提交，恢复。
- redo prepare，无 binlog：认为事务未完成，回滚。
- redo commit：事务已提交。

面试表达：

两阶段提交解决的是存储引擎日志和 server 层日志的一致性问题。它不是分布式事务的两阶段提交，而是 MySQL 内部为了同时保证崩溃恢复和主从复制一致所做的提交协议。

七、复制、高可用、备份与灾备

30. MySQL 主从复制如何工作？

```
Primary 提交 Binlog
-> Replica IO/Receiver 获取日志
-> 写 Relay Log
-> SQL/Applier 线程应用事务
-> 更新复制位点/GTID
```

复制通常异步，存在延迟和故障时的数据丢失窗口。并行复制提高应用能力，但大事务、热点冲突、DDL 和从库资源不足仍会造成延迟。

31. GTID 解决什么问题？

GTID 为已提交事务提供全局身份，简化复制定位、故障切换和拓扑变更，避免人工维护文件与 Position。

它不自动保证零丢失，也不替代拓扑一致性检查。切换前仍需确认候选节点已执行集合、复制健康和业务写入隔离。

32. 异步、半同步和 Group Replication 如何取舍？

- 异步：延迟低、可用性高，但主库故障可能丢未复制事务。
- 半同步：至少一个副本收到日志后返回，降低丢失概率，故障/网络异常会增加写延迟或退化。
- Group Replication/MGR：基于组通信和一致性协议管理成员与事务认证，自动化更强但运维和冲突模型更复杂。

选择取决于 RPO、RTO、跨地域延迟、团队能力和写可用性要求。

33. 主从延迟如何排查？

- 主库是否有大事务、突发写、DDL 或 Binlog 量增长。
- IO 接收慢还是 SQL 应用慢。
- Relay Log 是否堆积，Applier 是否有热点冲突。
- 从库 CPU、磁盘、Buffer Pool 和网络是否不足。
- 是否有备份、报表或 DDL 争抢资源。

强一致读临时回主；根治通过拆小事务、并行复制、优化从库资源和隔离任务。不要只依赖单一延迟秒数指标。

34. 故障切换如何保证正确？

```
隔离旧 Primary 写入
-> 收集各副本 GTID/位点和健康状态
-> 选择数据最新且可承载的候选
-> 提升并重建复制拓扑
-> 应用刷新路由/连接
-> 校验写入、复制和业务数据
-> 处理旧主回归
```

必须先防双写，再选主。客户端重试要带幂等键；切换后通过业务流水和对账确认是否存在丢失或重复。

35. 备份、恢复和 PITR 如何设计？

结论先行： 备份的价值由恢复演练证明。常用完整备份加持续 Binlog，实现恢复到指定时间点。

- 明确 RPO/RTO、保留周期、加密和异地保存。
- 备份与线上 IO 隔离，校验完整性。
- 恢复：还原基础备份，再按顺序回放 Binlog 到目标时间。
- 定期演练恢复时长、应用连接、权限和数据对账。
- 复制不是备份，误删会复制到所有副本。

深入：MySQL 主从复制原理是什么？

结论先行：

主库写入 binlog，从库 IO 线程拉取 binlog 写 relay log，SQL 线程重放 relay log，实现数据复制。

分层展开：

流程

- 复制启动后，从库 IO 线程连接主库，并携带 file/position 或 GTID 位点请求增量日志。
- 主库 dump 线程从指定位置读取 binlog；主库暂时没有新日志时保持连接等待。
- 主库事务提交阶段完成 binlog 写入后，新的 binlog event 才会被持续发送给从库。

- 从库 IO 线程接收 binlog，并先写入本地 relay log。
- 从库 SQL/applier 线程读取 relay log，执行事务并修改从库数据。

完整链路：

```
主库事务提交
-> 主库 binlog
-> 主库 dump 线程
-> 网络传输
-> 从库 IO 线程
-> 从库 relay log
-> 从库 SQL/applier 线程
-> 从库 InnoDB 数据
```

为什么需要 relay log?

relay log 是从库接收主库日志后的本地中转日志，不是主库 binlog 的另一种命名。它主要解决以下问题：

- **解耦接收和执行**：IO 线程负责接收并落 relay log，SQL 线程负责读取和应用，接收速度和执行速度可以不同。
- **吸收短时流量波动**：主库发送较快时，日志先在 relay log 中堆积，SQL 线程再逐步消费。
- **支持断点恢复**：IO 线程可能已经接收到较新的日志，而 SQL 线程只执行到较早位置；重启后可以从 relay log 的应用位点继续执行。
- **支持并行回放**：从库可以从 relay log 中拆分事务，由多个 worker 并行应用。
- **便于定位延迟**：可以分别观察“日志是否已经拉到从库”和“日志是否已经应用完成”。

从库自己的 binlog 默认不是 relay log 的替代品：

```
relay log：保存上游传入、等待本地应用的复制事件
binlog：保存本机作为复制源时对下游提供的变更事件
```

如果要做级联复制，可以开启：

```
log_replica_updates=ON
```

旧版本参数名为：

```
log_slave_updates=ON
```

此时从库应用上游事务后，会将对应变更写入自己的 binlog，供下游从库复制；但复制链路仍然是：

```
上游 binlog -> 本机 relay log -> 本机应用 -> 本机 binlog -> 下游
```

复制中的时间关系：

```
T1：主库执行写操作
T2：主库提交阶段写 binlog
T3：主库向从库发送 binlog
T4：从库 IO 线程写 relay log
T5：从库 SQL 线程应用事务
```

因此：

```
主库提交成功
!= 从库已经收到 binlog
!= 从库已经执行完成
```

IO 线程正常但 SQL 线程跟不上时，会表现为 relay log 堆积；IO 线程停止则优先排查网络、权限、复制位点、主库 binlog 文件和连接状态。

如何判断 IO 线程和 SQL 线程的进度？

```
show replica status\G
```

旧版本可以使用：

```
show slave status\G
```

重点关注：

- **Replica_IO_Running**：IO 线程是否正常运行。
- **Replica_SQL_Running**：SQL/applier 线程是否正常运行。
- **Read_Source_Log_Pos**：IO 线程从主库读取到的位置。
- **Exec_Source_Log_Pos**：SQL 线程已经应用到的位置。
- **Seconds_Behind_Source**：粗略的应用延迟指标，不能单独作为准确判断依据。

如果读取位点明显领先执行位点，通常说明日志已经传到从库，但 SQL/applier 线程应用速度不足。

复制模式

- 异步复制：性能好但可能丢数据。
- 半同步复制：至少一个从库确认后返回，降低丢失风险。
- 组复制：更强一致和高可用能力。

常见问题

- 主从延迟。
- 复制中断。
- 数据不一致。
- 大事务导致延迟。

面试追问：如何降低主从延迟？

避免大事务，优化从库 SQL 执行，开启并行复制，合理拆库拆表，读请求对一致性敏感时读主库。

深入：读写分离有什么坑？

结论先行：

读写分离能扩展读能力，但会引入主从延迟导致的读写不一致，需要按业务一致性选择读主、延迟判断或会话级一致性。

分层展开：

典型问题

- 写完马上读从库读不到。
- 从库延迟导致状态回退。
- 复杂查询压垮从库。

解决方案

- 写后短时间读主。
- 根据 GTID 或位点判断从库追平。
- 核心链路读主。
- 从库按业务隔离。

工程建议

- 不要把所有读都丢给从库。
- 监控复制延迟。
- 客户端路由要可配置。

面试追问：读写分离能提高写能力吗？

不能。写仍然主要受主库限制。写扩展通常需要分库分表、业务拆分或架构调整。

八、性能优化、容量规划与分库分表

36. 数据库并发和连接池如何估算？

```
concurrency ~= peak QPS * average DB latency seconds
instance connection budget <= DB safe connections / app instance count
```

Little's Law 只给初始估算，还要考虑慢 SQL 分位数、事务占用、后台任务和故障余量。连接池越大不代表吞吐越高，过多并发会增加上下文切换、锁竞争和内存。

37. 存储容量如何估算？

```
annual rows = peak or forecast daily rows * retention days
data bytes = rows * measured average row bytes
total bytes = data + all indexes + binlog/undo/temp headroom
```

使用真实表和索引实测，不只按字段长度相加；还需考虑页填充、主键进入二级索引、历史增长、备份空间、DDL 临时空间和副本倍数。

38. 慢 SQL 如何系统优化？

1. 从慢日志和链路确定业务影响、调用频率与参数。
2. 用执行计划判断扫描、回表、排序、临时表和 Join。
3. 检查数据分布、统计信息、锁等待和 Buffer Pool/IO。
4. 选择索引、SQL 改写、Seek 分页、批量、预聚合或缓存。
5. 在真实量级压测读写影响，灰度发布并观察回归。

不能只优化单次耗时，还要看调用次数和索引写放大。

39. 什么时候需要读写分离？

读流量显著高且允许复制延迟时，可把报表、列表等读分发到副本；写后立即读、状态机决策和强一致校验仍读 Primary 或使用一致性路由。

读写分离会引入延迟、路由、故障摘除和副本容量问题，不是所有查询自动发从库。

40. 什么时候需要分库分表？

先做索引、SQL、归档、缓存、读扩展和硬件治理。只有单库写入、容量、DDL/备份窗口或热点长期达到稳定上限，才进入分片。

分片前必须设计 Shard Key、全局 ID、跨分片查询、事务、扩容、数据校验、路由版本和回滚。分片把数据库问题转化为分布式系统问题。

41. 分片和迁移容量如何估算？

```
base shards >= max(
    peak write TPS / measured safe TPS per shard,
    forecast data / safe data per shard
)

migration time ~= data to copy / effective copy bandwidth
```

有效带宽要扣除线上流量、校验、重试和限速；双写难以天然一致，常用存量复制 + 增量日志 + 校验 + 灰度切流，并保留回退窗口。

深入：什么时候需要分库分表？

结论先行：

分库分表是为了解决单库单表容量、写入吞吐、索引膨胀和维护成本问题，不应过早引入。

分层展开：

触发因素

- 单表数据量过大，索引高度和维护成本上升。
- 写 QPS 接近单库瓶颈。
- 历史数据和热数据混在一起。
- DDL、备份、恢复变慢。

拆分方式

- 垂直拆分：按业务域拆库。
- 水平拆分：按用户 ID、订单 ID、时间等分片。
- 冷热拆分：热表保近期数据，历史归档。

代价

- 跨分片查询困难。
- 分布式事务复杂。
- 全局唯一 ID。
- 扩容迁移成本高。

面试追问：分表键怎么选？

选择高频查询条件、分布均匀、稳定不变、能减少跨分片访问的字段，例如用户维度业务常用 user_id。

深入：分库分表后如何生成全局 ID？

结论先行：

常见方案有雪花算法、号段模式、数据库自增步长、UUID，业务上通常优先雪花或号段。

分层展开：

雪花算法

- 时间戳 + 机器 ID + 序列号。
- 趋势递增，性能高。
- 依赖时钟，需处理回拨。

号段模式

- 从数据库批量申请 ID 段。
- 本地内存发号。
- 性能高且可控。

UUID

- 生成简单。

- 无序且较长。
- 不适合作为 InnoDB 主键热点写。

面试追问：雪花算法时钟回拨怎么办？

可短暂等待、切换备用 worker、记录逻辑时钟，严重回拨时拒绝发号并告警。

深入：如何做数据迁移和扩容？

结论先行：

数据迁移要保证不停机、可校验、可回滚，常用双写、增量同步、数据校验、灰度切流。

分层展开：

迁移流程

- 全量导历史数据。
- 增量同步新变更。
- 双写新旧链路。
- 校验数据一致性。
- 灰度读新库。
- 完成切流并保留回滚窗口。

风险点

- 双写失败。
- 增量乱序。
- 数据校验成本高。
- 切流后回滚复杂。

治理

- 操作幂等。
- 记录迁移状态。
- 分批执行。
- 核心指标监控。

面试追问：双写如何保证一致？

双写很难天然强一致，需要本地事务 + outbox、MQ 补偿、定期校验和人工兜底。核心数据切换要谨慎。

九、生产治理、安全与 Go 工程实践

42. MySQL 监控体系包括什么？

层次	指标
应用	QPS、错误、P99、连接池等待、超时

层次	指标
SQL	Top SQL、扫描/返回行、慢查询、排序、临时表
事务锁	活跃/长事务、锁等待、死锁、Undo 历史
实例	CPU、内存、Buffer Pool、磁盘、Redo、连接
复制	GTID/位点、Relay、应用延迟、错误和拓扑
业务	状态成功率、幂等冲突、补偿和对账差异

43. Online DDL 和 MDL 有什么风险？

Online DDL 仍需要 Metadata Lock。前方长事务可能让 DDL 等待，排队的 DDL 又阻塞后续 DML，形成请求堆积。

上线前确认版本和算法、表大小、磁盘临时空间、主从延迟、长事务和回滚；设置锁等待超时，小流量窗口执行并监控。大表可使用在线变更工具，但触发器、负载和切表同样要评估。

44. MySQL 安全如何治理？

- TLS 保护传输，账号按应用和环境隔离。
- 最小权限，不给业务账号 DDL、文件和管理权限。
- 凭证轮换并通过密钥系统管理，禁止写入代码和日志。
- 审计高风险查询、权限变更、导出和删除。
- 敏感字段加密/脱敏，备份同样加密和控权。
- 网络隔离、连接来源限制和异常登录告警。

45. Go database/sql 连接池如何配置？

- `SetMaxOpenConns` 受数据库总连接预算约束。
- `SetMaxIdleConns` 保留稳定复用但避免大量闲置。
- `SetConnMaxLifetime` 小于代理/数据库连接回收周期并加抖动。
- `SetConnMaxIdleTime` 回收长期空闲连接。
- 监控 `Stats()` 中 `InUse`、`Idle`、`WaitCount` 和 `WaitDuration`。

池满先查慢 SQL、长事务和泄漏，不直接扩大。多实例连接预算相加后必须低于数据库安全上限。

46. Go 如何正确处理超时、事务和结果集？

```
ctx, cancel := context.WithTimeout(parent, 800*time.Millisecond)
defer cancel()

tx, err := db.BeginTx(ctx, nil)
```

```
if err != nil { return err }
defer tx.Rollback()

// All statements use ctx and tx.
return tx.Commit()
```

- 使用 `QueryContext/ExecContext` 传递请求截止时间。
- `Rows` 必须及时 `Close`，循环后检查 `rows.Err()`。
- 事务内所有 SQL 必须走 `tx`，不要误用 `db` 逃出事务。
- `defer Rollback` 作为异常兜底，Commit 成功后 Rollback 无效。
- 网络超时后的提交结果可能不确定，重试必须有幂等键或状态查询。

47. Prepared Statement、批量写和连接泄漏怎么治理？

- 高频稳定 SQL 可使用参数化/预处理，避免拼接注入；注意服务端 Prepared 与连接绑定及缓存规模。
- 批量 Insert 控制包大小和事务行数，避免超大事、Redo/Binlog 峰值和复制延迟。
- 禁止循环单条查询形成 N+1，按 ID 分批查询。
- 连接泄漏常见于未关闭 Rows、事务未结束和 Context 缺失。
- 故障重试使用退避和总时限，避免连接风暴。

48. 线上故障如何分类排查？

SQL 慢：计划、扫描、回表、排序、IO
 锁慢：阻塞源、长事务、索引、死锁
 连接慢：池等待、泄漏、慢 SQL、连接风暴
 复制慢：大事、Applier、资源、DDL
 实例慢：CPU、Buffer Pool、Redo、磁盘、备份

先限制问题接口、暂停低优先级任务、摘除落后副本或终止非核心阻塞事务；保留证据后修复，最后用业务成功率和账验证恢复。

深入：数据库连接池怎么配置？

连接池参数不能直接按照双十一流量拍脑袋设置，核心是根据数据库最大连接数、应用实例数、SQL 平均耗时和数据库 CPU 余量计算。单实例最大连接数通常控制在几十以内，先通过 $QPS \times \text{平均响应时间}$ 估算并发，再结合数据库连接预算分配。连接池必须设置获取连接超时、SQL 超时、最大连接生命周期和空闲回收，避免连接池满后请求无限阻塞。大促系统不会靠扩大连接池解决问题，而是通过缓存、限流、MQ 削峰、读写分离和分片保护数据库。

结论先行：

连接池不是越大越好，核心是让应用并发和数据库承载能力匹配，避免连接泄漏、排队过长和慢 SQL 把连接池打满。

分层展开：

关键参数

- 最大打开连接数：限制同时打到 MySQL 的连接上限。
- 最大空闲连接数：减少频繁建连。
- 连接最大生命周期：避免长连接被网关、LB 或 MySQL 侧异常关闭。
- 连接空闲时间：回收长时间不用的连接。
- 获取连接等待时间：避免请求无限阻塞。

排查指标

- 连接池活跃连接数。
- 等待连接的请求数和等待时间。
- 慢 SQL 数量。
- 事务执行时间。
- MySQL 当前连接数和线程状态。

面试追问：连接池满了应该加连接吗？

不一定。先看是否有慢 SQL、长事务、下游阻塞、连接泄漏或突发流量。盲目加连接可能让 MySQL 并发更高、锁竞争更严重，最终把数据库压垮。

补充模块：线上治理与故障排查：发现问题、定位问题、解决问题

本模块回答“线上出问题怎么办”。慢 SQL、锁等待、连接池、主从延迟、DDL、CPU/IO 要按现象归类排查。

深入：如何优化慢 SQL？

结论先行：

慢 SQL 优化要先看执行计划和数据分布，再从索引、SQL 改写、分页方式、表结构和业务访问模式治理。

分层展开：

定位

- 慢查询日志。
- `EXPLAIN` 或 `EXPLAIN ANALYZE`。
- 查看扫描行数、回表、排序、临时表。

优化方向

- 建联合索引和覆盖索引。
- 避免 `select *`。
- 深分页改为游标或 seek 分页。

- 拆复杂 SQL。
- 冷热数据拆分。

工程治理

- 上线前 SQL 审核。
- 核心 SQL 压测。
- 慢 SQL 告警。
- 大表 DDL 走灰度或在线变更工具。

面试追问：深分页为什么慢？

`limit offset, size` 需要扫描并丢弃 offset 行，offset 越大越慢。可以用基于主键或时间的 seek 分页。

深入：大表 Online DDL 和 metadata lock 怎么治理？

结论先行：

大表变更最怕长时间锁表、阻塞写入和 metadata lock 堆积。线上变更要评估表大小、DDL 类型、MySQL 版本、业务峰谷、回滚方案和监控告警。

metadata lock 是什么：

MySQL 执行 DDL 时需要拿表的元数据锁。即使 DDL 本身支持 online，如果前面有长事务持有表引用，DDL 可能等待；DDL 一旦排队，又可能阻塞后续 DML，形成雪崩。

治理方式：

- 变更前查长事务和慢查询。
- 低峰期执行。
- 设置锁等待超时。
- 大表优先使用 online DDL 能力。
- 超大表可用 gh-ost / pt-online-schema-change。
- 分批回填字段，不在 DDL 中做重计算。
- 灰度发布读写新字段，保留回滚窗口。

gh-ost / pt-osc 思路：

- 创建影子表。
- 在影子表上完成新结构。
- 增量同步原表变更。
- 校验后短暂切换表名。

面试追问：大表加字段为什么危险？

危险不只在加字段本身，而在元数据锁、表重建、复制延迟、磁盘 IO、回滚困难和应用代码兼容性。正确做法是先评估 DDL 算法和锁级别，再设计灰度和回滚。

深入：MySQL 线上问题排障手册

结论先行：

MySQL 线上排障不是看到告警就统一限流或切主，而是先根据接口、数据库和复制指标判断故障类型，再执行针对性的止血动作。止血的目标是先阻止影响继续扩大，定位和修复则要在业务稳定后继续完成。

总流程：

发现告警 -> 判断影响面 -> 先限流/降级/切读主/暂停任务 -> 定位瓶颈 -> 修复 -> 复盘治理

总流程如何真正落地？

第一步：先确认哪里坏了

应用侧：接口 QPS、成功率、P95/P99、超时、错误码、连接池等待
MySQL 侧：CPU、内存、磁盘 IO、磁盘空间、网络、活跃连接
SQL 侧：慢 SQL 数量、TOP SQL、扫描行数、锁等待、临时表、排序
复制侧：主从延迟、relay log 堆积、IO/SQL 线程状态、复制错误
变更侧：新功能、新 SQL、新索引、DDL、批处理、定时任务

不要只看单个指标。例如 MySQL CPU 飙升但接口 QPS 没涨，可能是执行计划变化或全表扫描；接口 QPS 上涨但 MySQL CPU 平稳，可能主要被缓存或网关吸收。

第二步：根据现象选择止血动作

现象	第一优先级止血动作	不要直接做的事
新版本上线后慢 SQL 或错误增加	暂停发布、回滚应用或关闭功能开关	不要先盲目加索引
接口 QPS、MySQL CPU、连接数、IO 同时上升	网关限流、接口降级、关闭非核心查询、暂停低优先级任务	不要直接把连接池调大
慢 SQL 占满连接池	限制问题接口并发、暂停触发 SQL 的任务、设置超时	不要让请求无限等待连接
主从延迟大但主库正常	核心读切主、暂停读落后从库、摘除异常副本	不要继续把强一致读路由到落后从库
从库故障或复制中断	从读库列表摘除，追平后再接流量	不要把未追平从库重新加入读池
锁等待大量堆积	找阻塞事务，终止非核心长事务，暂停相关批任务	不要先扩大连接池
DDL 或 MDL 排队	暂停 DDL，必要时终止非核心长事务	不要继续提交更多 DDL
磁盘空间不足		

现象	第一优先级止血动作	不要直接做的事
	暂停日志和临时表增长源，清理可安全清理的数据并扩容	不要直接删除未知用途的数据库文件

这里的“限流”通常发生在网关、服务接口或任务消费者侧；“降级”通常是关闭报表、导出、非核心筛选等能力；“切读主”只针对读写分离，不能解决主库自身已经过载的问题。

第三步：根据现象建立故障假设

接口 QPS 上升 + MySQL CPU/IO 上升：
流量超过数据库承载，或新增流量触发了低效 SQL

QPS 基本不变 + 慢 SQL 突增：
执行计划变化、统计信息失真、数据分布变化或锁等待

只有某个账号/租户变慢：
数据倾斜、热点租户或单个索引范围过大

主库正常 + 从库延迟：
大事务、从库应用慢、DDL 或复制线程异常

发布后立即变慢：
新 SQL、新索引、参数变化或全量扫描

假设必须用指标、执行计划和实际数据验证，不能直接当作结论。

第四步：按流量、SQL、锁、数据和架构五个方向定位

1. 流量方向：比较故障前后的接口 QPS、请求参数、接口分布，确认是否某个接口、租户或缓存击穿贡献了主要增量。
2. SQL 方向：从慢日志或 APM 获取完整 SQL 和真实参数，使用 EXPLAIN，必要时在低风险环境使用 EXPLAIN ANALYZE，对比估算行数和实际扫描行数。
3. 锁方向：查看阻塞源事务、等待事务、锁定索引、锁模式、锁范围和事务持续时间，确认是否存在长事务、Gap Lock 或 Next-Key Lock。
4. 数据方向：统计 status、tenant_id、account_id、created_at 的数量分布，确认是否低区分度、热点租户、时间倾斜或统计信息过期。
5. 架构方向：只有在 SQL、索引、锁和访问模式都合理，但稳定流量仍长期超过单库能力时，才考虑读写分离、分片、归档或存储升级。

常用检查命令：

```
show full processlist;
show index from ad_task;
analyze table ad_task;
```

锁排查：

```
select * from information_schema.innodb_trx;
select * from performance_schema.data_locks;
select * from performance_schema.data_lock_waits;
```

数据分布：

```
select status, count(*) as cnt
from ad_task
where is_deleted = 0
group by status
order by cnt desc;

select account_id, count(*) as cnt
from ad_task
where is_deleted = 0
group by account_id
order by cnt desc
limit 20;

select date(created_at) as dt, count(*) as cnt
from ad_task
group by date(created_at)
order by dt desc
limit 30;
```

第五步：按根因选择修复方式

新 SQL 或索引缺失：

修复 SQL，设计匹配查询模式的联合索引

低区分度字段：

不只建 status 单列索引，结合账号、租户、时间设计联合索引

热点租户或账号：

增加租户/账号过滤，做限流、热点拆分、分片或归档

历史数据拖慢当前查询：

增加时间边界，改 seek 分页，冷热拆分和归档

单库容量已经长期到顶：

评估读写分离、分库分表、缓存、MQ 和读模型

因此，接口 QPS 上升后不能直接判断“要升级存储架构”。正确顺序是：

确认流量是否真实增加

- > 确认是否缓存击穿
- > 确认新增请求是否触发低效 SQL
- > 确认是否存在热点或数据倾斜
- > 确认单库是否达到稳定容量上限
- > 再决定索引、拆分、归档或升级架构

1. 慢 SQL 飙升

排查点	看什么	处理
慢日志	SQL、耗时、扫描行数、返回行数	找 TOP SQL
EXPLAIN	type、key、rows、Extra	判断索引、回表、排序、临时表
数据分布	是否热点租户/账号/状态	拆 key、加组合索引、冷热拆分
访问模式	是否新功能全量扫表	SQL 改写、分页改 seek、异步化

2. 锁等待/死锁

常见原因：

- 事务太长。
- 更新条件没走索引。
- 多事务加锁顺序不一致。
- 范围更新触发 gap lock / next-key lock。

处理方式：

- 先找阻塞源事务，必要时 kill 非核心长事务。
- 缩短事务边界，不在事务里调用外部接口。
- 更新条件必须命中索引。
- 统一资源加锁顺序。
- 应用层对死锁和锁等待超时做幂等重试。

3. 连接池耗尽

不要立刻加连接数，先判断：

- 是慢 SQL 占住连接？
- 是事务未提交？
- 是下游阻塞导致连接长时间不释放？
- 是突发流量超过 DB 承载？
- 是连接泄漏？

处理：

- 限流降级保护 DB。
- 优化慢 SQL。
- 设置合理超时。
- 控制事务耗时。
- 根据实例数和 DB 最大连接数重新分配连接池。

4. 主从延迟

常见原因：

- 主库大事务。
- 从库 SQL 重放慢。
- DDL 或大批量更新。
- 从库机器配置不足。
- 复制线程异常。

处理：

- 核心写后读走主库。
- 延迟过大时暂停读从或摘除落后从库。
- 避免大事务，批量任务分片分批。
- 开启并行复制并优化从库资源。
- 监控 seconds behind master、relay log 堆积、复制错误。

5. 大表 DDL 阻塞

风险：

- metadata lock 等待。
- DDL 排队后阻塞后续 DML。
- 表重建导致 IO 飙升。
- 主从延迟扩大。

治理：

- 变更前查长事务。
- 设置锁等待超时。
- 低峰执行。
- 大表用 gh-ost/pt-osc。
- 先加 nullable 字段，再灰度写入，再回填，再切读。

6. CPU/IO 飙升

可能原因：

- 大量全表扫描。
- filesort/temporary 暴增。
- Buffer Pool 命中率下降。
- redo/binlog 刷盘压力。
- 临时表落盘。
- 备份、归档、DDL、报表任务抢资源。

处理：

- 暂停非核心任务。
- 限流入口。
- 优先治理 TOP SQL。

- 报表迁移到从库/数仓/ES。
- 大任务分批、错峰、可暂停。

面试表达：

我排查 MySQL 问题不会只说“加索引”。我会先判断是 SQL 慢、锁慢、连接池慢、复制慢还是 IO 慢；先通过限流、降级、读写路由、暂停任务止血，再用慢日志、EXPLAIN、事务/锁视图、连接池指标和主从延迟指标定位，最后做索引、SQL、事务边界、任务拆分和容量治理。

十、业务架构与项目实战

49. 订单和库存如何保证正确？

- 订单号、支付流水号使用唯一约束保证幂等。
- 库存用条件更新：`stock >= quantity`，根据影响行数判断成功。
- 订单、扣减和流水若要求强一致，放在同一短事务。
- 不在持锁事务中调用支付或库存外部 RPC。
- 高并发入口可用缓存预扣和消息削峰，但 MySQL 条件更新、流水和对账是最终兜底。
- 失败通过状态机和补偿恢复，不直接回滚已完成外部副作用。

50. MySQL 与消息队列如何保证最终一致？

```
本地事务：业务表 + Outbox
-> Publisher 可靠投递
-> Consumer 幂等处理
-> 重试 / 死信
-> 状态扫描 / 对账 / 人工修复
```

“写 DB + 发消息”不是天然原子操作。Outbox 或 CDC 让事件可恢复，但通常仍是至少一次投递，消费端唯一约束、状态机和版本判断不能省略。

51. 广告投放工单分配如何设计？

- 工单、分配轮次、团队配额和操作流水建模为明确关系。
- 业务唯一键防止同一剧目/任务重复进入同轮分配。
- 通过 Version 乐观锁或条件更新抢占轮次和配额。
- 冲突有限重试并重新读取，不覆盖其他实例结果。
- 分配状态与关键流水同事务，通知和报表异步。
- 联合索引匹配项目、状态、负责人和更新时间列表。
- 监控冲突率、事务耗时、死锁、分配失败和规则公平性。

52. 设备事件消费如何写 MySQL?

- 事件以设备 ID 和事件 ID 幂等，唯一索引防重复。
- 使用 `INSERT ... ON DUPLICATE KEY UPDATE` 时确认是否真的允许覆盖，并通过 Version 防旧事件回退。
- 批量写降低 RTT，但限制批次和事务大小。
- 状态表保存最新值，事件流水异步归档或进入分析系统。
- 热点设备、索引写放大和主从延迟需要单独监控。
- 积压恢复按数据库安全写 TPS 限速，不能把消费能力直接压给主库。

53. 慢 SQL 项目经历如何表达?

我先通过链路和慢日志确定是投放工单列表，而不是泛泛看数据库 CPU；用真实租户参数检查执行计划，发现单列状态索引选择性低，查询还需要按负责人和更新时间过滤排序。随后设计匹配访问路径的联合索引，并把深分页改为基于时间和 ID 的 Seek 分页。上线前用生产量级数据验证扫描行、P99 和索引写入成本，灰度后观察连接池等待和主从延迟。最终不仅降低单次耗时，也减少了高峰连接占用。整个过程体现的是业务查询建模、证据定位和上线治理，而不是一句“加索引”。

54. 如何用两分钟回答 MySQL 架构题?

我先明确实体关系、权威数据和事务边界，用主键、唯一键和状态机保证基础正确性；再根据高频过滤、排序和分页设计联合索引，并通过执行计划和生产量级压测验证。并发写优先条件更新或乐观锁，必要时使用短悲观事务，外部调用不放在事务内。持久化依赖 Redo、Undo、Binlog 和两阶段提交，高可用按 RPO/RTO 选择复制与切换方案，并通过备份加 Binlog 做 PITR。容量上根据 QPS、延迟、数据增长和索引空间计算连接、存储和分片时点。Go 服务统一 Context 超时、连接池预算、事务封装和幂等重试。线上从 SQL、锁、连接、IO 和复制建立证据链。我的管理经验只帮助风险识别和协作推进，岗位定位仍是负责业务架构和交付的资深后端 IC。

补充模块：MySQL 实战设计：会用、用好、知道边界

本模块回答“怎么把 MySQL 用到业务设计里”。重点是事务边界、状态机、幂等、条件更新和最终一致。

深入：订单系统数据库怎么设计?

结论先行：

订单系统要围绕用户查询、商家查询、状态流转、支付履约、分库分表和一致性补偿设计。

分层展开：

核心表

- 订单主表。
- 订单明细表。
- 支付单。
- 履约单。
- 状态流转表。

关键设计

- 订单号全局唯一。
- 状态机控制流转。
- 金额字段用整数分。
- 幂等号防重复提交。

查询模型

- 用户订单列表。
- 商家订单列表。
- 订单详情。
- 售后和支付查询。

稳定性

- 下单链路短事务。
- 支付回调幂等。
- MQ 异步通知。
- 定时任务补偿。

面试追问：订单状态为什么要有流转表？

主表保存当前状态，流转表保存历史轨迹，方便审计、排障和补偿。

深入：如何保证库存扣减正确？

结论先行：

库存扣减要通过条件更新、幂等、事务和补偿保证不超卖；高并发场景可引入 Redis 预扣和异步落库。

分层展开：

数据库方案

- `update sku set stock = stock - n where id = ? and stock >= n`。
- 根据影响行数判断是否成功。
- 同事务记录扣减流水。

锁的工程使用

- 优先使用带条件的原子 UPDATE，让 InnoDB 自动对目标索引记录加 X 锁，并通过影响行数判断扣减是否成功。

- 必须先查后改时，在同一个显式事务中使用 SELECT ... FOR UPDATE，当前读读取最新已提交版本并持有锁到提交或回滚。
- 扣库存、创建订单、写扣减流水需要强一致时放在同一个短事务中；不要在持锁事务中调用外部 RPC。
- 抢具体座位时使用座位唯一索引加条件更新，保证一张座位只能被一个订单抢占。
- 秒杀入口使用限流、Redis 预扣和 MQ 削峰，MySQL 通过条件更新、唯一索引、幂等订单号和对账补偿做最终兜底。
- 无论使用 Redis 锁还是数据库锁，最终库存和订单状态都必须由 MySQL 约束和事务保证。

高并发方案

- Redis 预扣库存。
- MQ 异步落库。
- 定时对账补偿。
- 热点商品单独隔离。

关键风险

- 重复扣减。
- 回滚库存失败。
- Redis 和 DB 不一致。
- 活动结束后的对账。

面试追问：乐观锁版本号和条件扣减怎么选？

库存扣减通常条件更新更直接；版本号适合完整对象并发更新，但热点库存频繁冲突时会增加重试成本。

深入：MySQL 和 MQ 如何保证最终一致？

典型场景：

知乎统一发布服务中，发布主流程成功后，需要通知下游内容状态、审核、推荐、搜索等系统。如果 DB 更新成功但消息发送失败，就会出现状态不一致。

可靠方案：

- 本地事务同时写业务表和消息表。
- 后台任务扫描未发送消息并投递 MQ。
- 消费端用业务唯一键做幂等。
- 消息发送成功后标记消息表状态。
- 长时间失败的消息进入告警和人工补偿。

面试总结：

我一般不把“写 DB + 发 MQ”当成天然原子操作，而是用本地消息表或事务消息思想，把不确定性显式建模出来，再用补偿任务保证最终一致。

结论先行：

连接池配置要根据服务实例数、DB 最大连接数、SQL 耗时和峰值 QPS 计算，不能越大越好。

分层展开：

关键参数

- 最大打开连接数。
- 最大空闲连接数。
- 连接最大生命周期。
- 空闲连接最大时间。

配置原则

- 总连接数不能打爆 DB。
- 慢 SQL 会占用连接，导致排队。
- 连接生命周期要小于数据库或代理超时。

监控指标

- in-use 连接。
- idle 连接。
- wait count。
- wait duration。
- SQL P99。

面试追问：连接池满了应该加连接吗？

不一定。先看是否有慢 SQL、事务过长、下游阻塞。盲目加连接可能把 DB 压垮。

补充模块：项目实战表达：把 MySQL 讲成业务一致性能力

本模块用于项目深挖，突出你在广告投放、任务状态、工单分配、事件消费中的数据库设计和治理能力。

深入：枫叶互动里哪些数据适合放 MySQL？

结论先行：

在广告投放平台里，MySQL 更适合承载结构化强、需要事务一致性、查询关系明确的数据，比如投放工单、分配规则、绩效统计、轮次状态、配对历史、权限审计等。

项目例子：投放工单自动分配

短剧投放需要把剧目分配给不同投放组，并考虑主投/辅投、轮次、公平性、周末均衡、历史配对等规则。这个场景有明确的实体关系和事务边界，更适合 MySQL。

表设计思路：

- `delivery_work_order`：工单主表。
- `delivery_group_stat`：投放组统计。

- `delivery_round_state`：轮次状态。
- `delivery_pair_history`：历史配对。
- `delivery_round_log`：分配日志。

为什么不用 MongoDB?

因为这里不是字段频繁变化的半结构化配置，而是强规则、强约束、强一致的业务流程。MySQL 的事务、唯一索引、条件更新和执行计划更适合。

深入：投放工单并发分配如何保证一致性？

业务问题：

多个运营人员或自动任务可能同时触发工单分配。如果没有并发控制，可能出现同一剧目重复分配、投放组统计错误、轮次状态错乱。

方案：

- 工单创建和分配统计放在同一个事务里。
- 对轮次状态或组统计使用 version 乐观锁。
- 更新时带条件：`where id = ? and version = ?`。
- 如果 affected rows 为 0，说明发生并发冲突，重新加载最新状态并重试。
- 重试次数有限，超过后返回业务错误或进入人工处理。

回答示例：

```
update delivery_round_state
set current_round = ?, version = version + 1, updated_at = now()
where id = ? and version = ?;
```

追问：为什么不用悲观锁？

如果冲突不高，乐观锁吞吐更好，锁持有时间短。悲观锁适合强冲突、必须串行的场景，但会降低并发能力。投放分配可以接受少量重试，所以用乐观锁更合适。

深入：360 事件消费里 INSERT ... ON DUPLICATE KEY UPDATE 怎么优化？

业务背景：

家庭防火墙设备会上报上下线、固件版本、App 行为等事件。高峰 QPS 较高，部分数据需要按设备维度更新最新状态。如果每条事件都先查再写，会增加 DB 往返和并发冲突。

方案：

使用唯一键约束设备维度，例如 `device_id + event_type`，写入时使用 upsert：

```
insert into device_event_state(device_id, event_type, event_time, payload, updated_at)
values (?, ?, ?, ?, now())
on duplicate key update
  event_time = values(event_time),
  payload = values(payload),
  updated_at = now();
```

优化点：

- 唯一键设计要精准，避免误合并。
- 批量写入减少网络往返。
- 只更新必要字段，避免无意义写放大。
- 对热点设备或热点事件可先用 MQ 削峰。
- 如果主库压力高，评估异步落库、分表或冷热分离。

深入：MySQL 慢 SQL 如何结合你的项目回答？

推荐案例：投放工单列表/绩效报表慢查询

排查步骤：

1. 看慢日志，确认 SQL、耗时、扫描行数、返回行数。
2. `explain` 看 type、key、rows、Extra。
3. 判断 where 条件是否符合联合索引最左前缀。
4. 检查是否存在函数、隐式类型转换、`like '%xx'`、大 offset。
5. 区分是单条 SQL 问题，还是连接池、锁等待、主库压力问题。

优化方向：

- 工单列表按业务高频筛选设计联合索引，例如状态、负责人、项目、更新时间。
- 报表类查询尽量走离线/准实时聚合表，不直接扫明细。
- 大分页改成游标分页或 `search_after` 类似方案。
- 对复杂检索需求迁移 ES，MySQL 保留权威数据。

02-03 | MongoDB文档数据库

技术 | MongoDB 文档数据库面试题

面向 9 年以上 Golang Web 后端业务研发岗位。重点不是记命令，而是围绕聚合文档建模、索引和查询计划、读写一致性、副本集、分片、容量治理与业务边界形成完整判断。

一、MongoDB 学习地图与资深回答框架

1. MongoDB 的知识图谱主线是什么？

一句话结论： 从“为什么选择文档模型”出发，沿着聚合根建模、BSON 存储、索引查询、并发事务、副本集、分片、生产治理和业务实战逐层深入。

业务访问模式与能力边界

- > Database / Collection / BSON Document
- > 聚合根、嵌入与引用
- > WiredTiger、Cache、Checkpoint、Journal
- > 索引、Query Planner、Aggregation
- > 原子更新、事务、Read/Write Concern
- > Replica Set、Oplog、选举和回滚
- > Sharding、Chunk、Balancer、Shard Key
- > 容量、治理、排障和项目表达

2. 资深后端如何回答 MongoDB 问题？

采用“结论 -> 访问模式 -> 原理 -> 方案 -> 风险 -> 落地指标”的结构。

示例：嵌入还是引用？

- **结论：** 一起读取、一起更新且规模有界的数据优先嵌入；生命周期独立、关系多对多或会无限增长的数据使用引用。
- **访问模式：** 先说明核心查询和更新路径，而不是先谈范式。
- **原理：** 嵌入可用单文档原子性减少查询，引用可控制文档大小和更新放大。
- **方案：** 主信息嵌入，明细流水独立集合，必要时冗余少量展示字段。
- **风险：** 大数组、热点文档、重复数据同步和跨集合一致性。
- **指标：** 文档大小、P99、扫描文档数、更新冲突、工作集和增长速度。

3. MongoDB 场景设计题如何展开？

1. 明确实体边界、读写比例、查询组合、排序、分页和增长模型。
2. 确定聚合根，选择嵌入、引用和必要冗余。
3. 由高频查询反推联合索引和字段顺序。
4. 明确单文档原子性是否足够，事务是否真的必要。
5. 选择读偏好、读关注和写关注，说明一致性与延迟权衡。
6. 评估副本集、分片键、容量、备份、归档和故障恢复。
7. 给出 Go 连接池、超时、幂等、监控、降级与迁移方案。

4. MongoDB 高频题如何串联？

层次	核心问题	必须讲出的关键词
定位	为什么用 MongoDB	聚合文档、灵活结构、访问模式
建模	嵌入还是引用	一起读写、有界性、原子边界
存储	WiredTiger 做什么	Cache、MVCC、Checkpoint、Journal
索引	为什么走/不走索引	ESR、覆盖、选择性、Explain
一致性	如何保证正确	单文档原子、事务、Concern、版本
高可用	主节点挂了怎么办	Oplog、选举、回滚、Majority
扩展	Shard Key 怎么选	基数、均匀、定向、单调热点
治理	慢和抖动怎么查	扫描比、Cache、Ticket、磁盘、复制延迟
实战	项目如何表达	边界、权衡、指标、迁移和复盘

二、使用场景、文档建模与能力边界

5. MongoDB 适合哪些业务场景？

- 字段变化较快、不同类型记录结构存在差异。
- 数据天然以聚合文档整体读写，如内容、素材、配置、用户画像。
- 需要嵌套对象、数组和丰富的单文档更新表达式。
- 查询主要围绕聚合根，跨实体 Join 和强约束较少。
- 希望通过副本集和分片水平扩展读写与容量。

适合不等于“没有 Schema”；生产系统仍需应用层校验、Schema Validation、索引和演进策略。

6. 什么场景不适合 MongoDB?

- 大量跨实体事务、外键约束和复杂关系查询。
- 财务账务等要求严格关系模型和审计约束的核心事实。
- 数据访问高度随机，无法形成稳定聚合边界或索引组合。
- 数组或嵌套对象会无限增长，导致巨型热点文档。
- 团队只是为了“字段随便加”而引入，缺少容量和治理能力。

7. MongoDB、MySQL、ES、Redis 和 Kafka 如何分工?

组件	最擅长	典型边界
MySQL	事务、关系、约束、权威结构化数据	灵活嵌套和全文检索成本较高
MongoDB	聚合文档、灵活模型、单文档原子更新	复杂 Join 与全局强约束较弱
Elasticsearch	全文召回、相关性、组合过滤和聚合	不作为交易事实源
Redis	热点缓存、计数、限流和短期状态	不保存大规模主数据
Kafka	变更事件、异步解耦、削峰和重放	不提供随机业务查询

选择 MongoDB 应回答“文档聚合带来了什么业务收益”，而不是只说 JSON 写起来方便。

8. MongoDB 与 MySQL 建模思想有什么不同?

- MySQL 通常从实体关系、范式、约束和事务边界出发。
- MongoDB 通常从访问模式和聚合根出发，允许为读性能做反范式冗余。
- MySQL 通过 Join 在查询时组合；MongoDB 倾向在写入时把一起读取的数据组织到同一文档。
- MongoDB 仍可引用和使用 `$lookup`，但不应把关系模型原样搬过去。

9. 什么是聚合根，为什么它是 MongoDB 建模核心?

聚合根是业务上具有一致生命周期和事务边界的一组数据的入口。例如广告任务包含基本配置、定向摘要和当前状态，可作为一个聚合；执行流水则增长无界，应独立集合。

好的聚合根同时满足：高频一起读取、更新边界清晰、大小可控、热点可接受、生命周期相近。

10. 嵌入文档和引用如何选择?

优先嵌入： 一对一或有界一对少、一起查询、一起更新、需要单文档原子性。

优先引用： 一对多无界、多对多、子实体独立访问/更新、共享实体、文档可能接近大小上限。

折中方式是引用主实体，同时冗余名称、状态等展示快照；用事件同步冗余字段，并接受短暂最终一致。

11. 如何处理不断增长的数组？

不要将日志、评论、执行记录、成员历史无限 push 到一个文档。可采用：

- 子项独立集合，使用 parent_id + 时间联合索引。
- Bucket Pattern 按数量或时间分桶。
- 主文档只保留最新 N 条摘要和计数。
- 冷数据归档到历史集合或对象存储。

这同时避免文档搬迁、更新冲突、单文档大小限制和热点写。

12. Schema 灵活是否意味着不需要版本管理？

恰恰相反。需要：

- 用 `schema_version` 标记文档版本。
- 新代码同时兼容旧字段一段时间。
- 写新读旧、后台迁移、校验、灰度和回滚。
- 对核心字段启用 Schema Validation。
- 禁止相同字段在不同文档中出现混乱类型。

三、BSON、WiredTiger、存储与请求执行链路

13. BSON 与 JSON 有什么区别？

BSON 是二进制文档格式，除字符串、数字、布尔和数组外，还支持 ObjectId、Date、Decimal128、Binary 等类型，便于数据库编码、遍历和驱动映射。

代价是字段名会随文档重复存储，某些类型比紧凑二进制格式更占空间。Golang 中要明确 `int32/`
`int64`、时间、Decimal 和空值语义，避免跨语言类型漂移。

14. ObjectId 的组成和使用边界是什么？

ObjectId 具备全局低冲突生成和大致时间有序特性，适合默认 `_id`。但它不等于严格连续 ID，也不应依赖其时间部分代替业务时间。

若业务已有稳定唯一 ID，可直接用作 `_id`，减少一次唯一索引并方便幂等；分片场景仍要评估 `_id` 与 Shard Key 的关系。

15. WiredTiger 的核心职责是什么？

- 管理 B-Tree、页面、压缩和磁盘文件。
- 使用 Cache 保存内部页、数据页和脏页。

- 通过 MVCC 提供快照读和并发控制。
- 使用 Journal 与 Checkpoint 实现持久化和崩溃恢复。
- 写操作主要采用文档级并发控制和乐观冲突检测。

数据库慢不只看 CPU，还要看工作集是否装入 Cache、脏页回写、Checkpoint、磁盘延迟和读写排队。

16. WiredTiger Cache 与操作系统页缓存是什么关系？

WiredTiger Cache 保存解压后的页面和脏数据，操作系统页缓存还可能缓存压缩后的数据文件。把机器内存全部分给 WiredTiger 会挤压 OS 和其他进程。

应观察 Cache used、dirty bytes、eviction、pages read into cache、pages written、应用延迟和磁盘，而不是只盯一个使用率。

17. Checkpoint 和 Journal 分别保证什么？

- Journal 记录尚未进入稳定 Checkpoint 的写入意图，用于崩溃恢复。
- Checkpoint 将某个一致时间点的脏页状态持久化，并推进可恢复基线。

写关注决定客户端等待哪些确认；Journal/Checkpoint 解决节点内部持久化，副本集 Majority 解决跨节点确认，它们不是同一个层次。

18. 一条查询请求如何执行？

1. Driver 依据拓扑、Read Preference 和 Server Selection 选择节点。
2. `mongod` 解析过滤、排序、投影和会话语义。
3. Query Planner 生成候选计划，选择索引扫描或集合扫描。
4. 执行器从索引定位 RecordId，必要时 FETCH 文档并过滤。
5. WiredTiger 从 Cache 或磁盘读取页面。
6. 结果按 Batch 返回，Driver Cursor 继续 `getMore`。

慢查询可能出在选节点、排队、计划、扫描、FETCH、排序、磁盘或客户端消费任一阶段。

19. 一条写请求如何执行？

1. Driver 选择 Primary 并携带 Write Concern、Session 和超时。
2. Primary 校验文档、索引和唯一约束，执行更新并处理写冲突。
3. 变更写入本地存储和 Oplog。
4. Secondary 拉取 Oplog 并按顺序应用。
5. 达到 Write Concern 后向客户端确认。

网络超时时结果可能未知，应用必须依赖稳定业务 ID、条件更新或事务编号保证重试幂等。

20. MongoDB 为什么也会有锁和写冲突？

WiredTiger 并非无锁。MongoDB 在全局、数据库、集合层使用意向锁，在文档层做并发控制；两个事务修改同一文档可能发生 write conflict 并重试。

长事务、大批更新、DDL、热点单文档和索引构建仍会影响并发。业务设计不能因为“文档级锁”就忽略热点。

四、索引、Query Planner、Explain 与聚合查询

21. MongoDB 有哪些常见索引？

- 单字段和联合索引。
- 唯一索引、稀疏索引和 Partial Index。
- Multikey 数组索引。
- TTL 索引。
- Text、Geospatial、Hashed、Wildcard 索引。
- Hidden Index，用于评估隐藏后的影响。

索引不是越多越好：每个索引都消耗磁盘、Cache 和写入维护成本。

22. 联合索引如何按照 ESR 原则设计？

ESR 是常用启发式：Equality、Sort、Range。

```
{ tenant_id: 1, status: 1, created_at: -1 }
```

若查询固定等值过滤租户和状态，再按时间倒序，此顺序通常合理。但 ESR 不是定律：字段选择性、查询比例、范围大小、排序是否可避免内存排序都要用 Explain 和压测验证。

23. 什么是最左前缀和索引边界？

联合索引按字段顺序组织。缺少前导字段，后续字段通常无法形成高效连续边界；前导范围条件之后的字段也可能无法继续缩小扫描范围。

面试中应结合具体查询说明 `indexBounds`、`keysExamined` 和 `docsExamined`，而不是只背“最左匹配”。

24. Multikey 索引有什么特殊限制和风险？

数组字段会为多个元素生成索引键，数组越大写放大越明显。联合索引中多个数组字段会受到限制，数组元素组合查询还需 `$elemMatch` 保证条件落在同一元素。

大数组不应靠加索引硬撑，应重新评估文档边界和子集合建模。

25. Partial、Sparse 和 TTL 索引如何选择？

- Partial Index 只索引满足表达式的文档，控制更精确，适合只查询有效状态的数据。
- Sparse Index 跳过缺失字段，语义较粗，唯一约束要特别小心。
- TTL Index 后台异步删除过期文档，删除不是到点即时发生，不适合精确业务定时器。

26. 覆盖索引为什么快？

当过滤、排序和返回字段都可从索引获得时，可以避免回表 FETCH 完整 BSON 文档，减少 Cache 和磁盘访问。

代价是索引更宽、写入更慢、内存占用更大。仅对高频且字段稳定的查询设计覆盖索引。

27. Explain 必须看哪些指标？

- `winningPlan`：COLLSCAN、IXSCAN、FETCH、SORT 等阶段。
- `totalKeysExamined` 与 `totalDocsExamined`。
- `nReturned`，用于计算扫描/返回比。
- 是否出现内存排序、重复键消除和大量 FETCH 后过滤。
- 执行时间、索引边界和各 Shard 表现。

一个查询“用了索引”不等于高效；若扫百万 key 只返回十条，仍需优化。

28. Query Planner 为什么可能选错索引？

数据分布变化、参数差异、相似索引过多、统计信息和计划缓存都可能导致某个计划在部分条件下退化。

应先保存 Explain 和真实参数，比较候选索引与扫描量；必要时调整索引、查询形态或清理/分析计划缓存。`hint` 适合作为受控手段，不应成为长期掩盖建模问题的默认方案。

29. 深分页为什么慢，如何优化？

`skip N` 仍需扫描并丢弃前 N 条，页数越深成本越高。推荐基于稳定排序键做 Seek Pagination：

```
db.tasks.find({
  tenant_id: "t1",
  created_at: { $lt: lastCreatedAt }
}).sort({ created_at: -1, _id: -1 }).limit(50)
```

游标应同时携带 `created_at` 和 `_id`，解决相同时间的稳定翻页。随机跳页需产品或预计算方案配合。

30. Aggregation Pipeline 如何优化？

- 尽早 `$match` 和 `$project`，缩小文档和字段。

- 让过滤与排序命中联合索引。
- 谨慎使用 `$lookup`、`$unwind`、`$group` 和大范围 `$sort`。
- 限制结果集和内存，观察是否落盘。
- 复杂报表可预聚合、异步计算或交给分析系统。
- 在分片集群中关注各 Shard 执行和 mongos 归并成本。

五、原子更新、事务、并发控制与一致性语义

31. MongoDB 的原子性边界是什么？

单文档写入天然原子，包括更新文档内多个字段和有界数组。合理聚合建模可以把很多业务一致性收敛到单文档，无需事务。

跨文档、跨集合或跨 Shard 操作才考虑多文档事务；事务不是弥补错误文档边界的默认工具。

32. 如何实现乐观并发控制？

使用版本号或当前状态进入更新条件：

```
db.tasks.updateOne(
  { _id: taskId, version: expectedVersion, status: "pending" },
  { $set: { status: "running" }, $inc: { version: 1 } }
)
```

检查 `matchedCount/modifiedCount` 判断是否竞争失败。它适合状态机、库存配额、任务领取等场景，并天然支持幂等重试。

33. 多文档事务如何工作，代价是什么？

事务基于 Session、快照视图、WiredTiger MVCC 和副本集/分片协调提交。它能提供 ACID，但会延长快照和锁生命周期、增加缓存压力、冲突、网络往返和分片协调成本。

原则：事务要短、小、有界；禁止在事务中调用慢外部服务；为 `UnknownTransactionCommitResult` 和 `TransientTransactionError` 设计正确重试。

34. Read Concern、Write Concern 和 Read Preference 分别控制什么？

机制	控制的问题
Read Concern	能读到什么一致性/持久性级别的数据
Write Concern	写入需等待多少副本及持久化确认
Read Preference	请求发往 Primary 还是 Secondary

三者必须组合讨论。读 Secondary 并不自动提高一致性；写 Majority 后立即读延迟副本，仍可能读不到最新数据。

35. w: majority 保证了什么，没保证什么？

它要求写入达到多数确认边界，提高节点故障后保留写入的概率。它不保证业务跨系统事务、不消除网络超时的结果未知，也不意味着任意 Secondary 立即可见。

应用仍需幂等键、合理超时、读关注和补偿机制。

36. 如何实现“写后读到自己刚写的数据”？

可选方案：

- 写后读 Primary。
- 使用因果一致 Session，结合合适的 Read/Write Concern。
- 将写入结果直接返回并在短时间内由调用方使用。
- 对允许最终一致的页面展示“处理中”状态。

不要为了所有查询都强制最强语义；按订单确认、后台列表、统计报表分别定义一致性。

37. MongoDB 事务与 MySQL 事务如何比较？

两者都可提供 ACID，但 MongoDB 的优势仍是聚合文档和单文档原子性，MySQL 的优势是成熟关系约束、Join 和事务建模。若业务长期依赖大量跨文档事务和复杂关联，通常说明 MySQL 更贴合主模型，而不是证明 MongoDB 也能勉强实现。

38. 幂等写如何设计？

- 用业务唯一键或事件 ID 建唯一索引。
- Upsert 时把唯一条件放入过滤器。
- 状态迁移使用 `current_status + version` 条件更新。
- 消费事件记录 `last_event_version`，拒绝旧版本。
- 网络超时后先按业务键查询结果，再决定是否重试。

六、副本集、Oplog、选举与 Change Streams

39. 副本集的核心对象和链路是什么？

副本集通常包含一个 Primary 和多个 Secondary。Primary 接收写入并追加 Oplog，Secondary 持续拉取和应用 Oplog，成员通过心跳感知状态并在 Primary 故障时选举。

它解决高可用和数据冗余，但应用仍需处理短暂不可写、连接切换、重试和读延迟。

40. Oplog 是什么，为什么容量重要？

Oplog 是有界的复制操作日志，Secondary 用它追赶 Primary，Change Streams 也依赖可用的变更历史。

若 Secondary 离线时间超过 Oplog 覆盖窗口，就无法增量追赶，需要 Initial Sync。容量应按峰值写入、停机维护窗口和异常恢复时间估算，并监控时间窗口而非只看字节。

41. Primary 故障后会发生什么？

1. 成员通过心跳发现 Primary 不可达。
2. 满足条件的 Secondary 发起选举，获得多数票后成为新 Primary。
3. Driver 发现拓扑变化并选择新 Primary。
4. 客户端在选举窗口内可能收到不可写或超时错误。
5. 旧 Primary 恢复后回到 Secondary，未被多数确认的分叉写可能回滚。

业务要设置合理 Server Selection Timeout，并仅对幂等操作进行有界重试。

42. 什么是 Rollback，如何降低风险？

旧 Primary 上尚未进入新多数派历史的写入，在重新加入时可能被回滚。降低风险的方法包括使用 Majority Write Concern、合理副本拓扑、稳定网络、监控复制延迟和避免在失去多数时强行重配。

关键业务还需通过对账和业务幂等处理异常，不能只依赖数据库副本。

43. 为什么通常推荐奇数个投票成员？

多数派需要超过半数投票。三个投票成员可容忍一个故障，五个可容忍两个；增加偶数投票成员通常不增加同等故障容忍，却增加部署和协调成本。

投票成员、数据副本和只读扩展不是同一概念，拓扑需结合故障域和成本设计。

44. Secondary 读有什么收益和风险？

收益是分担可容忍陈旧的读取、就近读取和报表负载。风险是复制延迟、读到可回滚数据、读写不一致和 Secondary 被重查询拖慢复制。

订单确认等强写后读走 Primary；报表和推荐可按最大陈旧度使用 Secondary，并隔离分析节点。

45. Change Streams 如何工作，适合什么场景？

Change Streams 在复制日志之上提供集合、数据库或集群级变更订阅，事件包含 resume token，可用于缓存失效、搜索索引同步和审计通知。

它不是业务消息队列的完全替代：需要处理重复、断线恢复、Oplog 窗口、权限、事件顺序边界和下游背压。重要业务通常再进入 Kafka，获得更长保留、消费组和重放能力。

46. 副本集如何备份和恢复？

- 小规模可用逻辑备份，但恢复时间随数据量增长。
- 大规模通常使用一致性快照、托管备份或存储快照。
- 需要支持时间点恢复时保留 Oplog/增量日志。
- 备份必须异地、加密、校验并定期恢复演练。
- 明确 RPO、RTO、恢复顺序、索引重建时间和应用回切步骤。

七、Sharding、Shard Key、Chunk 与水平扩展

47. MongoDB 分片集群由哪些组件组成？

- **mongos**：无状态查询路由与结果归并入口。
- Config Server Replica Set：保存集群元数据和 Chunk 分布。
- Shard：真正保存数据，每个 Shard 通常是副本集。
- Balancer：迁移 Chunk 以改善分布。

分片解决容量和吞吐扩展，但引入路由、迁移、跨 Shard 查询和事务复杂度。

48. Shard Key 为什么是最重要的设计？

Shard Key 决定文档分布、查询能否定向、写入是否热点、Chunk 能否均衡以及未来扩容上限。一个差的 Shard Key 很难低成本修改。

评估四维：

1. **基数**：能否形成足够多分区。
2. **频率**：值分布是否均匀，是否有超级大租户。
3. **单调性**：是否把新增写集中到尾部 Chunk。
4. **查询相关性**：高频查询是否携带 Shard Key，从而定向路由。

49. Range Sharding 和 Hashed Sharding 如何取舍？

- Range 保留相邻值局部性，适合范围查询，但单调递增键易产生写热点。
- Hashed 分布更均匀，适合点查和写扩散，但范围查询会广播。
- 复合 Shard Key 可用租户、哈希或时间组合平衡路由与写入。

选择必须基于真实查询矩阵和最大租户模型，不能只说哈希更均匀。

50. 什么是 Scatter-Gather 查询？

查询不包含可定向的 Shard Key 条件时，mongos 将请求发往多个甚至全部 Shard，再归并结果。它会随 Shard 数放大 CPU、网络、尾延迟和排序聚合成本。

治理方法是调整 API 约束、索引和 Shard Key，使核心查询可定向；无法定向的运营报表应隔离或预计算。

51. Chunk、Balancer 和迁移如何工作？

数据按 Shard Key 范围组织为 Chunk。Balancer 根据分布和 Zone 约束触发迁移，将数据从源 Shard 复制到目标 Shard，提交元数据后清理旧数据。

迁移消耗网络、磁盘和 Cache，业务高峰期要限速或设置窗口。监控不能只看 Chunk 数，还要看数据大小、迁移队列、热点和查询分布。

52. Jumbo Chunk 是什么，如何治理？

当 Chunk 因低基数、不可分割键分布或超大相同键值而无法正常拆分迁移时，可能形成 Jumbo。根因往往是 Shard Key 数据分布不合理。

短期可评估手动拆分、迁移或归档；长期需重新设计 Shard Key、Resharding 或重建集合。操作前必须验证版本能力和回滚路径。

53. 什么时候才应该分片？

- 单副本集存储、IO、CPU 或写吞吐经过优化后仍接近上限。
- 工作集远超单机内存且业务无法归档拆分。
- 需要跨机器扩展容量，并有稳定 Shard Key。

不要把分片作为慢查询的第一解决方案。无索引查询分片后只会变成并行扫描更多机器。

八、性能优化、容量规划与 Golang 工程实践

54. MongoDB 容量如何估算？

```
文档平均 BSON 大小 x 文档数量
+ 全部索引大小
+ Oplog
+ Journal/临时空间
+ 副本倍数
+ 增长与安全余量
```

同时估算工作集：热点文档页、热点索引页和写入脏页能否进入 Cache。容量方案必须在单节点故障、Initial Sync、备份和索引构建时仍有余量。

55. 如何系统优化慢查询？

1. 保存真实过滤、排序、投影、耗时和业务上下文。
2. Explain 检查 winningPlan、扫描/返回比和排序。
3. 调整查询和联合索引，减少 FETCH 与返回字段。
4. 治理深分页、无界数组、`$lookup` 和大范围聚合。
5. 检查 Cache、eviction、读写排队、磁盘、CPU 和复制延迟。
6. 最后再考虑读扩展、分片或数据归档。

56. 如何处理热点文档和热点租户？

- 把高频计数拆成分桶计数或 Redis 聚合后异步落库。
- 将无界子项拆成独立集合。
- 对超大租户使用更细分区键或独立集合/集群。
- 条件更新失败采用有界退避，避免并发自旋。
- 通过 tenant_id、document_id、operation 指标识别倾斜。

57. 大批量写入如何设计？

- 使用 `bulkWrite`，根据业务选择 ordered/unordered。
- 控制批次字节数、并发和连接池，建立背压。
- 预先创建正确索引，避免导入中途反复建索引。
- 逐项解析错误，区分重复键、校验失败和临时错误。
- 大迁移要限速，监控复制延迟、Oplog 窗口、Cache 和磁盘。

58. Golang MongoDB Driver 如何治理？

- 复用全局 `mongo.Client`，不要每请求 Connect/Disconnect。
- 所有操作使用 `context` deadline，并区分服务选择、连接、Socket 和业务超时。
- 按压测配置 `maxPoolSize/minPoolSize/maxConnecting` 等参数，避免连接风暴。
- Cursor 必须关闭并检查 `cursor.Err()`，限制 Batch 和返回字段。
- 事务回调只包含数据库操作，正确重试临时事务错误和提交结果未知。
- 日志记录 collection、operation、duration、matched/modified、error labels 和 trace，脱敏查询参数。

59. MongoDB 压测要覆盖什么？

必须使用真实文档大小、数组分布、索引和读写混合，分别测热点与均匀流量、Primary 切换、Secondary 延迟、Checkpoint、索引构建和迁移。

输出 QPS、P50/P95/P99、扫描比、Cache/eviction、读写排队、磁盘延迟、复制延迟、Oplog 窗口、连接池等待和故障恢复时间。

九、生产治理、安全与故障排查

60. MongoDB 监控体系包括什么？

层次	核心指标
业务	成功率、P99、数据新鲜度、状态冲突、积压
查询	慢查询、扫描/返回比、COLLSCAN、内存排序
存储	Cache、dirty、eviction、Checkpoint、磁盘延迟
并发	连接、活跃操作、读写排队、锁和 write conflict
复制	成员状态、复制延迟、Oplog 窗口、选举和 rollback
分片	Shard 分布、Chunk、迁移、热点、Scatter-Gather
容量	数据/索引增长、磁盘水位、备份和恢复耗时

61. MongoDB 线上变慢如何排查？

1. 确认时间线、影响租户、接口、集合、读写类型和 P99。
2. 查 profiler/慢日志和 currentOp，保存真实查询与计划。
3. 判断是扫描、排序、锁/冲突、连接池等待还是磁盘 IO。
4. 检查 Cache eviction、dirty、Checkpoint、Ticket/排队和 CPU。
5. 检查复制延迟、选举、Balancer、索引构建和批量任务。
6. 先限流重查询、暂停迁移/批任务或切换降级，再做索引和模型根治。

62. 连接数打满或请求排队如何处理？

先区分应用连接池等待、服务端连接过多和存储引擎读写排队。排查发布是否创建的多 Client、请求是否未关闭 Cursor、慢查询是否长期占连接，以及实例扩容是否同时造成连接风暴。

止血是限制入口和批任务、回收异常实例、降低每实例连接上限；根治是单例 Client、合理池大小、deadline、慢查询治理和连接指标。

63. 复制延迟如何排查？

- Primary 峰值写入和大事务是否超过 Secondary 应用速度。
- Secondary 磁盘、CPU、Cache 是否不足。

- Secondary 是否承载重查询、备份或索引构建。
- 网络延迟和丢包是否异常。
- Oplog 窗口是否足以覆盖延迟。

延迟期间避免让强一致读继续走 Secondary；优先限制重任务，保护 Oplog 窗口，必要时扩容或重新同步落后节点。

64. 磁盘水位高和数据增长失控如何处理？

按数据、索引、Oplog、日志和临时文件拆分占用。短期停止非核心写入和大索引构建，清理可确认的过期数据并扩磁盘；长期通过 TTL、归档、索引精简、压缩评估、分片和容量告警治理。

禁止在未确认备份和业务范围时直接删除集合或文件。

65. MongoDB 安全如何治理？

- 开启认证、TLS 和最小权限，区分应用、运维、备份和审计账号。
- 数据库不得直接暴露公网，限制网络来源和管理端口。
- 凭证进入密钥系统并定期轮换。
- 敏感字段按合规要求加密、脱敏或使用字段级保护能力。
- 审计 DDL、用户权限、批量导出、删除和恢复操作。
- 应用层强制租户过滤，防止越权查询。

66. MongoDB 故障复盘如何组织？

给出“现象 -> 直接原因 -> 触发条件 -> 放大因素 -> 止血 -> 恢复 -> 防复发”。例如慢查询只是直接原因，缺少查询预算、索引评审、压测覆盖和扫描比告警才是治理缺口。

复盘必须落到负责人和可验证动作：索引变更、容量阈值、故障演练、备份恢复、连接池基线和发布检查。

十、业务架构与项目实战

67. 广告任务或素材文档如何建模？

主文档： 任务 ID、租户、项目、状态、当前配置、素材摘要、创建与更新时间；这些字段经常一起读取且更新边界清晰。

独立集合： 执行流水、审核历史、投放结果和大规模明细，它们增长无界并需要独立分页。

索引： 由后台列表查询反推，例如 `{tenant_id, status, updated_at}`；名称全文搜索复杂时同步到 ES。

一致性： 状态迁移使用状态 + version 条件更新，事件通过 Kafka 幂等传播。

68. 如何设计任务状态流转?

```
pending -> running -> succeeded
                -> failed
pending/running -> cancelled (按业务规则)
```

更新条件必须包含当前状态和版本, 检查 `matchedCount`; 同时写入独立状态流水, 事件带 `task_id`、`version` 和 `event_id`。重复消费同一版本直接返回成功, 旧版本不允许覆盖新状态。

69. MongoDB 到 ES 的同步链路如何设计?

- MongoDB 是完整业务文档, ES 是搜索读模型。
- 变更通过 Outbox/Change Stream 进入 Kafka。
- 消费者按 ID 回查最新文档, 构建精简 ES Document。
- 使用业务版本防止乱序, Bulk item 失败分类重试。
- 对账任务校验数量、版本、状态和删除标记。
- Mapping 变化时新建索引、全量导入、追增量、Alias 灰度切换。

70. 用户画像为什么可能适合 MongoDB?

画像字段按用户类型和业务阶段变化, 标签、偏好和设备摘要天然形成单用户聚合文档。读取常以 `user_id` 为入口, 适合点查和部分字段更新。

但事件明细不应无限嵌入; 高频实时计数可先在 Redis/流处理聚合, 离线分析进入数仓, MongoDB 只保存服务在线查询的画像快照。

71. 如何进行 MongoDB 灰度迁移?

1. 定义旧库与 MongoDB 的字段映射和一致性口径。
2. 全量迁移使用稳定游标、限速和断点续传。
3. 增量使用事件或 CDC, 写入带版本并幂等。
4. 双读影子对比关键字段和耗时, 允许明确的延迟窗口。
5. 按租户/流量切读, 再切写; 保留回滚链路。
6. 完成后对账、观察、归档旧数据, 最后下线双写。

72. 如何用两分钟回答 MongoDB 架构题?

我先把 MongoDB 定位为围绕聚合根的业务文档存储, 而不是无 Schema 的万能库。建模从访问模式出发: 一起读写且规模有界的数据嵌入, 生命周期独立或增长无界的数据引用并独立分页; 高频查询按照过滤、排序和范围设计联合索引, 用 Explain 验证扫描/返回比。正确性优先利用单文档原子更新和版本条件, 跨文档事务保持短小; 再按业务选择 Read Concern、Write Concern 和 Read Preference。高可用由副本集、Oplog、选举和备份承担, 容量超过单副本集后才依据基数、均匀性、

单调性和查询路由选择 Shard Key。线上重点监控 P99、慢查询、Cache、磁盘、连接、复制延迟、Oplog 窗口和分片热点。需要全文检索时同步到 ES，需要事件解耦时进入 Kafka，热点状态交给 Redis，每个组件只承担自己最擅长的职责。

技术 | Elasticsearch 搜索引擎面试题

面向 9 年以上 Golang Web 后端业务研发岗位。重点不是背 API，而是说明：为什么引入 ES、一次请求如何执行、搜索与写入如何权衡、数据如何可靠同步、集群如何规划，以及线上故障如何止血和复盘。

一、Elasticsearch 学习地图与资深回答框架

1. Elasticsearch 的知识图谱主线是什么？

一句话结论：以“可重建的搜索读模型”为顶层定位，沿着文档建模、写入索引、分布式检索、可靠同步、集群治理和业务落地逐层展开。

业务需求与边界

- > 文档与 Mapping
- > Analyzer 与倒排索引
- > Query / Filter / 排序 / 聚合
- > Primary / Replica / Segment / Translog
- > Shard 路由与分布式归并
- > 数据同步与索引重建
- > 性能容量与生产治理
- > 业务选型和项目表达

复习时始终追问五件事：

1. **业务目标：**是全文召回、组合筛选、相关性排序，还是统计分析？
2. **数据模型：**哪些字段检索、过滤、排序、聚合，哪些字段只展示？
3. **链路原理：**写入如何生成 Segment，搜索如何跨分片归并？
4. **工程代价：**实时性、一致性、资源、复杂度分别牺牲什么？
5. **生产闭环：**如何容量评估、监控、降级、重建和复盘？

2. 资深后端如何回答 ES 面试题？

采用“结论 -> 原理 -> 方案 -> 边界 -> 落地”的金字塔结构。

示例：为什么不能把 ES 当主数据库？

- **结论：**ES 应作为可重建的搜索读模型，交易事实仍由 MySQL 或 MongoDB 持有。

- **原理：** ES 为搜索与分析优化，依赖近实时刷新、分片副本和最终一致同步，不擅长跨文档强事务与复杂约束。
- **方案：** 主库先提交，通过 Outbox、CDC 或 Kafka 传播变更，消费者幂等更新 ES。
- **边界：** 短暂延迟、重复和乱序必须由版本、对账与重建兜底。
- **落地：** 监控同步延迟、Bulk 失败、版本差异和索引健康，ES 故障时降级而不阻塞交易。

3. 如何回答 ES 场景设计题？

按六步展开：

1. 明确搜索目标、数据量、QPS、P99、结果新鲜度和可用性目标。
2. 确认主数据源与 ES 的职责边界。
3. 设计索引、Mapping、Analyzer、路由、分页与排序。
4. 设计全量构建、增量同步、幂等、乱序和删除处理。
5. 估算分片、节点、磁盘、Heap、查询与写入吞吐。
6. 给出限流、降级、灰度、回滚、对账、快照和故障恢复方案。

4. ES 高频问题应如何串联复习？

层次	核心问题	必须讲出的关键词
定位	为什么用 ES	搜索读模型、近实时、可重建
建模	Mapping 怎么设计	text/keyword、nested、dynamic、Doc Values
原理	为什么搜索快	倒排索引、FST、Posting、BKD、页缓存
查询	DSL 怎么优化	filter、bool、分页、聚合、Profile
写入	为什么刚写搜不到	refresh、Segment、Translog
集群	分片怎么规划	路由、Primary、Replica、恢复成本
一致性	DB 如何同步 ES	Outbox/CDC、Kafka、幂等、版本、对账
治理	线上慢怎么查	证据、DSL、分片、线程池、Heap、磁盘
实战	项目如何表达	约束、权衡、指标、灰度、收益

二、使用场景、能力边界与组件选型

5. Elasticsearch 最适合解决什么问题？

结论： 适合大规模文档的全文搜索、多条件过滤、相关性排序、聚合分析和地理位置检索。

典型场景：

- 商品、内容、素材、工单和运营后台搜索。
- 日志、指标、链路等可观测数据分析。
- 标签、时间、状态、关键词组合筛选。
- 自动补全、模糊匹配、同义词和多语言检索。
- 搜索结果上的分桶、趋势和 TopN 聚合。

判断依据不是“数据量大”，而是查询模型是否具有**检索性**：召回条件复杂、排序相关、组合过滤多、关系 Join 少。

6. 什么场景不适合使用 ES？

- 强事务、唯一约束、外键和跨实体一致性是核心要求。
- 高频单行点更新远多于搜索，且每次更新都要求立即可见。
- 复杂多表 Join 和临时关系查询占主导。
- 数据不能从其他事实源恢复，又缺少严格备份恢复体系。
- 数据规模和查询都很简单，数据库索引已经满足 SLA。

资深表达： ES 的引入会增加同步链路、容量成本、数据一致性和运维复杂度；只有搜索收益超过这些成本时才值得使用。

7. ES、MySQL、MongoDB、Redis 和 Kafka 如何分工？

组件	核心职责	不应承担的主要职责
MySQL	事务、约束、关系模型、权威业务事实	大规模全文相关性检索
MongoDB	灵活文档、聚合根、变化较快的业务模型	复杂全文召回和全局强事务
Redis	热点缓存、计数、限流、短期状态	大规模持久搜索和唯一事实源
Kafka	事件传播、削峰、解耦、重放	随机查询和业务事实存储
Elasticsearch	搜索、过滤、排序、聚合、派生读模型	交易主库和跨系统强一致事务

典型链路：


```
MySQL / MongoDB (权威数据)
-> Outbox / CDC
-> Kafka (可靠传播与重放)
-> Elasticsearch (搜索读模型)
-> Redis (热点结果或短期缓存)
```

8. ES 与 MySQL 全文索引如何选择？

MySQL 优先： 数据量不大、搜索条件简单、事务一致性更重要、团队不希望增加新基础设施。

ES 优先： 需要相关性评分、复杂 Analyzer、多字段召回、模糊匹配、高亮、聚合或水平扩展。

不要只比较单次耗时，还要比较：功能上限、数据同步成本、查询稳定性、扩容路径和故障恢复复杂度。

9. ES 与 MongoDB 查询如何选择？

MongoDB 适合作为业务文档的主存储，擅长按确定字段和索引查询完整聚合文档；ES 擅长把文档拆成可检索词项后进行召回、评分和聚合。

常见组合是 MongoDB 保存素材或任务完整文档，ES 只保存检索、过滤、排序和列表展示需要的字段。详情页按 ID 回查 MongoDB，避免 ES 文档无限膨胀。

三、核心对象、请求链路与分布式检索

10. ES 的核心对象如何串起来？

```
Cluster
-> Node (master-eligible / data / ingest / coordinating)
-> Index
-> Primary Shard + Replica Shard
-> Lucene Index
-> Segment
-> Document -> Field -> Term
```

- Index 是逻辑数据集合。
- Shard 是数据分布、并行和故障恢复的基本单元，本质是 Lucene Index。
- Segment 是不可变的倒排索引文件集合。
- Replica 是 Primary 的完整副本，用于容灾并可分担搜索。
- 协调节点负责请求分发、结果归并和 Fetch。

11. 一条写入请求如何执行？

1. 客户端请求任意节点，该节点成为协调节点。

2. 根据 `_id` 或自定义 routing 计算目标 Primary Shard。
3. 请求转发到 Primary，完成校验、版本控制和 Lucene 内存缓冲写入，同时记录 Translog。
4. Primary 将操作并行转发到 Replica。
5. 满足活跃分片和确认条件后向客户端返回。
6. 后续 refresh 生成可搜索 Segment；flush 推进提交点并轮转 Translog；merge 合并小 Segment。

关键边界： 写入返回成功、持久化完成和搜索可见是三个不同概念。

12. 一条搜索请求如何执行？

搜索通常分为 `query then fetch` 两阶段：

1. 协调节点解析索引和路由，选择每个目标 Shard 的 Primary 或 Replica 副本。
2. Query Phase 在各 Shard 本地执行查询和排序，返回 DocID、score、sort values 等候选结果。
3. 协调节点合并各 Shard 的局部 TopK，形成全局 TopK。
4. Fetch Phase 向命中的 Shard 拉取 `_source` 或指定字段。
5. 协调节点组装最终响应。

因此，分片过多、`from + size` 过大、返回文档过重和复杂聚合都会放大 CPU、网络 and 协调节点内存。

13. ES 如何决定文档属于哪个 Shard？

概念上可理解为：

```
shard = hash(routing) % number_of_primary_shards
```

默认 routing 是 `_id`。自定义 routing 可以让同一租户或业务实体落到同一 Shard，查询时指定 routing 可减少扇出。

风险是 routing key 分布不均会制造热点；设计前必须评估基数、数据倾斜、最大租户和未来迁移能力。

14. Primary 和 Replica 分别解决什么？

- Primary 负责该 Shard 写入操作的排序和复制协调。
- Replica 提供冗余副本，Primary 故障时可提升为新 Primary，也可承接搜索。
- Replica 增加能提升读能力和可用性，但会增加写复制、磁盘和恢复成本。
- Replica 不是备份；误删除会同步到所有副本，仍需 Snapshot。

15. 协调节点为什么可能成为瓶颈？

它承担请求广播、候选结果维护、全局排序、聚合归并和响应序列化。大分页、高基数聚合、跨大量 Shard 查询或返回大 `_source` 时，协调节点会承受大量 Heap、CPU 和网络压力。

治理方法：减少目标 Shard、限制结果窗口、使用 `search_after`、压缩返回字段、限制聚合桶、隔离重查询，并为专用协调节点做容量评估。

四、Mapping、Analyzer、倒排索引与相关性

16. ES 为什么搜索快？

结论： ES 通过“写时建立索引、读时按词项定位”的方式，把全文扫描变成有序词典和 Posting List 的集合运算。

关键结构：

- **倒排索引：** Term -> 包含该 Term 的文档列表。
- **Term Dictionary/FST：** 快速定位词项并压缩公共前缀。
- **Posting List：** 保存 DocID、词频、位置等信息，可跳跃和压缩。
- **Doc Values：** 面向列的磁盘结构，服务排序、聚合和脚本访问。
- **BKD Tree：** 支持数值、日期和地理字段的范围与空间查询。
- **Filesystem Cache：** Lucene 大量利用操作系统页缓存。

代价是写放大、磁盘空间、近实时可见和 Mapping 设计复杂度。

17. text 和 keyword 有什么区别？

类型	索引方式	典型用途
<code>text</code>	经过 Analyzer 切分为多个 Term	全文召回、相关性评分
<code>keyword</code>	整体作为一个精确值	过滤、排序、聚合、去重

名称常设计为多字段：`name` 用 `text` 搜索，`name.keyword` 用于精确过滤或排序。ID、状态、枚举、手机号等通常使用 `keyword`，不要误设为 `text`。

18. Analyzer 的执行链路是什么？

Character Filter -> Tokenizer -> Token Filter

- Character Filter 在分词前清洗或映射字符。
- Tokenizer 决定如何切词。

- Token Filter 做小写、停用词、词干、同义词等处理。

索引时 Analyzer 决定存什么 Term；查询时 Search Analyzer 决定拿什么 Term 去匹配。两者不一致可能是业务需要，也可能导致完全搜不到。

19. 中文分词、同义词和停用词如何设计？

- 先根据业务语料定义召回目标，不要只比较分词器名称。
- 用 `_analyze` 对真实词库测试切词结果。
- 索引侧尽量稳定，搜索侧同义词更新更灵活。
- 同义词要区分等价词和单向扩展，防止召回爆炸。
- 停用词可能误删品牌、型号中的有效字符。
- 上线前用离线查询集评估召回率、准确率和 NDCG 等指标。

20. Mapping 设计有哪些关键原则？

1. 先列出查询矩阵：每个字段是否检索、过滤、排序、聚合、展示。
2. 明确字段类型，避免依赖无限制动态映射。
3. 控制字段数量，防止 Mapping Explosion。
4. 精确字段用 keyword/numeric/date，全文字段用 text。
5. 排序聚合使用 Doc Values；大文本不参与检索时关闭索引。
6. 只保存搜索需要的数据，详情可回查主库。
7. 重要 Mapping 变更通过新索引 + Alias 切换，不在旧索引上冒险修改。

21. dynamic mapping 有什么风险？

动态字段可能把日期识别成字符串、把 ID 识别成数值，或者因用户自定义 key 产生数十万字段，导致集群状态膨胀、Heap 压力和查询异常。

生产中对核心索引应使用显式 Mapping；变化字段可用 `dynamic: strict/false`、`flattened`、键值数组或独立扩展集合承载。

22. object、nested 和父子关系如何选择？

- `object` 会把对象数组扁平化，无法保持数组元素之间的字段关联。
- `nested` 把每个子对象作为隐藏 Lucene 文档，可保证同一子对象内的组合条件，但增加写入、查询和存储成本。
- `join` 父子关系允许独立更新，但查询代价高、路由限制多，应谨慎使用。
- 高频查询场景常优先反范式冗余，接受通过同步更新换取读性能。

23. query 和 filter 有什么区别？

- Query Context 判断“匹配度多高”，参与 `_score`。
- Filter Context 判断“是否匹配”，不计算评分，更适合状态、租户、时间、权限等精确条件，也更有缓存机会。

常见设计是 `bool.must` 放全文召回，`bool.filter` 放结构化约束，既表达清晰又减少无意义评分。

24. match、term、match_phrase 和 multi_match 如何选择？

- `match` 会分析查询文本，适合 `text` 全文检索。
- `term` 不分析输入，适合 keyword、数值、状态等精确值。
- `match_phrase` 要求词项位置满足短语关系，召回更精确但成本更高。
- `multi_match` 用于多字段召回，可按字段设置 boost。

最常见错误是在 `text` 字段上用 `term` 查自然语言，或对 `keyword` 字段用 `match` 后误判结果。

25. BM25 相关性评分如何理解？

评分核心考虑：查询词在当前文档中的频率、在全局文档中的稀有程度、字段长度归一化和字段 boost。它适合文本相关性，但不是完整业务排序。

生产排序往往是：

文本相关性

- + 质量分
- + 新鲜度衰减
- + 热度/转化率
- + 业务置顶和风控约束

需要用离线标注集和线上 A/B 测试验证，不能只凭开发者感觉调 boost。

五、Query DSL、分页、排序与聚合设计

26. bool 查询如何组织复杂业务条件？

- `must`：必须匹配且参与评分。
- `filter`：必须满足但不评分。
- `should`：提高召回或评分，可配 `minimum_should_match`。
- `must_not`：排除条件。

资深回答要补充：先限制租户、状态和时间范围，再做全文召回；将稳定过滤与评分逻辑分离；防止用户输入直接拼接脚本或任意 DSL。

27. 深分页为什么慢，如何治理？

`from + size` 要求每个 Shard 取出本地前 `from + size` 条，再由协调节点全局归并。页数越深，CPU、Heap 和网络成本越大。

方案选择：

- 普通交互分页：限制最大页深。
- 下一页浏览：`search_after` + 稳定唯一排序键。
- 一致性遍历：PIT + `search_after`。
- 离线全量导出：使用适合批量遍历的机制并限速，避免冲击在线集群。
- 运营确需随机跳页：评估产品改造或预计算，不能无限放开结果窗口。

28. `search_after` 为什么需要稳定排序？

它以“上一页最后一条的 `sort values`”为下一页游标。若排序值不唯一或翻页期间索引刷新导致视图变化，会出现重复或漏数据。

通常使用业务排序字段加唯一 ID 作为 tie-breaker，并结合 PIT 固定搜索视图。游标应编码查询条件版本，防止用户更换条件后继续使用旧游标。

29. 排序和字段返回如何优化？

- 排序字段使用 keyword/numeric/date，并确保有 Doc Values。
- 不需要评分时关闭 `track_scores`，大结果集谨慎使用 `_score` 排序。
- 只返回必要 `_source` 字段，或使用 `docvalue_fields/stored_fields`。
- 避免脚本排序和运行时字段进入高 QPS 主链路。
- 固定过滤查询可使用 preference 改善缓存局部性，但要考虑热点。

30. 聚合查询为什么容易拖垮集群？

聚合要在各 Shard 建立局部桶并在协调节点归并。高基数字段、大时间范围、过多 Shard、嵌套聚合和过大的 `size` 会放大内存与网络。

治理方法：

- 先通过 filter 和时间范围缩小数据集。
- 在 keyword/numeric/date 字段上聚合。
- 限制桶数量和层级，使用 composite aggregation 做可分页桶遍历。
- 报表与在线搜索隔离索引、节点或计算链路。
- 对固定历史数据利用请求缓存或预聚合。

31. 如何用 Profile、Explain 和慢日志定位查询问题？

- 慢日志先找到具体索引和 DSL。
- Profile 分析查询在各 Lucene 阶段的耗时，但本身有开销，只用于受控诊断。
- Explain 查看单文档为什么得到某个评分，不用于大规模性能分析。
- 同时结合 `_nodes/hot_threads`、线程池拒绝、Shard 分布、Heap、GC 和磁盘延迟判断是否为资源问题。

32. 商品或内容搜索如何设计？

1. 主库保存商品、内容和库存等权威数据。
2. ES 文档冗余标题、类目、品牌、标签、状态、时间和展示字段。
3. Analyzer 负责召回；filter 负责租户、状态、权限和上下架。
4. 排序综合相关性、质量、新鲜度和业务规则。
5. 库存、价格等高频强一致字段可在召回后回查，避免持续打写 ES。
6. 无结果时做纠错、同义词、热门内容或类目降级。

六、写入、Refresh、Translog、Merge 与数据同步

33. refresh、flush 和 merge 分别解决什么？

机制	主要作用	核心代价
Refresh	将内存缓冲形成新 Segment，使文档可搜索	频繁执行产生大量小 Segment
Flush	建立持久化提交点并轮转/清理 Translog	磁盘 IO 和短时资源消耗
Merge	合并 Segment、回收删除空间、减少查询开销	CPU、磁盘 IO 和写放大

大批量导入可临时调大 refresh interval，控制 Replica 和 Bulk 并发；导入完成后恢复设置并校验，不能把临时配置遗留在线上。

34. ES 如何保证单次写入可靠性？

- Primary 对操作排序并写入本地结构与 Translog。
- 操作复制到 Replica，确认策略决定返回时需要多少活跃副本。
- Translog 支持节点崩溃后的操作恢复。
- Primary 故障后由满足条件的副本接管。

这些机制只保证 ES 集群内部的可靠性边界，不保证 MySQL/MongoDB 与 ES 之间的原子一致，也不能替代 Snapshot。

35. Bulk API 如何正确使用？

- 以“批次字节数、单条大小、并发数、服务端拒绝率”共同调优，不能只追求条数。
- HTTP 200 不代表每个 item 成功，必须逐条解析失败项。
- 对 429、临时网络错误做有界指数退避；Mapping 错误等永久失败进入隔离队列。
- 文档 ID 使用稳定业务 ID，更新操作必须幂等。
- 生产者速度超过 ES 承载能力时要背压，不能无限堆积内存。

36. 更新和删除为什么会产生写放大？

Lucene Segment 不可变。更新通常是“标记旧文档删除 + 写入新文档”，删除也先记录删除标记，空间在后续 merge 才回收。因此高频更新、大文档和嵌套文档会放大磁盘、Merge 和网络成本。

设计上应减少无意义更新、拆分高频变化字段、合并批量变更，并避免把实时计数器直接放入 ES。

37. MySQL/MongoDB 到 ES 如何保证最终一致？

推荐链路：

业务事务提交权威数据

- > Outbox 或 CDC 可靠产生事件
- > Kafka 持久化、削峰和重放
- > ES Consumer 按业务 ID 幂等 upsert/delete
- > 失败重试 + 隔离队列
- > 定时对账 + 全量重建

关键点：

- 消息带实体 ID、事件 ID、业务版本、操作类型和发生时间。
- 消费者可回查权威源，避免旧消息携带的快照覆盖新状态。
- 使用外部版本或业务版本拒绝乱序旧写。
- 删除 404 可按目标状态视为幂等成功。
- 对账比较数量、版本、关键字段和抽样业务结果。

38. 为什么不建议业务线程直接双写 DB 和 ES？

DB 成功而 ES 失败、ES 成功而 DB 回滚、网络超时结果未知都会产生不一致。跨两个系统无法靠普通本地事务原子提交。

如果规模较小只能同步双写，也应仍以 DB 为事实源，记录待同步状态或 Outbox，并允许异步补偿；不能把一次 HTTP 成功当作最终一致性证明。

39. 如何安全地重建索引？

1. 创建带版本的新索引和明确 Mapping。

2. 从权威源按稳定游标全量导入，限速并逐条检查 Bulk 失败。
3. 同时记录或消费增量变更，解决全量期间的数据变化。
4. 校验总量、版本、关键字段、抽样搜索和性能。
5. 原子切换 Alias，灰度观察并保留回切路径。
6. 稳定后再删除旧索引，删除前确认 Snapshot 和保留周期。

七、集群、分片、副本、高可用与灾备

40. 分片数量如何设计？

分片数同时决定并行度、单分片大小、集群元数据、恢复速度和查询扇出。

评估维度：

- 预计主分片数据量和增长周期。
- 单分片搜索与写入吞吐压测结果。
- 节点故障后副本恢复时间。
- 单次查询命中的 Shard 数。
- 租户或时间维度的路由与生命周期。

原则是先通过真实数据压测得到单 Shard 能力，再反推数量；不要迷信固定 GB 经验值，也不要为未来无限预留小分片。

41. 分片过多和过少分别有什么问题？

过多： 集群状态和 Heap 开销大、查询广播与归并重、线程池竞争、恢复任务多、小 Segment 多。

过少： 单 Shard 过大、迁移恢复慢、并行度不足、热点难以拆分，达到上限后扩展困难。

治理包括 rollover、shrink/split、reindex、按时间或租户拆索引，但每种操作都要评估停机、磁盘和双写成本。

42. 节点角色如何规划？

- Master-eligible 节点负责集群元数据和选主，不应承担重查询。
- Data 节点持有 Shard 并执行读写，可按热温冷数据分层。
- Ingest 节点执行预处理管道，复杂处理应考虑独立计算链路。
- Coordinating-only 节点可隔离请求归并，但不是无限扩容的万能层。

小集群不必过度拆角色；规模扩大后再根据资源隔离和故障域规划。

43. ES 如何处理节点故障和脑裂风险？

集群通过 Master 维护成员与 Shard Allocation，Primary 丢失后从合格 Replica 中提升新 Primary，并重新补齐副本。生产上应：

- 部署奇数个独立故障域的 Master-eligible 节点。
- 配置合理发现机制，不允许不同环境误组成集群。
- 通过 allocation awareness 分散 Primary 和 Replica。
- 网络分区期间优先保证多数派安全，避免双主写入。
- 恢复后检查未分配 Shard、数据路径和版本兼容性。

44. Yellow、Red 分别意味着什么？

- Yellow：所有 Primary 已分配，但至少一个 Replica 未分配，通常仍可服务但容灾下降。
- Red：至少一个 Primary 未分配，相关数据不可用或不完整。

排查不能停在颜色，要用 allocation explain 找到具体原因：节点离线、磁盘水位、分配规则、版本、损坏、同故障域约束或副本数超过节点数。

45. Snapshot 为什么仍然必要？

Replica 只能抵抗节点故障，无法抵抗误删除、错误更新、逻辑污染和整个集群损坏。Snapshot 应存放在独立可靠仓库，并定期做恢复演练。

灾备设计要明确 RPO、RTO、索引优先级、恢复顺序、密钥权限和跨区域带宽。若 ES 可从主库重建，也要估算全量重建耗时是否满足 RTO。

46. ILM、Rollover 和数据分层解决什么问题？

对日志和时序数据，可按大小、文档数或时间 rollover，随后进入热、温、冷、冻结和删除阶段。它将容量治理制度化，避免单索引无限增长。

业务搜索索引未必适合纯时间切分；应根据查询是否跨周期、更新是否频繁和 Alias 路由决定。

八、性能优化、容量规划与 Golang 工程实践

47. ES 容量如何估算？

原始数据量

- x 索引膨胀系数
- x (1 + Replica 数)
- x 增长周期
- + Merge/恢复临时空间
- + 安全余量

索引膨胀系数必须用真实 Mapping、Analyzer 和样本导入测量。节点规划还要同时满足：

- 磁盘水位与故障恢复余量。
- 单节点 Heap 和操作系统页缓存。
- 峰值搜索、写入、Merge 和 Snapshot IO。
- N-1 节点故障下的容量和 SLA。

48. Heap 为什么重要，为什么也不能给得无限大？

Heap 承担集群元数据、查询、聚合、缓存和索引缓冲；过小会频繁 GC 或触发 Circuit Breaker。过大则延长 GC，并挤占 Lucene 依赖的操作系统页缓存。

应根据节点角色和实测调优，重点观察 Heap 使用趋势、老年代、GC 暂停、Breaker、字段数量、分片数量和高基数聚合，而不是只看某一刻百分比。

49. 如何系统优化查询性能？

按层次处理：

1. **业务层**：缩小时间和租户范围，限制分页、聚合和返回字段。
2. **DSL 层**：filter 替代无意义评分，避免脚本、通配前缀和复杂 nested。
3. **Mapping 层**：正确字段类型、Doc Values、冗余查询字段。
4. **Shard 层**：减少查询扇出，修复热点和不合理分片。
5. **资源层**：检查线程池、CPU、Heap、GC、磁盘和网络。
6. **架构层**：搜索与报表隔离、预计算、缓存或独立集群。

50. 如何系统优化写入性能？

- 使用合适 Bulk 大小和有限并发。
- 批量导入时降低 refresh 频率，必要时临时调整 Replica，完成后恢复。
- 简化 Analyzer、Ingest Pipeline、脚本和高成本 Mapping。
- 减少大文档、高频局部更新和 nested 更新。
- 保证磁盘低延迟、节点负载均衡和足够 Merge 余量。
- 监控 write/bulk 线程池拒绝、Merge、Refresh、Translog、GC 和磁盘水位。

51. Golang ES 客户端如何治理？

- 所有请求使用 `context` deadline，禁止无界等待。
- 复用 HTTP Transport，配置连接池、空闲连接和 TLS。
- 对查询与写入设置不同超时、并发和重试策略。
- 只对明确可重试且幂等的请求退避重试，避免重试风暴。

- Bulk Worker 必须有有界队列、背压、逐项错误处理和优雅停机。
- 记录 index、operation、status、took、attempt、trace_id，但避免打印敏感全文和巨大 DSL。

52. 如何对 ES 做压测？

使用接近生产的数据分布、Mapping 和查询混合，不只测单一简单 DSL。分别测稳态、峰值、批量导入、节点故障和数据恢复。

输出至少包括吞吐、P50/P95/P99、超时率、拒绝率、CPU、Heap、GC、磁盘延迟、Merge、缓存命中、Shard 热点和 N-1 场景结果。

九、生产治理、安全与故障排查

53. ES 监控体系应该覆盖什么？

层次	核心指标
业务	搜索成功率、无结果率、结果新鲜度、同步延迟
接口	QPS、P95/P99、超时、错误码、降级率
查询	慢日志、目标 Shard、分页深度、聚合桶、返回大小
写入	Bulk 成功率、逐项失败、吞吐、积压、重试
集群	Health、未分配 Shard、节点数、主节点变化
资源	CPU、Heap、GC、磁盘水位/延迟、网络、线程池拒绝
数据	主库/ES 数量、版本、关键字段和删除状态差异

54. ES 查询慢或接口超时如何排查？

1. 确认影响范围、开始时间、索引、租户、DSL 和 trace。
2. 判断 ES span 是否真的慢，排除客户端连接池、网络和业务等待。
3. 检查慢日志、分页、聚合、脚本、nested、返回字段和查询 Shard 数。
4. 检查 CPU、Heap、GC、线程池拒绝、热点线程、磁盘和 Segment Merge。
5. 对比发布、流量、索引重建、Kafka 重放和报表任务时间线。
6. 先限流重查询、关闭非核心聚合、缩小范围或走降级，再做 Mapping/索引改造。

55. 写入慢、Bulk 失败或同步积压如何排查？

先把链路拆开：事件产生 -> Kafka lag -> Consumer -> Bulk -> Primary/Replica -> Refresh/Merge。

重点检查：Bulk item 错误、429 拒绝、Mapping 冲突、批次和并发、refresh、Replica、磁盘水位、Merge、GC、热点 Shard。止血时降低消费者并发、暂停全量重建和低优先级同步，保护权威主库，不能盲目加消费者把 ES 打垮。

56. Heap、GC、Circuit Breaker 或 OOM 如何处理？

立即限制高基数聚合、深分页、大结果集和脚本查询，暂停大规模导入。然后定位是字段/分片膨胀、聚合桶、缓存、Ingest、Mapping Explosion 还是持续 Merge。

扩 Heap 只能暂时缓解；根治通常是缩小查询、修正 Mapping、减少 Shard、隔离负载或扩 Data Node，同时给操作系统保留页缓存。

57. ES 与主库数据不一致怎么查？

主库是否提交成功？

- > Outbox/CDC 是否产生事件？
- > Kafka 是否积压或丢在错误 Topic？
- > Consumer 是否失败、乱序或被永久错误阻塞？
- > ES Bulk item 是否失败？
- > Alias 是否指向正确索引？
- > 查询是否因 refresh 尚不可见？

恢复时以主库为准，按版本幂等回放或局部重建；大规模回灌必须限速。修复后补对账、告警和故障注入测试。

58. ES 安全和多租户如何治理？

- 强制认证、TLS、最小权限和凭证轮换。
- 禁止业务账号拥有删除集群、修改安全设置等高危权限。
- 租户隔离可使用共享索引 + tenant_id、按大租户独立索引或独立集群，依据规模和合规选择。
- 应用层必须强制注入租户过滤，不能依赖前端传参自觉。
- 限制查询复杂度、结果窗口、脚本、通配索引和批量导出。
- 审计索引删除、Alias 切换、Snapshot 恢复和权限变更。

59. 生产故障的标准止血与复盘框架是什么？

止血顺序： 控制入口 -> 隔离重请求 -> 保护主库和 ES -> 恢复关键 Shard/节点 -> 分批恢复同步 -> 校验业务结果。

证据最小集： 时间线、Index/Alias、Shard、节点、DSL、业务参数、错误响应、trace、线程池、Heap/GC、磁盘、发布和上下游指标。

复盘输出： 直接原因、触发条件、放大机制、为何未提前发现、恢复为何耗时，以及监控、容量、限流、变更流程和演练的改进项。

十、业务架构与项目实战

60. 广告素材库为什么引入 ES?

背景： 素材和广告任务需要按名称、项目、类型、媒体、状态、创建人和时间组合查询，并支持关键词召回和排序。继续在 MongoDB/MySQL 堆叠条件会扩大主库索引和资源压力。

方案：

- MySQL/MongoDB 保留权威业务数据。
- Kafka 传播变更，ES 保存列表检索字段和必要展示字段。
- `resource_name` 使用 text + keyword，ID、状态、类型和租户使用 keyword，时间使用 date。
- 查询强制租户和权限过滤，详情回查主库。
- ES 故障时保留基础列表或热门结果降级，不影响素材上传与投放主链路。

价值： 查询能力和主库稳定性都得到提升，同时承认同步延迟和额外治理成本。

61. MongoDB/MySQL 到 ES 的同步项目如何表达?

先讲问题：直接双写存在失败窗口、消息重复乱序和删除遗漏。

再讲方案：主库提交后由 Outbox/CDC 产生 Kafka 事件，消费者按实体 ID 回查最新状态，通过 Bulk 幂等 upsert/delete；事件包含业务版本，旧版本拒绝覆盖；永久失败进入隔离队列，对账任务扫描版本差异。

最后讲指标：同步 P99 延迟、Bulk item 失败率、Kafka lag、对账差异数、重建耗时和故障恢复 RTO。

62. ES 查询灰度迁移如何设计?

1. 保留 Mongo/MySQL 与 ES 双查询路径。
2. 通过用户、租户或流量比例灰度。
3. 影子请求对比数量、关键 ID、排序和耗时，不把影子结果返回用户。
4. 对可接受的近实时差异设置窗口，避免误报。
5. 监控 ES 错误和同步延迟，支持快速回切。
6. 全量后保留一段观察期，再下线旧查询和冗余索引。

这体现的是迁移能力，而不只是“会写 DSL”。

63. 如何设计一个高可用的内容搜索服务?

- API 层做参数校验、租户权限、复杂度限制和 deadline。
- Query Builder 将业务条件映射为稳定 DSL，禁止任意透传。
- ES 索引按内容类型和生命周期规划，热点查询可短时缓存。

- 排序融合相关性、时效、质量和风控状态。
- ES 超时返回热门内容、类目结果或上次成功缓存。
- 主库写入不依赖 ES；同步链路可重试、可对账、可重建。
- 以 P99、无结果率、点击率、同步延迟和降级率评估系统。

64. 如何回答“你在 ES 项目中解决过什么线上问题”？

使用 STAR + 技术闭环：

- **场景：** 某运营列表 P99 上升并伴随 ES CPU 飙升。
- **任务：** 在不影响投放主链路的情况下恢复查询并定位根因。
- **行动：** 用 trace 和慢日志定位到深分页叠加高基数聚合；先限制分页、关闭非核心聚合并按租户限流，再用 search_after、预聚合和索引字段改造；补充 DSL 指纹和桶数量监控。
- **结果：** 用真实数字说明 P99、错误率、资源使用和最大可承载 QPS 的改善。
- **复盘：** 说明为什么原容量模型漏掉该查询，以及如何通过评审、压测和预算防止复发。

65. 如何用两分钟回答 ES 架构题？

我把 ES 定位为可重建的搜索读模型，而不是交易主库。建模时从查询矩阵出发，明确全文、过滤、排序、聚合和展示字段，用 text/keyword、nested 和 Analyzer 控制召回与成本。写入上由 MySQL 或 MongoDB 先提交，通过 Outbox/CDC 和 Kafka 可靠传播，消费者按业务 ID 与版本幂等更新 ES，并用对账和新索引加 Alias 支持重建。查询链路重点控制分片扇出、深分页、高基数聚合和返回体；集群按真实压测规划 Shard、Replica、Heap、磁盘和 N-1 容量。线上通过业务指标、慢日志、线程池、Heap/GC、磁盘和同步延迟分层监控，故障时限流重请求、保护主库、分批恢复并校验数据。项目效果最终用搜索 P99、成功率、同步延迟、差异率和业务转化来衡量。

技术 | 消息队列 MQ 面试题

面向 9 年以上 Golang Web 后端工程师。本文以 Kafka 为主线，按照“场景与边界 → 核心模型 → 生产 → 存储复制 → 消费 → 一致性 → 性能容量 → 治理排障 → 业务架构”的知识图谱组织。

回答统一遵循金字塔结构：结论先行 → 核心机制 → 完整流程 → 工程取舍 → 生产实践 → 面试追问。

一、Kafka 学习地图与资深面试回答框架

1. Kafka 的知识图谱主线是什么？

结论先行：

Kafka 是一个以 Partition 为并行和顺序边界、以副本机制保证可用性、以 Consumer Group 扩展消费能力、以 Offset 管理消费进度的分布式日志系统。

场景与边界

- 核心模型
- Producer
- Broker
- Consumer
- 可靠性与一致性
- 性能与容量
- 治理与排障
- 业务架构与选型

核心关系：

吞吐上限	← Partition 数量、批量、压缩、Broker 资源
局部顺序	← 相同业务 Key 进入同一 Partition
写入可靠性	← acks + ISR + min.insync.replicas + 幂等 Producer
消费可靠性	← 业务成功后提交 Offset + 消费端幂等
水平扩展	← Partition 在 Broker 和 Consumer 之间分摊
最终一致	← 可靠事件 + 幂等 + 重试 + 补偿 + 对账

2. 资深后端如何回答 Kafka 问题？

结论先行：

不要只背配置或定义。先回答它解决什么问题，再解释实现机制，最后说明失效边界、工程取舍和线上证据。

题型	回答顺序
概念题	定义 → 对象关系 → 适用边界 → 常见误区
原理题	数据结构 → 正常流程 → 异常流程 → 设计代价
配置题	配置作用 → 参数联动 → 吞吐/延迟/可靠性取舍 → 监控验证
排障题	影响面 → 指标和证据 → 定向止血 → 根因 → 恢复验证
架构题	业务目标 → 事件模型 → 正确性 → 容量 → 治理 → 降级补偿

面试追问：高级岗位和中级岗位的回答差别是什么？

中级岗位通常能说明 API 和常见方案；高级岗位还要能确定业务边界、量化容量、处理故障、设计补偿，并说明为什么没有选择另一种方案。

二、使用场景与能力边界

3. Kafka 解决什么问题？

结论先行：

Kafka 主要解决高吞吐事件流中的异步解耦、削峰填谷、多订阅分发、最终一致和数据重放问题，但会引入延迟、重复、积压和运维复杂度。

核心能力：

- 异步：把通知、索引、导出等非主链路任务移出同步请求。
- 解耦：生产者只发布稳定事件，不感知每个下游实现。
- 削峰：Broker 持久化突发流量，消费者按可承受速度处理。
- 多订阅：不同 Consumer Group 独立处理同一事件流。
- 重放：按 Offset 重新消费，重建索引、读模型或修复数据。
- 事件驱动：业务状态变化通过事件推进后续流程。

工程取舍：

- 同步强一致变为可控的不一致窗口。
- 调用栈变成跨服务异步链路，定位需要消息身份和全链路追踪。
- Broker 能承接积压，但不能代替容量规划和下游保护。
- 至少一次投递下，重复是正常现象，消费者必须幂等。

4. 什么场景不适合使用 Kafka？

结论先行：

结果必须立即返回、需要跨外部系统强原子事务、消息量很小但路由复杂、要求精确单条定时触发时，不应只因为“解耦”就引入 Kafka。

典型边界：

场景	原因	更合理的方向
登录鉴权结果	主流程必须同步获得结果	RPC/本地校验
扣款与账务记账	需要严格业务正确性	本地事务、状态机、对账；事件只做后续驱动
任意条件历史查询	Kafka 按日志位置读取	数据库、搜索或分析存储
精确到时执行单条任务	原生模型不是定时队列	调度系统、时间轮或专用延迟能力
小流量复杂路由	分区日志模型不擅长丰富路由	按团队现有消息体系评估

面试追问：Kafka 能长期保存数据，为什么不是数据库？

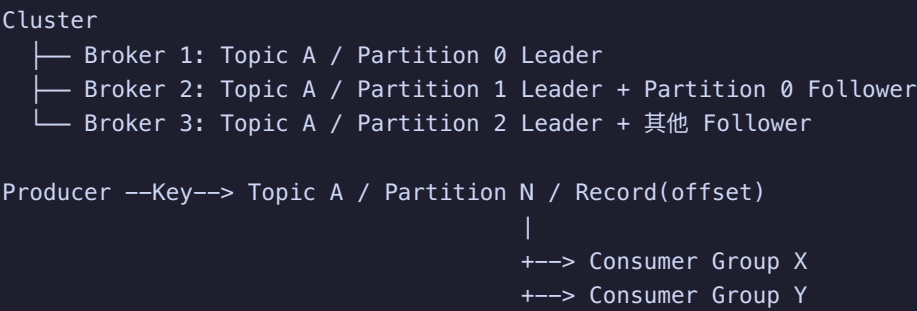
它擅长按 Partition 和 Offset 顺序读写，不提供数据库式任意索引、关联查询、约束和事务模型。长期保留日志不等于适合作为业务查询源。

三、核心数据模型

5. Kafka 的核心对象如何串起来？

结论先行：

Topic 表达一类事件，Partition 是存储、并行和顺序边界，Record 是日志记录，Offset 是分区内位置，Consumer Group 是一个逻辑订阅者，Replica 提供数据冗余。



对象	职责	常见误区
Broker	保存分区副本并处理请求	Broker 数量不等于消费并发度
Topic	事件流的逻辑分类	Topic 不是越细越好
Partition	存储分片、并行度和顺序边界	跨 Partition 不保证全局顺序

对象	职责	常见误区
Record	Key、Value、Header、Timestamp 等组成的记录	大文件不应直接放入消息体
Offset	Record 在 Partition 中的位置	不是全局消息 ID，也不是业务成功证明
Replica	Partition 的冗余副本	副本存在不代表始终同步
Consumer Group	一个逻辑订阅方，组内分摊 Partition	不同业务不应为节省资源共用 Group

6. Partition 为什么是 Kafka 最核心的设计？

结论先行：

Partition 同时决定生产吞吐、消费并行度、局部顺序、数据分布和故障恢复粒度，因此分区设计本质上是业务正确性与容量设计的交点。

同一 Consumer Group 中：
 一个 Partition 同一时刻只归一个 Consumer 成员
 有效消费并行度 $\leq$ Partition 数量
 Consumer 数量 $>$ Partition 数量时，多余成员空闲

工程取舍：

- 分区少：并行能力不足，但顺序和治理简单。
- 分区多：吞吐上限更高，但文件、元数据、复制、选举和 Rebalance 成本增加。
- 扩分区会改变 Key 到 Partition 的映射；对依赖同一 Key 历史顺序的业务要提前评估。

面试追问：分区数增加后，相同 Key 仍然有序吗？

扩容前后的哈希取模结果可能不同，新旧消息可能位于不同 Partition。单个 Partition 内仍有序，但跨扩容时点不能自动保证该 Key 的全历史顺序。

7. 一条消息的完整生命周期是什么？

完整流程：

1. 业务生成事件和稳定业务 Key
2. Producer 序列化并选择 Partition
3. 消息进入客户端批次，由 Sender 发往 Leader
4. Leader 追加日志，Follower 拉取复制
5. 满足 acks 条件后 Producer 收到结果
6. Consumer Group 为成员分配 Partition
7. Consumer Fetch/Poll 消息并执行幂等业务
8. 业务成功后提交下一条待消费 Offset

- 9. 监控记录端到端延迟、失败、Lag 和业务结果
- 10. 失败消息进入重试、死信、补偿或对账链路

生产实践：

消息必须带 `msg_id`、业务 ID、事件类型、Schema 版本、发生时间和 `trace_id`，才能把生产、存储、消费和业务结果串成可审计链路。

四、Producer 生产机制

8. Producer 的完整发送链路是什么？

结论先行：

Producer 不是业务线程直接逐条写 Broker，而是经过序列化、分区、客户端聚批和异步网络发送；吞吐、延迟、可靠性与内存压力都在这条链路上相互制约。

```
业务线程
-> Serializer
-> Partitioner
-> RecordAccumulator (按 Topic-Partition 聚合批次)
-> Sender (网络线程)
-> Broker Leader
-> ISR 副本复制
-> 按 acks 条件返回 ACK
```

核心机制：

1. Serializer 把 Key 和 Value 转成字节。
2. Partitioner 根据显式分区、Key 或客户端策略选择 Partition。
3. RecordAccumulator 为每个 Topic-Partition 聚合批次。
4. Sender 选择已就绪批次，经网络连接发给目标 Leader。
5. Leader 追加日志并根据可靠性配置等待副本。
6. 成功回调或可重试错误进入重试；不可重试错误返回业务。

面试追问：业务发送方法返回成功意味着什么？

只代表达到了 Producer 配置所定义的 Broker 确认条件，不代表消费者业务完成，更不代表外部数据库已最终一致。

9. acks、ISR 和 `min.insync.replicas` 如何联动？

结论先行：

可靠业务通常使用 `acks=all`，并让 `min.insync.replicas` 至少为 2、Replication Factor 通常为 3；这样在同步副本不足时拒绝写入，用可用性换数据安全。

配置	含义	主要风险
<code>acks=0</code>	不等待 Broker 确认	客户端几乎无法感知丢失
<code>acks=1</code>	Leader 写入后确认	Leader 在副本追平前故障可能丢数据
<code>acks=all</code>	等待当前 ISR 达到确认要求	ISR 不足时写入失败，延迟也更高
<code>min.insync.replicas</code>	<code>acks=all</code> 时要求的最少同步副本	过低保护不足，过高降低故障时可写性

生产实践：

不能只看 `acks`。还要联合监控 ISR、Under Replicated Partitions、生产错误和副本延迟，并明确故障时选择“拒绝写入”还是“降级可靠性”。

10. Producer 如何通过批量和压缩提高吞吐？

结论先行：

`batch.size` 决定单批容量，`linger.ms` 允许等待更多消息形成批次，压缩减少网络 and 磁盘字节；更大批次通常提高吞吐，但会增加延迟和客户端内存占用。

配置	影响	调优判断
<code>batch.size</code>	单 Partition 批次上限	小批次浪费请求，大批次占内存
<code>linger.ms</code>	等待聚批时间	提升批量率，但增加端到端延迟
<code>compression.type</code>	网络、Broker 存储和 CPU	按消息类型和 CPU/网络瓶颈压测
<code>buffer.memory</code>	Producer 总缓冲区	满时业务线程会阻塞或失败
<code>max.block.ms</code>	获取元数据或缓冲空间最多阻塞多久	防止请求线程无限等待

工程取舍：

调优必须同时观察平均批次大小、压缩率、发送延迟、缓冲等待、错误率、Broker CPU 和网络，不能只追求更高 TPS。

11. 幂等 Producer 和事务 Producer 分别解决什么？

结论先行：

幂等 Producer 解决单个 Producer 会话内因网络重试产生的重复写；事务 Producer 进一步原子提交多个 Kafka 分区写入及消费位点，但都不等于业务请求全局幂等。

核心机制：

- 幂等写通过 Producer 身份、序列号和 Broker 端校验识别重试重复。

- `max.in.flight.requests.per.connection` 会影响并发请求和重试顺序；使用客户端推荐的幂等配置组合，避免手工参数破坏保证。
- 事务通过 `transactional.id` 识别 Producer，异常实例会被 fencing。
- 消费者使用 `read_committed` 才会屏蔽已中止事务的数据。

边界：

用户重复点击、服务重复调用、进程重启后的业务重复、消费者重复执行，仍需业务幂等键、唯一约束和状态机处理。

12. Producer 如何处理热点 Key、背压和优雅停机？

结论先行：

热点 Key 会让单个 Partition 成为瓶颈；缓冲区耗尽说明 Broker 或网络跟不上生产速度。解决时要先区分业务顺序要求，再决定拆 Key、限流还是扩容。

生产实践：

- 必须有序：保留稳定 Key，优化单分区消费与下游写入，必要时按更细业务实体拆分。
- 不要求严格顺序：Key 加可控散列后缀，消费端再聚合。
- Producer 设置发送超时、错误分类、重试上限和熔断，不能在请求线程无限重试。
- 监控缓冲区等待、批次大小、发送错误、Throttle 时间和热点分区。
- Go 服务停机时先停止接收新请求，等待已接收发送结果，Flush/Close 后退出；失败事件应有 Outbox 或补偿来源。

面试追问：为什么不能看到发送失败就无限重试？

无限重试会占满内存和连接，放大 Broker 故障，并拖住业务线程。重试必须受投递超时和业务时效约束，可靠事件还要有持久化补偿来源。

五、Broker 存储、复制与集群架构

13. Kafka 的日志文件如何组织？

结论先行：

每个 Partition 是一份追加日志，目录下按 Segment 分段；每个 Segment 由数据文件和稀疏索引组成，便于滚动、删除、恢复和定位。

```
topic-a-0/
  00000000000000000000.log
  00000000000000000000.index
  00000000000000000000.timeindex
  00000000001234567890.log
```

```
00000000001234567890.index
00000000001234567890.timeindex
```

文件	作用
<code>.log</code>	顺序保存 Record Batch
<code>.index</code>	相对 Offset 到文件物理位置的稀疏索引
<code>.timeindex</code>	时间戳到相对 Offset 的稀疏索引

查找流程：

目标 Offset

- > 根据 Segment 起始 Offset 定位 Segment
- > 在稀疏 `.index` 中找到不大于目标值的最近索引项
- > 跳到 `.log` 对应物理位置
- > 顺序扫描到目标 Record Batch

14. Kafka 为什么吞吐高？

结论先行：

高吞吐来自顺序追加、Page Cache、批量、压缩、零拷贝和 Partition 并行的共同作用，不是“磁盘一定比内存快”。

核心机制：

- 顺序追加减少随机寻道和复杂原地更新。
- Page Cache 由操作系统统一缓存热日志，避免应用维护巨大对象堆。
- Producer 和 Consumer 都以 Record Batch 为主要传输单位。
- 压缩按批次执行，减少网络与磁盘字节。
- 文件向网络发送可利用 `sendfile` 等零拷贝路径，减少用户态拷贝和上下文切换。
- Partition 让不同 Broker、磁盘和消费者并行工作。

工程取舍：

批量和压缩会消耗 CPU 并增加等待；零拷贝路径也受加密等条件影响。真实性能必须通过端到端压测和系统指标验证。

15. Retention 和 Log Compaction 有什么区别？

结论先行：

Retention 按时间或空间删除旧 Segment；Log Compaction 按 Key 保留较新的值。前者适合事件日志，后者适合可重建的状态流。

```
行为事件、审计记录、操作流水 -> cleanup.policy=delete
账户最新配置、实体最新状态   -> cleanup.policy=compact
需要状态快照且限制历史窗口   -> compact,delete
```

边界：

- Compact 是后台异步过程，不保证重复 Key 立即只剩一条。
- Value 为空通常表示 tombstone，消费者必须处理删除语义。
- Compact 不等于数据库唯一约束，也不能解决业务重复执行。
- Retention 删除以 Segment 为单位，不是消息到点立即逐条删除。

16. Leader、Follower、ISR、LEO 和 HW 是什么？

结论先行：

Leader 处理分区读写，Follower 拉取复制；LEO 表示副本日志末端位置，ISR 是保持同步的副本集合，HW 是默认可见的高水位边界。

```
Leader:    0 1 2 3 4 5 6    LEO=7
Follower1: 0 1 2 3 4 5 6    LEO=7
Follower2: 0 1 2 3 4 5      LEO=6

ISR = {Leader, Follower1}
HW  = 已被 ISR 可靠推进的可见边界
```

生产实践：

- Replication Factor 表示副本目标数量，不代表所有副本当前健康。
- ISR 缩小要结合磁盘延迟、网络、GC、Broker 负载和副本分布排查。
- 副本要跨故障域分布，避免同机架故障同时损失多份数据。

17. Leader 故障后如何恢复？

结论先行：

Controller 从可选副本中选举新 Leader，客户端刷新元数据后恢复请求；可靠性取决于是否从 ISR 选举，以及故障前消息是否达到提交条件。

异常流程：

```
Broker 失联
-> Controller 感知副本和 Broker 状态变化
-> 为受影响 Partition 选择新 Leader
-> 更新元数据并传播
-> Producer/Consumer 刷新元数据
-> Follower 继续追赶并恢复 ISR
```


工程取舍：

允许非 ISR 副本参与非干净选举可以提高极端故障下的可用性，但可能丢失已在旧 Leader、尚未同步的数据。核心事件通常优先一致性，不轻易启用这种降级。

18. KRaft 解决什么问题？

结论先行：

KRaft 使用 Kafka 自身的元数据日志和 Raft 风格仲裁管理 Controller 元数据，统一集群元数据体系，减少外部协调组件和双系统运维复杂度。

核心角色：

- Controller Quorum 维护 Broker、Topic、Partition、副本和配置等元数据。
- Active Controller 处理元数据变更，其他 Controller 参与仲裁并保持日志同步。
- Broker 注册并接收元数据变化，负责实际数据读写和复制。

生产实践：

- Controller 与数据 Broker 的资源、故障域和监控要明确规划。
- 重点监控 Controller 选举、元数据日志同步、请求延迟和 Broker 注册状态。
- 升级或迁移必须按部署版本支持的官方路径演练，不能把元数据切换当成普通配置变更。

面试追问：Controller 管元数据，是否也承载业务消息？

职责不同。Controller 管理集群元数据和选举，业务 Record 仍由对应 Partition Leader 所在 Broker 处理。

六、Consumer 与消费组

19. Consumer Group、Offset 和消费并行度是什么关系？

结论先行：

一个 Consumer Group 表示一个逻辑订阅者，组内成员分摊 Partition；Offset 按 Group、Topic、Partition 记录恢复位置，有效并行度不超过 Partition 数量。

```
Topic: 6 Partitions
Group A: 3 Consumers -> 每个成员约 2 个 Partition
Group B: 6 Consumers -> 每个成员约 1 个 Partition
Group C: 8 Consumers -> 2 个成员空闲
```

Offset 误区：

- 提交的是下一条待消费位置。
- Offset 前进不等于业务数据已经正确写入。

- 先提交后处理可能丢业务；先处理后提交可能重复。
- 不同 Group 的 Offset 互相独立。

20. Consumer 的 Fetch、Poll 和业务处理如何配合？

结论先行：

Consumer 从 Broker 批量 Fetch，应用通过 Poll 取得记录并处理；单批大小、处理耗时、提交时机和心跳存活必须统一设计。

核心参数：

配置	作用	风险
<code>fetch.min.bytes</code>	Broker 累积到一定数据再返回	提高吞吐但可能增加等待
<code>fetch.max.wait.ms</code>	Fetch 最长等待时间	与最小字节共同决定拉取延迟
<code>max.poll.records</code>	单次交给应用的最大记录数	太大可能处理超时
<code>max.poll.interval.ms</code>	两次 Poll 的最大间隔	超过后成员可能被移出组
<code>session.timeout.ms</code>	会话失效时间	太小易抖动，太大故障发现慢
<code>heartbeat.interval.ms</code>	心跳频率	必须和会话超时协调

生产实践：

先用消费耗时分位数确定批量上限，再压测吞吐。不要靠无限增大 Poll 间隔掩盖慢 SQL、外部 API 或锁竞争。

21. Consumer Group 如何完成分区分配？

结论先行：

成员找到 Group Coordinator 后加入组，协调出成员和订阅信息，再同步分配结果；成员、订阅或 Partition 变化会触发重新分配。

```
FindCoordinator
-> JoinGroup：上报成员和订阅
-> 选出 Group Leader/确定协议
-> 计算 Partition Assignment
-> SyncGroup：下发分配结果
-> 稳定消费和心跳
```

策略	特点	适用判断
Range	按 Topic 连续范围分配	多 Topic 时可能倾斜

策略	特点	适用判断
RoundRobin	跨成员轮询分配	较均衡，但变更可能移动较多 Partition
Sticky	尽量均衡且保留原分配	降低状态和缓存迁移
Cooperative Sticky	增量协作式迁移	减少全量停止，但客户端策略必须兼容

22. 为什么会发生 Rebalance，如何降低影响？

结论先行：

Rebalance 是成员或 Partition 变化后的所有权重分配；它本身不是故障，但频繁发生会导致暂停、重复处理和状态迁移。

常见原因：

- 实例扩缩容、重启、发布或 OOM。
- 心跳超时、网络抖动、长时间 GC。
- 单批处理超过 `max.poll.interval.ms`。
- Topic 增加 Partition 或订阅变化。
- Coordinator 故障。

治理方式：

- 控制单批记录数和最慢处理时间。
- 使用 Cooperative Sticky 降低全量迁移。
- 稳定实例可设置 `group.instance.id` 使用静态成员，减少短暂重启引发的抖动。
- Rebalance 回调中暂停拉取、保存可安全提交的 Partition 进度、释放分区级资源。
- 发布时优雅退出，避免所有实例同时离组。

23. Go Consumer 如何设计 Worker Pool？

结论先行：

Go 中可以把拉取与业务处理解耦，但必须保留 Partition 所有权和 Offset 连续提交语义；不能把消息随意扔进全局 Worker Pool 后按完成顺序提交。

```

Poll Loop
-> 按 Topic-Partition 分发
-> Partition Worker / 有界队列
-> 幂等业务处理
-> 记录连续完成水位
-> 仅提交连续安全 Offset

```

工程取舍：

- 要求同 Key 有序：同 Partition 串行或按 Key 路由到固定 Worker。
- 不要求顺序：可并行处理，但必须跟踪空洞，不能因后面的消息先完成就越过失败 Offset 提交。
- Worker 队列必须有界；满时 Pause 对应 Partition 或降低拉取，形成背压。
- 下游调用设置超时、并发上限、退避和熔断。
- 停机时停止 Poll，等待在途任务，提交连续完成水位，再离组。

24. 毒消息如何处理？

结论先行：

毒消息不能无限阻塞 Partition，也不能无审计跳过。应区分暂时失败和永久失败，使用有界重试、死信、告警和人工修复闭环。

处理流程：

解析/业务失败

- > 分类：临时错误 / 永久错误 / 未知错误
- > 临时错误：退避后进入重试 Topic
- > 永久错误：死信 Topic + 告警
- > 未知错误：有限次重试后转死信
- > 修复后以隔离 Group 重放

七、可靠性、顺序与一致性

25. 如何保证消息尽量不丢失？

结论先行：

消息可靠性必须覆盖生产、Broker、消费和业务补偿四层。任何单一配置都不能证明端到端业务不丢。

层次	核心措施
生产	Outbox/可靠来源、acks=all、幂等、有限重试、发送结果审计
Broker	多副本、合理最小同步副本、跨故障域、ISR 和磁盘监控
消费	业务成功后提交 Offset、幂等、失败重试、优雅停机
业务	状态扫描、对账、补偿、死信处理、可控重放

工程判断：

工程上追求的是低丢失概率、可发现、可追溯、可补偿。对支付和资产类事件，业务账本和对账系统比“MQ 发成功了”更接近最终正确性证明。

26. 为什么会重复消费，如何保证幂等？

结论先行：

至少一次投递下，重复是正常结果。幂等应落在最终业务资源，而不是只在消费进程内记忆消息 ID。

重复来源：

- Producer 超时重试。
- 消费成功但 Offset 提交失败。
- Rebalance、进程崩溃或网络抖动后重新拉取。
- 业务上游重复产生同一事件。
- 人工重放或补偿扫描。

幂等手段：

- 数据库唯一约束：适合创建类操作。
- 状态机条件更新：如 `pending -> paid`，不允许状态倒退。
- 业务幂等键：订单号、支付流水号、任务 ID。
- 版本号：只接受比当前版本新的事件。
- 消费记录表：记录消息 ID、结果和处理时间。
- Redis 去重：适合短期低风险优化，不能作为核心正确性的唯一依赖。

27. 如何保证消息顺序？

结论先行：

Kafka 只保证单 Partition 内的追加顺序。业务通常需要同一实体局部有序，而不是所有消息全局有序。

```
order_id / campaign_id / device_id
-> 稳定 Key
-> 同一 Partition
-> Partition 内按 Offset 消费
-> 消费端串行或连续水位提交
```

异常兜底：

- 消息携带业务版本或事件时间。
- 更新使用 `where version < incoming_version` 条件。
- 旧消息到达时丢弃或转审计，不覆盖新状态。
- 失败消息若转入重试 Topic，会改变与后续消息的时间关系；严格顺序业务应阻塞该业务 Key、使用状态机，或把失败实体隔离处理。

面试追问：全局有序怎么实现？

可以只使用一个 Partition 并单线程消费，但吞吐和可用性扩展受限。除非业务真的要求全局顺序，否则优先设计局部顺序。

28. At-most-once、At-least-once 和业务 Exactly-once 怎么理解？

语义	典型提交顺序	结果
At-most-once	先提交 Offset，再处理	可能丢，不轻易重复
At-least-once	先处理，成功后提交	不轻易丢，可能重复
Kafka 内部 Exactly-once	事务写入 + 位点原子提交 + <code>read_committed</code>	Kafka 事务边界内一次可见

关键边界：

Kafka 事务不能原子覆盖 MySQL、MongoDB、Elasticsearch、Redis 或外部 HTTP API。所谓**业务 Exactly-once**仍要依靠业务幂等、数据库约束、状态机、Outbox、补偿和对账实现可证明的最终结果。

29. 本地事务和 Kafka 消息如何保证一致？

结论先行：

常用方案是 Transactional Outbox：业务数据和事件记录在同一个本地数据库事务中提交，再由独立 Publisher 或 CDC 投递 Kafka。

本地事务：

- 更新业务表
- 插入 Outbox 事件
- 一起 Commit

异步投递：

- 扫描 Outbox 或订阅 CDC
 - > 发布 Kafka
 - > 记录发送结果
 - > 失败重试

消费端：

- 幂等处理
 - > 成功后提交 Offset
 - > 失败重试/死信

工程取舍：

- Outbox 通用、边界清晰，但要治理表增长、扫描竞争和发送状态。
- CDC 减少业务侵入，但增加变更捕获平台、Schema 和顺序治理。
- 两者通常仍是至少一次投递，消费者幂等不能省略。

30. 延迟、重试和死信如何设计？

结论先行：

Kafka 原生更适合日志流。延迟和失败重试通常通过分级重试 Topic、时间轮/调度器或外部调度能力实现，并在最终失败后进入死信闭环。

```
主 Topic
-> 临时失败
-> retry-1m -> retry-10m -> retry-1h
-> 超过最大次数
-> dead-letter Topic
-> 告警 / 人工修复 / 隔离重放
```

生产实践：

- 重试消息保留原 Topic、Partition、Offset、`msg_id`、业务 ID、错误类型和重试次数。
- 使用指数退避和随机抖动，避免故障恢复瞬间形成重试风暴。
- 永久参数错误不要反复重试。
- 延迟触发后重新查询业务状态，例如关闭订单前确认仍未支付。
- 死信必须有负责人、处理时限和可审计操作入口。

八、性能优化与容量规划

31. Partition 数量如何估算？

结论先行：

先通过接近生产环境的压测得到单 Partition 写入和消费能力，再分别按生产峰值和目标消费速度计算，取较大值并预留故障与增长余量。

```
partition count >= max(
    producer peak throughput / measured partition write throughput,
    required consumer throughput / measured partition consume throughput
)
```

示例：

生产峰值为 200 MB/s，压测单 Partition 稳定写入 20 MB/s，需要至少 10 个；消费目标为 300 MB/s，单 Partition 消费 25 MB/s，需要至少 12 个。基础值取 12，再结合 Broker 数量、故障降容和未来增长预留。

必须修正的变量：

- 消息大小和 Key 倾斜。
- 压缩率与 CPU。

- 副本复制和跨机架网络。
- 单消费者业务耗时及下游容量。
- Broker 故障后剩余节点承载能力。
- 分区过多带来的元数据和恢复成本。

32. 磁盘和 Broker 容量如何估算？

结论先行：

先算保留窗口内原始数据，再乘副本和安全余量；实际规划还要考虑压缩、索引、重放、磁盘水位和节点故障后的迁移空间。

```
raw storage = peak bytes per second * retention seconds
cluster storage = raw storage * replica factor * headroom factor
```

示例：

若压缩后持续写入 50 MB/s，保留 3 天，原始保留量约为 $50 \text{ MB/s} \times 259200 \text{ s}$ ；再乘副本因子 3 和不低于业务要求的安全余量。不能让正常状态就逼近磁盘高水位，否则副本恢复和流量波动没有空间。

Broker 数量还要考虑：

- 单节点磁盘吞吐、容量和磁盘数量。
- 单节点网卡的生产、消费和复制总流量。
- Leader 分布和热点流量。
- 一个 Broker 故障后剩余 Broker 是否仍能承载峰值。
- 机架、可用区和维护窗口。

33. 消息积压多久可以恢复？

结论先行：

只有恢复消费速率大于持续流入速率时，Lag 才会下降；恢复时间取决于两者的净差值，而不是单看消费者数量。

```
recovery time = accumulated messages /
                (recovery consume rate - incoming rate)
```

如果流入 8 万条/秒，扩容后可消费 12 万条/秒，净恢复速度只有 4 万条/秒。积压 1.44 亿条理论上需要约 1 小时，实际还要加入失败重试、下游限流和流量波动余量。

面试追问：扩消费者为什么可能无效？

有效并行度受 Partition 数限制；瓶颈也可能在数据库、外部 API、单 Key 顺序、锁竞争或热点 Partition。盲目扩容可能把下游压垮。

34. Producer、Consumer 和 Broker 如何系统调优？

层次	调优方向	必须同时观察
Producer	批量、等待、压缩、并发请求、缓冲	端到端延迟、错误、缓冲等待、CPU、网络
Consumer	Fetch 批量、Poll 数量、业务并发、批量写	Lag、处理分位数、失败率、下游压力、Rebalance
Broker	磁盘、网络、请求线程、副本和 Leader 均衡	请求队列、磁盘延迟、ISR、Throttle、Page Cache
数据模型	消息大小、Key 分布、Topic/Partition 数量	热点、大消息、文件句柄、元数据规模

压测原则：

- 使用接近真实的消息大小、Key 分布、压缩方式和副本配置。
- 同时压生产、消费和副本复制，不只测空消费者下的写入峰值。
- 测稳定吞吐和 P99 延迟，而不是短时间峰值。
- 注入 Broker 故障、消费者重启和下游变慢，验证降容状态。
- 输出容量基线、告警水位、扩容触发条件和恢复时间。

九、生产治理、安全与故障排查

35. Kafka 监控体系如何设计？

结论先行：

监控要覆盖 Producer、Broker、Consumer 和业务结果四层，并把指标放在同一时间线上，不能只看 Lag 或发送成功率。

层次	核心指标
Producer	成功率、错误类型、重试、发送延迟、批次、压缩率、缓冲等待、Throttle
Broker	入出流量、请求延迟和队列、磁盘、网络、ISR、离线分区、Leader 分布
Consumer	Lag 总量和增长率、消费速率、失败率、处理分位数、Rebalance、死信
业务	状态推进成功率、不一致数量、补偿数量、事件端到端延迟、对账差异

告警原则：

Lag 总量必须结合增长速度和业务时效。报表消息积压十分钟可能可接受，支付履约事件积压一分钟就可能需要高优先级响应。

36. 消息积压如何排查和止血？

结论先行：

先判断是生产突增、消费变慢、局部分区热点还是 Broker 问题，再在下游承受范围内扩容和恢复，不能看到 Lag 就直接加机器。

```
所有 Group 同时变慢
  -> Broker / 网络 / 集群级问题

单个 Group 变慢
  -> 消费应用 / 下游依赖问题

单个 Partition 变慢
  -> 热点 Key / 毒消息 / Partition Worker 卡住

生产速率突增
  -> 业务流量 / 重放任务 / 异常重复生产
```

处理步骤：

1. 确认影响 Topic、Group、Partition、业务时效和增长速度。
2. 对齐生产速率、消费速率、错误率、处理耗时和下游指标。
3. 止血：隔离低优先级、限重放、转移毒消息、降低下游并发。
4. 确认 Partition 足够且下游有余量后再扩消费者。
5. 恢复时观察净消化速度、错误率、死信和下游水位。

37. 频繁 Rebalance 如何排查？

证据顺序：

1. 对齐 Rebalance 与发布、重启、OOM、GC、CPU 和网络事件。
2. 检查 `max.poll.interval.ms`、Session Timeout 和 Coordinator 错误。
3. 查看每批消息数、最慢消息、DB/HTTP 调用和 Worker 队列。
4. 检查实例是否频繁扩缩容、健康检查是否过于敏感。
5. 对比分配策略和 Rebalance 前后的 Partition 移动量。

修复方向：

- 缩小单批任务或把长任务改为有状态异步任务。
- 使用协作式分配和稳定成员降低迁移。
- 发布时错峰优雅退出。
- 修复慢依赖和无界 Worker，不用无限增大超时掩盖问题。

38. ISR 缩小、离线分区和 Broker 故障如何排查？

结论先行：

先保护剩余健康副本和核心流量，再判断是磁盘、网络、资源争抢、数据倾斜还是节点故障；不要在证据不足时删除日志或强制重置状态。

排查顺序：

- 查看 Partition Leader、Replica、ISR 和 Offline Partition。
- 对齐 Broker 磁盘容量、I/O 延迟、网络吞吐、请求队列、CPU 和进程停顿。
- 检查 Leader 是否集中、单 Broker 是否承载热点、大规模重放或副本迁移。
- 查看 Controller/KRaft 元数据、Leader Election 和 Broker 注册事件。
- 核对 `acks` 与最小同步副本是否导致拒绝写入。

恢复原则：

暂停非核心重放和迁移，先让副本追平并恢复 ISR；确认 Leader 分布和错误率稳定后，再分批恢复流量。

39. Offset 重置和消息重放有哪些风险？

结论先行：

Offset 重置会造成重复或跳过数据，必须先保存原位点、明确范围、验证消费者幂等，并使用隔离 Group 控制重放速度。

安全流程：

```
确认修复目标和时间范围
-> 保存当前 Group 各 Partition Offset
-> 评估重复执行和外部副作用
-> 使用隔离 Group / 隔离环境小范围验证
-> 限速重放并监控下游
-> 对账结果
-> 决定是否切换或回滚
```

40. Kafka 安全和多租户如何治理？

结论先行：

安全治理至少包括传输加密、身份认证、资源授权和资源配额；多租户还要通过命名、所有权、容量和故障域避免相互影响。

机制	作用
TLS	加密客户端、Broker 和跨集群传输
SASL	对客户端和 Broker 身份认证

机制	作用
ACL	限制 Topic、Group、Cluster 等资源操作
Quota	限制租户生产、消费或请求速率，避免抢占集群

治理清单：

- Topic 命名包含环境、业务域、事件语义和版本。
- 每个 Topic 有 Owner、数据等级、保留时间、峰值和处置预案。
- 禁止大对象直接进入 Kafka，文件放对象存储，消息传引用和校验信息。
- 重要与非重要业务通过 Topic、配额或集群隔离。
- 凭证定期轮换，最小权限授权，审计高风险操作。

41. 灾备、跨集群和滚动升级如何考虑？

结论先行：

Kafka 副本主要解决单集群节点故障，不自动等同于跨地域灾备。灾备要明确复制方向、RPO/RTO、切换步骤、消费者位点和业务去重。

生产实践：

- 跨集群复制需要规划 Topic 配置、ACL、消息身份和消费位点映射。
- 双向复制要防止消息环路和重复。
- 灾备演练必须包含生产切换、消费恢复、重复消息和业务对账。
- 滚动升级前检查版本兼容、协议和元数据模式，先小范围节点验证。
- 升级期间监控 ISR、Leader Election、请求错误、Rebalance 和端到端业务指标。
- Kafka 不是业务数据库备份；权威数据和审计账本仍需自己的备份策略。

42. Kafka 故障如何复盘？

复盘结构：

影响：哪些业务、持续多久、数据是否丢失或重复
 时间线：告警、人工动作、恢复节点
 根因：直接原因、触发条件、为何未被提前发现
 止血：哪些动作有效，哪些动作放大了风险
 恢复：如何验证 Lag、业务成功率、对账和下游压力
 改进：监控、容量、发布、隔离、预案和演练

高级岗位表达重点是自己如何建立证据链、做风险判断和推进业务恢复，而不是强调人员管理。

十、业务架构与项目实战

43. Topic、Partition、Key 和 Consumer Group 如何设计？

结论先行：

先定义稳定事件语义和业务顺序维度，再按吞吐与恢复目标确定 Partition，按逻辑下游划分 Consumer Group，不能从组件默认值倒推业务模型。

设计问题：

1. 这是事实事件、任务命令还是状态快照？
2. 谁是权威数据源，是否需要历史重放？
3. 同一什么实体必须有序？
4. 峰值 TPS、字节流量、保留时间和消息时效是多少？
5. 有哪些独立下游，各自允许多大 Lag？
6. 失败后如何重试、补偿、对账和人工修复？

生产实践：

- 高低优先级拆 Topic，避免报表拖慢交易或投放主链路。
- 每个逻辑下游使用独立 Group。
- Key 选择订单、广告任务、设备等需要局部顺序的实体 ID。
- Topic 配置、Schema、Owner、SLA 和告警一起纳入发布评审。

44. 消息信封和 Schema 如何演进？

推荐消息信封：

```
{
  "msg_id": "uuid",
  "msg_type": "campaign_created",
  "schema_version": 2,
  "occurred_at": 1730000000,
  "producer": "ad-api",
  "trace_id": "trace-xxx",
  "business_key": "campaign-123",
  "data": {},
  "extensions": {}
}
```

演进原则：

- 优先新增可选字段并提供默认语义，避免直接删除或改名。
- Producer 先兼容旧 Consumer，Consumer 先支持新旧版本，再切换生产。
- 明确字段缺失、空值和删除事件的含义。

- 重要事件使用 Schema Registry 或团队统一的 JSON/Protobuf/Avro 兼容检查流程。
- 灰度 Consumer 验证后再扩大生产流量。
- Schema 变化要能关联发布版本、Owner 和回滚方案。

45. 如何设计可靠的支付成功事件?

业务目标:

支付成功后推进订单、履约、积分和通知，同时避免回调重复、消息丢失和重复履约。

设计方案:

支付回调

- > 以支付流水号幂等
- > 本地事务更新支付单 + 写 Outbox
- > Publisher 发布 payment-succeeded
- > order_id 作为 Key
- > 履约、积分、通知使用独立 Group

正确性:

- 支付流水号唯一约束，状态只允许合法前进。
- 消费者按订单号和事件类型幂等。
- 履约成功后再提交安全 Offset。
- Outbox 重试、消费重试、支付渠道对账和业务补偿共同闭环。
- 监控支付成功到履约完成的端到端延迟，而非只看发送成功。

降级:

消息系统短暂不可用时，支付本地事务仍可提交 Outbox；通知可延后，核心履约通过恢复投递和对账补齐。

46. 广告投放异步任务如何设计?

业务目标:

用户提交广告创建任务后快速返回，后台调用多个媒体 API，并处理限流、超时、不确定状态和重复创建风险。

方案:

- Topic 表达“广告创建任务已就绪”，以 `campaign_id` 或 `task_id` 为 Key。
- 任务表或 Mongo 文档保存 `pending -> publishing -> success/failed/unknown` 状态机。
- 消费者通过条件更新抢占任务，重复消息看到终态直接确认。
- 媒体支持幂等 Key 时透传业务唯一 ID；否则保存请求身份和平台返回 ID。
- API 成功但本地更新失败时进入 `unknown`，调用媒体查询接口反查，不能盲目重试创建。

- 按媒体或优先级隔离 Topic/Group，设置并发上限、退避、熔断和配额。
- 监控任务成功率、端到端耗时、平台错误码、重试和未知状态数量。

项目表达：

强调自己负责业务边界、状态机、幂等和故障恢复设计，以及如何与产品和外部平台联调；管理经验只体现风险识别和协作能力，不把职业方向描述为管理岗位。

47. MySQL/MongoDB 到 Elasticsearch 如何保证最终一致？

结论先行：

数据库是权威数据源，Kafka 传递可追踪变更，索引消费者幂等写入 Elasticsearch，并通过版本校验、补偿扫描和全量重建保证最终一致。

```
业务事务 / Change Stream / CDC
-> Outbox 或变更事件
-> Kafka
-> ES Indexer Group
-> 回查权威数据
-> Bulk Upsert/Delete
-> 记录同步版本和结果
```

关键设计：

- Key 使用文档或业务实体 ID，保证同一实体尽量局部有序。
- 消息带操作类型、数据版本、`msg_id` 和 `trace_id`。
- Upsert 前比较版本，旧事件不覆盖新索引。
- 消费者优先回查权威源构建最新文档，降低乱序影响。
- Bulk 部分失败必须逐项识别并重试，不能整批视为成功。
- 删除 404 可按幂等成功处理，但要保留审计。
- 定时扫描版本落后数据；大规模修复使用隔离 Group 或新索引重建后切别名。

48. 家庭防火墙设备事件如何削峰？

业务目标：

设备集中上线或网络恢复会产生突发事件，系统需要保护设备状态、App 推送和统计链路，核心状态优先于非实时分析。

方案：

- 按 `device_id` 分区，保证同一设备状态局部有序。
- 设备状态、通知、统计使用独立 Group；高低优先级必要时拆 Topic。
- 状态消费者采用版本或设备序列号，防止旧事件覆盖新状态。
- 批量写数据库和缓存，推送链路设置独立限流与重试。

- 容量按设备集中恢复峰值而不是日均流量规划。
- 积压时优先恢复状态更新，统计和非紧急通知可降级或延后。

面试追问：为什么不能只扩消费者？

可能受 Partition 数、单设备热点、数据库写入、推送接口限流和顺序要求限制。扩容前必须验证下游余量和净恢复速度。

49. Kafka、RocketMQ、RabbitMQ 如何做业务选型？

结论先行：

不按组件名做结论，先看吞吐、延迟、重放、顺序范围、延迟消息、事务边界、路由复杂度、团队运维能力和已有基础设施。

维度	Kafka	RocketMQ	RabbitMQ
核心定位	高吞吐、可重放的分布式日志流	交易和业务消息能力较完整	传统消息队列和灵活路由
消费模型	Partition + Group + Offset	Topic/Queue + 消费组	Exchange + Queue + ACK
重放	强，适合索引和读模型重建	支持，按其消费和存储模型治理	通常不作为长期日志重放首选
局部顺序	单 Partition	单队列/分片	单 Queue，可用并发受限
延迟消息	通常需要额外调度设计	业务能力较成熟	可通过 TTL/DLX 等实现，需评估精度和治理
事务边界	Kafka 内部事务	事务消息和回查机制	依赖确认与业务补偿设计
路由	Topic/Partition/Group，较简单	Topic、Tag 等	Exchange、Binding、Routing Key 灵活
典型优势	大事件流、多订阅、重放、CDC	电商交易、延迟、事务和重试	中等吞吐、任务队列、复杂路由

结合项目选择：

- 广告行为流、设备事件、CDC、Elasticsearch 索引构建：优先考虑 Kafka。
- 强依赖事务消息回查、延迟和交易消息治理：评估 RocketMQ。
- 消息量中等但路由规则复杂、临时队列或任务分发突出：评估 RabbitMQ。
- 团队已有成熟平台时，迁移成本、SDK、监控、值班和故障经验也是重要约束。

50. 如何用两分钟回答 Kafka 项目题？

推荐模板：

这个场景的目标是把非主链路异步化并承接峰值流量。我先定义稳定事件和权威数据源，再按业务实体选择 Partition Key，按峰值吞吐和恢复目标规划 Partition。生产侧用本地事务加 Outbox 保证事件可恢复，Broker 侧使用多副本和同步副本约束，消费侧在业务成功后提交 Offset，并通过唯一约束、状态机或版本号保证幂等和防乱序。失败进入分级重试和死信，同时有补偿扫描和对账。线上监控端到端延迟、Lag 增长率、失败率、ISR 和业务完成率；积压时先保护下游，再按净恢复速度扩容。选型时还会评估延迟、路由、重放、团队运维能力和事务边界，而不是只看吞吐。

资深 IC 加分点：

- 能量化峰值、Partition、磁盘和恢复时间。
- 能说清异常状态、补偿入口和上线验证。
- 能用一次真实故障说明证据链和改进结果。
- 能推动接口、Schema、SLA 和监控标准落地。
- 管理经验只作为拆解问题、识别风险和跨团队协作的补充优势。

04-01 | 微服务与RPC治理

技术 | 微服务与 RPC 治理面试题

面向 9 年以上 Golang Web 后端业务研发岗位。重点是服务为什么拆、边界如何定、RPC 失败如何治理、数据如何协作、变更如何安全发布，以及怎样用工程指标证明微服务没有变成分布式单体。

一、微服务学习地图与资深回答框架

1. 微服务与 RPC 的知识图谱主线是什么？

一句话结论：微服务不是把代码拆成更多进程，而是围绕业务能力建立可独立演进、交付和治理的边界，并承担由网络调用和数据分治带来的复杂度。

业务复杂度与组织边界

- > 领域拆分、服务职责和数据所有权
- > API / HTTP / gRPC / Protobuf 契约
- > 注册发现、配置、负载均衡和健康检查
- > Deadline、重试、幂等、限流、熔断、降级
- > 本地事务、事件驱动和工作流
- > 可观测性、发布、灰度、回滚和兼容
- > 安全、容量、容灾和成本
- > 线上排障与项目表达

2. 资深后端如何回答微服务问题？

采用“业务问题 -> 边界原则 -> 调用与数据 -> 失败治理 -> 发布运维 -> 指标收益”的结构。

示例：为什么拆订单服务？

- **业务问题：** 订单生命周期、交易 SLA 和迭代节奏与商品、营销不同。
- **边界：** 订单服务拥有订单状态与流水，其他服务不能直接改表。
- **调用与数据：** 同步 API 完成必须即时确认的操作，领域事件传播状态变化。
- **失败治理：** 幂等、状态机、Outbox、deadline、限流和补偿。
- **发布运维：** 接口兼容、灰度、回滚、数据迁移和独立容量。
- **收益：** 发布频率、故障域、P99、扩容效率改善；同时承认链路和一致性复杂度增加。

3. 微服务架构题的回答步骤是什么？

1. 量化业务规模、团队协作、变化频率和 SLA。

2. 识别领域边界、核心用例和数据所有权。
3. 设计同步 API 与异步事件，限制调用链深度。
4. 定义超时、重试、幂等、熔断、降级和过载保护。
5. 设计一致性、状态机、补偿、对账和重放。
6. 设计可观测性、灰度发布、兼容、回滚和故障演练。
7. 用交付、稳定性和成本指标验证拆分价值。

4. 微服务高频题如何串联？

层次	核心问题	关键词
拆分	为什么拆、拆多细	领域能力、变化率、数据所有权
契约	HTTP 还是 gRPC	语义、兼容、流式、调试、生态
路由	如何找到实例	注册发现、健康、LB、连接复用
韧性	下游变慢怎么办	deadline、重试预算、熔断、隔离
数据	跨服务如何一致	本地事务、Outbox、Saga、对账
观测	如何定位跨服务问题	RED、Trace、结构化日志、业务 ID
发布	如何避免变更事故	向后兼容、灰度、回滚、Feature Flag
安全	服务间如何信任	mTLS、身份、授权、密钥、审计
实战	怎么证明设计有效	SLA、故障域、交付效率、成本

二、服务拆分、领域边界与架构演进

5. 单体、模块化单体和微服务如何选择？

- 单体部署简单、调用本地、事务自然，适合早期和复杂度有限的系统。
- 模块化单体通过清晰包边界和数据访问约束控制耦合，是很多团队的优先起点。
- 微服务适合业务边界较稳定、团队需要独立交付、容量和 SLA 差异明显的场景。

如果没有独立演进和治理需求，拆成微服务只会增加网络、数据一致性、测试和运维成本。

6. 服务边界如何划分？

优先依据业务能力、聚合和变化原因，而不是按 Controller/DAO 技术层拆分。一个服务应有：

- 清晰业务职责和语言。

- 独立数据所有权。
- 可独立发布和扩缩容。
- 相对内聚的变更集合。
- 可接受的同步调用数量和团队认知负担。

领域边界不是一次设计永久不变，需要用事件风暴、调用数据和变更历史持续校准。

7. 什么是限界上下文？

同一业务词在不同上下文可有不同模型。例如“用户”在账户域是认证主体，在广告域是投放客户，在结算域是合同与账务主体。限界上下文明确模型语义、规则和责任，服务通常围绕它划分，但不必机械一一对应。

8. 服务粒度过细有什么问题？

- 一次请求跨多个服务，尾延迟和故障概率叠加。
- 本地事务变成分布式一致性。
- 发布、测试、版本和运维对象暴增。
- 团队频繁跨服务修改，边界并未降低耦合。
- 小服务各自占连接池和基础资源，成本上升。

若两个服务总是一起改、一起发布、共享数据库且频繁同步调用，通常应重新评估合并。

9. 什么是分布式单体？

系统虽然拆成多个进程，但服务共享数据库、调用链强耦合、必须一起发布、任何一个失败都影响全链路。这既失去单体简单性，又没有微服务独立性。

治理方向是明确数据所有权、减少同步依赖、建立兼容契约、允许独立发布，并合并没有独立业务价值的小服务。

10. 服务能否共享数据库？

长期目标应是服务拥有自己的数据，其他服务通过 API 或事件访问。共享数据库会绕过业务规则、导致 Schema 变更互相影响、难以独立扩容和审计。

迁移期可以共享实例但分 Schema/账号，并禁止跨域直接写；通过 CDC、事件和渐进迁移逐步收回所有权。

11. 如何从单体渐进迁移到微服务？

使用 Strangler Pattern：

1. 先模块化和补可观测性，找到高收益边界。
2. 在网关或应用路由层切出一个低耦合能力。

3. 通过 Anti-Corruption Layer 隔离旧模型。
4. 先同步历史数据，再双写/CDC 增量并对账。
5. 灰度流量，保留回切。
6. 稳定后删除旧路径，而不是永久双写。

12. 如何判断拆分是否成功？

- 服务可独立开发、测试、发布和回滚。
- 跨团队变更和发布协调次数下降。
- 故障影响范围和恢复时间下降。
- 可按服务容量独立扩缩容。
- 业务 P99、可用性和交付周期改善。
- 基础设施、人力和调用复杂度没有失控。

三、HTTP、gRPC、Protobuf 与接口契约

13. HTTP REST 与 gRPC 如何选择？

维度	HTTP/JSON	gRPC/Protobuf
可读与调试	直观、生态广	需工具和生成代码
契约	OpenAPI 可描述	IDL 强约束、代码生成
性能	文本编码开销较高	二进制、HTTP/2、多路复用
流式	支持方式较分散	原生四种 RPC 模式
浏览器/外部 API	更自然	常需网关或 gRPC-Web
内部服务	通用	强类型、高吞吐场景常见

外部开放 API 常优先 HTTP/JSON，内部稳定服务可优先 gRPC；也可通过网关提供双协议。

14. gRPC 的四种调用模式是什么？

- Unary：一请求一响应，最常见。
- Server Streaming：一个请求持续返回流。
- Client Streaming：客户端持续发送，服务端最终响应。
- Bidirectional Streaming：双方独立流式发送。

流式并不自动更快，要处理背压、生命周期、断线恢复、消息顺序、内存和负载均衡黏性。

15. Protobuf 为什么高效?

它使用字段编号和紧凑二进制编码, 省去重复字段名; 生成代码提供类型约束, 未知字段可支持兼容转发。代价是可读性较差、需要 IDL 与生成流程、某些动态结构表达不如 JSON 直接。

性能收益仍应端到端压测, 数据库和业务处理可能才是主要瓶颈。

16. Protobuf 如何保证兼容演进?

- 已发布字段号永不复用, 删除字段用 `reserved` 保留编号和名称。
- 新增可选字段并提供明确默认语义。
- 不随意改变字段类型和语义。
- Enum 新增值时旧客户端必须能处理未知值。
- 先发布能兼容新旧数据的服务端/消费者, 再启用新字段。
- 在 CI 中检查 Breaking Change。

17. API 如何做向后兼容?

优先增加字段和能力, 不改变旧字段含义; 服务端对缺失字段提供兼容; 删除采用“废弃 -> 观测使用量 -> 通知 -> 双版本 -> 下线”。

版本号只能解决识别问题, 不能替代兼容设计。内部 API 也必须有消费者清单和契约测试。

18. 接口应该返回领域对象还是专用 DTO?

使用专用 API DTO 隔离内部存储与领域实现, 防止数据库字段直接成为永久外部契约。DTO 只暴露调用方需要的信息, 明确可空性、单位、时区、分页和错误语义。

19. 错误码如何设计?

分为: 参数/认证/权限、资源不存在、业务冲突、限流过载、依赖失败和内部错误。错误响应包含稳定机器码、可读信息、trace_id 和可选重试提示, 不暴露堆栈和内部敏感信息。

gRPC 状态码要映射业务语义; `UNAVAILABLE` 可能重试, `INVALID_ARGUMENT` 不应重试, `ALREADY_EXISTS/ABORTED` 要结合幂等和并发语义。

20. 分页接口如何设计?

大数据集使用基于稳定排序键的 Cursor, 令牌包含排序值、查询条件版本和必要签名; 限制 page_size 和返回字段。Offset 适合浅分页和小集合, 不适合深分页。

21. API Gateway 和 BFF 分别解决什么?

- Gateway 处理统一入口、认证、路由、限流、协议转换、观测和灰度。
- BFF 为特定客户端聚合和裁剪数据, 减少移动端/网页多次调用。

不能把所有业务编排塞进网关；核心业务规则仍属于领域服务，避免网关成为新单体。

四、服务发现、配置、负载均衡与连接治理

22. 服务发现的基本流程是什么？

```
实例启动 -> 注册地址和元数据 -> 健康/租约维护  
客户端或网关查询实例列表 -> 负载均衡选择 -> 发起调用  
实例下线 -> 先摘流量 -> 完成在途请求 -> 注销
```

注册中心保存控制面信息，不应进入每次请求的数据面关键路径；客户端需要本地缓存和异常兜底。

23. 客户端发现和服务端发现有什么区别？

- 客户端发现由调用方获取实例列表并负载均衡，少一跳但客户端复杂、语言 SDK 多。
- 服务端发现由 LB/Proxy 统一路由，客户端简单但增加代理层和运维成本。
- Service Mesh 属于代理化方案，将流量治理下沉到 Sidecar/节点代理。

24. 健康检查应该检查什么？

- Liveness：进程是否需要重启，不能因为临时下游失败就重启。
- Readiness：是否可以接收新流量，包括初始化、预热和关键资源状态。
- Startup：慢启动期间避免被误杀。
- gRPC 可暴露标准 Health Service，由服务更新 SERVING 状态。

健康检查要轻量、有超时，不能每次深查所有依赖造成探测风暴。

25. 常见负载均衡算法如何选择？

- Round Robin：实例同质、请求相近。
- Weighted Round Robin：实例容量不同。
- Least Request/Connection：请求耗时差异明显。
- Consistent Hash：需要会话或缓存局部性，但可能热点。
- EWMA/自适应：依据历史延迟和排队选择更快实例。

无论算法如何，都需配合异常实例摘除、慢启动和连接池治理。

26. gRPC 长连接如何影响负载均衡？

HTTP/2 多路复用让大量 RPC 复用少量连接。如果客户端只建立一个连接并在连接建立时选定实例，流量可能长期落在单实例。

需要使用合适的 Name Resolver、LB Policy、连接池或代理；扩容后还要让客户端及时发现并建立到新实例的 Subchannel。

27. 配置中心应如何治理？

- 配置有 Schema、默认值、环境和作用域。
- 变更经过权限、审计、评审和灰度。
- 客户端使用本地缓存，配置中心故障不应让服务立即不可用。
- 动态配置要验证范围和边界，支持快速回滚。
- 关键参数如超时、限流、开关要有变更事件和指标关联。

配置中心不是保存密码的普通 KV；密钥应进入专用 Secret 系统。

28. 优雅启动和优雅停机怎么做？

启动时先完成配置、连接池、缓存预热和依赖检查，再置 Ready。停机时先置 Not Ready/摘流量，停止接收新请求，等待在途 RPC 和消费者处理，提交 Offset/Flush 数据，超时后退出。

Golang 要处理 SIGTERM、Context Cancellation、HTTP Server Shutdown、gRPC GracefulStop 的超时兜底和后台 Goroutine 回收。

29. DNS、注册中心和本地缓存不一致怎么办？

客户端要容忍实例列表短暂陈旧：连接失败快速摘除并后台刷新，列表为空时短时使用最后成功快照，但不能无限信任。监控解析错误、实例收敛时间和过期实例调用率。

五、Deadline、重试、幂等、限流、熔断与隔离

30. 为什么每个 RPC 都必须设置 Deadline？

没有 Deadline 的调用可能无限等待，占用 Goroutine、连接、内存和上游请求，最终级联过载。Deadline 应根据端到端 SLO、历史 P99 和下游预算设置，并沿调用链传播。

服务端收到取消后必须停止数据库、HTTP 和后台工作，避免调用方放弃后仍浪费资源。

31. 超时预算如何分配？

若入口剩余 800ms，不能给三个串行下游各 800ms。需预留网关、排队、业务处理和响应时间，再按关键路径分配；并行调用预算取最大分支而非简单相加。

预算应动态使用剩余 Deadline，避免中间服务重新创建更长超时。

32. 哪些 RPC 可以重试？

只重试瞬时错误、幂等操作、未超过总 Deadline 且未触发过载保护的调用。使用有限次数、指数退避、抖动和 Retry Budget。

创建/扣款等写操作必须有幂等键；如果服务端已收到请求但响应丢失，透明重试也可能重复执行。

33. 如何实现接口幂等？

- 调用方生成业务唯一 request_id。
- 服务端用唯一索引或幂等表持久化请求状态和结果。
- 同一 key 参数摘要不同则拒绝，防止误复用。
- 并发请求看到处理中状态时等待、查询或返回明确语义。
- 幂等记录和业务写入处于同一本地事务。

34. 限流应该放在哪些层？

- CDN/WAF：恶意和大流量初筛。
- Gateway：租户、用户、API 全局配额。
- 服务实例：本地 QPS 与并发保护。
- 依赖侧：数据库、第三方 API 和热点资源配额。

令牌桶允许突发，并发限制更适合保护慢调用；全局精确和本地高可用常组合使用。

35. 熔断器的状态机是什么？

Closed 正常统计；错误率/慢调用达到阈值后进入 Open 快速失败；等待后进入 Half-Open，只允许少量探测，成功则关闭，失败再次打开。

熔断按下游方法或实例池隔离，设置最小请求量，避免低流量误判；恢复要渐进，不能瞬间放回全部流量。

36. 舱壁隔离解决什么问题？

将不同依赖、租户或业务等级放入独立连接池、并发池、队列或实例组，避免一个慢依赖占满全部资源。Golang 中可用有界 Worker Pool、Semaphore 和独立 HTTP/gRPC Client 实现。

37. 降级如何保持业务语义清晰？

- 推荐不可用返回热门内容，而不是伪造个性化结果。
- 搜索不可用返回基础筛选或缓存，并标识数据时间。
- 非核心统计缺失可省略字段。
- 核心写失败必须明确失败或处理中，不能返回假成功。

降级要定义触发、恢复、缓存陈旧上限和数据补偿。

38. 如何防止调用链放大？

- 限制同步调用深度和扇出数量。
- 批量 API 替代循环 N+1 RPC。
- 并行请求设置总并发和共享 Deadline。
- 请求级缓存/合并相同下游调用。
- 不在多层重复重试。
- 将非关键动作异步事件化。

39. 第三方 API 如何治理？

使用独立 Client、连接池、deadline、限流、熔断和并发隔离；请求带外部幂等号；超时后通过查询接口确认结果；错误分类重试，长期失败进入任务与人工处理；记录供应商成功率、P99、配额和成本。

六、数据所有权、事件驱动、事务与 workflow

40. 微服务之间为什么不能直接改对方数据库？

直接改表绕过业务规则，使 Schema 和发布强耦合，责任与审计模糊，也让服务无法独立迁移。应通过命令 API 请求对方执行规则，或订阅其公开事件构建本地读模型。

41. 同步调用和异步事件如何选择？

- 调用方必须立即知道结果、操作短且交互式：同步 RPC。
- 可延迟、需削峰、一对多传播、希望解耦：异步事件。
- 核心命令常同步确认受理，后续工作异步执行。

异步不是自动可靠；需要事件契约、幂等、顺序、重试、死信、对账和可观测性。

42. Command 和 Event 有什么区别？

- Command 表达“请某个服务做某事”，有明确目标和可能失败。
- Event 表达“某个事实已经发生”，发布方不应要求某个消费者必须采取特定内部动作。

把事件命名为 `OrderCreated` 比 `NotifyInventoryToDeduct` 更解耦；但关键流程仍需确认消费者结果和超时治理。

43. 跨服务一致性如何设计？

单服务内使用本地事务保存业务事实和 Outbox；事件发布到 Kafka；消费者用 Inbox/唯一键幂等处理，并产生下一事件。长流程通过显式状态机或 Saga 编排，失败执行补偿，最终由对账发现遗漏。

强一致核心应尽量收敛到同一服务边界，而不是在所有服务间追求分布式锁。

44. Saga 编排和协同如何选择？

- Choreography：服务订阅事件自行反应，耦合低，但流程分散、排障困难。
- Orchestration：编排器显式下发步骤和记录状态，流程清晰，编排器复杂且需高可用。

短流程可协同，跨多个服务、超时和人工节点的关键流程常用编排。无论哪种都要幂等、补偿和审计。

45. Outbox Pattern 解决什么问题？

在同一本地事务中更新业务数据并插入 Outbox，后台 Relay/CDC 发布事件，消除“数据库提交成功但消息未发送”的窗口。

它仍可能重复发布，消费者必须幂等；还要治理 Outbox 积压、分区、清理、事件版本和发布延迟。

46. 事件 Schema 如何演进？

- 使用稳定 event_type、event_id、aggregate_id、version、occurred_at 和 trace context。
- 新增字段保持旧消费者可忽略，删除字段经过废弃周期。
- 不随意改变字段语义和单位。
- Schema Registry/CI 做兼容检查。
- 消费者先兼容新旧版本，再升级生产者。

47. 如何处理事件重复、乱序和迟到？

重复用 event_id/业务键去重；同一实体按 Key 分区；事件带 version，旧版本拒绝覆盖；缺版本可回查事实源或触发重建；迟到事件按业务窗口处理。

删除也要带版本或 Tombstone，防止迟到更新复活数据。

48. CQRS 适合什么场景？

写模型承载业务规则和事务，读模型为搜索、列表、报表按查询需求冗余。适合读写模型差异大、查询复杂、可接受最终一致的系统。

代价是模型同步、重复数据、延迟、对账和运维。普通 CRUD 不应为了模式完整而引入 CQRS。

49. 如何治理长时间业务流程？

使用持久化 workflow 状态机记录步骤、输入、尝试次数、Deadline 和补偿；每步幂等，定时扫描超时任务，支持人工暂停/重试/跳过；外部副作用保存请求号和响应证据。

不要用内存 Goroutine 或一次长 RPC 承载跨分钟、小时的流程。

七、可观测性、发布治理与变更安全

50. 微服务可观测性应该覆盖什么？

- 接口 RED：Rate、Errors、Duration。
- 实例 USE：Utilization、Saturation、Errors。
- Trace：入口到 DB、缓存、MQ、第三方的关键路径。
- 结构化日志：service、version、instance、trace_id、business_id、error_code。
- 业务不变量：订单状态、同步延迟、差异数和积压。
- 变更事件：发布、配置、扩容、路由和依赖切换。

51. Trace Context 如何传播？

在 HTTP Header、gRPC Metadata 和消息 Header 中传播 trace/span、request_id 和必要 baggage。异步消息创建 Consumer Span 并链接 Producer 上下文。

Baggage 只放小且必要的信息，敏感数据和高基数字段不能无界传播。

52. 日志如何避免“很多但没用”？

记录关键状态变化、依赖结果、错误分类和关联 ID；使用稳定字段而非拼接文本；控制级别、采样和敏感信息；一次错误由最理解语义的层记录，避免每层重复堆栈。

53. SLI、SLO 和 Error Budget 是什么？

- SLI 是实际测量值，如成功率和 P99。
- SLO 是目标，如月度 99.95% 成功率。
- Error Budget 是目标允许的失败额度，用于权衡发布速度和稳定性。

SLO 应面向用户体验，不只看容器存活；预算消耗过快时暂停高风险变更并治理可靠性。

54. 灰度发布如何设计？

按内部用户、租户、地域、设备或流量比例逐步放量；灰度版本与基线对比错误率、P99、资源和业务指标；设置自动暂停/回滚阈值；确保同一会话和状态流程路由一致。

数据库和消息 Schema 必须先兼容，再灰度应用，避免回滚后旧版本读不了新数据。

55. 蓝绿、滚动和金丝雀发布如何选择？

- 滚动成本低，版本短期混跑，要求兼容。
- 蓝绿切换和回滚快，但需要双份容量和数据变更策略。
- 金丝雀小流量验证风险，适合高频发布，但监控必须能区分版本。

选择取决于容量、兼容、风险和回滚速度，不应只由平台默认决定。

56. Feature Flag 如何治理？

Flag 要有负责人、范围、默认值、到期时间和回滚语义；变更需审计和灰度；代码同时支持开关两侧并做测试；功能稳定后及时删除 Flag，避免组合爆炸。

57. 数据库变更如何做到向后兼容？

采用 Expand-Contract：先新增字段/表并让旧代码可运行；新代码双读/双写或后台回填；验证后切读；最后停止旧写并删除旧结构。DDL、应用发布和数据迁移分阶段，每阶段可观察可回滚。

58. 消费者发布如何避免重复和丢处理？

停机前停止拉取、完成在途处理、提交 Offset 并关闭资源；新旧版本混跑时事件 Schema 兼容；每个消费者幂等；Rebalance 和超时期间不让同一批任务失控并发。大变更可新建 Consumer Group 影子消费验证。

八、安全、性能、容量与 Golang 工程实践

59. 微服务身份认证和授权如何分层？

- 入口验证用户身份和 Token，传递可信主体信息。
- 服务间使用工作负载身份和 mTLS，不能只信内网 IP。
- 每个服务在资源边界做授权，防止越权横向访问。
- 权限遵循最小化，敏感操作审计。

用户身份与服务身份是两层概念，不能用一个共享密钥替代全部授权。

60. mTLS 解决什么，没解决什么？

mTLS 提供双向身份认证和传输加密，可确认连接双方工作负载身份。它不自动决定“该服务能否访问某个订单”，仍需授权策略；也不解决应用层注入、越权和敏感日志。

61. Secret 如何管理？

密钥存专用 Secret Manager/KMS，通过短期凭证或工作负载身份获取；禁止写入代码、镜像、普通配置和日志；定期轮换并支持双密钥过渡；限制读取范围并审计。

62. 微服务容量如何估算？

```
实例数 = 峰值 QPS / 单实例安全 QPS × 冗余系数
并发 ≈ QPS × 平均延迟
下游调用 QPS = 入口 QPS × 单请求调用次数 × 重试放大
```

还要计算连接池、响应大小、CPU/内存、消息积压恢复、N-1 容量和发布期间双版本容量。单实例安全 QPS 来自真实依赖和请求混合压测。

63. 如何降低 RPC 尾延迟？

- 减少串行调用和扇出。
- 治理慢实例、连接建立、DNS 和连接池等待。
- 缩小请求/响应，使用批量接口。
- 为热点数据缓存或预计算。
- 控制 GC、Goroutine、锁和下游查询。
- Hedging 只用于幂等读且有额外流量预算，不能默认开启。

64. Golang HTTP/gRPC 服务如何做资源治理？

- 入口设置读写、Header、Idle 和业务 Deadline。
- 限制请求体、并发、Goroutine 和有界队列。
- 复用 `http.Transport` / `grpc.ClientConn`，不要每请求建连接。
- 数据库和下游调用传播 Context。
- Interceptor/Middleware 统一认证、Trace、指标、恢复和错误映射。
- 优雅停机并等待在途任务，后台 Goroutine 可取消可回收。

65. gRPC Interceptor 适合做什么？

适合统一认证、授权、Trace、Metrics、日志、Panic Recovery、限流和错误规范化。业务规则、复杂重试和不同方法的降级不应全部塞进一个全局 Interceptor。

顺序很重要：恢复通常最外层，Trace 要覆盖后续处理，日志和指标应得到最终状态码。

66. Service Mesh 的收益和代价是什么？

收益是统一 mTLS、流量路由、观测和部分重试熔断，减少多语言 SDK 重复。代价是代理资源、网络跳数、控制面复杂度、排障层次和配置风险。

Mesh 解决通用流量治理，不理解业务幂等、补偿和降级语义；团队规模和异构程度不足时不必急于引入。

九、生产故障排查与稳定性治理

67. 微服务接口 P99 突然升高如何排查？

1. 确认影响接口、租户、地域、版本和时间线。
2. Trace 判断耗时在排队、应用、RPC、DB、缓存还是第三方。
3. 检查流量、错误、实例负载、GC、Goroutine、连接池和队列。
4. 检查下游 P99、超时、重试和熔断状态。
5. 对比发布、配置、扩容、证书和路由变化。
6. 先限流、降级、回滚或隔离慢依赖，再做代码/索引根治。

68. 大量 RPC DeadlineExceeded 如何定位？

区分 Deadline 本身过短、下游处理慢、客户端排队、连接建立/DNS、网络、服务端未响应取消和上游剩余预算不足。查看 Client/Server Span、剩余 Deadline、状态码、连接池等待和下游资源。

不要第一反应只把超时调大，否则会增加在途请求并放大故障。

69. 服务发现异常如何处理？

判断注册中心不可用、实例心跳失效、DNS 缓存、客户端 Resolver 未更新还是实例 Ready 状态错误。短期使用最后成功实例快照、快速摘除连接失败节点、停止扩散；恢复后验证实例列表收敛和连接重建。

70. 发布后错误率上升如何止血？

立即冻结放量，比较新旧版本的错误码、P99、资源和业务指标；若满足阈值直接回滚或关 Feature Flag。检查 Schema/配置兼容和下游依赖，回滚后继续观察积压与数据副作用，不能只看 Pod 恢复。

71. 消息积压导致业务延迟如何处理？

拆分生产速率、消费速率、失败重试和单条耗时；识别毒消息、下游 DB/ES 变慢和分区热点。止血时暂停低优先级生产/重放、隔离毒消息、控制消费者并发保护下游；计算净恢复速率和预计恢复时间。

72. 级联故障如何切断？

关闭多层重试，限制入口和并发，熔断慢依赖，启用缓存/静态降级，释放卡住连接和 Goroutine。按最底层依赖到入口逐层恢复，并渐进放量，避免刚恢复就被积压打回故障。

73. 微服务稳定性治理清单是什么？

- 每个外部调用有 Deadline、错误分类和幂等策略。
- 每个队列、连接池和并发池有界。

- 每个核心接口有 SLO、降级和容量基线。
- 每次发布可灰度、可回滚、可区分版本指标。
- 每条关键事件链可追踪、可重放、可对账。
- 定期做实例故障、依赖变慢、注册中心异常和备份恢复演练。

74. 故障复盘如何避免只归因于个人？

分析直接错误之外的系统条件：为什么评审没发现、测试为何没覆盖、权限为何允许、监控为何晚报、降级为何无效、回滚为何慢。改进要进入自动检查、平台能力、容量模型、演练和责任流程，而不只是“以后注意”。

十、业务架构、项目实战与高级 IC 表达

75. 广告投放平台如何划分服务？

可按业务能力划分账户/授权、广告资产、投放任务、调度执行、媒体适配、报表等边界。投放任务拥有状态机；媒体适配封装外部平台差异；报表消费事件构建读模型。

同步链路只做校验和受理，耗时发布通过工作流/Kafka 异步执行。服务间不能直接改对方表，状态通过 API 与领域事件协作。

76. 第三方媒体发布 RPC 如何设计？

- 请求包含 task_id、platform、account_id、request_id 和业务版本。
- 调用前做账号级限流和并发隔离。
- 使用明确 Deadline、错误分类和退避。
- 超时后用 request_id 查询媒体侧结果，避免重复创建。
- 结果持久化并产生状态事件，永久失败进入人工处理。
- 指标按媒体、账号、错误码统计成功率、P99、限流和重试。

77. 如何把一个同步长接口改造成异步任务？

1. API 校验并幂等创建任务，快速返回 task_id 和 accepted 状态。
2. 本地事务写任务与 Outbox，事件进入 Kafka。
3. Worker 领取并按状态机执行，每一步保存进度。
4. 客户端轮询、SSE/WebSocket 或回调获取状态。
5. 超时、重试、取消和人工介入都有明确语义。
6. 监控排队时间、执行时间、成功率和积压恢复时间。

78. 微服务拆分项目如何证明不是过度设计？

展示拆分前的具体痛点：发布互相阻塞、故障域过大、某模块容量独特、代码变更冲突或数据职责混乱。再给出拆分后的量化变化：独立发布比例、发布周期、故障影响、实例成本、P99 和跨团队协作次数。

同时坦诚新增的 RPC、最终一致性和运维成本，以及如何用平台和规范控制。

79. 如何表达跨团队推动能力但保持业务研发定位？

我有一年多带团队和协作经验，所以在做核心业务研发时，会主动推动接口契约、风险清单、容量评审、灰度方案和故障复盘落地。我能理解排期和协作成本，也能和产品、前端、测试及基础设施团队对齐。但我的职业方向仍是高级业务研发/架构职责，希望继续亲自承担核心设计、编码和线上问题，而不是转为纯管理角色。

项目回答要具体说明自己写过什么、做过什么取舍和解决过什么故障，管理经验只作为推动复杂交付的加分项。

80. 如何回答“是否所有服务都要上 gRPC、Mesh 和 Kafka”？

不需要。协议和基础设施由需求决定：外部开放 API 可能更适合 HTTP；低频简单调用不必为微小性能收益上 gRPC；团队语言单一且治理简单时 SDK 已足够，不必上 Mesh；必须立即返回且无异步价值的操作不必上 Kafka。

高级判断是以业务收益、故障模型、团队能力和总成本做选择，而不是追求组件齐全。

81. 如何用两分钟回答微服务架构题？

我不会把微服务理解为把单体拆成很多进程，而是围绕业务能力、数据所有权和独立变化率建立边界。先确认团队规模、发布冲突、容量差异和 SLA，边界不清时优先模块化单体。服务间即时确认走有明确契约的 HTTP/gRPC，非关键传播走事件；限制同步链路深度和 N+1 调用。每个 RPC 都传播 Deadline，重试只用于幂等瞬时错误并受 Retry Budget 控制，同时配置并发上限、熔断、隔离和业务降级。数据由服务自己拥有，单服务内用本地事务，跨服务通过 Outbox、Kafka、幂等消费者、状态机、补偿和对账保证最终一致。发布遵循 Schema 向后兼容、灰度和快速回滚；运行时用 RED、Trace、结构化日志和业务不变量定位问题。最后用独立发布率、P99、可用性、故障域、恢复时间和资源成本证明拆分是否真正带来收益。

04-02 | 分布式与高并发架构

技术 | 分布式与高并发架构面试题

面向 9 年以上 Golang Web 后端业务研发岗位。重点是从故障模型和业务目标推导一致性、幂等、事务、协调、流量治理与容量方案，而不是背 CAP、锁和中间件名词。

一、分布式系统学习地图与资深回答框架

1. 分布式与高并发的知识图谱主线是什么？

一句话结论： 分布式系统是在网络、节点和时钟都不完全可靠的条件下，通过冗余、协议和业务约束实现可接受的一致性、可用性、性能与成本。

业务目标：正确性 / 可用性 / 延迟 / 吞吐 / 成本

- > 故障模型：超时、重复、乱序、分区、崩溃、慢节点
- > 一致性模型：线性一致、因果、最终一致
- > 身份与时间：ID、版本、幂等键、逻辑时钟
- > 协调：Quorum、共识、Leader、Lease、Lock、Fencing
- > 事务：本地事务、Outbox、TCC、Saga
- > 流量：缓存、限流、熔断、降级、背压
- > 高并发设计：读扩展、写扩展、削峰和热点治理
- > 容量、容灾、可观测性和项目表达

2. 资深后端如何回答分布式问题？

采用“目标 -> 约束 -> 失败场景 -> 不变量 -> 方案 -> 权衡 -> 验证”的结构。

示例：如何防止库存超卖？

- **目标：** 已确认订单的扣减总量不能超过可售库存。
- **约束：** 多实例并发、请求重试、MQ 重复、下游超时。
- **不变量：** `available >= 0`，同一订单只扣一次。
- **方案：** 数据库条件更新或原子脚本完成预扣，订单号唯一约束保证幂等，超时后按订单号查结果；取消通过状态机补偿。
- **权衡：** Redis 预扣吞吐高但需要与数据库对账；DB 条件更新简单但热点行上限较低。
- **验证：** 并发压测、重复/乱序/超时注入、库存账实对账和故障恢复演练。

3. 架构题为什么要先定义不变量？

不变量是任何故障下都必须成立的业务规则，例如：

- 钱不能凭空增加或消失。
- 同一订单只能成功支付一次。
- 已售库存不能超过总库存。
- 已发布版本不能被旧事件覆盖。

先定义不变量，才能判断哪些操作必须强一致，哪些状态允许最终一致，哪些重试需要幂等，哪些失败必须人工介入。

4. 如何区分功能需求和非功能需求？

- 功能需求：下单、扣库存、发布、查询、取消。
- 非功能需求：峰值 QPS、P99、可用性、RPO/RTO、数据规模、一致性窗口、安全和成本。

高级架构答案必须量化非功能需求。没有 QPS、数据量和 SLA 的“分库分表、加缓存、上 MQ”只是组件罗列。

5. 分布式题的通用失败清单是什么？

```
请求没到
请求到了但响应没回来
请求被执行多次
请求乱序到达
节点崩溃或暂停
网络分区或高延迟
主从切换与旧主恢复
时钟漂移
容量耗尽和级联过载
人工误操作与错误发布
```

每个方案至少回答：检测方式、业务影响、自动恢复、人工恢复和数据校验。

二、CAP、BASE、一致性模型、时间与故障语义

6. CAP 定理应该如何准确理解？

CAP 讨论的是发生网络分区时，系统无法同时保证线性一致性和所有请求可用：

- C：对外表现得像单副本，读到符合线性顺序的值。
- A：每个到达非故障节点的请求最终得到非错误响应。
- P：即使节点间消息丢失或延迟，系统仍继续处理。

现实分布式系统必须面对 P，因此真正的选择是分区期间对具体操作偏 C 还是偏 A，而不是永久给系统贴 CP/AP 标签。

7. BASE 与 ACID 是什么关系？

BASE 强调基本可用、软状态和最终一致，是跨服务大规模系统常见工程思想；ACID 描述事务原子性、一致性、隔离性和持久性。

它们不是非此即彼：单个服务内部可用 ACID 保证核心事实，服务间通过事件和补偿实现最终一致。

8. 强一致、线性一致、顺序一致和最终一致有什么区别？

- **线性一致**：每个操作像在调用与返回之间某一时刻原子发生，且尊重真实时间先后。
- **顺序一致**：所有进程看到相同全序，但不要求尊重真实时间。
- **因果一致**：有因果关系的操作顺序一致，无关操作可不同序。
- **最终一致**：停止新写后，各副本最终收敛，不规定收敛前读到什么。

面试中“强一致”必须具体化，不要把所有较强语义都混称强一致。

9. 读己之写、单调读和因果一致分别解决什么？

- **读己之写**：用户提交后刷新页面能看到自己的修改。
- **单调读**：同一会话不会先看到新值再看到旧值。
- **因果一致**：回复不会出现在原帖之前，状态 B 不会早于导致 B 的状态 A。

很多业务不需要全局线性一致，只需会话粘滞、版本令牌或因果 Session 即可降低成本。

10. 网络超时为什么不等于执行失败？

超时只说明调用方在期限内没有收到响应，可能是：请求未到、服务执行中、已成功但响应丢失、下游提交后进程崩溃。

因此重试前必须判断操作是否幂等；关键写操作要用业务请求号查询最终状态，不能看到 timeout 就直接再扣一次。

11. 系统时钟为什么不可靠？

机器时钟会漂移，NTP 校时可能跳变，跨节点时间戳不能天然表示因果顺序。墙上时钟适合展示和粗粒度过期，不适合作为唯一并发正确性依据。

版本控制可使用数据库序列、日志 Offset、Term/Index、Lamport Clock 或业务版本；Lease 还要考虑时钟漂移和暂停。

12. Lamport Clock 和 Vector Clock 解决什么？

- Lamport Clock 用逻辑计数建立“若 A 因果先于 B，则 $L(A) < L(B)$ ”的顺序，但不能判断并发。

- Vector Clock 记录各参与方版本，可识别先后或并发冲突，但元数据随参与方增加。

业务系统常用更简单的实体版本号、事件序号和单分区顺序来实现同类目标。

13. 什么是故障检测器的局限？

在异步网络中，无法仅凭一段时间没收到心跳区分“节点已死”和“节点很慢/网络分区”。超时阈值越短切换越快但误判越多，越长则故障恢复越慢。

因此高可用不是只调心跳，还需多数派、Term、Fencing、幂等和恢复校验防止误判造成双写。

三、身份、幂等、去重、顺序、超时与重试

14. 什么是幂等，为什么分布式系统必须做幂等？

幂等是同一业务操作执行一次或多次，最终业务效果相同。网络超时、客户端重试、MQ 至少一次投递、进程重启都会制造重复。

幂等不是“接口返回一样”，而是余额、库存、订单状态和外部副作用不被重复改变。

15. 幂等键如何设计？

优先使用调用方生成且在业务语义内唯一的 request_id/order_id/event_id：

- 数据库唯一索引阻止重复创建。
- 幂等表记录 key、请求摘要、状态和响应。
- 状态机条件更新限制合法迁移。
- 消费端 Inbox 记录已处理事件。

需要定义作用域、有效期、请求参数不一致时的处理和并发中的“处理中”语义。

16. 去重表和唯一索引如何选择？

- 唯一索引最简单可靠，适合结果直接落同一数据库。
- 去重表可记录处理状态和响应，适合复杂操作，但需和业务写在同一本地事务中。
- Redis `SET NX` 可做快速挡板，但过期、故障和双写使其不适合单独承担资金类最终正确性。

17. 如何处理请求结果未知？

为写请求提供查询状态接口，以业务 ID 查询：不存在才重试，成功则返回原结果，处理中则稍后查询，失败且可重试才重新执行。

服务端要持久化请求状态，并使“记录请求 + 修改业务数据”处于同一本地事务或原子边界。

18. 重试的正确条件是什么？

同时满足：错误是瞬时的、操作幂等、仍在总 deadline 内、下游有恢复可能、系统未过载。

策略使用有限次数、指数退避、随机抖动和 Retry Budget。参数错误、权限错误、业务冲突和资源长期不足不应重试。

19. 为什么重试会引发重试风暴？

下游变慢后上游超时重试，流量倍增使下游更慢；多层服务各自重试又产生乘法放大。

治理：只在一个合适层重试、限制尝试次数、共享 deadline、设置 Retry Budget、配合限流熔断，并让下游通过背压/ `Retry-After` 明确拒绝。

20. Timeout 和 Deadline 有什么区别？

Timeout 是允许本次操作执行多久，Deadline 是整条调用最晚结束时间。跨服务应传播 Deadline，并给网络、业务处理和返回预留预算。

下游发现剩余时间不足应尽早失败，服务端也必须响应 Context Cancellation，停止数据库查询和后台工作。

21. 消息顺序如何保证？

先定义顺序范围，通常只要求同一订单、设备或用户内有序，而非全局有序。将相同业务键路由到同一分区，由单消费序列处理；事件携带实体版本，消费端拒绝旧版本。

并发 Worker 必须按 key 串行或分片，不要在取到消息后无约束并行破坏顺序。

22. 全局唯一 ID 有哪些方案？

方案	优点	代价
数据库自增/号段	简单、趋势递增	中心依赖或号段管理
UUID	本地生成、低冲突	较大、索引局部性较差
Snowflake 类	趋势递增、吞吐高	时钟回拨、节点号管理
Redis/协调服务	全局递增	额外中心依赖

ID 方案要考虑唯一性、趋势、信息泄露、分库路由、长度、时钟回拨和故障降级。

四、Quorum、共识、Leader、Lease、锁与 Fencing

23. Quorum 读写的基本思想是什么？

在 N 个副本中，写等待 W 个、读查询 R 个。若 $W + R > N$ ，读写集合至少相交，但是否得到最新值还取决于版本选择、失败和实现细节。

Quorum 是副本交集思想，不自动等于线性一致；还需要处理并发写、旧主、读修复和成员变更。

24. 共识算法解决什么问题？

共识使多个节点即使发生故障和消息延迟，也对一系列值或日志顺序达成一致。它常用于选主、元数据、配置、锁服务和复制状态机。

它不直接解决业务跨系统事务，也不能在失去多数时同时保持写可用和一致。

25. Raft 的核心流程是什么？

- 节点处于 Follower、Candidate、Leader 状态。
- 选举通过递增 Term 和多数票产生 Leader。
- 客户端写入先进入 Leader Log，再复制到多数节点后提交。
- 已提交日志按顺序应用到状态机。
- 日志匹配和 Term 规则避免旧 Leader 覆盖新历史。

真正工程难点还包括快照、日志压缩、成员变更、磁盘持久化和慢节点。

26. 为什么多数派系统通常使用奇数节点？

三个节点可容忍一个故障，五个可容忍两个；四个节点仍需三个多数，只能容忍一个故障。增加偶数节点通常不增加故障容忍，却增加复制成本。

节点应跨独立故障域部署，否则物理上多个节点可能同时失效。

27. Leader Election 与业务主节点有什么风险？

网络分区或长 GC 可能使旧 Leader 不知道自己已失去领导权，继续操作外部资源，形成双主副作用。因此仅有“选主成功”不够，外部写操作需要 Term/Epoch 或 Fencing Token 拒绝旧 Leader。

28. Lease 与 Lock 有什么区别？

- Lock 表示互斥所有权。
- Lease 是带有效期、需要续约的临时所有权，持有方失联后能自动释放。

Lease 适合服务注册、Leader 和任务租约，但进程暂停、网络延迟和时钟漂移可能让客户端误以为仍持有。执行外部副作用仍应配合 Fencing。

29. 分布式锁的正确性要求是什么？

1. 互斥：同一资源同一时刻只有合法持有者。
2. 所有权：只有持有者能释放，value 使用唯一 token。
3. 有界存活：持有者崩溃后锁最终释放。
4. 续约：长任务在确认仍持有时延长 Lease。
5. Fencing：下游拒绝旧持有者。
6. 可观测：记录等待、持有、续约失败和超时。

30. Fencing Token 为什么重要？

锁服务每次授予锁时返回单调递增 token。资源服务只接受 token 大于已见 token 的请求。即使旧持有者因暂停后恢复并继续执行，也会被资源层拒绝。

没有资源端校验的 Fencing，锁过期后旧客户端仍可能与新客户端同时写。

31. Redis 锁、etcd/ZooKeeper 锁和数据库锁如何选择？

- Redis 锁性能高，适合可补偿、短临界区，需正确 token、过期和续约。
- etcd/ZooKeeper 基于共识与 Lease/临时节点，语义较强但吞吐和运维成本更高。
- 数据库条件更新/行锁最接近业务事实，适合低到中等并发核心数据。
- 能用唯一约束、原子更新、队列串行化解决时，优先避免分布式锁。

32. 一致性哈希解决什么问题？

普通取模在节点数变化时会重映射大量 Key。一致性哈希将节点和 Key 放在环上，节点变化只迁移相邻区间；虚拟节点改善分布和异构权重。

它常用于缓存、路由和分片，但仍需处理热点 Key、节点容量差异、复制和迁移期间双读双写。

五、分布式事务、最终一致性与补偿

33. 分布式事务有哪些主流方案？

方案	一致性/适用	主要代价
2PC/XA	强原子提交、资源支持 XA	阻塞、协调者和资源占用
TCC	高价值、动作可 Try/Confirm/Cancel	业务侵入、空回滚/悬挂/幂等
Saga	长事务、多步骤业务	中间状态可见、补偿不等于回滚
Outbox + MQ	跨服务事件最终一致	延迟、重复、乱序、对账

方案	一致性/适用	主要代价
本地消息表	基础设施简单	扫描、清理和吞吐治理

选择依据是不变量、一致性窗口、吞吐、参与方能力、补偿可行性和运维复杂度。

34. 2PC 的流程和问题是什么？

Prepare 阶段协调者询问参与者是否能提交，参与者预留资源并记录日志；Commit 阶段根据所有投票决定提交或回滚。

问题是锁和资源持有时间长、协调者故障、网络分区下阻塞、参与者吞吐下降。它适合同构资源和低延迟强原子场景，不适合任意微服务调用链。

35. TCC 要解决哪些异常？

- Try 预留资源，Confirm 真正确认，Cancel 释放资源。
- 三个动作都必须幂等。
- 空回滚：Try 未执行但 Cancel 到达，需识别并安全返回。
- 悬挂：Cancel 先到，迟到 Try 不得再次预留。
- Confirm/Cancel 失败要持续重试和人工兜底。

36. Saga 的补偿为什么不等于数据库回滚？

每一步本地事务提交后对外可见，失败时通过语义相反的补偿动作恢复业务，例如取消订单、退还额度。期间其他业务可能已经观察并依赖中间状态，补偿也可能失败。

因此需要状态机、幂等、可重试、人工介入和审计，而不能把 Saga 描述成自动 ACID。

37. Transactional Outbox 如何工作？

```

同一本地事务：
  更新业务表
  + 插入 outbox(event_id, aggregate_id, version, payload)

异步 Relay：
  扫描/CDC Outbox -> 发布 Kafka -> 标记或归档
  
```

它消除“DB 已提交但消息未产生”的窗口。消息仍可能重复，因此 Relay 和 Consumer 都要幂等，Outbox 还需分片、归档和积压监控。

38. 消费 MQ 更新数据库如何保证一致？

消费端在同一本地事务中完成：插入 Inbox/幂等记录、修改业务数据、记录待发下游事件。事务成功后才提交 Offset。

如果提交 Offset 失败会重复投递，但 Inbox 阻止重复副作用。业务 Exactly-once 来自幂等和状态约束，不来自口号。

39. 如何处理乱序事件？

- 按实体 ID 路由同一分区，尽量保证源端顺序。
- 事件携带实体 version/sequence。
- Consumer 只接受大于当前版本的事件。
- 有缺口时暂存、回查事实源或触发重建。
- 删除使用 Tombstone 或状态版本，防止旧更新复活已删除数据。

40. 对账和补偿系统为什么是最终一致性的组成部分？

重试只能处理已知失败，无法发现漏事件、人工改数、程序 Bug 和长期数据漂移。对账应比较业务主键、状态、金额/数量、版本和时间窗口，输出可追踪差异并按风险自动或人工修复。

没有可重建、可对账和可人工处置的最终一致性方案是不完整的。

六、缓存、热点、限流、熔断、降级与背压

41. Cache Aside 如何工作，主要不一致窗口是什么？

读：先查缓存，未命中查 DB 并回填。写：先更新 DB，再删除缓存。

不一致来自并发回填、删除失败、主从延迟和多级缓存。常用版本、短 TTL、MQ/CDC 失效、延迟双删和对账降低概率，但不能宣称绝对强一致。

42. 缓存穿透、击穿和雪崩如何区分？

- 穿透：查询不存在数据，持续打到后端；用参数校验、空值缓存、Bloom Filter。
- 击穿：单个热点 Key 失效，大量请求同时回源；用互斥重建、逻辑过期、请求合并。
- 雪崩：大量 Key 同时失效或缓存整体故障；用 TTL 抖动、多级缓存、限流降级和数据库保护。

43. 热点 Key 和热点资源如何治理？

- 识别 Top Key、租户、接口和分区。
- 读热点可本地缓存、复制、拆 Key 或预计算。
- 写热点可分桶、合并写、队列串行化或按业务维度拆分。
- 超级租户可独立资源池或隔离集群。
- 方案必须提供聚合与一致性机制，不能只“随机打散”。

44. 常见限流算法如何选择？

- 固定窗口简单但边界突刺。
- 滑动窗口更精确但状态成本更高。
- 漏桶输出平滑，适合保护固定速率下游。
- 令牌桶允许可控突发，最常用于 API。
- 并发限制直接控制在途请求，适合慢依赖。

限流维度包括全局、租户、用户、接口和资源；返回明确错误并支持降级，而非静默丢弃核心请求。

45. 本地限流和分布式限流如何选择？

本地限流延迟低、无中心依赖，但实例间只能近似公平；分布式限流可全局精确，但增加网络依赖和热点。

常用组合是入口网关做租户全局配额，服务实例做本地并发与突发保护，核心资源再做细粒度保护。

46. 熔断解决什么问题？

当下游错误率、慢调用或并发超过阈值时，熔断器快速失败，避免资源被持续占用和级联故障；半开阶段用少量探测判断恢复。

熔断必须按依赖/方法隔离，配合 deadline、限流、重试预算和降级结果。阈值要基于调用量窗口，低流量偶发错误不应误触发。

47. 降级应该如何设计？

- 核心写链路优先，关闭推荐、聚合和装饰信息。
- 搜索失败返回热门或基础列表。
- 非核心同步转异步或延迟处理。
- 只读缓存允许短暂陈旧并标识时间。
- 明确触发条件、恢复条件、数据补偿和产品体验。

降级不是返回空成功；调用方必须知道结果语义是否完整。

48. 背压为什么重要？

当下游处理速度低于上游产生速度时，背压让压力在可控边界显现，而不是无限堆积内存、连接和队列。实现包括有界队列、并发信号量、消费速率调节、429/503、暂停拉取和生产端限速。

每个队列都要有容量、等待时间、拒绝策略和监控。

七、高并发读写、秒杀、订单、Feed 与计数设计

49. 高并发系统的通用拆解方法是什么？

```
入口: CDN / WAF / 网关 / 限流
-> 无状态服务: 水平扩展 / 连接池 / deadline
-> 缓存: 热点 / 请求合并 / 降级
-> MQ: 削峰 / 异步 / 重放
-> 存储: 索引 / 分片 / 批量 / 幂等
-> 治理: 容量 / 监控 / 演练 / 预案
```

每层都要说明容量上限和失败后压力会流向哪里。

50. 秒杀系统如何设计？

1. CDN/本地缓存承载静态信息。
2. 网关按活动、用户、设备限流，验证码和资格校验挡流量。
3. Redis/Lua 原子校验资格、去重和预扣，快速返回排队中。
4. Kafka/RocketMQ 削峰，订单消费者幂等创建订单并落库。
5. 数据库条件更新或预留记录保证最终库存不超卖。
6. 失败释放预扣，超时订单关闭，对账修正 Redis 与 DB。

还要防刷、热点 Key、消息积压、重复下单和活动结束后的容量回收。

51. 库存扣减有哪些实现方案？

- DB 条件更新: `UPDATE ... SET stock=stock-n WHERE stock>=n`，正确简单，热点吞吐受限。
- Redis Lua 预扣: 吞吐高，需要持久化订单、补偿和对账。
- 分桶库存: 降低单点热点，需处理余量碎片和汇总。
- 队列串行化: 顺序清晰，延迟和单分区吞吐受限。

最终选择由峰值、正确性、实现成本和可补偿性决定。

52. 订单系统如何保证状态正确？

- 用明确状态机约束合法迁移。
- 更新条件包含当前状态和 version。
- 创建使用 order_id 唯一索引。
- 支付、库存、履约通过 Outbox/Saga 最终一致。
- 每次状态变化记录不可变流水。
- 超时关单与支付回调竞争通过条件更新裁决。

53. 高并发读系统如何扩展？

依次考虑：浏览器/CDN 缓存、服务本地缓存、Redis、数据库读副本、搜索读模型、预计算和分片。每增加一层缓存都要定义 Key、TTL、失效、回源、穿透和降级。

不要一开始就分库；先优化数据模型、索引、请求合并和热点。

54. 高并发写系统如何扩展？

- 缩短同步主链路，只保留必须立即确认的事实。
- 批量、合并写和异步事件降低随机 IO。
- 按业务键分区，保证局部顺序和水平扩展。
- 通过 Shard Key/分库键分散写入。
- 热点计数分桶并周期聚合。
- 所有写入具备幂等、重放和对账能力。

55. Feed 流如何设计？

- 写扩散：发布时把内容 ID 推入粉丝 Inbox，读快但大 V 写放大严重。
- 读扩散：读取时聚合关注者 Outbox，写轻但读延迟和计算高。
- 混合：普通用户写扩散，大 V 读扩散，读取时合并、去重和排序。

需要处理游标分页、删除/屏蔽、热点、时间线一致性和缓存。

56. 点赞、播放量等计数如何设计？

强准确且低并发可数据库原子更新；高频计数可 Redis 分桶或流式聚合，周期批量落库。用户是否点赞用唯一关系记录，展示总数允许短暂最终一致。

必须定义去重口径、迟到事件、回放、跨天边界和账实对账。

57. 短链接服务如何设计？

- 发号器生成唯一短码，映射持久化并缓存热点。
- 跳转服务无状态，先查本地/Redis，再查数据库。
- 访问日志异步进入 Kafka，不阻塞跳转。
- 防止恶意 URL、枚举扫描和热点链接攻击。
- 过期与删除要同步缓存，分析数据单独存储。

八、性能、容量、弹性、容灾与成本

58. 容量评估的基本公式是什么？

```
峰值 QPS = 日均请求量 x 峰谷系数 / 有效秒数
服务实例数 = 峰值 QPS / 单实例安全 QPS x 冗余系数
存储 = 日增量 x 保留天数 x 副本/索引系数 + 临时空间
带宽 = 峰值 QPS x 平均请求/响应字节
积压恢复时间 = backlog / (恢复消费速率 - 新增速率)
```

所有单机能力必须来自接近生产的压测，并在 N-1、发布和依赖变慢场景下验证。

59. 为什么只看平均值会误判容量？

平均 QPS 掩盖峰值，平均延迟掩盖长尾，平均文档大小掩盖大对象，平均租户掩盖超级租户。容量至少看 P95/P99、最大值、时间分布、Key 分布和故障期间容量。

60. Little's Law 如何用于并发估算？

稳态下：

```
在途请求数 L = 到达率 λ x 平均响应时间 W
```

若 1000 QPS、平均耗时 0.1 秒，平均约 100 个在途请求。下游变慢到 1 秒，在途变成约 1000，连接、Goroutine 和内存随之膨胀，因此 deadline、并发限制和背压是容量设计的一部分。

61. 弹性扩容为什么不是瞬时解法？

实例启动、镜像拉取、预热、服务发现、连接池建立、JIT/缓存填充都需要时间。若告警触发时系统已雪崩，扩容可能来不及，还可能给数据库制造连接风暴。

需要基础冗余、提前预测、预热、最大扩容速率和下游容量联动。

62. 多可用区和异地多活如何选择？

- 同城多 AZ 通常延迟低，适合多数派和自动故障切换。
- 异地灾备重点是备份、异步复制和受控切换。
- 异地多活要解决流量路由、数据归属、冲突、一致性和回切，成本最高。

先定义 RPO/RTO 和业务是否允许降级，再决定架构，不要为“多活”而多活。

63. RPO 和 RTO 分别是什么？

- RPO：最多允许丢失多长时间的数据。
- RTO：故障后允许多长时间恢复服务。

它们直接决定同步/异步复制、备份频率、跨地域方案、演练投入和成本。口头说“高可用”却没有 RPO/RTO 不完整。

64. 如何控制架构成本？

按业务等级分配可用性、缓存、副本和灾备；冷热分层、数据归档、自动缩容和容量归属；评估中间件的机器、网络、存储和人力成本。

成熟架构不是组件最多，而是在 SLA 内以可理解、可运维的方式实现目标。

九、可观测性、稳定性治理与故障排查

65. 分布式系统如何建立可观测性？

- Metrics 看趋势和告警：RED (Rate、Errors、Duration) 与 USE (Utilization、Saturation、Errors) 。
- Logs 提供结构化事件和业务上下文。
- Traces 串联跨服务关键路径。
- Events 记录发布、扩容、配置和故障切换。
- Business invariants 监控库存、金额、状态和数据差异。

三大信号必须通过 trace_id、request_id、order_id 等关联。

66. 线上故障如何快速分层？

```
是单接口、单租户、单机房还是全局？
-> 流量是否变化？
-> 应用 CPU/内存/Goroutine/GC 是否异常？
-> 连接池、线程池、队列是否饱和？
-> DB/Redis/MQ/ES 是否变慢？
-> 网络、DNS、证书和服务发现是否异常？
-> 最近是否发布、改配置、迁移或重放？
```

先保存证据和止血，再追根因；不要在影响范围未知时直接重启全部实例。

67. 如何识别级联故障？

典型信号是下游延迟上升、上游连接和 Goroutine 增长、超时重试放大、线程池/队列饱和，最终多个服务同时不可用。

止血要切断放大链：关闭重试、限流、熔断慢依赖、降级非核心功能、释放卡住资源，并按依赖顺序恢复。

68. 如何设计稳定性防线？

1. 设计阶段：SLO、容量、故障模型和降级语义。
2. 开发阶段：deadline、幂等、有界队列和资源隔离。
3. 发布阶段：灰度、自动观测、回滚和配置审计。
4. 运行阶段：监控、值班、预案、容量巡检。
5. 演练阶段：故障注入、备份恢复和数据对账。
6. 复盘阶段：修系统机制，不要求个人小心。

69. 混沌工程应该如何开展？

从明确稳态指标和小故障域开始，注入延迟、超时、实例退出、网络丢包、只读、磁盘不足和消息重复。必须有停止条件、回滚、观察和业务批准。

目的不是制造事故，而是验证假设：限流是否生效、重试是否放大、主从是否切换、数据是否可校验。

70. 故障复盘最重要的输出是什么？

除时间线和根因外，还要说明：为什么故障被允许发生、为什么影响被放大、为什么监控没提前发现、为什么恢复这么慢。改进项必须可验证，有负责人、期限和验收指标。

十、业务架构与项目实战

71. 广告投放任务如何设计分布式执行？

- MySQL/MongoDB 保存任务事实和状态机。
- 调度器使用租约领取分片，任务更新用 version 条件防双执行。
- Kafka 按账户或任务 Key 分区，Worker 幂等执行并记录 event_id。
- 外部媒体 API 使用 deadline、限流、退避和请求号查询结果。
- 状态变化通过 Outbox 传播，失败进入重试/人工处理。
- 指标包括任务延迟、重复率、外部 API 成功率、积压和状态差异。

72. 短剧发布链路如何保证最终一致？

发布涉及元数据、媒资、审核、搜索索引和 CDN。核心状态在主库按状态机推进；每一步本地提交后发事件，消费者按内容 ID 和版本幂等处理。搜索和 CDN 允许短暂延迟，失败持续补偿；只有必需步骤完成才将版本标记为可见。

下线事件使用更高版本，防止迟到发布事件让内容重新上线。

73. 设备事件高并发接入如何设计？

1. 网关鉴权、限流和请求大小限制。
2. 事件带 device_id、sequence、event_id 和发生时间。
3. Kafka 按 device_id 分区，保证设备内顺序。
4. 消费端批量写存储，event_id 唯一去重。
5. 迟到和缺序事件按业务窗口处理，设备最新状态用版本条件更新。
6. 冷数据归档，查询和统计使用不同读模型。

74. 如何设计一个可靠的计费/额度系统？

账本使用不可变流水和唯一业务单号，余额是流水聚合或事务内维护的快照。扣减使用数据库事务、行锁/条件更新保证不为负；外部请求超时按业务单号查询，不直接重扣。跨服务通过 Outbox 传播，定期做流水、余额和外部账单三方对账。

资金类系统优先正确和可审计，不能把 Redis 分布式锁作为唯一真相。

75. 如何表达你的 1 年多管理经验但不应聘管理岗？

将其表达为高级 IC 的工程优势：

- 能推动接口契约、容量评审、故障复盘和跨团队协作。
- 能把复杂项目拆成可交付阶段，识别风险并推动闭环。
- 理解排期、人员和业务优先级，但职业选择是继续深耕业务架构与核心研发。
- 项目回答以自己承担的设计、编码、排障和结果为主，管理经验只作为协同补充。

76. 如何用两分钟回答分布式架构题？

我会先量化业务 QPS、P99、可用性、RPO/RTO 和一致性要求，再定义库存、金额、状态等不可破坏的不变量。因为网络超时不代表失败、消息会重复乱序、节点和时钟会异常，所以写链路首先用业务 ID、唯一约束、版本和状态机保证幂等；单服务内用本地事务，跨服务根据业务选择 Outbox、Saga 或 TCC，并提供重试、对账和人工补偿。高并发上通过缓存、请求合并、MQ 削峰、批量写、分区和热点拆分扩展，但每层都配置 deadline、并发上限、背压、熔断和降级。涉及 Leader、Lease 或锁时还要用 Term/Fencing 防旧持有者。最后用真实压测规划 N-1 容量，通过指标、日志、Trace、业务对账、灰度发布和故障演练验证，而不是只描述理想流程。

05-01 | Linux操作系统与系统排障

技术 | Linux 操作系统与系统排障面试题

面向 9 年以上 Golang Web 后端业务研发岗位。本篇回答应用之下的系统问题：进程如何运行、CPU 为什么排队、内存去了哪里、IO 为什么变慢、容器为何被限制，以及怎样用证据完成生产排障。

一、Linux 学习地图与系统排障框架

1. Linux 后端知识图谱是什么？

```
Process / Thread / Syscall
-> Scheduler / CPU / Load / Context Switch
-> Virtual Memory / Page / Cache / OOM
-> VFS / Filesystem / Block IO
-> FD / Pipe / Signal / IPC
-> Namespace / Cgroup / Container
-> Metrics / Commands / Evidence
-> Incident Response / Capacity / Recovery
```

2. 资深后端如何回答系统排障题？

采用“业务现象 -> 资源饱和 -> 进程证据 -> 内核机制 -> 止血 -> 根治”。先看成功率、P99、QPS 和影响范围，再判断 CPU、内存、IO、网络或依赖，不从背命令开始。

3. Linux 排障的标准六步是什么？

1. 确认时间、实例、主机、容器和业务影响。
2. 保存发布、配置、日志、指标、进程和内核证据。
3. 限流、隔离、回滚或扩容控制影响。
4. 从系统全局到进程、线程、调用栈逐层收敛。
5. 修复后渐进恢复并校验业务。
6. 补容量、告警、运行手册和演练。

4. USE 方法如何使用？

对每类资源看 Utilization、Saturation、Errors：CPU 看使用和运行队列，内存看使用/OOM/回收压力，磁盘看利用率/队列/错误，网络看带宽/丢包/重传。

利用率不高也可能饱和，例如单核热点、磁盘队列和连接池等待。

5. 常用第一现场命令是什么？

```
uptime
top -H -p <pid>
vmstat 1
mpstat -P ALL 1
pidstat -p <pid> 1
iostat -xz 1
free -h
ss -s
dmesg -T | tail
```

命令输出必须与同一时间窗口的业务指标、版本和容器限制关联。

二、进程、线程、系统调用、信号与生命周期

6. 进程和线程有什么区别？

进程拥有独立虚拟地址空间和资源表；同进程线程共享地址空间、文件描述符等，但各有寄存器、栈和调度实体。Linux 内核用 Task 表示可调度实体，创建时通过 Clone Flags 决定共享范围。

7. 用户态和内核态如何切换？

系统调用、中断和异常使 CPU 进入内核态。切换本身及数据复制、调度会产生成本。高频小 IO、日志和锁竞争可能表现为 System CPU 高。

8. fork、exec 和 Copy-on-Write 如何工作？

Fork 创建子进程并共享物理页，页表标记只读；任一方写入时复制页面。Exec 用新程序映像替换当前进程。大内存进程 Fork 虽不立即复制全部数据，但页表、COW 写放大和内存峰值仍需评估。

9. Zombie 和 Orphan Process 是什么？

子进程退出后父进程未 Wait，保留退出状态成为 Zombie；父进程先退出，子进程由系统指定的 Reaper 接管。大量 Zombie 消耗 PID 表项，说明父进程生命周期处理错误。

10. Linux Signal 如何使用？

Signal 是异步通知。SIGTERM 用于可处理终止，SIGKILL 无法捕获，SIGHUP 常用于重载，SIGCHLD 通知子进程变化。Handler 中只能执行安全操作，应用通常转为设置状态或唤醒主循环。

11. 为什么 SIGKILL 不适合正常发布？

进程没有机会停止接流量、完成请求、提交 Offset 和 Flush 数据。正常流程先 SIGTERM 与宽限期，超时才 SIGKILL；业务写仍需幂等抵抗非正常退出。

12. System Call 慢如何定位？

用 `strace -ttT -p` 受控观察调用耗时和错误，用 `pidstat -w` 看切换，结合应用 Profile。Strace 有开销，生产短时采集；高频 Futex、read/write、fsync、connect 分别指向不同问题。

13. 文件描述符是什么？

进程 FD Table 的整数索引指向内核 Open File Description，再关联文件、Socket、Pipe 等。Dup 后多个 FD 可共享 Offset/状态；Fork 也会继承。泄漏会导致 `EMFILE`。

三、CPU 调度、负载、上下文切换与性能

14. Load Average 表示什么？

近 1/5/15 分钟可运行和不可中断等待任务的平均数量，不等于 CPU 使用率。Load 高可能是 CPU Run Queue，也可能是大量 D 状态 IO 等待。

与 CPU 核数、`vmstat r/b`、`mpstat` 和任务状态一起判断。

15. User、System、lowait、Steal 分别说明什么？

- user：用户态执行。
- system：内核态工作。
- iowait：CPU 空闲但有 IO 等待，解释需谨慎。
- steal：虚拟化环境被 Hypervisor 占用。

还需看 IRQ/SoftIRQ、Guest 和 Cgroup Throttling。

16. CPU 100% 如何分层排查？

确认全核还是单核，找进程与线程，区分 User/System/SoftIRQ/Steal，再用语言 Profile 或 Perf 定位调用栈；对照流量、发布和后台任务。先限流/回滚，扩容前检查下游。

17. CPU 不高但接口很慢有哪些原因？

锁/Channel/连接池排队、下游 IO、磁盘、DNS、CPU Throttling、单线程热点、Goroutine/线程睡眠。低使用率不代表没有饱和，必须看 Queue 和等待时间。

18. Context Switch 为什么昂贵？

切换需保存状态、调度、可能破坏 Cache/TLB 局部性。自愿切换多通常是阻塞等待，非自愿切换多可能是时间片或竞争。大量线程、锁和小任务会放大。

19. 进程状态 R、S、D、T、Z 如何理解？

R 可运行，S 可中断睡眠，D 不可中断等待常见于 IO，T 停止/跟踪，Z 僵尸。大量 D 状态结合磁盘/NFS 指标排查，不能 Kill 立即解决正在内核等待的任务。

20. CPU Affinity 和 NUMA 何时重要？

高吞吐低延迟、跨 NUMA 内存访问和中断不均时重要。普通 Web 服务优先让调度器工作；绑核前用 Perf/NUMA 指标证明，容器 CPU Set 也要一致。

21. Perf 能解决什么问题？

采样 User/Kernel 调用栈、硬件事件、Cache Miss、分支等。使用 `perf top/record/report` 定位语言 Profile 看不到的 Syscall、内核和 Native 热点；符号、权限和采样开销需准备。

22. Run Queue 长如何治理？

降低入口和并发、关闭非核心 CPU 工作、扩 CPU/实例、消除热点锁和忙循环；检查 Cgroup Quota 和单核倾斜。恢复时渐进放量，避免积压瞬间重新排队。

四、虚拟内存、页、回收、Page Cache 与 OOM

23. 虚拟内存解决什么问题？

每进程看到独立连续地址空间，由页表映射物理页或文件；提供隔离、按需分配、共享和换页。VSS 大不等于实际物理占用，RSS/PSS 更接近实际驻留。

24. Page Fault 是什么？

访问页表未建立当前映射时触发。Minor Fault 不需磁盘，可能分配零页或建立已有页映射；Major Fault 需从存储读取，延迟高。高 Fault 要结合 mmap、启动预热和内存压力判断。

25. RSS、PSS、VSS 和 Working Set 有什么区别？

VSS 是虚拟地址范围；RSS 是驻留物理页，重复计共享页；PSS 按共享比例分摊；容器 Working Set 常近似当前不可轻易回收部分。不同工具口径不同，排障先确认定义。

26. Page Cache 为什么既好又会“吃内存”？

文件读写使用空闲内存缓存页面，提升 IO；内存压力时可回收。因此 `free` 低不等于不足，要看 available、回收、Swap、Fault 和应用延迟。

27. Anonymous Memory 和 File-backed Memory 有什么区别？

堆、栈等匿名页无文件副本，压力下写 Swap 或 OOM；文件映射页可从原文件重读，脏页需回写。Go Heap 主要是匿名内存，mmap/共享库是文件映射或特殊映射。

28. Swap 的收益和风险是什么？

Swap 可避免短时 OOM、容纳冷匿名页，但大量换入换出会产生抖动。数据库/低延迟服务通常严格控制；不能简单把 Swap=0 当绝对规则，要结合工作负载和容器策略。

29. 内核如何回收内存？

在 Watermark 压力下回收 Page Cache、写回脏页、交换匿名页；Direct Reclaim 会让申请线程参与，影响延迟。观察 pgscan/pgsteal、kswapd、Major Fault 和 PSI Memory。

30. OOM Killer 如何选择进程？

内核根据内存占用、`oom_score_adj` 等选择牺牲者；Cgroup 内存超限可在组内 OOM。查看 Dmesg、Cgroup Event、退出码和容器事件，应用往往没有机会日志。

31. Linux 内存高如何排查？

先主机/容器总量，再按进程 RSS/PSS，拆匿名、文件缓存、Slab、共享内存和 Huge Page；结合应用 Heap、线程栈、mmap/CGO。止血前保留映射和 Profile。

32. 内存碎片是什么？

物理页虽有空闲但难以找到所需连续块，或应用分配器 Size Class 内部浪费。看 Buddy Info、Compact、THP 与应用 Heap；扩内存不是唯一方案，需修对象尺寸和生命周期。

33. Transparent Huge Pages 有什么影响？

大页减少 TLB Miss，但分配/合并和回收可能带来延迟抖动。数据库等工作负载常有明确建议；生产按组件和实测配置，不全局照抄。

五、VFS、文件系统、磁盘与 Block IO

34. 一次文件读取经过哪些层？

应用 read -> VFS -> 文件系统 -> Page Cache；命中直接复制，未命中提交 Block IO 到设备，完成后唤醒任务。Direct IO 会绕过部分 Page Cache，但有对齐和复杂度。

35. Inode 和 Dentry 分别是什么？

Inode 保存文件元数据和数据块映射，不含文件名；Dentry 表示路径组件到 Inode 的关联并缓存路径解析。FD 指向打开文件状态。磁盘空间有但无法建文件可能是 Inode 耗尽。

36. df 和 du 为什么可能不一致？

已删除但仍被进程打开的文件仍占空间，du 路径不可见而 df 统计；还可能有挂载点、稀疏文件、Reserved Block。用 `ls -lsof +L1` 查 Deleted Open File。

37. Buffering、Flush、fsync 有什么区别？

应用 Buffer、语言 Runtime、Page Cache 和设备 Cache 是不同层。Flush 常只把用户缓冲写入内核；fsync 请求文件数据/元数据达到持久化边界，实际保证还取决于文件系统和硬件。

38. 顺序 IO 和随机 IO 为什么差异大？

顺序访问利于预读、合并和设备吞吐；随机 IO 增加寻址、队列和放大，SSD 虽改善仍有 IOPS/写放大限制。数据库/MQ 通过顺序日志和批量降低成本。

39. iostat 重点看什么？

设备吞吐、IOPS、平均等待、队列和利用率。不同版本字段定义有差异；高 `%util` 在并行设备不必然饱和，应结合 await、queue、应用延迟和设备能力。

40. 磁盘 IO 高如何排查？

确认设备和挂载，找进程/线程，区分读写、吞吐/IOPS、同步写和后台回写；对照日志、备份、Compaction、数据库和发布。先暂停非核心批任务，保护数据库和日志。

41. IO Wait 高一定是磁盘慢吗？

不一定，可能 NFS、Block Device、存储网络或任务结构；lowait 统计本身受 CPU 是否有其他工作影响。用 D 状态、设备指标、文件系统和远端存储证据确认。

42. 日志为什么可能拖垮服务？

同步格式化/写盘、日志暴增、磁盘满、采集 Agent 阻塞和大量小写。使用级别/采样/异步有界队列、轮转和磁盘配额；错误日志也不能无限重试打印。

43. 磁盘满的正确处理顺序是什么？

确认占用和增长源，停止非核心写与日志风暴，扩容或安全清理可再生数据；检查 Deleted Open File；恢复后验证数据库和文件系统。禁止未经确认直接删除数据目录。

六、FD、Pipe、Socket、IPC、epoll 与资源边界

44. FD 耗尽有什么表现？

`too many open files`、无法 Accept/打开日志/连接依赖。查进程 FD Limit、`/proc/<pid>/fd` 类型和增长；常见原因是 Body、Rows、文件和 Socket 未关闭，或连接无上限。

45. Soft/Hard Limit 如何理解？

Soft 是进程当前限制，可在权限范围调整；Hard 是上限。Systemd、容器和 Shell 启动链可能有不同 Limit，不能只看交互式 `ulimit`。

46. Pipe 的语义是什么？

内核有界字节缓冲，读空阻塞、写满阻塞；所有写端关闭后读到 EOF。Shell Pipeline、子进程 Stdout 和应用通信若不消费会互相阻塞。

47. Unix Domain Socket 与 TCP Socket 如何选择？

同机 IPC 可用 Unix Socket，省网络协议部分开销并支持文件权限；跨机必须网络 Socket。容器/代理架构还要考虑挂载、权限和故障边界。

48. epoll 的核心模型是什么？

将感兴趣 FD 注册到内核集合，就绪时返回事件，避免每轮线性扫描全部 FD。它通知“可进行 IO”，应用仍需非阻塞读写到 EAGAIN 并维护连接状态。

49. Level Trigger 与 Edge Trigger 有什么区别？

LT 只要仍就绪就持续通知，编程简单；ET 状态边缘变化通知，必须一次读/写到 EAGAIN，否则可能漏后续处理。不能只说 ET 一定更快，复杂度和事件量要实测。

50. epoll 惊群是什么？

多个等待者被同一事件唤醒但只有少数获得工作，造成无效调度。内核和服务端使用独占唤醒、Accept 分配和 SO_REUSEPORT 等机制缓解；具体实现结合版本。

51. PSI 解决什么问题？

Pressure Stall Information 统计任务因 CPU、内存或 IO 资源不足而停顿的时间，比单纯使用率更直接体现资源压力。可用于过载告警和扩缩容，但需结合业务 SLO。

七、Namespace、Cgroup、容器与 Kubernetes 系统边界

52. Container 与 VM 的核心区别是什么？

容器共享宿主机 Kernel，通过 Namespace 隔离视图、Cgroup 限制资源、Filesystem Layer 提供环境；VM 有独立 Guest Kernel。容器更轻但内核故障域共享。

53. 常见 Namespace 有哪些？

PID、Network、Mount、UTS、IPC、User、Cgroup、Time 等，分别隔离进程号、网络、挂载、主机名、IPC、身份等。隔离不是安全全部，还需 Capability、Seccomp、LSM 和网络策略。

54. Cgroup CPU Limit 如何工作？

Quota/Period 限制时间窗口 CPU，达到后 Throttle；CPU Request/Weight 影响竞争份额。应用平均 CPU 未达但 Throttling 高时 P99 仍会抖动。

55. Cgroup Memory Limit 与 OOM 如何工作？

统计组内匿名、Page Cache 等口径，达到限制且无法回收时触发 Cgroup OOM。看 `memory.events`、current、stat 和 PSI；Go GOMEMLIMIT 要低于容器 Limit 并留非 Go 余量。

56. Pod 内看到的 CPU/内存为什么和宿主机不同？

Namespace/Cgroup 视图、指标口径和工具版本不同；`free/top` 未必按限制展示。排障同时看容器 Cgroup、Pod 指标和 Node 压力。

57. Kubernetes Pod 被 Evict 与 OOMKilled 有什么区别？

OOMKilled 常是容器/节点内核杀进程；Eviction 是 Kubelet 因节点内存、磁盘、PID 等压力驱逐 Pod。事件、退出码和 Node Condition 可区分，治理路径不同。

58. CPU Request 和 Limit 如何影响服务？

Request影响调度和竞争权重，Limit 可能 Throttle。低延迟服务需压测是否使用 Limit、如何设置 GOMAXPROCS 和副本；不能只靠提高 Limit掩盖低效或下游瓶颈。

59. 容器磁盘问题有哪些？

Writable Layer、EmptyDir、日志和镜像占用可能触发 Ephemeral Storage Eviction；OverlayFS 小文件/写放大也影响。大数据使用持久卷/对象存储并监控 Inode 与空间。

八、系统观测、命令选择与证据保存

60. /proc 能提供什么？

进程状态、Cmdline、FD、Maps、Smaps、Limits、IO、Net 和系统级内存/压力。它是许多命令的数据源；容器中需确认 PID Namespace 和挂载权限。

61. top、pidstat、vmstat 如何分工？

Top 快速全局/线程定位；pidstat 连续观察进程 CPU、IO、切换、Fault；vmstat 看运行队列、内存、Swap、IO 和 CPU 全局趋势。不要只截一个瞬时值。

62. sar 的价值是什么？

保留历史 CPU、内存、IO、网络趋势，适合事故发生后回看；前提是采集已启用、采样频率合适。云/容器环境还需平台历史指标。

63. eBPF 适合解决什么？

低侵入观测 Syscall、调度、网络、Block IO 和内核事件，可按 PID/Cgroup 聚合。工具强大但需 Kernel/BTF/权限和开销控制；先明确问题，不因流行直接上生产探针。

64. 如何安全保存故障证据？

记录时间、Host/Pod/Container、PID、Version、Cgroup Limit；保存命令完整输出、Profile、Trace 和 Dmesg；避免只截 Top 截图。敏感数据受控存储，采集动作和开销记录。

65. 运行手册中的命令应怎样写？

写目标、命令、预期字段、危险性、权限和回滚；禁止只有一串无解释命令。定期演练，避免工具未安装或容器权限不足。

九、生产故障分支、止血、恢复与复盘

66. 接口大量超时如何分层？

入口流量/LB -> 应用队列/CPU/GC -> 连接池 -> DB/缓存/RPC -> 主机 CPU/内存/IO/网络 -> 最近变更。先 Trace 定位慢层，再选系统工具，避免重复调查。

67. 主机 Load 很高但 CPU 空闲怎么办？

看 `vmstat b`、D 状态和 Wchan，检查磁盘/NFS/块设备；`lsof`、`Mount` 和 `Dmesg` 确认证据。暂停重 IO 作业并保护写链路；重启不一定能中断内核等待。

68. 内存压力时如何止血？

限制入口和批任务、回滚泄漏版本、缩小缓存/并发、扩容或迁移；保留 Heap/Smaps/Dmesg。避免手工 Drop Cache 作为常规修复，它可能制造 IO 风暴。

69. 磁盘延迟突增如何止血？

暂停备份/Compaction/批量重放等非核心任务，降低日志，限写入并保护数据库；确认设备故障和云盘限额。恢复按优先级渐进，检查数据一致性和 fsync 错误。

70. 系统故障复盘应回答什么？

为什么资源达到上限、为何未提前告警、哪一步放大、为何恢复慢。改进落到容量阈值、Cgroup、应用限流、日志配额、发布门禁、备份和演练，而非只写“增加机器”。

十、业务场景与项目实战

71. Go 服务发布后 P99 抖动如何结合 Linux 排查？

按版本比较业务和 Runtime 指标，再看 Cgroup Throttling、Run Queue、Context Switch、Fault 和 IO；Profile 定位代码。若新版本分配增加导致 GC CPU 和 CPU Throttle，先回滚，再减少分配/调整资源并灰度验证。

72. 广告批量任务导致整机异常如何处理？

任务无限并发使 CPU、FD 和数据库连接排队。入口暂停任务，保留 Profile/系统指标；改为 Cgroup 独立资源、有界 Worker、连接预算和任务队列；监控队列年龄、拒绝、CPU PSI 和 FD。

73. 日志打满磁盘如何表达？

先停止日志风暴和非核心写，扩容/安全清理并处理 Deleted Open File；根因可能重试循环每次打印大响应。修复为错误聚合/采样、有界异步日志、轮转和磁盘水位告警，验证故障时日志不会再放大。

74. 容器 OOM 但 Go Heap 不高如何回答？

同时检查 RSS、线程栈、CGO/mmap、Page Cache 和 Sidecar；Cgroup Event确认 OOM；Smaps 和进程映射找非 Heap 占用。GOMEMLIMIT 仅控制 Runtime口径，需为非 Go 内存留余量。

75. 如何用两分钟回答 Linux 排障？

我先从业务成功率、P99、QPS 和时间线确定影响，再按 USE 判断 CPU、内存、磁盘和网络是否饱和。CPU 看核分布、Run Queue、User/System/Steal/Throttle，再到进程线程和 Profile；内存区分 RSS、匿名页、Page Cache、回收、Cgroup 与 OOM；IO 从应用、VFS、Page Cache、文件系统到块设备看队列和延迟；资源泄漏看 FD、进程状态和 /proc。止血时先控制入口、暂停非核心任务或回滚，避免重启丢证据、扩容打爆下游。恢复后渐进放量并校验数据，复盘把问题落到容量、告警、Cgroup、发布门禁和运行手册。

05-02 | 计算机网络与HTTP工程

技术 | 计算机网络与 HTTP 工程面试题

面向 9 年以上 Golang Web 后端业务研发岗位。本篇从报文如何跨主机到 HTTP/RPC 如何稳定调用，串联 IP、TCP、UDP/QUIC、Socket、DNS、TLS、代理、负载均衡、连接池与生产故障。

一、网络学习地图与资深回答框架

1. 后端网络知识图谱是什么？

```
Link / ARP
-> IP / Route / ICMP / MTU
-> TCP / UDP / QUIC
-> Socket / Nonblocking / epoll
-> DNS / TLS
-> HTTP/1.1 / HTTP/2 / HTTP/3 / gRPC
-> Proxy / LB / NAT / Connection Pool
-> Security / Diagnostics / Incidents
```

2. 资深后端如何回答网络题？

采用“所在层 -> 报文/状态流程 -> 可靠性机制 -> 性能代价 -> 应用配置 -> 线上证据”。例如 TIME_WAIT 不只讲定义，还要讲为何产生、谁主动关闭、连接复用和端口耗尽如何验证。

3. 网络故障为什么要分层？

DNS、路由、握手、TLS、协议、应用和下游都可能表现为 timeout。分层能选择正确证据：解析耗时、SYN 重传、TLS Alert、HTTP Status、应用 Trace。

4. 延迟由哪些部分组成？

排队、传播、传输、处理，以及 DNS、TCP/TLS 握手、代理和应用依赖。小请求常由 RTT/排队主导，大响应受带宽和拥塞主导。

5. 网络排障第一现场看什么？

错误类型、目标 Host/IP/Port、协议、DNS、连接建立、TLS、TTFB、重传、丢包、连接状态和代理日志；保留客户端与服务端同一 Trace/时间线。

二、链路层、IP、ARP、路由、ICMP 与 MTU

6. OSI/TCP-IP 分层的工程价值是什么？

分层提供职责和故障定位：链路传 Frame，IP 尽力交付 Packet，TCP 提供可靠字节流，HTTP 定义应用消息。真实实现可跨层优化，但排障仍需明确问题在哪层。

7. 同网段通信如何经过 ARP？

主机通过路由判断目标在本地链路，查询 ARP/邻居缓存得到目标 MAC；未知时广播请求，收到应答后封装 Ethernet Frame。跨网段则解析下一跳网关 MAC，而非远端 MAC。

8. IP 协议提供什么保证？

无连接、尽力交付，不保证到达、顺序和不重复。TTL 防路由环，Checksum/上层校验检测错误，可靠性由 TCP 或应用承担。

9. 路由表如何选择路径？

按最长前缀匹配选目标网段，再结合优先级/Metric 和策略路由确定接口与下一跳。多网卡、容器网段和 VPN 问题常来自路由/Source IP 不一致。

10. ICMP 有什么作用？

传递不可达、超时、Fragmentation Needed 等控制信息；Ping 使用 Echo。防火墙完全丢弃 ICMP 可能破坏 Path MTU Discovery。Ping 通不代表 TCP/HTTP 一定通，反之亦然。

11. MTU 和 MSS 有什么区别？

MTU 是链路层最大 IP Packet 尺寸，MSS 是 TCP Payload 上限，通常从 MTU 减 IP/TCP Header。隧道增加头部会降低有效 MTU。

12. IP 分片和 PMTUD 如何工作？

IPv4 可能由路由器分片，任一 Fragment 丢失导致整个包失败；IPv6 路由器不分片。PMTUD 通过 ICMP 发现路径 MTU。ICMP 被丢可能形成 MTU Black Hole：小包通、大包卡。

13. NAT 解决什么并带来什么问题？

转换地址/端口以共享公网或连接网络，维护状态表。高并发可能耗尽 SNAT Port，Idle Timeout 关闭长连接，源 IP 在代理后丢失。需监控 NAT 表、端口和代理 Header。

14. Anycast、ECMP 和多路径有什么影响？

Anycast 将同一 IP 从多地宣告，ECMP 在等价路径散流；路径可能随哈希/路由变化，导致延迟和连接迁移差异。长连接通常保持五元组路径，QUIC Connection ID 更利于迁移。

三、TCP 连接、可靠传输、流控与拥塞控制

15. TCP 三次握手为什么是三次？

双方交换并确认初始序列号与收发能力：SYN、SYN-ACK、ACK。两次无法让服务端确认客户端已收到其序列信息，也难抵抗旧重复连接请求。

16. 四次挥手为什么通常是四次？

TCP 全双工，两方向独立关闭。收到 FIN 只表示对端不再发送，本端可能仍有数据，先 ACK，处理完再 FIN；某些情况下 ACK 与 FIN 可合并。

17. TIME_WAIT 为什么存在？

主动关闭方等待足够时间，确保最后 ACK 可重传，并让旧连接延迟 Segment 消失。大量 TIME_WAIT 常见于短连接；优先连接复用和合理架构，不粗暴修改内核语义。

18. CLOSE_WAIT 多说明什么？

对端已发 FIN，本端 Socket 尚未 Close，通常应用未关闭 Response Body/连接或处理卡住。定位持有 FD 的进程和调用栈，调内核超时不能根治应用泄漏。

19. TCP 如何保证可靠有序？

Sequence、ACK、Checksum、重传、接收缓存和乱序重组。可靠不等于业务请求只执行一次：连接断开和应用重试仍会重复副作用。

20. 超时重传和快速重传有什么区别？

RTO 到期重传，依据 RTT 动态估计；收到多个 Duplicate ACK 可触发快速重传，较早判断中间 Segment 丢失。现代算法还有 SACK 等改进，具体由系统实现。

21. 滑动窗口和流量控制是什么？

Receiver 通告可用窗口，Sender 控制未确认数据，避免压垮接收缓存。Zero Window 时发送方探测；应用不读取会让窗口变小并反向产生背压。

22. 拥塞控制解决什么？

通过 Congestion Window 控制网络在途数据，基于丢包/延迟等信号调整，保护共享网络。实际发送上限受 `min(cwnd, rwnd)` 等约束。算法如 CUBIC/BBR 取舍不同。

23. TCP 粘包是什么？

TCP 是字节流，不保留消息边界。一次 Send 不对应一次 Receive。应用用长度前缀、分隔符、固定长度或 HTTP/Protobuf Framing，解析器处理半包、多包和恶意长度。

24. Keepalive 和 HTTP Keep-Alive 是一回事吗？

TCP Keepalive 是内核探测长期空闲连接存活；HTTP Keep-Alive 是复用连接发送多个请求。应用还需要请求 Deadline 和 LB Idle Timeout，不能只靠 TCP Keepalive。

25. SYN Flood 如何防护？

攻击制造大量半连接。内核 SYN Cookie/队列治理、LB/WAF、源限速和容量共同防护。调大 Backlog 只能缓解，需监控 SYN_RECV、Drop 和攻击流量。

26. listen backlog 涉及哪些队列？

实现概念上有半连接和已完成等待 Accept 的队列，应用 Accept 慢或参数/内核上限不足会丢连接。排查同时看 SYN、Accept Queue、应用 Event Loop 和 CPU。

四、UDP、QUIC 与实时/低延迟通信

27. UDP 与 TCP 的核心区别是什么？

UDP 保留 Datagram 边界、无连接状态和内建可靠/有序/拥塞控制，Header 小、可一对多；应用按需实现重传、顺序、分片和安全。不是“UDP一定快”，可靠功能仍有成本。

28. UDP 适合什么场景？

DNS、实时音视频、游戏、遥测和基于 UDP 的 QUIC。允许部分丢失或应用有专门协议时合适；交易请求通常不直接手写 UDP 可靠层。

29. UDP 丢包在哪里发生？

发送/接收 Socket Buffer、网卡队列、内核处理、网络拥塞、MTU/分片、防火墙和应用读取不及时。需要端到端序号和各层 Drop 指标。

30. QUIC 解决什么问题？

在 UDP 上实现加密、可靠 Stream、拥塞控制和连接迁移；整合 TLS，减少握手；各 Stream 独立传输，避免 TCP 层一个丢包阻塞所有 Stream。代价是用户态 CPU、生态和网络设备兼容。

31. QUIC 为什么仍然会拥塞？

它仍共享网络并执行拥塞控制，只是多 Stream 的应用层队头阻塞改善。单个 Stream 内仍需有序，带宽、丢包和服务端 CPU 仍是限制。

32. 连接迁移如何实现？

QUIC 使用 Connection ID 而非仅依赖五元组识别连接，客户端 IP/网络变化后可验证新路径并继续。服务端负载均衡需能按 Connection ID 路由或共享状态。

五、Socket、非阻塞 IO、epoll 与服务器模型

33. Socket 是什么？

应用访问网络协议栈的内核对象，通过 FD 操作；TCP Socket 有状态、收发 Buffer 和地址元组。创建、Bind、Listen、Accept、Connect、Read/Write、Close 构成基本生命周期。

34. 阻塞和非阻塞 Socket 有什么区别？

阻塞调用无数据/空间时让线程睡眠；非阻塞立即返回 EAGAIN，由 Event Loop 在就绪后重试。非阻塞不等于无等待，只是等待由调度器管理。

35. IO 多路复用解决什么？

一个/少量线程等待大量 FD 的就绪事件，避免每连接一线程。Select/Poll 线性扫描，epoll 使用注册的 Interest/Ready 结构提高大量 FD 场景效率。

36. epoll LT 和 ET 如何选择？

LT 简单稳健，只要仍就绪就通知；ET 事件更少但必须非阻塞并读写到 EAGAIN。框架已封装时遵循框架，不为理论性能自行改底层。

37. Reactor 和 Proactor 有什么区别？

Reactor 通知“可以做 IO”，应用执行 Read/Write；Proactor 通知异步 IO 已完成。Linux 常见高性能网络库以 Reactor 为主，io_uring 提供更广异步提交/完成模型。

38. Go Netpoller 如何映射这些机制?

Go `net` 使用非阻塞 FD 和平台 Poller；G 等待网络时 Park，事件就绪后 Ready，不长期占用 OS Thread。业务仍需 Deadline、连接/并发上限和慢客户端治理。

39. Socket Buffer 越大越好吗?

大 Buffer 可吸收 RTT/带宽和突发，但增加单连接内存、排队和延迟；小 Buffer 限吞吐。结合 Bandwidth-Delay Product、连接数和业务消息压测，系统 Max 也限制应用设置。

40. Zero Copy 是什么?

减少用户态/内核态之间复制和上下文切换，例如 sendfile、splice、mmap 等。适合文件传输；TLS、内容变换和跨平台会限制。端到端 Profile 后再用。

41. Backpressure 如何沿网络传播?

应用消费慢 -> Receive Buffer 满 -> TCP Window 缩小 -> Sender Write/Buffer 排队 -> 上游请求积压。每层都应有有界 Buffer 和 Deadline，不能无限内存队列切断信号。

六、DNS、TLS、证书与安全连接

42. DNS 查询链路是什么?

应用/系统 Resolver 查本地缓存和 Hosts，递归 Resolver 依次获取 Root、TLD、权威结果并缓存。实际可有搜索域、Stub、Sidecar 和 Service Discovery。

43. TTL 如何影响变更?

TTL 控制缓存时长，低 TTL 收敛快但增加查询，高 TTL 稳定但迁移慢。客户端长连接即使 DNS 更新也可能继续旧 IP，因此发布/容灾要同时治理连接生命周期。

44. DNS 为什么会成为性能瓶颈?

Resolver 慢、缓存 Miss、搜索域多次尝试、并发限制、网络和权威故障。记录 DNS Duration/Error，使用缓存和冗余 Resolver，但避免永久缓存失效地址。

45. TLS 握手大致发生什么?

协商版本/Cipher，服务器发送证书，客户端验证域名、链、有效期和信任根，双方完成密钥交换并建立会话密钥。现代 TLS 减少往返并支持会话恢复。

46. 对称和非对称加密如何配合？

非对称用于身份与密钥协商，成本高；建立会话后用对称加密传数据。证书把公钥与身份由 CA 签名关联。

47. 证书验证为什么会失败？

过期/未生效、域名不匹配、链不完整、根不信任、SNI 错、时钟偏差和撤销/策略。不能生产跳过验证，应修信任、证书和时间。

48. mTLS 与普通 TLS 有什么区别？

普通 TLS 主要验证服务端，mTLS 还要求服务端验证客户端证书，适合服务身份。它提供认证和加密，不自动提供资源级授权。

49. TLS 如何影响性能？

握手 RTT、非对称计算、记录加解密和证书验证。连接复用、会话恢复、硬件/库优化可降低；大规模短连接最明显。不能为性能关闭验证。

50. 0-RTT 有什么安全边界？

早期数据可能被重放，只适合幂等读或有重放防护的操作。交易写不能因降低一次 RTT 忽视业务幂等。

七、HTTP/1.1、HTTP/2、HTTP/3、gRPC 与长连接

51. HTTP/1.1 的核心特性和限制是什么？

文本 Header、Persistent Connection、Chunked 等；同连接请求响应顺序约束导致应用层队头问题，浏览器常开多连接。连接复用减少握手，但 Idle 与池大小需治理。

52. HTTP/2 如何多路复用？

将连接上的消息拆为二进制 Frame，按 Stream ID 交错传输，并有 Header Compression 和流控。一个 TCP 丢包仍会阻塞连接上所有 Stream 的字节到达。

53. HTTP/3 为什么改善队头阻塞？

基于 QUIC，各 Stream 独立可靠，某 Stream 丢包不阻塞其他 Stream；同时整合 TLS 和连接迁移。服务器、客户端和中间网络支持度及 CPU 需要评估。

54. HTTP Keep-Alive 和连接池如何配置？

按单实例并发和下游总预算配置 Max Idle/Per Host/Total，设置 Idle Timeout 和连接年龄；确保 Body 读取/关闭；监控池等待、建连和复用。池越大不等于吞吐越高。

55. Chunked、Content-Length 和流式响应有何差异？

已知长度可发 Content-Length；未知长度 HTTP/1.1 可 Chunked；HTTP/2/3 用 Frame 结束语义。流式响应要 Flush、Backpressure、取消和代理 Buffer 配置。

56. HTTP 状态码 502、503、504 如何理解？

502 代理收到无效/失败上游响应；503 服务不可用/过载；504 代理等待上游超时。实际由网关定义，必须结合代理日志、Upstream Timing 和应用 Trace。

57. gRPC 为什么依赖 HTTP/2？

利用二进制 Frame、多路复用、双向 Stream、Header/Trailer 和流控。gRPC Status 在 Trailer，代理和观测工具需正确支持。

58. WebSocket 和 SSE 如何选择？

WebSocket 双向、适合交互；SSE 基于 HTTP 单向服务端推送、浏览器重连简单。两者都是长连接，需心跳、慢客户端、连接上限、重连游标和发布治理。

59. HTTP 幂等和网络重试有什么关系？

GET/PUT 等定义上的幂等仍需正确实现；POST 可用 Idempotency-Key。网络超时只表示未知，自动重试前必须知道服务端是否可能已执行。

60. Header 为什么也需要限制？

超大 Header/Cookie 会耗内存、带宽和解析 CPU，并可能触发代理差异。服务端、网关统一限制；Trace/Baggage 不传播无界业务数据。

八、代理、负载均衡、NAT、服务发现与连接治理

61. 正向代理和反向代理有什么区别？

正向代理代表客户端访问外部，反向代理代表服务端接收用户并转发后端。API Gateway/LB 常是反向代理，承担 TLS、路由、限流和观测。

62. L4 和 L7 负载均衡如何选择？

L4 按连接/五元组，性能高且协议透明；L7理解 HTTP/gRPC，可按路径/Header路由和重试，但解析成本与语义复杂。架构可组合。

63. 常见负载算法有哪些？

Round Robin、Weighted、Least Request/Connection、Consistent Hash、EWMA/自适应。长连接下连接均匀不等于请求均匀；一致性哈希可能热点。

64. 健康检查为什么可能误判？

阈值太短把慢当死，深度探测依赖异常导致全部摘除，检查路径与业务路径不同产生假健康。区分 Liveness、Readiness 和被动错误，设置抖动和慢启动。

65. 代理重试为什么危险？

代理不知道完整业务幂等，重试写请求可能重复；多层重试放大。仅对安全方法/明确条件、有限次数和总 Deadline 重试，并输出 attempt 指标。

66. X-Forwarded-For 和 Forwarded 如何信任？

客户端可伪造，只有来自可信代理的 Header 可接受；网关清洗并追加，应用按可信跳数解析。安全审计保留直接 Peer 与逻辑 Client IP。

67. 服务发现更新为何不立即生效？

DNS/注册缓存和现有长连接仍指向旧实例。客户端 Resolver 更新地址后还需连接轮换和 LB 创建新 Subchannel；下线先摘流量、等待连接收敛再终止。

68. SNAT Port 耗尽如何发生？

大量到同一目标短连接需要唯一源端口/五元组，NAT 表与端口范围耗尽导致 Connect 失败。连接复用、增加出口 IP、扩大资源和降低短连接共同治理。

九、网络安全、观测与故障诊断

69. 网络问题常用哪些命令？

```
ip addr
ip route get <ip>
ss -s
ss -tanp
dig +trace <domain>
curl -v --trace-time <url>
openssl s_client -connect host:443 -servername host
tcpdump -i any -nn host <ip>
```

生产抓包要限定 Host/Port/时长和权限，注意敏感数据。

70. 如何用 tcpdump 判断 TCP 问题？

看 SYN/SYN-ACK/ACK、Retransmission、Duplicate ACK、RST、Zero Window 和 RTT；同时抓客户端/服务端更可靠。抓包未看到应用内部排队，必须与 Trace 结合。

71. connection refused、timeout、reset 有何差异？

Refused 常是目标可达但端口无监听/主动拒绝；Connect Timeout 常是丢包、路由、防火墙或积压；Reset 是连接被一方/中间设备异常关闭。错误仍需结合阶段和抓包。

72. 丢包和重传如何定位？

按客户端、主机网卡、交换/云网络、服务端逐段；看 TCP Retrans、NIC Drop/Error、Softnet、队列和带宽。重传也可能接收端应用不读导致，不全是物理网络。

73. 如何防 DDoS 和应用层攻击？

边缘清洗/CDN/WAF、L4 SYN防护、L7认证限流、请求大小/复杂度限制、连接与并发上限、缓存和降级。源站隐藏和容量预案同样重要。

74. 网络故障复盘需要哪些证据？

目标域名/IP/端口、协议版本、DNS答案、路由、握手/TLS/TTFB分段、连接状态、重传、代理 Upstream 日志、Trace、发布和网络变更。只写“网络抖动”不是根因。

十、业务架构与生产案例

75. 广告创建接口大量超时如何排查？

按入口网关、Go服务、HTTP连接池、DNS/TLS、媒体 API和账号限流拆 Trace；确认是池等待、平台 429还是网络。止血按平台熔断、任务异步排队并保护其他媒体；恢复限速回放。

76. 大量 TIME_WAIT 如何处理？

确认主动关闭方和目标分布，检查每请求建 Client、Keep-Alive 被禁、Body 未读、LB Idle 不匹配；修复连接复用和池参数。只有端口确实瓶颈时再评估系统参数和出口扩展。

77. HTTP/2 单连接故障为什么影响大？

大量 Stream 复用一个 TCP，丢包/连接关闭影响多个请求；Server Max Concurrent Streams和客户端连接数也会排队。可使用合理连接池、Keepalive和重连，但连接过多又增加资源。

78. 小包正常、大响应超时如何处理？

检查 MTU Black Hole、代理 Body/Buffer/Timeout、带宽和客户端读取；用不同 Payload、Ping DF/Tracepath和抓包验证。不要只调应用 WriteTimeout。

79. DNS 切流为何仍有请求到旧集群？

递归/本地缓存未过 TTL，客户端长连接未重建，代理有独立缓存。迁移前降低 TTL，旧集群保留过渡，连接设置合理年龄，监控新旧流量并渐进下线。

80. 如何用两分钟回答网络架构题？

我按链路、网络、传输、加密和应用层定位。域名先经 DNS，路由与 ARP找到下一跳；TCP通过三次握手、序列/ACK、流控和拥塞控制提供可靠字节流，应用必须定义消息边界；TLS建立身份和会话密钥；HTTP/1.1复用连接，HTTP/2在单TCP多路复用，HTTP/3用QUIC改善跨Stream队头阻塞。生产上代理/LB、NAT、连接池和服务发现都会影响路径，所以我监控 DNS、Connect、TLS、TTFB、池等待、重传和状态码。故障时用Trace先找慢阶段，再用ss、dig、抓包和代理日志取证；重试只用于幂等请求并受总Deadline控制，恢复时渐进放流和轮换旧连接。

技术 | 系统架构设计方法论面试题

面向 9 年以上 Golang Web 后端业务研发岗位。本篇不从“堆组件”开始，而是从业务目标、不变量和故障模型推导数据、流量、一致性、高可用、容量、安全与演进方案。

一、系统设计地图与资深回答框架

1. 系统架构知识图谱是什么？

```
Business Goal / User Journey
-> Scale / SLO / RPO / RTO / Cost
-> Domain / State Machine / Invariant
-> API / Data / Read-Write Path
-> Consistency / Idempotency / Transaction
-> Cache / MQ / Partition / Search
-> Availability / Degrade / Disaster Recovery
-> Security / Observability / Evolution
-> Capacity / Trade-off / Verification
```

2. 系统设计题的标准步骤是什么？

1. 澄清用户、功能范围和成功标准。
2. 量化 QPS、数据量、P99、可用性、RPO/RTO。
3. 定义核心实体、状态机和业务不变量。
4. 画 API、读写链路和数据模型。
5. 找瓶颈并按需要引入缓存、MQ、分片和读模型。
6. 覆盖超时、重复、乱序、故障和降级。
7. 做容量、监控、迁移、成本和验证闭环。

3. 为什么先定义不变量？

不变量决定一致性边界，例如库存不为负、同一订单只支付一次、已下线内容不被旧事件重新发布。没有不变量就无法判断事务、锁、幂等和补偿应放在哪里。

4. 架构图应该画什么？

先画用户和核心请求，再画服务边界、事实存储、缓存/MQ/搜索等派生组件及数据方向；标出同步/异步、所有者和故障边界。不要一开始画满网关、注册中心等无关盒子。

5. 如何表达 Trade-off？

说明选择优化了什么、牺牲了什么、为何在当前约束下值得、触发什么条件会重选。例如异步化提升吞吐和可用性，但引入延迟、重复、对账和运维成本。

二、需求澄清、SLO、规模与约束

6. 功能需求和非功能需求如何拆？

功能是发布、查询、下单、支付；非功能是峰值、P99、可用性、数据正确性、保留周期、安全和成本。资深答案必须量化后者。

7. QPS 如何估算？

从 DAU/业务次数/时段得到平均，再乘峰谷、活动和重试系数；拆读写和不同 API。给区间并说明假设，随后通过压测校正。

8. 数据量如何估算？

日新增 = 用户/设备 × 每日事件 × 平均大小；总存储再乘保留、索引、副本、压缩和安全余量。冷热、删除、重建和备份临时空间也要考虑。

9. SLI、SLO、SLA 如何区分？

SLI 是测量，SLO 是内部目标，SLA 是对外承诺和责任。设计要明确成功口径、统计窗口和排除项；健康检查成功不等于用户请求可用。

10. P99 为什么影响架构？

调用链串行和扇出会累积尾延迟；平均值无法决定 Deadline和容量。需减少同步层级、隔离慢依赖、缓存/预计算并设置端到端预算。

11. RPO 和 RTO 如何决定灾备？

RPO决定可丢数据窗口，RTO决定恢复时间；进而选择同步/异步复制、备份频率、跨地域和演练投入。高目标增加延迟、复杂度和成本。

12. 如何给业务分级？

按收入/正确性/用户影响分核心写、核心读、非核心体验和离线任务；分别设置 SLO、资源、降级和恢复顺序。不能所有功能都按最高等级建设。

13. 如何识别硬约束？

合规、预算、上线时间、现有团队、中间件能力、地域和第三方配额。方案必须在现实约束下可交付，而不是理论最优。

三、领域建模、状态机、API 与数据所有权

14. 如何识别核心实体和聚合？

从用户旅程、业务语言、生命周期和一致性边界识别。聚合内规则强一致，跨聚合用 ID/事件协作。实体不是数据库表的一比一翻译。

15. 状态机为什么重要？

显式列出合法迁移、触发者、幂等、超时和补偿，防止任意更新状态。每次迁移用当前状态+版本条件裁决并记录流水。

16. API 设计从什么开始？

从业务命令和查询语义，不从表 CRUD。定义请求 ID、权限、幂等、状态码、分页、异步任务和兼容；限制批量和复杂度。

17. 服务边界如何决定？

按业务能力、数据所有权、变化率、SLA 和团队认知拆分。频繁一起改/发布、共享事务和深同步链说明边界可能过细。

18. 数据所有权意味着什么？

只有所属服务能直接写事实数据，其他服务通过 API/事件协作。读性能可构建本地冗余模型，但必须标明来源、版本和重建方式。

19. 关系模型和文档模型如何选择？

关系、约束、事务和灵活查询优先 MySQL；聚合文档、结构变化和整体访问优先 MongoDB。搜索用 ES、热点用 Redis、事件用 Kafka，各自不越界。

20. 范式与反范式如何取舍？

事实模型优先正确和单一来源；读模型可按查询冗余。反范式降低读成本但增加同步和修复，必须有版本、事件和对账。

21. 如何设计审计与流水？

关键状态变化写不可变流水，包含业务 ID、前后状态、操作者、请求/事件 ID、时间和原因。流水支持对账、追责和重建，不只打印日志。

四、读写链路、缓存、异步化、搜索与数据分发

22. 读链路如何逐层优化？

正确索引/模型 -> 只读必要字段 -> 本地/Redis缓存 -> 读副本 -> 搜索/CQRS读模型 -> 预计算/CDN。每层定义命中、失效、回源、降级和一致性。

23. Cache Aside 的边界是什么？

读缓存未命中回源，写 DB 后删缓存。并发回填和删除失败会短暂不一致；用 TTL、版本、CDC/MQ失效、热点互斥和对账治理，不能宣称绝对强一致。

24. 什么时候异步化？

非必须即时完成、可持久化重试、需削峰或一对多传播时。异步任务必须有状态、幂等、错误、取消、超时、补偿和可观测，不是随手 `go func`。

25. MQ 在架构中承担什么？

事件持久化、解耦、削峰和重放。不能替代事实存储；引入重复、乱序、积压和 Schema 演进，Consumer 要幂等并保护下游。

26. 什么时候引入 ES？

全文召回、复杂过滤、相关性排序和聚合超过主库能力时。ES 是可重建读模型，主库先提交，通过 Outbox/CDC/MQ同步并对账。

27. CQRS 适合什么？

写规则和查询模型差异大、读多且允许最终一致时。代价是双模型、同步和运维；普通 CRUD 不需为模式而使用。

28. 热点数据如何治理？

识别 Top Key/租户；读热点复制、本地缓存、请求合并；写热点分桶、批量、队列串行或拆分大租户。拆散后要有聚合和一致性。

29. 深分页和大结果如何处理？

Cursor/Seek、限制 Page/Fields、异步导出和对象存储。禁止在线接口无限 Offset 和一次返回全量；导出任务有资源配额。

30. 数据分发为什么需要版本？

事件会重复乱序，派生存储用实体版本拒绝旧更新；删除也带更高版本/Tombstone。版本由事实源生成，不能只依赖发生时间。

五、一致性、幂等、事务、锁与补偿

31. 如何确定一致性级别？

按不变量区分强一致核心事实、读己之写、因果或最终一致。订单金额与库存更强，搜索/报表可延迟。避免全系统统一最高级别。

32. 幂等如何设计？

业务 request_id/event_id、唯一索引、状态机条件更新、Inbox和结果查询。定义 Key作用域、参数摘要、处理中和过期；记录与业务写同一事务。

33. 网络超时为什么需要状态查询？

超时是结果未知。创建/支付等通过业务 ID查询不存在/处理中/成功/失败，再决定重试；不能见 Timeout 就再执行副作用。

34. 跨服务一致性有哪些方案？

2PC、TCC、Saga、Outbox+MQ。根据不变量、参与方、吞吐、补偿可行性和一致性窗口选。多数业务以本地事务+Outbox+幂等+对账为主。

35. Outbox 为什么重要？

业务更新和事件记录同一本地事务，Relay/CDC异步发布，消除 DB成功消息未产生窗口。发布和消费仍会重复，必须幂等和治理积压。

36. 分布式锁什么时候必要？

需要跨实例互斥且无法用唯一约束、条件更新、分区串行化时。锁需 token、Lease、续约、Owner释放和 Fencing；锁不替代业务幂等。

37. 乐观锁和悲观锁如何选择？

冲突少用版本条件，失败重试；冲突高且临界操作短可数据库锁。锁内不调用慢外部服务，监控冲突、等待和死锁。

38. 补偿为什么不等于回滚？

跨服务已提交状态可能被观察，补偿是新的业务动作，如退款/释放，不抹去历史。补偿也会失败，需要幂等、重试、人工和审计。

39. 对账系统检查什么？

按业务主键比较状态、金额/数量、版本和时间窗口，产生可追踪差异并自动/人工修复。它发现重试看不到的漏事件、Bug和人工改数。

六、高可用、过载保护、容灾与恢复

40. 高可用由哪些层构成？

多实例/故障域、健康与摘流量、依赖冗余、Deadline/限流/熔断/降级、数据副本/备份、灰度和演练。单纯多副本不是高可用。

41. 为什么所有调用都要 Deadline？

无界等待占连接、线程/Goroutine和内存，形成级联故障。端到端预算向下传播，下游响应取消；超时不是重试理由本身。

42. 过载保护的顺序是什么？

Admission Control、并发上限、有界队列、背压、限流、熔断和降级。拒绝要早、明确、有业务优先级；无限队列是延迟 OOM。

43. 降级如何设计？

核心写优先；读可用缓存/热门/简化结果并标识陈旧；非核心统计关闭；写未完成返回处理中而非假成功。定义触发、恢复和补偿。

44. 多可用区如何部署？

实例、副本和主控跨独立故障域；考虑多数派、网络延迟和容量 N-1。避免主副本在同机架，配置 Allocation Awareness。

45. 异地灾备和多活如何选择？

先按 RPO/RTO决定备份、异步复制、热备或多活。多活需流量归属、冲突、一致性和回切，成本最高；不是所有业务都需要。

46. Backup 和 Replica 有什么区别？

Replica抵抗节点故障，但误删/逻辑污染同步传播；Backup提供历史恢复点。备份必须异地、加密、校验并定期恢复演练。

47. 级联故障如何切断？

控制入口、关闭重试放大、熔断慢依赖、降级非核心、释放卡住资源，保护事实存储；从底层依赖向上渐进恢复并限制积压回放。

48. 如何设计故障演练？

定义稳态指标、最小故障域和停止条件，注入延迟、超时、实例退出、网络分区、磁盘满、消息重复；验证告警、降级、恢复和数据正确性。

七、容量、性能、分片、扩展与成本

49. 容量评估基本公式是什么？

```
实例数 = 峰值 QPS / 单实例安全 QPS × 冗余
并发 ≈ QPS × 平均延迟
存储 = 日增 × 保留 × 索引/副本系数 + 临时余量
带宽 = 峰值 QPS × 平均报文大小
```

能力来自真实混合压测，并验证 N-1、发布和依赖变慢。

50. 如何找到系统瓶颈？

负载阶梯中观察哪个资源先饱和：CPU、锁、连接池、DB、Redis、MQ、网络或第三方配额；用 Trace 和 Queue Wait解释 P99。瓶颈随流量和数据分布移动。

51. 什么时候读写分离？

读压力大且可容忍复制延迟，查询已优化后。必须处理写后读、路由、延迟和故障切换；不用于解决慢 SQL 本身。

52. 什么时候分库分表？

单库容量/吞吐经模型、索引、归档和读扩展后仍不足，并有稳定分片键时。它引入跨分片查询、事务、全局ID和迁移成本。

53. 分片键如何选择？

基数、均匀性、单调热点、查询路由和扩容迁移。最大租户而非平均租户决定倾斜；核心查询最好携带分片键。

54. 如何处理超级租户？

资源配额、独立分片/集群、二级分区、按租户限流和成本归属。共享架构必须承认极端租户会破坏平均模型。

55. 弹性扩容为什么不是即时救命？

启动、预热、发现和连接池需时间，还会增加下游连接。要基础冗余、预测、慢启动、下游联动和提前限流。

56. 如何控制架构成本？

按业务等级设置副本/SLO，冷热分层和归档，优化实例利用率、网络 and 存储；计算中间件机器与人力。最成熟的方案不是组件最多。

八、安全、可观测性、运维与治理

57. 威胁建模从哪里开始？

识别资产、信任边界、入口、攻击者和影响；覆盖认证、授权、注入、越权、数据泄露、供应链和拒绝服务。安全要求进入设计而非上线前补。

58. 多租户隔离有哪些层次？

应用行级、Schema/库、实例/集群和账号/网络隔离。隔离越强成本越高；无论哪层，租户 ID从可信身份注入并做资源配额。

59. 可观测性三支柱如何落地？

Metrics看趋势/告警，Trace定位链路，Logs给事件上下文，再加业务不变量和变更事件。用 Trace/Business ID关联，控制高基数和敏感数据。

60. 架构评审检查什么？

目标/约束、不变量、边界、数据、失败场景、容量、安全、观测、灰度/回滚、成本和备选。评审输出决策与 Owner，不只开会。

61. 如何治理配置和 Feature Flag？

强类型、范围校验、权限/审计、灰度和回滚；Flag有 Owner和过期，稳定后删除。配置中心故障使用最后快照，不让全部服务启动失败。

62. 运行手册包含什么？

告警含义、确认步骤、止血、风险、恢复、数据校验和升级联系人。命令可执行并定期演练；危险重置/删除不是第一步。

63. 如何用 SLO 驱动治理？

按用户请求定义 SLI，Error Budget消耗过快时限制高风险发布并治理主要失败来源。避免只用CPU/存活作为服务目标。

九、架构演进、迁移、灰度、回滚与决策记录

64. 为什么架构演进优于一次重写？

大重写需求仍变化、验证周期长、双系统成本和切换风险高。通过边界、适配、双读/影子和 Strangler 小步迁移，每步可验证回滚。

65. 数据迁移的完整流程是什么？

Schema兼容 -> 全量稳定游标/断点 -> 增量CDC/事件 -> 版本幂等 -> 对账 -> 双读影子 -> 灰度切流 -> 观察 -> 下线旧链路。

66. Expand-Contract 如何用于 Schema 变更？

先新增兼容结构，新旧代码都可运行；回填并切读写；确认旧消费者消失后删除。DDL、应用和数据迁移分发布。

67. 灰度发布怎样设计？

按租户、用户、地域或比例；对比新旧错误、P99、资源和业务；自动暂停阈值；状态流程保持版本一致；支持快速回滚和数据副作用处理。

68. ADR 记录什么？

背景、约束、备选、决策、后果和复审条件。用于重要选型和边界，帮助后人理解为何而非只看结果。

69. 如何识别过度设计？

没有真实规模/独立演进需求却引入分片、微服务、多活和复杂消息编排；团队无法运维；收益无法量化。优先最简单可演进方案，并写触发升级条件。

70. 技术债如何进入架构治理？

具体描述风险、业务影响、偿还成本和触发条件，分核心高风险与普通维护；结合需求渐进修复，重大债独立里程碑。不是把所有不喜欢的代码称债。

十、系统设计案例与两分钟表达

71. 高并发商品详情页如何设计？

CDN/本地/Redis多级读，Cache Aside和热点保护；商品事实在DB，库存价格按一致性独立查询；ES负责搜索不进入交易；缓存故障限流回源并降级；容量按热点、命中和DB保护评估。

72. 订单支付链路如何设计？

订单状态机和唯一业务号；本地事务写订单/Outbox；支付请求带幂等号，超时查询状态；回调幂等；库存/履约用Saga/事件；不可变流水和三方对账保证资金正确。

73. 内容发布系统如何设计？

元数据、媒资、审核、搜索和CDN分步骤 workflow；主库版本是事实，Outbox事件驱动下游；下线高版本防旧事件复活；查询走ES/缓存，核心发布不依赖搜索；支持灰度和回滚。

74. 广告投放任务系统如何设计？

任务状态机、账号级限流、持久化队列和按账号/任务分区；Worker幂等，第三方超时按request_id查询；平台适配隔离；失败可重试/人工；指标含排队、平台成功率、429和状态差异。

75. 埋点事件平台如何设计？

入口鉴权/限流/大小，事件带device_id、event_id、sequence；Kafka按Key分区，批量落湖/存储，去重与迟到窗口；Schema Registry；冷热分层和重放；在线状态与离线分析拆模型。

76. 如何用两分钟回答系统设计题？

我先澄清用户旅程和功能边界，量化峰值QPS、数据量、P99、可用性、RPO/RTO和成本，再定义核心实体、状态机与不可破坏的不变量。主链路先用最简单事实存储和正确索引完成，按瓶颈引入缓存、MQ、搜索、读模型或分片，并说明一致性、重复乱序和运维代价。写请求用业务ID、唯一约束和版本幂等，单服务本地事务，跨服务用Outbox、事件、补偿和对账。所有依赖有Deadline、有界并发、限流熔断和业务降级，数据有副本、备份和演练。最后用压测规划N-1容量，以Metrics/Trace/业务不变量观测，通过兼容Schema、灰度和回滚演进，并明确方案何时需要升级。

06-02 | 项目复盘与资深面试表达

技术 | 项目复盘与资深面试表达

面向 9 年以上 Golang Web 后端业务研发岗位。目标是把真实经历表达成复杂业务 Owner 能力：理解业务、做出技术判断、亲自落地核心代码、治理线上风险并推动结果；管理经验只作为协作优势。

一、候选人定位与表达地图

1. 你的核心职业定位是什么？

资深 Golang 业务后端 IC/技术骨干：能独立负责复杂业务域，从需求分析、方案、编码、上线到复盘闭环；不应聘纯管理岗，但具备推动复杂项目落地的协作能力。

2. 面试表达知识图谱是什么？

```
Positioning / Resume
-> Business Context / Scale / Problem
-> Ownership / Decision / Implementation
-> Reliability / Incident / Trade-off
-> Metrics / Result / Reflection
-> Collaboration / Influence
-> Follow-up / System Design / Career Choice
```

3. 为什么项目表达不是背故事？

面试官通过追问验证一致性：数据规模、时间线、职责、代码、指标和故障必须互相支撑。应建立项目事实卡和决策链，而不是记华丽句子。

4. 资深候选人和普通候选人的表达差异是什么？

普通候选人说用了什么；资深候选人说业务为何需要、替代方案、失败边界、容量、上线迁移、指标和复盘，并明确自己亲自完成的部分。

二、自我介绍、简历主线与岗位匹配

5. 9 年+ Go 后端自我介绍结构是什么？

工作年限/主栈 -> 三段业务经历主线 -> 擅长的复杂问题 -> 代表成果 -> 当前岗位诉求。控制在 1.5~2 分钟，给面试官留下可追问钩子。

6. 推荐自我介绍如何组织？

我有 9 年以上 Web 后端经验，近年的主技术栈是 Golang。做过出海广告投放、内容发布和高并发设备事件等业务，比较擅长复杂状态流、缓存与消息一致性、第三方平台治理和线上稳定性。我既做方案，也持续承担核心编码和故障处理；之前有一年多团队协作经验，提升了方案拆解和跨团队推进能力，但我的职业方向仍是资深业务研发，希望负责重要业务域和技术难题。

具体项目和指标替换成真实事实，不编造。

7. 简历项目应该按什么顺序？

与目标岗位最相关、个人 Ownership 最大、技术深度最高的优先；每个项目写业务、规模、职责、关键动作和量化结果。技术栈列表不能替代贡献。

8. 如何把三段经历串成主线？

- 枫叶互动：广告投放、第三方 API、任务和素材平台。
- 知乎：动态模板、统一发布、插件与并发治理。
- 360：设备事件、高并发消费、广告/运营平台。

共同主线是复杂业务平台、异步链路、外部依赖和稳定性，而非三段孤立公司介绍。

9. 如何回答“为什么考虑这个岗位”？

连接目标业务与自身经历，说明希望解决的具体问题和能贡献的能力；同时确认岗位是一线业务研发/技术 Owner，而非只说公司平台大。

10. 如何回答离职原因？

客观、简短、向前看：业务/组织变化、成长方向和岗位匹配。避免贬低前团队和披露敏感信息，不把“不想管理”说成逃避责任。

11. 没有精确指标怎么办？

使用可信区间、相对变化或可验证替代指标，如 P99 从秒级到数百毫秒、DB QPS下降约某比例、故障频率明显下降；明确是估算，不编造精准数字。

三、项目介绍、STAR、金字塔与事实卡

12. 项目介绍的六层结构是什么？

业务目标、规模约束、你的职责、关键挑战、方案与权衡、结果与复盘。先两分钟全景，再根据追问深入，不在开场讲全部源码。

13. STAR 如何避免变成流水账？

Situation/Task简洁交代约束，Action占主要篇幅并突出你的决策和实现，Result用指标，最后增加Reflection。每个 Action 对应一个问题和证据。

14. 什么是项目事实卡？

记录时间、用户/规模、架构、核心表/Topic、QPS、P99、职责、上线过程、故障、结果和可公开边界。每个数字知道来源和统计口径。

15. 项目两分钟版本应该包含什么？

一句业务定位、两个规模、你的 Owner 范围、一个最难问题、三项关键方案、两个结果和一个复盘。目的是让面试官选择追问方向。

16. 如何介绍技术栈而不罗列？

把组件放在链路和职责中：MySQL承载事实与状态，Kafka传播任务，Redis限流/热点，ES检索，Go Worker有界并发。每个组件都说明为何、边界和替代方案。

17. 如何回答“项目最大的难点”？

选择约束冲突而非单纯工作量，例如既要高吞吐又不能超过媒体账号配额、既要异步又要状态可追踪。讲不变量、失败场景、权衡、落地和指标。

18. 如何回答“如果重做会怎么改”？

基于当时信息肯定原方案合理性，再指出规模/认知变化后的改进、迁移顺序和收益。不能把原项目全盘否定，也不能回答“没什么可改”。

四、Ownership、职责边界与技术决策表达

19. 如何回答“你负责到什么程度”？

明确需求澄清、方案、核心模块编码、联调、上线、监控和复盘哪些亲自承担，哪些与他人协作。用具体文件/模块/决策证明，不说模糊“全程负责”。

20. 如何区分团队成果和个人贡献？

先承认团队结果，再说自己的关键决策、代码和推动动作；不抢功，也不只说“我们”。例如“整体由三人完成，我负责状态机、任务消费者和灰度方案”。

21. 如何表达技术选型？

业务需求 -> 候选方案 -> 决策维度 -> 选择 -> 代价 -> 复审条件。例：Mongo与MySQL不比较“谁快”，比较聚合模型、事务、查询和团队能力。

22. 面试官质疑方案时怎么办？

先确认约束是否不同，再基于事实讨论；承认合理替代和边界，不为维护面子硬辩。若当时确有缺陷，说明发现、修复和制度改进。

23. 如何证明自己写过核心代码？

能讲数据结构、接口、关键伪代码、事务/并发边界、错误处理、测试和一次真实 Bug；避免泄露公司源码。追问深度应与职责一致。

24. 如何表达方案评审能力？

讲一次发现遗漏不变量、容量、幂等、监控或回滚的案例，以及如何推动修改。重点是风险被提前消除，而不是“我经常评审别人”。

25. 如何表达推进能力而不变成项目经理？

聚焦技术接口、风险和交付：拆依赖、定义契约、建立联调环境、灰度和验收；同时亲自承担关键实现。排期管理只是支撑技术落地。

五、架构深挖、原理追问与权衡表达

26. 项目架构图怎样讲？

从用户请求和事实数据开始，沿同步/异步链路讲组件职责、数据所有权和故障边界；然后讲最关键的写/读路径。不要逐个念盒子。

27. 面试官问“为什么不用另一组件”怎么答？

回到当时规模、模型、SLA、团队和迁移成本；说明候选优势及为何不匹配。再给未来触发条件，体现开放判断。

28. 如何回答一致性追问？

定义业务不变量和一致性窗口，讲本地事务、Outbox、幂等版本、补偿和对账；明确哪些状态可短暂不一致以及用户体验。

29. 如何回答容量追问？

给峰值QPS、请求大小、单实例安全能力、连接/数据库预算、存储增量和N-1冗余；数字来自监控/压测，未知时说明估算公式。

30. 如何回答高可用追问？

故障检测、摘流量、冗余、Deadline/限流/熔断/降级、数据副本/备份和演练；说明某依赖不可用时核心业务具体返回什么。

31. 如何回答安全追问？

认证、资源授权、租户隔离、输入限制、Secret、审计和数据合规；结合真实接口，不只说HTTPS。

32. 如何回答成本追问？

机器/存储/网络和人力，冷热分层、资源利用、简化组件与业务等级。高级设计要承认预算，不以“多副本”作为所有答案。

六、线上故障、性能优化与复盘表达

33. 线上故障故事的结构是什么？

影响与时间线 -> 第一证据 -> 止血 -> 根因 -> 修复 -> 恢复校验 -> 防复发。Action中区分自己做的和团队协作。

34. 如何避免把相关性说成根因？

用指标/日志/Trace/Profile、复现和修复后验证建立证据链。例如发布与CPU同时发生只是线索，Profile定位具体函数才接近根因。

35. 如何表达止血成熟度？

先控制用户影响和保护事实存储，保存证据；选择可逆动作，知道扩容/重试会把压力转移何处；恢复渐进并校验积压和数据。

36. 性能优化案例必须有哪些数字？

负载条件、P95/P99、QPS、CPU、内存/分配或数据库指标，以及优化前后。只报百分比但无基线和流量可能误导。

37. 没有经历重大事故如何回答？

可讲险情、性能退化、数据差异或演练，但必须真实。说明如何发现和预防升级，不能编造灾难故事。

38. 复盘如何处理个人失误？

承认具体判断和影响，说明当时信息、及时行动和机制改进；不甩锅也不把系统性问题全揽为个人。重点是学习和防复发。

39. 如何回答最有价值的性能优化？

选端到端收益，不选微技巧。比如素材补偿从全量加载/无限Goroutine改为分页、有界并发和Bulk，展示RSS、GC、P99和完成时间。

40. 如何回答数据不一致事故？

说明事实源和不变量，定位消息产生、积压、Consumer、版本/幂等和派生存储；以事实源修复并限速回放；补对账和重建能力。

七、结果、指标、业务价值与复盘深度

41. 技术结果怎样映射业务价值？

P99降低对应运营效率/转化，积压降低对应数据新鲜度，故障减少对应收入和用户体验，资源降低对应成本。不要只说CPU下降却不解释业务意义。

42. 哪些指标适合项目结果？

成功率、P99、吞吐、积压年龄、同步延迟、数据差异、故障频率、MTTR、发布周期、资源成本和业务转化。选择与问题直接相关的两三个。

43. 如何说明结果归因？

对比前后相同负载、灰度对照或A/B；说明同时发生的变化和统计窗口。团队项目不把全部业务增长归因个人技术改造。

44. 如何表达未达到预期的项目？

说明原假设、实际数据、偏差原因、及时止损和后续选择。能基于证据停止错误方向也是成熟度。

45. “亮点”应该是什么？

在真实约束下解决难问题的决策和闭环：不变量、故障、迁移、指标。使用热门技术不是亮点本身。

46. 如何准备项目追问树？

为每个项目准备业务、规模、模型、并发、一致性、性能、故障、安全、成本、协作和复盘十个方向，每个回答能继续下钻一层原理与证据。

八、协作影响力、管理经验与高级 IC 定位

47. 如何回答“带过团队为什么不继续管理”？

管理经验让我在任务拆解、风险识别和跨团队协作上更成熟，但我确认自己的长期优势和兴趣仍在复杂业务与核心系统建设。我希望承担高级业务研发/技术Owner职责，继续亲自做设计、编码和线上治理，同时用协作经验帮助项目落地，而不是转纯人员管理。

48. 如何避免让面试官担心稳定性？

清楚说明这是经过实践后的主动职业选择，而非短期退缩；展示近年持续编码、技术深度和目标岗位匹配，并接受高级 IC 的责任和协作要求。

49. 如何表达带人经验？

讲技术辅导、方案质量、Review、故障复盘和新人独立交付的改善，不强调人数和行政管理。选择一个具体案例和结果。

50. 如何处理跨团队冲突？

统一业务目标和事实，明确接口/Owner/Deadline，记录决策与风险；必要时升级但不绕过沟通。面试讲自己如何改变方案或推动折中，不把对方描述成阻碍。

51. 如何表达技术影响力？

标准被采用、平台复用、事故减少、交付更快和他人能独立维护。文档/分享只是手段，实际行为变化和业务结果才是影响力。

52. 如何回答“是否愿意带人”？

愿意做技术辅导、Review和项目协作，也能承担复杂项目推进；但岗位主线希望是一线业务研发，不以纯人员管理作为目标。

九、行为面试、压力追问与面试官沟通

53. 如何回答失败经历？

选择真实且可控的失败，讲自己的责任、判断偏差、修正和机制改变。不要选无关小事，也不要把失败包装成“太追求完美”。

54. 如何回答意见不一致？

描述双方目标和证据，如何做实验/压测/评审、最终决策与关系；若自己错了明确承认。重点是理性协作而非赢辩论。

55. 被追问不知道怎么办？

明确知识边界，给已知原理和推导，说明如何验证；不要编造源码细节。可以说“这个版本字段我不确定，但稳定语义是.....我会查官方源码/文档确认”。

56. 面试官打断时怎么办？

先给结论，确认他要业务、原理还是实现，再调整深度。不要坚持背完准备稿。回答控制在金字塔层次，允许随时下钻。

57. 可以问面试官什么？

岗位核心业务域、当前最大技术挑战、高级工程师Ownership、代码/值班比例、团队对架构和质量期待、前三个月成功标准。避免首轮只问福利和晋升。

58. 如何判断岗位是否偏管理？

询问日常时间分布、是否有人绩效/招聘职责、核心代码Ownership、值班和方案落地。岗位名不如实际职责重要。

59. 面试后如何复盘？

记录问题、回答、卡点、追问和可信度；区分知识缺口、表达结构和项目事实；补文档并再次口述。不要只记“过没过”。

十、三段经历、系统设计与最终表达模板

60. 枫叶互动广告投放平台如何讲？

业务：多个媒体平台的广告账户、素材、任务和状态回流。挑战：外部API限流/不稳定、异步状态、一致性和运营检索。方案：平台Adapter、账号级限流、有界任务、Outbox/Kafka、幂等版本、Mongo/MySQL事实、ES检索和对账。选择真实模块和指标深入。

61. 知乎动态模板/统一发布如何讲？

业务：发布流程与动态扩展。挑战：插件生命周期、第三方SDK、并发安全和兼容。方案：清晰接口、前中后插件阶段、同步/异步边界、Context/Recovery、Race定位共享Map、灰度和回滚。突出亲自实现。

62. 360设备事件消费如何讲？

业务：大量设备事件进入规则、存储和推送链路。挑战：峰值、顺序、重复、下游容量。方案：Kafka按设备Key、幂等事件ID、有界Consumer、批量写、状态版本、积压和毒消息治理。给出规模与故障案例。

63. 如何把经历映射目标内容/商业化岗位？

广告投放映射商业化和素材平台；内容发布映射生产/审核/分发；设备事件映射埋点和高并发数据链。强调可迁移的问题解决能力，不假装熟悉未做过的内部业务。

64. 如何准备系统设计题？

每题用需求、规模、不变量、API/数据、核心链路、一致性、容量、高可用、安全、迁移十项；准备短剧发布、播放页、广告投放、素材搜索、埋点回流等与经历相关案例。

65. 两分钟项目模板是什么？

这个项目解决【业务问题】，峰值/数据规模是【真实数字或区间】。我负责【明确边界】，最大的矛盾是【目标与约束】。我先定义【不变量/状态】，方案上采用【三项关键设计】，同时处理【失败、容量、灰度】。我亲自完成【核心实现】，上线后【两个指标结果】。这套方案的代价是【权衡】，如果规模达到【条件】，我会按【演进路径】升级。

66. 故障案例模板是什么？

【时间/发布】后出现【用户影响】，我先【止血并保存证据】。通过【指标/Trace/Profile】把问题收敛到【根因】，触发条件是【条件】，放大机制是【重试/队列/资源】。修复后我用【灰度/数据】验证，并补充【监控、容量、测试、预案】，最终【MTTR/错误/P99】改善。

67. 最终高级 IC 定位如何表达？

我的优势是能把复杂业务问题落到可靠的Go后端系统：先理解业务和数据，再做一致性、性能和稳定性设计，同时亲自承担核心实现和线上闭环。我有带团队经历，所以能更好推动跨团队交付，但当前明确希望继续做一线业务研发和技术Owner，用技术深度和业务结果产生影响。

68. 面试准备的最终检查清单是什么？

自我介绍两版；三个项目各有2分钟/深挖；两个故障、两个性能、一个失败、一个冲突；所有数字可解释；管理转IC回答稳定；目标岗位问题准备；任何公司敏感信息已脱敏。

07-00 | 算法面试总纲与Golang实现

技术 | 算法面试总纲与 Golang 实现

面向 9 年以上 Golang Web 后端岗位。算法是编码和思维门槛，本篇重点是识别题型、建立不变量、写出可靠 Go 代码并完成自测；具体 70 题和专项代码由配套练习册承载。

配套练习资料

- [后端算法面试 70 题高频分类清单]
- [后端算法面试 70 题高频母模板排序版]
- [动态规划与图遍历高频背诵清单]

一、算法学习地图与现场答题框架

1. 后端算法知识图谱是什么？

```
Complexity / Constraint
-> Array / Hash / Prefix
-> Two Pointer / Window / Binary Search
-> Linked List / Stack / Queue
-> Tree / Heap / Divide and Conquer
-> Graph / Union Find / Backtracking
-> Dynamic Programming / Greedy
-> Go Implementation / Test / Engineering Problem
```

2. 现场答题的标准七步是什么？

复述题意与输入 -> 问清边界 -> 给暴力和瓶颈 -> 识别模板/不变量 -> 说明复杂度 -> 编码 -> 手工用例和边界自测。不要拿到题立刻沉默写代码。

3. 什么是算法不变量？

循环/递归每一步都成立的条件，例如滑动窗口内满足约束、二分答案区间始终包含解、链表 prev 指向已反转部分、堆顶是当前最小候选。先说不变量能显著减少 Bug。

4. 卡住时怎样推进？

写小样例、画结构、给 $O(n^2)$ 暴力，问重复计算在哪里；从数据范围推复杂度；尝试排序、Hash、单调性和状态定义。向面试官同步假设而非一直沉默。

5. 资深岗位算法评价什么？

不仅是 AC，还看沟通、边界、代码结构、复杂度、测试和是否过度设计。高级候选人不应因工程经验丰富而忽视基础手写。

6. 复习应按题还是按母模板？

先掌握能覆盖多题的母模板，再用代表题和变形验证：反转、遍历、窗口、二分、堆、BFS/DFS、回溯和 DP。最后随机混合训练识别能力。

二、复杂度、约束与数据结构选择

7. 如何从数据规模反推复杂度？

大致思路： n 很小可指数/回溯，中等可 $O(n^2)$ ， 10^5 级通常 $O(n \log n)/O(n)$ ，更大需线性/数学。实际还受常数、语言和时限影响，面试给合理判断即可。

8. 时间复杂度为什么不是实际耗时？

它描述增长阶，忽略常数、Cache、分配和输入分布。两个 $O(n)$ 实现可能差很多；但面试先保证阶正确，再谈工程性能。

9. 空间复杂度如何计算？

关注额外空间、递归栈、Hash/Visited、输出是否计入按题意说明。原地算法也可能有 $O(\log n)$ 递归栈。

10. 均摊复杂度是什么？

少数昂贵操作摊到操作序列，如动态数组扩容、单调栈元素进出一次。它不是平均输入概率，而是序列总成本上界。

11. 如何选择 Array、Map、Set、Heap？

连续索引/顺序扫描用数组；存在性/映射用 Hash；动态 TopK/最值用 Heap；有序范围需排序、Tree 或二分；选择同时说明空间和是否需要稳定顺序。

12. 排序为什么经常是第一步？

排序把无序问题转成单调结构，便于双指针、区间合并、二分和去重，代价 $O(n \log n)$ 及可能丢原下标。需保留原位置时存 Pair。

13. 递归和迭代如何选择？

树/回溯递归表达自然，深度可控时优先清晰；链表长、图深或输入不可信时迭代避免栈风险。Go Goroutine 栈可增长但不是无限。

三、数组、哈希、字符串、前缀和与区间

14. 两数之和代表什么模板？

“遍历当前元素，查询此前状态”：Map记录值到下标，查 $target - x$ ，时间 $O(n)$ 、空间 $O(n)$ 。先查再放处理同一元素不能重复使用。

15. Hash 题的四类映射是什么？

值到位置、值到频次、状态到首次位置、前缀值到出现次数。看到查补数、分组、去重、最长连续和子数组计数优先考虑。

16. 前缀和解决什么？

将区间和转为 $prefix[r] - prefix[l]$ ；配合 Map 查此前前缀解决和为 K 的子数组。二维前缀处理矩形查询。注意初始前缀 0 和次数而非仅存在。

17. 区间问题的通用步骤是什么？

按起点/终点排序，维护当前合并区间或结束边界；明确闭开区间和相邻是否合并。会议室/重叠数可用扫描线或 Heap。

18. 字符串题在 Go 中有什么特殊边界？

Byte适合 ASCII/协议，Rune适合 Unicode Code Point；`range` 索引是字节偏移。字符计数数组比 Map快但只适固定字符集；切片必须落在 UTF-8边界。

19. 矩阵模拟如何少写 Bug？

明确行列范围、四边界和每轮收缩；方向数组统一邻接遍历；访问前检查边界。空矩阵和单行/单列必须自测。

20. 原地数组修改的常见模式是什么？

读写指针：read扫描，write指向下一个保留位置；适合删除/去重/过滤。每步维持 `[0, write)` 已是最终结果。

四、双指针、滑动窗口、二分与单调性

21. 左右双指针适合什么？

有序数组求两数、三数、盛水、回文；根据比较结果单调移动一侧。核心是证明移动不会错过解。

22. 快慢指针适合什么？

链表 midpoint/环/倒数位置，数组原地读写。要明确定义每个指针速度、距离和相遇条件。

23. 滑动窗口的识别条件是什么？

连续子数组/子串，左右边界能单调前进，窗口状态可增量维护。固定窗口与可变窗口分开；有负数时某些“和满足就缩”不再单调。

24. 可变窗口模板是什么？

右侧加入并更新状态；当不满足/过度满足时移动左侧并撤销；在正确时机更新答案。最长通常在满足时更新，最短常在收缩过程中更新。

25. 二分查找的本质是什么？

在单调谓词上寻找边界，而不只是查数。先定义“第一个满足条件”的区间和不变量，再写循环，可覆盖左右边界、旋转数组和答案二分。

26. 推荐的左边界二分模板是什么？

```
func lowerBound(nums []int, target int) int {
    l, r := 0, len(nums)
    for l < r {
        m := l + (r-l)/2
        if nums[m] >= target { r = m } else { l = m + 1 }
    }
    return l
}
```

区间始终是 `[l, r)`，返回第一个大于等于目标的位置。

27. 答案二分怎样识别？

题目求最小可行最大值/最大可行最小值，且给定答案 x 能 $O(n)$ 判断可行，谓词随 x 单调。边界来自可能答案范围，而不是数组下标。

28. 为什么二分最容易死循环？

区间定义混用、Mid取整和更新不缩小范围。写前决定闭区间或半开区间，每个分支保证长度严格减少，用 $0/1/2$ 元素测试。

五、链表、栈、队列与单调结构

29. 链表题写代码前问什么？

头节点会变吗、需要找位置吗、会断链吗、一个还是多个链表、返回头是谁。头可能变用 Dummy，修改 Next 前保存 Next。

30. 反转链表的核心不变量是什么？

prev 是已反转区间头，cur 是未处理首节点。保存 next $\rightarrow$ cur.Next=prev $\rightarrow$ 推进。区间反转用 [head, stop) 模板复用。

31. 环入口如何推导？

快慢相遇后，一个从头、一个从相遇点同速，下一次相遇是入口。面试可简述距离等式，并先保证空指针边界。

32. 栈适合什么题？

括号、表达式、DFS迭代、路径、单调结构和撤销。判断需要“最近未匹配/后进先出”时使用。

33. 单调栈解决什么？

寻找左/右第一个更大/更小、柱状图、每日温度、接雨水。元素通常进出各一次 $O(n)$ ；栈存下标便于距离和取值。

34. 单调队列解决什么？

滑动窗口最大/最小：队尾移除不可能成为答案的元素，队头移除过期下标。Deque保持值单调且索引在窗口内。

35. LRU 为什么是 HashMap + 双向链表？

Map $O(1)$ 定位节点，双链表 $O(1)$ 移动和淘汰尾部；Dummy简化边界。每次 Get/Put维护“头部最新、尾部最旧”不变量。

36. 合并 K 个有序链表如何选择？

小顶堆每次取最小 $O(N \log K)$ ，或分治两两归并同阶且常数不同。解释 K、总节点 N 和是否可修改输入。

六、二叉树、BST、堆、分治与 TopK

37. 树题为什么常用递归定义？

问题可由左右子树同类结果组合。先定义函数返回语义，再写终止、子问题和合并；不要边写边改变返回含义。

38. 前中后序分别适合什么？

前序先处理根，适合构建/路径；中序在BST得到有序；后序先得子树结果，适合高度、直径、平衡和删除。层序用BFS处理层级最短距离。

39. BST 题的核心性质是什么？

中序严格有序或节点受上下界约束。验证不能只比较直接孩子；重复值规则按题意。查第K小用中序计数。

40. 树的后序信息汇总模板是什么？

递归返回父节点需要的信息（高度/贡献/是否平衡），同时用外部变量更新全局最优（直径/路径和）。区分“向父返回”与“全局答案”。

41. Heap 的不变量是什么？

完全二叉树且父节点不大于/不小于孩子，只保证堆顶极值，不保证全序。插入上浮、删除顶下沉，均 $O(\log n)$ 。

42. TopK 如何选 Heap、Quickselect 或排序？

全排序 $O(n \log n)$ 简单；K小/流式用大小K Heap $O(n \log K)$ ；只需一次第K且允许修改用Quickselect平均 $O(n)$ 。需要有序结果还要最终排序。

43. Go container/heap 有什么坑？

实现 Len/Less/Swap/Push/Pop, Push/Pop方法使用指针接收者并操作切片尾；小顶/大顶由Less决定。业务代码封装类型，避免到处断言 `any`。

44. 分治和递归有什么区别？

分治将问题拆成相对独立子问题并合并，如归并排序、构树；递归只是实现方式。分治需分析子问题数量、规模和合并成本。

七、图、BFS/DFS、拓扑排序、并查集与回溯

45. 图的存储如何选择？

稀疏图邻接表 $O(V+E)$ 空间，稠密/快速边查询可邻接矩阵 $O(V^2)$ 。无向边双向添加；节点编号连续可Slice，否则Map。

46. BFS 和 DFS 如何选择？

无权最短路和层级扩散用 BFS；连通、路径枚举、拓扑递归和回溯用 DFS。两者都需明确Visited何时标记，BFS通常入队即标记防重复。

47. 多源 BFS 是什么？

所有源点同时入队、距离0，逐层扩散，得到离最近源的距离/时间，如腐烂橘子。不是分别从每个源跑BFS。

48. 拓扑排序解决什么？

有向无环图依赖顺序；Kahn用入度0队列，处理数少于节点数则有环；DFS用三色检测回边。课程、构建和任务依赖常见。

49. 并查集适合什么？

动态合并集合和判断连通。Parent+路径压缩+按秩/大小合并，近似 $O(1)$ ；适合无向连通、冗余边和账户合并，不适合直接求具体路径。

50. 回溯的核心模板是什么？

路径、选择列表、终止条件；做选择 $\rightarrow$ 递归 $\rightarrow$ 撤销。组合控制起点，排列用Used，重复元素先排序并在同层去重，剪枝要证明不会漏解。

51. 网格搜索常见 Bug 是什么？

行列混淆、Visited恢复错误、边界、重复访问和递归深度。连通遍历Visited不恢复；路径回溯离开时恢复。方向数组统一四邻接。

八、动态规划、贪心与状态设计

52. 如何识别动态规划？

求最优/计数/可行，存在重叠子问题和最优子结构；能定义有限状态并从较小状态转移。先写递归关系，再决定记忆化或迭代。

53. DP 五步是什么？

状态含义、转移、初始化、遍历顺序、返回值。再分析是否能压缩空间。状态定义不清时不要先写二维数组。

54. 背包问题如何区分？

0/1每物品一次，容量倒序；完全背包可重复，容量正序。求组合数/排列数时物品和容量循环顺序影响计数语义。

55. 子序列和子数组 DP 有什么区别？

子数组要求连续，状态常表示“以i结尾”；子序列可跳过，常按前缀或二维序列定义。最长公共子序列和最大子数组代表两类。

56. 什么时候用记忆化搜索？

状态转移自然递归、部分状态不会访问、迭代顺序难时。注意缓存Key、基线和递归深度；能清楚自底向上时迭代通常更省栈。

57. 贪心如何证明？

局部选择可交换为某个最优解，或证明选择后剩余仍是同类子问题。只凭“每次选最大看起来对”不够；无法证明时考虑DP。

58. DP 空间压缩有哪些风险？

覆盖仍需要的旧状态，遍历方向错误，二维依赖不完整。压缩前画出转移依赖，只在面试时间允许和正确性稳定时做。

九、Golang 编码、边界、自测与性能

59. Go 算法代码的基本规范是什么？

语义命名、函数短、先处理空输入、避免全局状态、整数溢出意识、注释不变量而非语法。面试中标准库可用就用，但关键结构如Heap按题意实现。

60. Slice 和 Map 有哪些算法陷阱？

Append可能扩容、子Slice共享数组、nil/空语义；Map遍历无序、并发不安全、读取零值需comma-ok。回溯结果必须复制当前Path，否则后续修改覆盖答案。

61. 如何设计自测用例？

最小/空、单元素、一般、重复、全相同、升降序、无解、边界值和触发核心分支。先口述预期，再跟代码逐步执行。

62. 如何防止整数溢出？

中点用 $l+(r-l)/2$ ，和/乘积按范围用int64；比较乘法可改除法但处理0；Go溢出不会自动异常。题目约束决定类型。

63. 递归代码如何避免共享状态污染？

每层明确选择/撤销，返回结果与全局最优分开；多次调用前初始化；并发环境不使用包级变量。追加Path到答案时复制。

64. 如何分析 Go 实现常数成本？

Heap分配、Map、String/Rune转换、递归、排序和复制；面试先复杂度正确，工程上线再Benchmark/Profile。不要为零分配写不可读Unsafe算法。

65. 面试代码写完如何讲复杂度？

定义n/V/E/K，解释每元素进出次数、排序/Heap/递归状态数；空间包含数据结构和调用栈。避免只报结论不说明来源。

十、后端工程手写题、复习路线与最终模板

66. 后端高频工程手写题有哪些？

LRU、Worker Pool、令牌桶/滑动窗口、TopK、并发安全Map、生产者消费者、单例、延时任务简化版和一致性哈希。既考数据结构，也考并发和工程边界。

67. Worker Pool 手写要包含什么？

有界Worker/Queue、Submit Context、Owner关闭、WaitGroup、Panic/Error、停止接收和优雅退出。若只写无限Goroutine不算生产级。

68. 限流器手写要关注什么？

算法状态、时间来源、并发安全、突发、精度和清理。令牌桶适合允许突发，滑动窗口更精确但状态多；测试注入Clock避免Sleep。

69. 一致性哈希手写要关注什么？

哈希环有序结构、虚拟节点、顺时针查找和回绕；处理Hash冲突、权重和节点增删。工程上还需复制、迁移和热点，面试代码可明确简化。

70. 第一轮复习顺序是什么？

链表反转/快慢 -> 树遍历/后序 -> Hash/前缀 -> 双指针/窗口 -> 二分 -> Heap/TopK -> BFS/DFS -> 回溯 -> DP。对应母模板练习册逐项默写。

71. 时间不足时如何取舍？

优先 S+ 题和14类母模板，保证能在20分钟写出并自测；再补动态规划和专项。不要平均刷大量相似题却不会独立复现。

72. 掌握一道题的标准是什么？

能识别模板、解释不变量和复杂度、无提示写出、覆盖边界、完成一种变形，并在隔天重新写。只看答案不算掌握。

73. 如何用两分钟回答算法题？

我先确认输入、返回和边界，用数据规模判断复杂度。先给暴力基线，再指出重复计算或单调性，选择Hash、排序双指针、窗口、二分、Heap、图遍历或DP。编码前定义循环/递归不变量和数据结构含义，写Go代码时处理空值、重复、整数范围和Slice复制。完成后用最小、一般、重复和极端用例走一遍，最后准确说明时间、额外空间和可选方案的权衡。

07-01 | 算法70题高频分类清单

后端算法面试 70 题高频分类清单

定位：按题型和优先级选择练习题；解题框架与 Golang 编码规范见[算法面试总纲]，通用代码模板见[高频母模板排序版]。

适用目标：字节跳动、腾讯等互联网公司后端岗位现场算法面试。

选题原则：优先考虑现场面试中容易出现的题型、常见原题和高频变形，以及解法模板的覆盖价值。

复习定位与优先级说明

- **S+：必须默写**
建议在 15~20 分钟内独立完成，能说明复杂度和关键边界，并处理常见变形。
- **S：必须掌握**
能够独立写出主体代码，理解模板，并能完成简单变形。
- **A：补齐模板**
至少能够快速识别题型，讲出解法并写出核心代码。

一、链表：12 题

优先级	LeetCode 题目	解法分类
S+	206. 反转链表	三指针迭代 / 递归
S+	21. 合并两个有序链表	双指针 + 虚拟头节点
S+	19. 删除链表的倒数第 N 个节点	快慢指针 + 虚拟头节点
S+	141. 环形链表	快慢指针
S+	142. 环形链表 II	快慢指针 + 环入口推导
S+	160. 相交链表	双指针路径对齐
S+	25. K 个一组翻转链表	分组 + 区间反转
S+	146. LRU 缓存	哈希表 + 双向链表
S	92. 反转链表 II	局部反转 + 虚拟头节点

优先级	LeetCode 题目	解法分类
S	23. 合并 K 个升序链表	小顶堆 / 分治归并
S	2. 两数相加	链表模拟 + 进位
A	148. 排序链表	链表归并排序

推荐刷题顺序：

21 → 206 → 19 → 141 → 142 → 160
→ 92 → 25 → 2 → 23 → 148 → 146

链表题模板组合速查：

链表题先不要急着写代码，先按这个顺序判断：

是否头节点会变? → dummy
是否要找倒数/中点/环? → 快慢指针
是否要改 next 指针? → 先保存 next
是否两个链表长度不同? → 路径对齐
是否有序链表合并/排序? → 归并
是否逐位相加? → dummy + carry
是否 0(1) 查询和淘汰? → HashMap + 双向链表

题目	模板组合
206 反转链表	三指针反转
92 反转链表 II	dummy + 找前驱 + 区间反转
25 K 个一组翻转	dummy + 找 kth + 区间反转 + 分组推进
19 删除倒数第 N 个	dummy + 快慢指针 + 删除
141 环形链表	快慢指针
142 环形链表 II	快慢指针相遇 + 环入口推导
160 相交链表	路径对齐
21 合并两个有序链表	dummy + 双指针归并
23 合并 K 个链表	mergeTwoLists + 分治 / 小顶堆
148 排序链表	快慢找中点 + 断链 + 递归归并
2 两数相加	dummy + carry + 同步遍历
146 LRU	HashMap + 双向链表

二、二叉树：13 题

优先级	LeetCode 题目	解法分类
S+	94. 二叉树的中序遍历	递归 / 迭代栈
S+	102. 二叉树的层序遍历	BFS + 队列
S+	104. 二叉树的最大深度	DFS / BFS
S+	98. 验证二叉搜索树	中序遍历 / 上下界递归
S+	236. 二叉树的最近公共祖先	后序 DFS
S	101. 对称二叉树	镜像递归 / BFS
S	226. 翻转二叉树	递归 / BFS
S	105. 从前序与中序遍历序列构造二叉树	递归分治 + 哈希定位
S	110. 平衡二叉树	后序 DFS + 高度返回
S	543. 二叉树的直径	后序 DFS + 全局最优
S	199. 二叉树的右视图	BFS / 右优先 DFS
A	230. 二叉搜索树中第 K 小的元素	中序遍历
A	124. 二叉树中的最大路径和	后序 DFS + 全局最大值

推荐刷题顺序：

```
94 → 104 → 226 → 101 → 102
→ 199 → 98 → 230 → 110 → 543
→ 236 → 105 → 124
```

三、数组、哈希、字符串、矩阵：8 题

优先级	LeetCode 题目	解法分类
S+	1. 两数之和	哈希查补数
S+	128. 最长连续序列	HashSet + 序列起点
S+	56. 合并区间	排序 + 区间合并

优先级	LeetCode 题目	解法分类
S	49. 字母异位词分组	排序签名 / 字符计数
S	54. 螺旋矩阵	四边界模拟
S	240. 搜索二维矩阵 II	从右上角 / 左下角搜索
S	415. 字符串相加	双指针 + 进位模拟
S	238. 除自身以外数组的乘积	前缀积 + 后缀积

推荐刷题顺序：

1 → 49 → 128 → 56
→ 415 → 238 → 54 → 240

四、双指针、滑动窗口、前缀和：8 题

优先级	LeetCode 题目	解法分类
S+	3. 无重复字符的最长子串	滑动窗口 + 哈希
S+	15. 三数之和	排序 + 左右双指针
S+	42. 接雨水	左右双指针 / 单调栈
S+	76. 最小覆盖子串	滑动窗口 + 字符计数
S+	560. 和为 K 的子数组	前缀和 + 哈希计数
S+	239. 滑动窗口最大值	单调队列
S	11. 盛最多水的容器	左右双指针
S	5. 最长回文子串	中心扩展 / 区间 DP

推荐刷题顺序：

11 → 15 → 42
→ 3 → 76
→ 560 → 239
→ 5

接雨水至少掌握双指针；第二遍复习时，再使用单调栈实现一次。

五、二分查找、排序、堆、TopK：8 题

优先级	LeetCode 题目	解法分类
S+	34. 在排序数组中查找元素的第一个和最后一个位置	左右边界二分
S+	33. 搜索旋转排序数组	判断有序区间 + 二分
S+	215. 数组中的第 K 个最大元素	快速选择 / 小顶堆
S+	912. 排序数组	快排 / 归并排序 / 堆排序
S	153. 寻找旋转排序数组中的最小值	二分查找
S	347. 前 K 个高频元素	哈希计数 + 小顶堆 / 桶排序
A	162. 寻找峰值	二分查找
A	295. 数据流的中位数	大顶堆 + 小顶堆

912 必须掌握的三种排序

快速排序
归并排序
堆排序

堆排序必须掌握

1. 建立大顶堆
2. 实现下沉 heapify
3. 交换堆顶与当前末尾
4. 缩小堆的有效范围
5. 对新的堆顶继续执行下沉

推荐刷题顺序：

34 → 33 → 153 → 162
→ 912 (快速排序)
→ 912 (归并排序)
→ 912 (堆排序)
→ 215 → 347 → 295

912 虽然只算一道 LeetCode 题，但快速排序、归并排序和堆排序必须分别手写。

六、栈、队列、单调结构：6 题

优先级	LeetCode 题目	解法分类
S+	20. 有效的括号	栈匹配
S+	739. 每日温度	单调递减栈
S	155. 最小栈	辅助栈 / 同步最小值
S	394. 字符串解码	栈 / 递归解析
S	227. 基本计算器 II	栈 + 运算符处理
A	232. 用栈实现队列	输入栈 + 输出栈

推荐刷题顺序：

20 → 232 → 155 → 739 → 394 → 227

学完每日温度后，再回头使用单调栈重写一次接雨水。

七、DFS、BFS、图、回溯：8 题

优先级	LeetCode 题目	解法分类
S+	200. 岛屿数量	网格 DFS / BFS
S+	207. 课程表	拓扑排序 / DFS 判环
S+	46. 全排列	回溯 + used 数组
S	78. 子集	回溯 / 选与不选
S	39. 组合总和	回溯 + start 下标 + 剪枝
S	79. 单词搜索	网格 DFS + 回溯
S	994. 腐烂的橘子	多源 BFS
A	547. 省份数量	并查集 / DFS 连通分量

推荐刷题顺序：

200 → 994
→ 46 → 78 → 39
→ 79
→ 207 → 547

该模块覆盖：

普通网格 DFS
多源 BFS
回溯搜索
拓扑排序
并查集

八、动态规划、贪心：7 题

优先级	LeetCode 题目	解法分类
S+	53. 最大子数组和	Kadane / 一维 DP
S	198. 打家劫舍	选与不选 DP
S	322. 零钱兑换	完全背包 DP
S	139. 单词拆分	字符串划分 DP + HashSet
S	55. 跳跃游戏	贪心维护最远可达位置
A	300. 最长递增子序列	DP / 贪心 + 二分
A	1143. 最长公共子序列	二维 DP

推荐刷题顺序：

53 → 198 → 55
→ 139 → 322
→ 300 → 1143

九、题目数量核对

高频题型	数量
链表	12
二叉树	13

高频题型	数量
数组、哈希、字符串、矩阵	8
双指针、滑动窗口、前缀和	8
二分查找、排序、堆、TopK	8
栈、队列、单调结构	6
DFS、BFS、图、回溯	8
动态规划、贪心	7
合计	70

十、总体复习顺序

第一阶段：链表
 第二阶段：数组、哈希、双指针、滑动窗口
 第三阶段：二分查找、排序、堆、TopK
 第四阶段：二叉树
 第五阶段：栈、队列、单调结构
 第六阶段：DFS、BFS、图、回溯
 第七阶段：动态规划、贪心

十一、时间不足时的取舍规则

1. 所有 **S+** 题必须完成，并至少复写一次。
2. 所有 **S** 题至少独立完成一次。
3. **A** 题只在 **S+**、**S** 已经稳定后再做。
4. **912. 排序数组** 虽然只算一道题，但快排、归并和堆排序必须分别手写。
5. 做过但第二天无法独立写出的题，不算真正完成。
6. 刷题时优先总结模板，不要只记忆某道题的代码。

十二、建议掌握的核心模板

1. 哈希查找和哈希计数
2. 左右双指针
3. 滑动窗口
4. 前缀和

5. 二分查找与边界二分
6. 快速排序、归并排序、堆排序
7. 小顶堆、大顶堆和 TopK
8. 链表虚拟头节点
9. 链表快慢指针
10. 链表区间反转
11. 二叉树递归遍历
12. 二叉树后序 DFS 返回子树信息
13. 二叉树层序 BFS
14. 普通栈、单调栈、单调队列
15. 网格 DFS / BFS
16. 多源 BFS
17. 回溯：选择、递归、撤销
18. 拓扑排序
19. 并查集
20. 一维 DP、背包 DP、二维 DP
21. 贪心维护最优边界

十三、最终结论

这 70 题的目标不是保证原题直接命中，而是覆盖后端现场算法面试中大多数常见题型和解法模板。

真正有效的完成标准是：

看到题目后能够快速识别所属模板
5 分钟内讲清楚解题思路
15~25 分钟写出完整代码
主动说明时间和空间复杂度
覆盖空输入、越界、重复元素等边界
题目稍作变化后仍然能够调整代码

当你能够把多道题归纳为同一个模板时，才算真正形成面试能力。例如：

206、92、25 → 链表反转模板
104、110、543、124 → 后序 DFS 模板
3、76 → 滑动窗口模板
34、33、153、162 → 二分查找模板
200、79、994 → 网格搜索模板
46、78、39 → 回溯模板

后端算法面试 70 题：高频母模板排序版（Golang）

定位：用少量母模板覆盖高频题和变形；整体解题流程见[算法面试总纲]，选题优先级见[70 题分类清单]。

目标：不是为每道题背一份答案，而是把 70 道题压缩成少量可复用的“母模板”。

本文严格只覆盖此前确定的 70 道题：

- 14 个高频母模板覆盖 58 题
- 12 个专项题覆盖剩余 12 题

排序依据是国内互联网后端现场面试中的相对常见程度、公开面经聚合、题型迁移价值和现场可考性，不代表字节或腾讯官方公布的精确次数。

一、总体优先级

S+：最先背熟

顺序	模板/专项	覆盖题目
1	链表区间反转母模板	206、92、25
2	二叉树后序信息汇总母模板	104、110、543、124、236
3	二叉树遍历/BST/层序母模板	94、98、101、226、230、102、199
4	链表快慢指针与路径对齐母模板	19、141、142、160
5	LRU 专项	146
6	滑动窗口母模板	3、76
7	二分查找母模板	34、33、153、162
8	哈希状态映射母模板	1、49、128、560
9	堆与 TopK 母模板	215、347、295
10	排序专项	912：快排、归并、堆排序

S: 高频补齐

顺序	模板/专项	覆盖题目
11	左右双指针母模板	11、15、42
12	栈与单调结构母模板	20、155、232、394、227、739、239
13	网格搜索母模板	200、79、994
14	链表归并母模板	21、23、148
15	回溯母模板	46、78、39
16	动态规划与贪心母模板	53、198、322、139、55、300、1143
17	拓扑排序专项	207
18	二叉树构造专项	105
19	合并区间专项	56

A: 最后补充

顺序	模板/专项	覆盖题目
20	并查集专项	547
21	单调矩阵搜索专项	240
22	螺旋矩阵专项	54
23	前缀积/后缀积专项	238
24	字符串加法专项	415
25	中心扩展专项	5
26	链表进位模拟专项	2

二、使用口诀

链表题先判断：是否变头、是否找位置、是否改指针、是否多链表
链表要删改构造：先上 dummy
链表要反转：先确定区间，再保存 next
链表倒数、环：快慢指针
链表有序合并：归并
链表长度不同找相交：路径对齐

链表逐位计算: dummy + carry
LRU : HashMap 定位, 双向链表维护顺序
树要子树信息: 后序 DFS
BST 有序性: 中序遍历
树要按层: BFS
连续最长最短: 滑动窗口
有序或局部单调: 二分
存在、频率、历史状态: 哈希
前 K 大: 小顶堆
两端收缩: 双指针
右侧第一个更大: 单调栈
固定窗口最大值: 单调队列
网格连通: DFS
多个起点扩散: 多源 BFS
列举所有方案: 回溯
状态依赖过去: 动态规划

三、公共定义

```
package algorithm

import (
    "container/heap"
    "sort"
)

type ListNode struct {
    Val  int
    Next *ListNode
}

type TreeNode struct {
    Val    int
    Left  *TreeNode
    Right *TreeNode
}

func min(a, b int) int {
    if a < b {
        return a
    }
    return b
}

func max(a, b int) int {
    if a > b {
        return a
    }
    return b
}

func abs(a int) int {
```

```

if a < 0 {
    return -a
}
return a
}

```

四、链表题通用思维框架

链表题不是“看到题套一个模板”，而是先判断操作本质，再组合模板。

4.1 第一反应：先问 5 个问题

1. 会不会改变链表结构？
2. 头节点会不会变？
3. 需要找某个位置吗？
4. 是一个链表，还是两个/多个链表？
5. 是否要求 $O(1)$ 查询、移动或淘汰？

问题	如果答案是“是”	优先想到
会改变结构吗？	删除、插入、反转、合并	dummy + 保存 next
头节点会变吗？	删除头、反转头、合并新链表	dummy
要找位置吗？	倒数、中点、环入口	快慢指针
两个链表有关联吗？	相交、合并、相加	路径对齐 / 归并 / 进位模拟
多个链表吗？	K 个链表	分治归并 / 小顶堆
要 $O(1)$ 查询和移动吗？	LRU	HashMap + 双向链表

4.2 通用解题步骤

- Step 1：判断是否需要 dummy
- Step 2：判断是否需要定位节点
- Step 3：判断是否需要断链/改指针
- Step 4：判断是否需要多个指针协作
- Step 5：写代码前先明确每个指针含义
- Step 6：最后检查头节点、nil、断链、返回值

4.3 Step 1: 先判断要不要 dummy

只要有下面任意一种情况，优先用 dummy：

可能删除头节点
可能返回新的头节点
需要拼接新链表
需要反转局部区间
需要统一处理边界

典型题：

19 删除倒数第 N 个节点
21 合并两个有序链表
2 两数相加
92 反转链表 II
25 K 个一组翻转链表

一句话：

链表题只要“头节点可能变”，先上 dummy。

基础写法：

```
dummy := &ListNode{Next: head}  
// ...  
return dummy.Next
```

构造新链表：

```
dummy := &ListNode{}  
tail := dummy  
// tail.Next = node  
// tail = tail.Next  
return dummy.Next
```

4.4 Step 2：再判断是不是“找位置”

链表不能随机访问，所以很多题本质是“找位置”。

要找什么	模板
倒数第 N 个	快慢指针，fast 先走 N 步
中点	快慢指针，fast 走 2 步，slow 走 1 步
是否有环	快慢指针相遇
环入口	相遇后 head 和 meet 一起走
反转区间前驱	dummy + 循环走 left-1 步

要找什么	模板
K 个一组尾节点	从 groupPrev 往后数 K 个

关键判断：

我要找到的是目标节点，还是目标节点的前一个节点？

删除题通常找“目标节点的前一个节点”；反转题通常找“区间前一个节点 + 区间后一个节点”。

4.5 Step 3：如果要改指针，先保存 next

只要看到反转、删除、插入、断开、拼接，先记一句：

改 `cur.Next` 前，先保存 `next`。

反转链表四步口诀：

存 next
改指向
prev 前进
cur 前进

对应代码：

```
next := cur.Next
cur.Next = prev
prev = cur
cur = next
```

4.6 Step 4：判断一个链表、两个链表、还是多个链表

一个链表常见任务：

反转
删除
找环
找中点
排序

优先模板：

dummy
快慢指针
三指针反转
归并排序

例如 148 排序链表不是单模板，而是组合：

快慢指针找中点 + 断链 + 递归排序 + mergeTwoLists

两个链表常见任务：

题目	本质	模板
21 合并两个有序链表	两个有序序列归并	dummy + 双指针
160 相交链表	长度不同但找同一节点	路径对齐
2 两数相加	两个链表同步模拟加法	dummy + carry

看到两个链表时，先问：

它们是要比较顺序？
还是要对齐路径？
还是要逐位模拟？

多个链表常见是 23 合并 K 个升序链表：

分治归并 / 小顶堆

如果用分治，本质还是复用 `mergeTwoLists`。

4.7 Step 5：写代码前定义指针语义

写代码前先说清楚这些指针：

dummy：虚拟头节点
prev：当前处理位置的前一个节点
cur：当前节点
next：保存 cur.Next，防止断链
fast：快指针
slow：慢指针
groupPrev：当前 K 组前一个节点
kth：当前 K 组最后一个节点
groupNext：当前 K 组后一个节点
tail：构造新链表的尾指针

例如 K 个一组翻转：

groupPrev：这一组前面的节点
kth：这一组最后一个节点
groupNext：下一组的开头

4.8 链表题的思想分类

思想	代表模板	解决问题	关键词	代表题
哨兵思想	dummy	头节点特殊处理、删除头、构造新链表	删除、插入、合并、新链表、局部反转	19、21、2、92、25
双指针定位思想	fast / slow	倒数位置、中点、环、环入口	倒数、中点、是否有环、入口	19、141、142、148
局部结构改造思想	三指针反转、区间反转、K 组反转	改变链表方向	反转、翻转、每 K 个、left 到 right	206、92、25
归并思想	mergeTwoLists、分治 merge	合并有序链表、排序链表	有序、合并、排序、K 个	21、23、148
路径对齐思想	A+B / B+A	两个链表长度不同但要找相同节点	相交、公共节点、长度不同	160
状态模拟思想	dummy + carry	逐位相加、进位、长度不同	数字、相加、进位、逐位	2
索引 + 顺序维护思想	HashMap + 双向链表	O(1) 查询 + 访问顺序维护	LRU、缓存、最近使用、淘汰	146

4.9 链表题快速决策树

题目要反转吗？

是 -> 三指针 / 区间反转 / K 组反转

否 -> 继续

题目要删除/插入/构造新链表吗？

是 -> dummy

否 -> 继续

题目要找倒数、中点、环吗？

是 -> 快慢指针

否 -> 继续

题目有两个链表吗？

是 ->

有序合并？ -> mergeTwoLists

找相交？ -> 路径对齐

数字相加？ -> carry 模拟

否 -> 继续

题目有 K 个链表吗？

是 -> 分治归并 / 小顶堆

否 -> 继续

题目要求 $O(1)$ 查询和淘汰吗?
是 $\rightarrow$ HashMap + 双向链表

4.10 12 道链表题的模板组合表

题目	模板组合
206 反转链表	三指针反转
92 反转链表 II	dummy + 找前驱 + 区间反转
25 K 个一组翻转	dummy + 找 kth + 区间反转 + 分组推进
19 删除倒数第 N 个	dummy + 快慢指针 + 删除
141 环形链表	快慢指针
142 环形链表 II	快慢指针相遇 + 入口推导
160 相交链表	路径对齐
21 合并两个有序链表	dummy + 双指针归并
23 合并 K 个链表	mergeTwoLists + 分治 / 小顶堆
148 排序链表	快慢找中点 + 断链 + 递归归并
2 两数相加	dummy + carry + 同步遍历
146 LRU	HashMap + 双向链表

4.11 写链表代码前的检查清单

1. 我要不要 dummy?
2. 我要返回 head 还是 dummy.Next?
3. 我要找目标节点, 还是目标节点前一个节点?
4. 有没有可能删除/反转头节点?
5. 改 Next 前有没有保存 next?
6. 是否需要断链?
7. 循环条件是 `cur != nil`, 还是 `fast != nil && fast.Next != nil`?
8. 最后有没有接回剩余链表?

尤其注意:

删除题: 找前一个节点
反转题: 保存 next
合并题: 移动 tail/cur
快慢题: 防止 fast.Next 空指针
K 组反转: 不足 K 个不要反转

4.12 推荐学习顺序

第一组：dummy + 基础指针

21 合并两个有序链表

2 两数相加

19 删除倒数第 N 个节点

第二组：反转

206 反转链表

92 反转链表 II

25 K 个一组翻转链表

第三组：快慢指针

141 环形链表

142 环形链表 II

148 排序链表

第四组：路径和归并

160 相交链表

23 合并 K 个升序链表

第五组：工程链表

146 LRU 缓存

最核心的一句话：

先判断是否需要哨兵统一边界；

再用快慢指针定位；

如果要改结构，先保存 next；

如果有序就归并；

如果两个链表长度不同就路径对齐；

如果要 $O(1)$ 查询和移动，就 HashMap + 双向链表。

母模板 1：链表区间反转

识别关键词

反转链表

反转 left 到 right

每 K 个一组反转

局部反转

核心不变量

previous：已经反转部分的头

current：当前待处理节点

next : 提前保存, 防止断链
stop : 本次反转区间的后一个节点

通用模板: 反转 [head, stop)

```
func reverseRange(head, stop *ListNode) *ListNode {
    previous := stop
    current := head

    for current != stop {
        next := current.Next
        current.Next = previous
        previous = current
        current = next
    }

    return previous
}
```

如何套题

206: 反转整个链表

```
func reverseList(head *ListNode) *ListNode {
    return reverseRange(head, nil)
}
```

92: 反转局部区间

1. dummy 指向 head
2. 找到 left 前一个节点 before
3. 找到 right 后一个节点 stop
4. before.Next = reverseRange(区间头, stop)

25: K 个一组反转

1. 从 groupPrev 向后找第 K 个节点
2. 不足 K 个直接结束
3. 保存 groupNext
4. reverseRange(groupHead, groupNext)
5. 旧 groupHead 变成新尾部

覆盖题目

- 1. 反转链表
- 1. 反转链表 II

- 1. K 个一组翻转链表

母模板 2：二叉树后序信息汇总

识别关键词

左右子树给当前节点返回信息
高度
是否平衡
直径
最大路径
最近公共祖先

核心思考

每次先问四个问题：

1. 左子树返回什么？
2. 右子树返回什么？
3. 当前节点如何合并？
4. 向父节点返回什么？

通用模板

```
func solveTree(root *TreeNode) int {  
    answer := 0  
  
    var dfs func(node *TreeNode) int  
  
    dfs = func(node *TreeNode) int {  
        if node == nil {  
            return 0  
        }  
  
        left := dfs(node.Left)  
        right := dfs(node.Right)  
  
        // 在这里使用 left、right 更新全局答案  
        answer = max(answer, left+right)  
  
        // 向父节点只能返回一条方向的信息  
        return max(left, right) + 1  
    }  
  
    dfs(root)  
    return answer  
}
```

各题只改两个地方

题目	全局答案怎么更新	向父节点返回什么
104 最大深度	不需要全局答案	<code>max(left, right)+1</code>
110 平衡二叉树	高度差大于 1 返回 -1	子树高度
543 二叉树直径	<code>left+right</code>	<code>max(left, right)+1</code>
124 最大路径和	<code>node.Val+left+right</code>	<code>node.Val+max(left, right)</code>
236 最近公共祖先	左右都非空返回当前节点	找到的目标节点

覆盖题目

- 1. 二叉树的最大深度
- 1. 平衡二叉树
- 1. 二叉树的直径
- 1. 二叉树中的最大路径和
- 1. 二叉树的最近公共祖先

母模板 3：二叉树遍历、BST 与层序

这类题不是计算复杂状态，而是“以正确顺序访问节点”。

3.1 DFS 遍历骨架

```
func dfs(root *TreeNode) {  
    if root == nil {  
        return  
    }  
  
    // 前序位置  
    dfs(root.Left)  
  
    // 中序位置  
    dfs(root.Right)  
  
    // 后序位置  
}
```

3.2 中序迭代模板

```
func inorderTraversal(root *TreeNode) []int {
    answer := make([]int, 0)
    stack := make([]*TreeNode, 0)
    current := root

    for current != nil || len(stack) > 0 {
        for current != nil {
            stack = append(stack, current)
            current = current.Left
        }

        current = stack[len(stack)-1]
        stack = stack[:len(stack)-1]

        answer = append(answer, current.Val)
        current = current.Right
    }

    return answer
}
```

3.3 层序 BFS 模板

```
func levelOrder(root *TreeNode) [][]int {
    if root == nil {
        return nil
    }

    queue := []*TreeNode{root}
    answer := make([][]int, 0)

    for len(queue) > 0 {
        size := len(queue)
        level := make([]int, 0, size)

        for i := 0; i < size; i++ {
            node := queue[0]
            queue = queue[1:]

            level = append(level, node.Val)

            if node.Left != nil {
                queue = append(queue, node.Left)
            }
            if node.Right != nil {
                queue = append(queue, node.Right)
            }
        }

        answer = append(answer, level)
    }
}
```

```
    return answer
}
```

如何套题

题目	使用方式
94 中序遍历	直接使用中序模板
98 验证 BST	中序结果必须严格递增
230 BST 第 K 小	中序访问到第 K 个节点
101 对称二叉树	同时比较左树左侧与右树右侧
226 翻转二叉树	遍历每个节点并交换左右孩子
102 层序遍历	直接使用 BFS
199 右视图	保存每层最后一个节点

覆盖题目

- 94、98、101、226、230、102、199

母模板 4：链表快慢指针与路径对齐

识别关键词

倒数第 N 个
链表是否有环
环入口
两个链表是否相交

4.1 固定距离模板

```
func removeNthFromEnd(head *ListNode, n int) *ListNode {
    dummy := &ListNode{Next: head}
    fast, slow := dummy, dummy

    for i := 0; i < n; i++ {
        fast = fast.Next
    }

    for fast.Next != nil {
        fast = fast.Next
    }
}
```

```

        slow = slow.Next
    }

    slow.Next = slow.Next.Next
    return dummy.Next
}

```

4.2 不同速度模板

```

func hasCycle(head *ListNode) bool {
    slow, fast := head, head

    for fast != nil && fast.Next != nil {
        slow = slow.Next
        fast = fast.Next.Next

        if slow == fast {
            return true
        }
    }

    return false
}

```

4.3 路径对齐模板

```

func getIntersectionNode(first, second *ListNode) *ListNode {
    a, b := first, second

    for a != b {
        if a == nil {
            a = second
        } else {
            a = a.Next
        }

        if b == nil {
            b = first
        } else {
            b = b.Next
        }
    }

    return a
}

```

覆盖题目

- 1. 删除链表的倒数第 N 个节点
- 1. 环形链表

- 1. 环形链表 II
- 1. 相交链表

母模板 5：滑动窗口

识别关键词

连续子串
最长或最短
窗口内字符出现次数
右指针扩张
左指针收缩

通用模板

```
func slidingWindow(s string) int {  
    count := make(map[byte]int)  
    left := 0  
    answer := 0  
  
    for right := 0; right < len(s); right++ {  
        in := s[right]  
        count[in]++  
  
        for /* 窗口不合法 */ {  
            out := s[left]  
            count[out]--  
            left++  
        }  
  
        // 最长问题：窗口合法后更新  
        answer = max(answer, right-left+1)  
    }  
  
    return answer  
}
```

两种模式

最长合法窗口

right 扩张
窗口非法时收缩
窗口重新合法后更新最大值

对应：3. 无重复字符的最长子串。

最短满足窗口

right 扩张到满足条件
满足后持续收缩
每次收缩前更新最小值

对应：76. 最小覆盖子串。

覆盖题目

- 1. 无重复字符的最长子串
- 1. 最小覆盖子串

母模板 6：二分查找

识别关键词

有序
 $O(\log n)$
第一个或最后一个
旋转数组
峰值
每次可以排除一半

6.1 找第一个满足条件的位置

```
func lowerBound(nums []int, target int) int {  
    left, right := 0, len(nums)  
  
    for left < right {  
        mid := left + (right-left)/2  
  
        if nums[mid] >= target {  
            right = mid  
        } else {  
            left = mid + 1  
        }  
    }  
  
    return left  
}
```

6.2 闭区间搜索

```
func binarySearch(nums []int, target int) int {
    left, right := 0, len(nums)-1

    for left <= right {
        mid := left + (right-left)/2

        if nums[mid] == target {
            return mid
        }

        if nums[mid] < target {
            left = mid + 1
        } else {
            right = mid - 1
        }
    }

    return -1
}
```

各题判断规则

题目	判断后保留哪一半
34 首尾位置	找第一个 <code>>= target</code> 和第一个 <code>> target</code>
33 搜索旋转数组	先判断哪一半有序
153 旋转数组最小值	比较 <code>nums[mid]</code> 和 <code>nums[right]</code>
162 峰值	比较 <code>nums[mid]</code> 和 <code>nums[mid+1]</code>

覆盖题目

- 34、33、153、162

母模板 7：哈希状态映射

识别关键词

是否出现过
出现次数
历史状态
分组
要求 $O(n)$

通用模板

```
func hashTemplate(nums []int) {
    index := make(map[int]int)
    count := make(map[int]int)
    set := make(map[int]struct{})

    for i, value := range nums {
        index[value] = i
        count[value]++
        set[value] = struct{}{}
    }
}
```

四种映射方式

题目	key	value
1 两数之和	数字	下标
49 字母异位词	字符计数签名	字符串列表
128 最长连续序列	数字	是否存在
560 和为 K 的子数组	前缀和	出现次数

前缀和计数模板

```
func subarraySum(nums []int, k int) int {
    count := map[int]int{0: 1}
    prefix := 0
    answer := 0

    for _, value := range nums {
        prefix += value
        answer += count[prefix-k]
        count[prefix]++
    }

    return answer
}
```

覆盖题目

- 1、49、128、560

母模板 8：堆与 TopK

识别关键词

第 K 大
前 K 高频
数据流中位数
只保留 K 个候选

Go 小顶堆定义

```
type IntMinHeap []int

func (h IntMinHeap) Len() int      { return len(h) }
func (h IntMinHeap) Less(i, j int) bool { return h[i] < h[j] }
func (h IntMinHeap) Swap(i, j int)  { h[i], h[j] = h[j], h[i] }

func (h *IntMinHeap) Push(value any) {
    *h = append(*h, value.(int))
}

func (h *IntMinHeap) Pop() any {
    old := *h
    n := len(old)
    value := old[n-1]
    *h = old[:n-1]
    return value
}
```

前 K 大通用模板

```
func topK(nums []int, k int) []int {
    h := &IntMinHeap{}
    heap.Init(h)

    for _, value := range nums {
        heap.Push(h, value)

        if h.Len() > k {
            heap.Pop(h)
        }
    }

    return append([]int(nil), (*h)...)
}
```

变形规则

题目	堆中保存什么
215 第 K 大	数字，保持堆大小为 K
347 前 K 高频	{数字, 频率}，按频率比较
295 数据流中位数	左侧大顶堆 + 右侧小顶堆

覆盖题目

- 215、347、295

母模板 9：左右双指针

识别关键词

从两端向中间
排序后找组合
移动较小的一侧
左右边界共同决定答案

通用模板

```
func twoPointers(nums []int) int {
    left, right := 0, len(nums)-1
    answer := 0

    for left < right {
        // 根据 nums[left]、nums[right] 计算答案

        if /* 左侧是当前瓶颈 */ {
            left++
        } else {
            right--
        }
    }

    return answer
}
```

变形规则

题目	移动规则
11 盛最多水	移动高度较小的一侧
15 三数之和	和小左移，和大右移
42 接雨水	先处理左右边界较低的一侧

覆盖题目

- 11、15、42

母模板 10：栈与单调结构

这个模块有同一个核心：延迟处理，等待后面的信息到来。

10.1 普通栈

```
stack := make([]int, 0)

stack = append(stack, value)    // push
top := stack[len(stack)-1]     // top
stack = stack[:len(stack)-1]   // pop
```

对应：

- 20 有效括号
- 155 最小栈
- 232 用栈实现队列
- 394 字符串解码
- 227 基本计算器 II

10.2 单调栈

识别关键词

右边第一个更大
等待多少天
元素暂时无法确定答案

```
func dailyTemperatures(values []int) []int {
    answer := make([]int, len(values))
```

```

stack := make([]int, 0)

for i, value := range values {
    for len(stack) > 0 &&
        value > values[stack[len(stack)-1]] {

        index := stack[len(stack)-1]
        stack = stack[:len(stack)-1]
        answer[index] = i - index
    }

    stack = append(stack, i)
}

return answer
}

```

对应：739 每日温度。

10.3 单调队列

识别关键词

固定窗口
每个窗口最大值
队头始终是最大值

```

func maxSlidingWindow(nums []int, k int) []int {
    deque := make([]int, 0)
    answer := make([]int, 0)

    for right, value := range nums {
        for len(deque) > 0 && deque[0] <= right-k {
            deque = deque[1:]
        }

        for len(deque) > 0 &&
            nums[deque[len(deque)-1]] <= value {
            deque = deque[:len(deque)-1]
        }

        deque = append(deque, right)

        if right >= k-1 {
            answer = append(answer, nums[deque[0]])
        }
    }

    return answer
}

```

对应：239 滑动窗口最大值。

覆盖题目

- 20、155、232、394、227、739、239

母模板 11：网格搜索

识别关键词

二维网格
上下左右
连通区域
路径不能重复经过
多个起点同时扩散

11.1 普通 DFS

```
directions := [][]int{
    {-1, 0},
    {1, 0},
    {0, -1},
    {0, 1},
}

var dfs func(row, col int)

dfs = func(row, col int) {
    if /* 越界或不能访问 */ {
        return
    }

    // 标记已访问

    for _, direction := range directions {
        dfs(row+direction[0], col+direction[1])
    }
}
```

对应：200 岛屿数量。

11.2 回溯搜索

进入格子：标记已访问
继续搜索四个方向
返回前：恢复现场

对应：79 单词搜索。

11.3 多源 BFS

把所有起点一次性加入队列
每轮处理当前层
处理完一层时间加一

对应：994 腐烂的橘子。

覆盖题目

- 200、79、994

母模板 12：链表归并

识别关键词

两个或 K 个有序链表
链表排序
 $O(n \log n)$

合并两个有序链表

```
func mergeTwoLists(first, second *ListNode) *ListNode {
    dummy := &ListNode{}
    tail := dummy

    for first != nil && second != nil {
        if first.Val <= second.Val {
            tail.Next = first
            first = first.Next
        } else {
            tail.Next = second
            second = second.Next
        }

        tail = tail.Next
    }

    if first != nil {
        tail.Next = first
    } else {
        tail.Next = second
    }

    return dummy.Next
}
```

变形规则

题目	使用方式
21 合并两个链表	直接使用模板
23 合并 K 个链表	分治地调用模板，或小顶堆
148 排序链表	找中点、递归排序两半、调用模板合并

覆盖题目

- 21、23、148

母模板 13：回溯

识别关键词

所有排列
所有组合
所有子集
列举全部方案
做选择、递归、撤销

通用模板

```
func backtrack(start int) {  
    if /* 到达结束条件 */ {  
        // 复制 path 保存答案  
        return  
    }  
  
    for i := start; i < len(nums); i++ {  
        if /* 当前选择不合法 */ {  
            continue  
        }  
  
        path = append(path, nums[i])  
  
        backtrack(/* 下一个状态 */)  
  
        path = path[:len(path)-1]  
    }  
}
```

三种开关

题目	防止重复选择的方式	下一层参数
46 全排列	<code>used[i]</code>	从 0 重新枚举
78 子集	<code>start</code>	<code>i+1</code>
39 组合总和	<code>start</code>	<code>i</code> ，允许重复使用

覆盖题目

- 46、78、39

母模板 14：动态规划与贪心

DP 不能只背代码，必须先定义状态。

通用五步

1. `dp[i]` 或 `dp[i][j]` 表示什么？
2. 当前状态从哪些历史状态转移？
3. 初始值是什么？
4. 按什么顺序遍历？
5. 最终答案在哪个状态？

14.1 一维状态压缩

```
previousTwo := 0
previousOne := 0

for _, value := range nums {
    current := max(previousOne, previousTwo+value)
    previousTwo = previousOne
    previousOne = current
}
```

对应：198 打家劫舍。

14.2 完全背包

```
dp := make([]int, target+1)

for current := 1; current <= target; current++ {
    for _, item := range items {
        if current >= item {
```

```

        // 从 dp[current-item] 转移
    }
}
}

```

对应：322 零钱兑换。

14.3 字符串分割 DP

```

dp[0] = true

for end := 1; end <= len(s); end++ {
    for start := 0; start < end; start++ {
        if dp[start] && dictionaryContains(s[start:end]) {
            dp[end] = true
        }
    }
}

```

对应：139 单词拆分。

14.4 双序列二维 DP

```

for i := 1; i <= len(first); i++ {
    for j := 1; j <= len(second); j++ {
        if first[i-1] == second[j-1] {
            dp[i][j] = dp[i-1][j-1] + 1
        } else {
            dp[i][j] = max(dp[i-1][j], dp[i][j-1])
        }
    }
}

```

对应：1143 最长公共子序列。

其余开关

题目	状态或贪心含义
53 最大子数组和	以当前位置结尾的最大和
55 跳跃游戏	当前能够到达的最远位置
300 最长递增子序列	长度为 i 的序列最小结尾

覆盖题目

- 53、198、322、139、55、300、1143

四、12 个专项题

这些题无法通过同一母模板覆盖很多题。不要为了追求“模板化”而强行归类。

专项 1: 146 LRU 缓存

关键词: get/put O(1)、最近使用、淘汰最久未使用
结构: 哈希表 + 双向链表

必须独立手写:

```
map[key]*Node
remove(node)
addFirst(node)
moveToFront(node)
removeLast()
```

专项 2: 912 排序数组

必须分别手写:

快速排序
归并排序
堆排序

堆排序核心

```
func heapSort(nums []int) {
    n := len(nums)

    for i := n/2 - 1; i >= 0; i-- {
        siftDown(nums, i, n)
    }

    for end := n - 1; end > 0; end-- {
        nums[0], nums[end] = nums[end], nums[0]
        siftDown(nums, 0, end)
    }
}

func siftDown(nums []int, root, size int) {
    for {
        left := root*2 + 1
        if left >= size {
            return
        }
    }
}
```

```

        largest := left
        right := left + 1

        if right < size && nums[right] > nums[left] {
            largest = right
        }

        if nums[root] >= nums[largest] {
            return
        }

        nums[root], nums[largest] = nums[largest], nums[root]
        root = largest
    }
}

```

专项 3: 207 课程表

关键词: 依赖、先修、有向图、是否有环
解法: 拓扑排序

构建邻接表
统计入度
入度为 0 的节点入队
不断删除节点并减少后继入度
最终访问数等于节点数说明无环

专项 4: 105 构造二叉树

前序第一个元素是根
中序确定左右子树范围
哈希表快速定位根在中序中的位置

专项 5: 56 合并区间

按照左端点排序
和最后一个区间重叠则扩张右端点
否则加入新区间

专项 6: 547 省份数量

关键词: 连通分量、朋友圈、集合合并
解法: 并查集

专项 7: 240 搜索二维矩阵 II

从右上角开始
当前值太大向左
当前值太小向下

专项 8: 54 螺旋矩阵

维护 top、bottom、left、right
每走完一条边收缩对应边界
每次反向遍历前检查边界

专项 9: 238 除自身以外数组的乘积

第一遍: 答案中写入左侧乘积
第二遍: 乘上右侧乘积

专项 10: 415 字符串相加

从两个字符串末尾向前
逐位相加
维护 carry
最后反转结果

专项 11: 5 最长回文子串

枚举每个中心
分别扩展奇数长度和偶数长度
记录最长区间

专项 12: 2 两数相加

同时遍历两个链表
对应位相加
维护 carry
使用 dummy 构造结果链表

五、70 题覆盖检查

14 个母模板覆盖 58 题

母模板	覆盖题目
链表区间反转	206、92、25
二叉树后序信息汇总	104、110、543、124、236
二叉树遍历/BST/层序	94、98、101、226、230、102、199
链表快慢指针/路径对齐	19、141、142、160
滑动窗口	3、76
二分查找	34、33、153、162
哈希状态映射	1、49、128、560
堆与 TopK	215、347、295
左右双指针	11、15、42
栈与单调结构	20、155、232、394、227、739、239
网格搜索	200、79、994
链表归并	21、23、148
回溯	46、78、39
动态规划与贪心	53、198、322、139、55、300、1143

12 个专项题覆盖剩余 12 题

146、912、207、105、56、547
240、54、238、415、5、2

合计：

$58 + 12 = 70$

六、实际复习顺序

第一轮：只背 S+ 母模板

链表区间反转
二叉树后序 DFS
二叉树遍历/BST/BFS
链表快慢指针
滑动窗口
二分查找
哈希状态映射
堆与 TopK
LRU
排序

第二轮：补 S 级

左右双指针
栈与单调结构
网格搜索
链表归并
回溯
动态规划
拓扑排序
构造二叉树
合并区间

第三轮：补专项 A 级

并查集
矩阵搜索
螺旋矩阵
前后缀乘积
字符串加法
中心扩展
链表进位

七、掌握标准

每个母模板必须做到：

看到关键词后 30 秒内识别模板
2 分钟内讲清楚状态和不变量
15~20 分钟内写完代表题
能说明该模板如何切换到同类其他题

例如：

看到“直径、最大路径、平衡”：

立即想到后序 DFS，先拿左右子树信息，再更新当前答案。

看到“连续子串、最长或最短”：

立即想到滑动窗口，右边扩张，左边恢复条件。

看到“第一个位置、旋转数组、峰值”：

立即想到二分，核心是证明哪一半可以排除。

看到“局部反转、每 K 个反转”：

立即想到 `reverseRange(head, stop)`。

07-03 | 动态规划与图遍历高频背诵清单 | Golang

动态规划与图遍历：高频背诵清单（Golang）

定位：动态规划、图遍历和回溯的专项复习册；题型识别和现场答题流程见[算法面试总纲]。

适用范围：此前确定的 70 道后端算法面试题。

目标：只背最有代表性的题目，用少量母题覆盖动态规划、图遍历和回溯中的高频模板。

一、动态规划：建议背 5 题

第一优先级：必须默写

1. 53 最大子数组和

代表模板

以当前位置结尾的最优值

核心状态

```
dp[i] = max(nums[i], dp[i-1] + nums[i])
```

Golang 模板

```
func maxSubArray(nums []int) int {
    current := nums[0]
    answer := nums[0]

    for i := 1; i < len(nums); i++ {
        current = max(nums[i], current+nums[i])
        answer = max(answer, current)
    }

    return answer
}
```

必须掌握的点

current 表示：以当前位置结尾的最大子数组和
answer 表示：全局最大子数组和

2. 198 打家劫舍

代表模板

选或不选

核心状态

选当前: $dp[i-2] + nums[i]$
不选当前: $dp[i-1]$

Golang 模板

```
func rob(nums []int) int {
    previousTwo := 0
    previousOne := 0

    for _, value := range nums {
        current := max(previousOne, previousTwo+value)
        previousTwo = previousOne
        previousOne = current
    }

    return previousOne
}
```

必须掌握的点

previousTwo : $dp[i-2]$
previousOne : $dp[i-1]$
current : $dp[i]$

3. 322 零钱兑换

代表模板

完全背包
每种硬币可以重复使用
求最少数量

核心状态

$dp[x]$: 凑出金额 x 所需的最少硬币数
 $dp[x] = \min(dp[x], dp[x - \text{coin}] + 1)$
 $dp[0] = 0$

Golang 模板

```
func coinChange(coins []int, amount int) int {
    const inf = int(^uint(0) >> 1)

    dp := make([]int, amount+1)

    for current := 1; current <= amount; current++ {
        dp[current] = inf

        for _, coin := range coins {
            if current >= coin && dp[current-coin] != inf {
                dp[current] = min(
                    dp[current],
                    dp[current-coin]+1,
                )
            }
        }
    }

    if dp[amount] == inf {
        return -1
    }

    return dp[amount]
}
```

4. 139 单词拆分

代表模板

字符串分割 DP
枚举最后一段

识别关键词

能否拆分
前缀是否合法
最后一段是什么

Golang 模板

```
func wordBreak(s string, wordDict []string) bool {
    dictionary := make(map[string]struct{})

    for _, word := range wordDict {
        dictionary[word] = struct{}{}
    }

    dp := make([]bool, len(s)+1)
    dp[0] = true

    for end := 1; end <= len(s); end++ {
        for start := 0; start < end; start++ {
            if !dp[start] {
                continue
            }

            if _, exists := dictionary[s[start:end]]; exists {
                dp[end] = true
                break
            }
        }
    }

    return dp[len(s)]
}
```

必须掌握的点

dp[end] 表示: s[0:end] 能否被字典中的单词拆分
枚举 start, 相当于枚举最后一个单词

5. 1143 最长公共子序列

代表模板

两个序列的二维 DP

核心状态

```
dp[i][j] :  
first 前 i 个字符与 second 前 j 个字符的最长公共子序列长度
```

状态转移

字符相等：看左上角
字符不等：看上方和左方

Golang 模板

```
func longestCommonSubsequence(first, second string) int {  
    m, n := len(first), len(second)  
  
    dp := make([][]int, m+1)  
    for i := range dp {  
        dp[i] = make([]int, n+1)  
    }  
  
    for i := 1; i <= m; i++ {  
        for j := 1; j <= n; j++ {  
            if first[i-1] == second[j-1] {  
                dp[i][j] = dp[i-1][j-1] + 1  
            } else {  
                dp[i][j] = max(  
                    dp[i-1][j],  
                    dp[i][j-1],  
                )  
            }  
        }  
    }  
  
    return dp[m][n]  
}
```

二、动态规划第二优先级

6. 300 最长递增子序列

建议掌握程度

记住核心思想和 `sort.Search` 写法，不必把它和普通二维 DP 混在一起背。

核心思想

```
tails[i] :  
长度为 i+1 的递增子序列中，最小的结尾值
```

Golang 模板

```
import "sort"  
  
func lengthOfLIS(nums []int) int {  
    tails := make([]int, 0)  
  
    for _, value := range nums {  
        index := sort.Search(  
            len(tails),  
            func(i int) bool {  
                return tails[i] >= value  
            },  
        )  
  
        if index == len(tails) {  
            tails = append(tails, value)  
        } else {  
            tails[index] = value  
        }  
    }  
  
    return len(tails)  
}
```

三、图遍历：建议背 4 题

第一优先级：必须默写

1. 200 岛屿数量

代表模板

```
网格 DFS  
连通分量计数
```

必须背的结构

```
越界判断  
当前格子是否合法
```


标记已访问
向四个方向递归

Golang 模板

```
func numIslands(grid [][]byte) int {
    if len(grid) == 0 {
        return 0
    }

    rows, cols := len(grid), len(grid[0])
    answer := 0

    var dfs func(row, col int)

    dfs = func(row, col int) {
        if row < 0 || row >= rows ||
            col < 0 || col >= cols ||
            grid[row][col] != '1' {
            return
        }

        grid[row][col] = '0'

        dfs(row-1, col)
        dfs(row+1, col)
        dfs(row, col-1)
        dfs(row, col+1)
    }

    for row := 0; row < rows; row++ {
        for col := 0; col < cols; col++ {
            if grid[row][col] == '1' {
                answer++
                dfs(row, col)
            }
        }
    }

    return answer
}
```

2. 994 腐烂的橘子

代表模板

多源 BFS
多个起点同时扩散
按层统计时间

核心区别

开始时，把所有起点一次性加入队列

Golang 模板

```
func orangesRotting(grid [][]int) int {
    rows, cols := len(grid), len(grid[0])
    queue := make([][2]int, 0)
    fresh := 0

    for row := 0; row < rows; row++ {
        for col := 0; col < cols; col++ {
            if grid[row][col] == 2 {
                queue = append(queue, [2]int{row, col})
            } else if grid[row][col] == 1 {
                fresh++
            }
        }
    }

    directions := [][]int{
        {-1, 0},
        {1, 0},
        {0, -1},
        {0, 1},
    }

    minutes := 0

    for len(queue) > 0 && fresh > 0 {
        size := len(queue)

        for i := 0; i < size; i++ {
            current := queue[0]
            queue = queue[1:]

            for _, direction := range directions {
                nextRow := current[0] + direction[0]
                nextCol := current[1] + direction[1]

                if nextRow < 0 || nextRow >= rows ||
                    nextCol < 0 || nextCol >= cols ||
                    grid[nextRow][nextCol] != 1 {
                    continue
                }

                grid[nextRow][nextCol] = 2
                fresh--
                queue = append(queue, [2]int{nextRow, nextCol})
            }
        }

        minutes++
    }
```

```

    }

    if fresh > 0 {
        return -1
    }

    return minutes
}

```

3. 207 课程表

代表模板

有向图
 依赖关系
 拓扑排序
 判断是否有环

必须背的四步

1. 建邻接表
2. 统计入度
3. 入度为 0 的节点入队
4. 处理节点并减少后继节点入度

Golang 模板

```

func canFinish(numCourses int, prerequisites [][]int) bool {
    graph := make([][]int, numCourses)
    indegree := make([]int, numCourses)

    for _, edge := range prerequisites {
        course := edge[0]
        prerequisite := edge[1]

        graph[prerequisite] = append(graph[prerequisite], course)
        indegree[course]++
    }

    queue := make([]int, 0)

    for course := 0; course < numCourses; course++ {
        if indegree[course] == 0 {
            queue = append(queue, course)
        }
    }

    visited := 0

    for len(queue) > 0 {

```

```

    current := queue[0]
    queue = queue[1:]
    visited++

    for _, next := range graph[current] {
        indegree[next]--

        if indegree[next] == 0 {
            queue = append(queue, next)
        }
    }
}

return visited == numCourses
}

```

4. 79 单词搜索

代表模板

网格 DFS + 回溯
访问后还要恢复

和岛屿数量的区别

岛屿数量：标记后不恢复
单词搜索：当前路径结束后必须恢复

Golang 模板

```

func exist(board [][]byte, word string) bool {
    rows, cols := len(board), len(board[0])

    var dfs func(row, col, index int) bool

    dfs = func(row, col, index int) bool {
        if index == len(word) {
            return true
        }

        if row < 0 || row >= rows ||
            col < 0 || col >= cols ||
            board[row][col] != word[index] {
            return false
        }

        original := board[row][col]
        board[row][col] = '#'

        found := dfs(row-1, col, index+1) ||

```

```

        dfs(row+1, col, index+1) ||
        dfs(row, col-1, index+1) ||
        dfs(row, col+1, index+1)

        board[row][col] = original

        return found
    }

    for row := 0; row < rows; row++ {
        for col := 0; col < cols; col++ {
            if dfs(row, col, 0) {
                return true
            }
        }
    }

    return false
}

```

四、图遍历第二优先级

5. 547 省份数量

代表模板

并查集
连通分量
集合合并

建议掌握程度

建议背 `Find` 和 `Union` 两个函数。

时间极度有限时，也可以直接用 DFS 解，不一定必须使用并查集。

Golang 模板

```

type UnionFind struct {
    parent []int
    count  int
}

func NewUnionFind(n int) *UnionFind {
    parent := make([]int, n)

    for i := range parent {
        parent[i] = i
    }
}

```

```

    return &UnionFind{
        parent: parent,
        count: n,
    }
}

func (u *UnionFind) Find(x int) int {
    if u.parent[x] != x {
        u.parent[x] = u.Find(u.parent[x])
    }

    return u.parent[x]
}

func (u *UnionFind) Union(a, b int) {
    rootA := u.Find(a)
    rootB := u.Find(b)

    if rootA == rootB {
        return
    }

    u.parent[rootB] = rootA
    u.count--
}

```

五、回溯：建议额外背 2 题

1. 46 全排列

代表模板

used 数组
每一层从所有元素中选择
顺序不同算不同答案

Golang 模板

```

func permute(nums []int) [][]int {
    answer := make([][]int, 0)
    path := make([]int, 0, len(nums))
    used := make([]bool, len(nums))

    var backtrack func()

    backtrack = func() {
        if len(path) == len(nums) {
            answer = append(answer, append([]int(nil), path...))
            return
        }
    }
}

```

```

    }

    for i := 0; i < len(nums); i++ {
        if used[i] {
            continue
        }

        used[i] = true
        path = append(path, nums[i])

        backtrack()

        path = path[:len(path)-1]
        used[i] = false
    }

    backtrack()

    return answer
}

```

2. 39 组合总和

代表模板

start 下标
 同一个元素可以重复使用
 排序后剪枝

Golang 模板

```

import "sort"

func combinationSum(candidates []int, target int) [][]int {
    sort.Ints(candidates)

    answer := make([][]int, 0)
    path := make([]int, 0)

    var backtrack func(start, remain int)

    backtrack = func(start, remain int) {
        if remain == 0 {
            answer = append(answer, append([]int(nil), path...))
            return
        }

        for i := start; i < len(candidates); i++ {
            value := candidates[i]

```

```

        if value > remain {
            break
        }

        path = append(path, value)

        backtrack(i, remain-value)

        path = path[:len(path)-1]
    }
}

backtrack(0, target)

return answer
}

```

和 78 子集的关系

78 子集不需要单独死背
只需把“每到一节点都保存一次 path”加入组合模板即可

六、最终最小背诵清单

动态规划必须背

53 最大子数组和
198 打家劫舍
322 零钱兑换
139 单词拆分
1143 最长公共子序列

动态规划第二优先级

300 最长递增子序列

图遍历必须背

200 岛屿数量
994 腐烂的橘子
207 课程表
79 单词搜索

回溯必须背

46 全排列
39 组合总和

图遍历第二优先级

547 省份数量

七、最小背诵数量

优先背熟 11 道代表题：

DP : 5 道
图遍历: 4 道
回溯: 2 道

这 11 道题覆盖：

一维 DP
选或不选 DP
完全背包
字符串分割 DP
二维序列 DP
普通网格 DFS
带恢复的网格 DFS
多源 BFS
拓扑排序
排列回溯
组合回溯

时间极度有限时，先练到闭眼能写：

200 岛屿数量
994 腐烂的橘子
207 课程表
53 最大子数组和
198 打家劫舍
322 零钱兑换

八、背诵标准

动态规划

每道题必须能说清楚：

1. 状态定义
2. 状态转移方程
3. 初始化
4. 遍历顺序
5. 最终答案

图遍历

每道题必须能说清楚：

1. 节点是什么
2. 边是什么
3. 何时标记 `visited`
4. 队列或递归中保存什么
5. 什么时候结束

回溯

每道题必须能说清楚：

1. 当前有哪些选择
2. 什么时候结束
3. 如何做选择
4. 如何递归
5. 如何撤销选择

九、面试现场识别口诀

连续最优：先想一维 DP
不能相邻选：选或不选
可以重复凑目标：完全背包
字符串能否拆分：枚举最后一段
两个字符串比较：二维 DP
岛屿和连通区域：网格 DFS
多个起点同时扩散：多源 BFS
依赖和先修关系：拓扑排序
网格路径要恢复：DFS + 回溯
所有排列：used 数组
组合不看顺序：start 下标